BÁO CÁO BÀI TẬP HW02: KIỂM THỬ MIỀN (DOMAIN TESTING) & PHÂN TÍCH GIÁ TRỊ BIÊN (BVA) TRÊN HỆ THỐNG ESHOP

Môn học: Kiểm thử Phần mềm (CS423 / CSC13003) – FIT@HCMUS
Sinh viên thực hiện: Nguyễn Quốc Kỳ
MSSV: 23120138 – **Lớp:** 23CLC02
Hệ thống thử nghiệm (SUT): EShop (Web, Admin, Mobile, Backend REST API)
Tài liệu đặc tả nguồn (SRS): `specs/system_requirements_specification.md`
Ngày báo cáo: 18/10/2026

1. TỔNG QUAN & PHƯƠNG PHÁP LUẬN KIỂM THỬ

1.1 Mục Tiêu & Cơ Sở Lý Thuyết

Báo cáo này trình bày quy trình thiết kế kịch bản kiểm thử hộp đen (Black-box Test Design) áp dụng hai kỹ thuật cốt lõi:

1. **Phân vùng tương đương (Equivalence Partitioning - EP) theo ISTQB FL v4.0 §4.2.1:**
   - Phân chia miền dữ liệu của từng biến đầu vào thành các lớp tương đương hợp lệ (**Valid Equivalence Partitions - VEP**) và các lớp tương đương không hợp lệ (**Invalid Equivalence Partitions - IEP**).
   - **Nguyên tắc Giả định Đơn lỗi (Single Fault Assumption):** Khi kiểm thử một lớp không hợp lệ (IEP), chỉ cho phép duy nhất một tham số mang giá trị sai; tất cả các tham số còn lại bắt buộc phải giữ giá trị hợp lệ cơ sở để tránh hiện tượng che giấu lỗi (error masking).
2. **Phân tích giá trị biên (Boundary Value Analysis - BVA) theo ISTQB FL v4.0 §4.2.2:**
   - Xác định hành vi của hệ thống tại các ranh giới của các tập hợp có thứ tự (ordered domains: số, độ dài chuỗi, ngày tháng, thời gian).
   - Áp dụng kỹ thuật **2-Value BVA** (On-point và Off-point) và mở rộng **3-Value BVA** (bổ sung In-point) với bước vi sai nhỏ nhất ϵ.
   - **Định nghĩa các điểm:**
     - **On-point:** Giá trị nằm chính xác trên đường biên giới.
     - **Off-point:** Giá trị gần nhất với biên nằm ở phía đối diện (ngoài vùng nếu On nằm trong, hoặc trong vùng nếu On nằm ngoài).
     - **In-point:** Giá trị nằm liền kề bên trong phân vùng.

1.2 Nguyên Tắc "Đặc Tả Là Căn Cứ Duy Nhất" (Specification as Sole Truth)

- Mọi điều kiện, thông báo lỗi, mã trạng thái HTTP và kết quả mong đợi (Expected Results) đều được trích dẫn trực tiếp từ tài liệu đặc tả `specs/system_requirements_specification.md`.
- Sự sai lệch giữa hành vi thực tế của SUT EShop và tài liệu đặc tả được xác định là một khiếm khuyết (Defect/Bug).

1.3 Danh Sách 4 Tính Năng Được Lựa Chọn

Pool	Mã FR	Tên chức năng	Phân hệ	Đặc điểm kiểm thử biên & miền dữ liệu
Pool A	**FR-02**	Đăng nhập & Khóa tài khoản *(Login & Account Lockout)*	Web / API	Biên số lần thử sai ($N \ge 3$); Biên thời gian phạt 30 giây ($29s, 30s, 31s$); Miền email/password.
Pool B	**FR-07**	Giỏ hàng *(Shopping Cart)*	Web	Biên số lượng ($0, 1, 2, \text{stock}$); Chuyển đổi trạng thái $1 \to 0$; Tính toán số học thành tiền ($Price \times Quantity$).
Pool C	**FR-17**	Quản lý Mã Giảm Giá *(Coupon CRUD - Admin)*	Admin / API	Ràng buộc bất đẳng thức: `min_order_amount >= 0`, `max_uses_per_user >= 1`, `discount_value > 0`, biên `percent` ($0%, 1%, 100%, 101%$), biên ngày `expired_at`.
Pool D	**FR-20 & FR-10**	Hủy đơn hàng theo trạng thái *(Cancel Order - Mobile)*	Mobile / API	Kiểm thử miền trạng thái hữu hạn theo State Machine: `pending`, `confirmed` (hủy được) vs. `shipping`, `delivered`, `canceled` (bị chặn hủy).

2. TÍNH NĂNG A (POOL A): FR-02 – ĐĂNG NHẬP & KHÓA TÀI KHOẢN

2.1 Bước 1: Nhận diện biến đầu vào & Ràng buộc (Step 1 Deliverables)

Trích dẫn đặc tả SRS FR-02:

- Người dùng nhập Email và Mật khẩu.
- Sau mỗi lần đăng nhập sai, hệ thống tăng bộ đếm lên đúng 1 đơn vị.
- Nếu đăng nhập sai từ 3 lần trở lên liên tiếp, tài khoản bị tạm khóa 30 giây (môi trường demo). Hệ thống trả về thông báo lỗi phù hợp; không để lộ chi tiết nguyên nhân.
- Đăng nhập thành công trả về JWT Token. Token được lưu phía client và gửi kèm tất cả các yêu cầu có xác thực qua header `Authorization: Bearer <token>`.
- Trường email phải dùng `type="email"` (có validate HTML5 format).

Variable ID	Variable Name	Data Type	Mandatory?	Precision (ϵ)	Specification Constraint (Quote)	Assumptions / Ambiguities
VAR-02-01	email	String	Có	1 char	Trích dẫn SRS FR-02: *"Người dùng nhập Email và Mật khẩu."* và *"Trường email phải dùng* `type="email"` *(có validate HTML5 format)."* (Đối chiếu FR-01: *"Email phải có định dạng hợp lệ (*`user@domain.com`*)"*)	Cần có tài khoản tồn tại trong CSDL (`test@eshop.com`); SRS không nêu độ dài tối đa $\to$ Áp dụng chuẩn RFC 5321 / RFC 3696: tối đa 254 ký tự; Trình duyệt validate format HTML5 trước khi submit; Backend `POST /api/login` cần validate tính hợp lệ.
VAR-02-02	password	String	Có	1 char	Trích dẫn SRS FR-02: *"Người dùng nhập Email và Mật khẩu."* (Đối chiếu FR-01: *"Tối thiểu 8 ký tự, có ít nhất 1 chữ hoa, 1 chữ thường, 1 chữ số và 1 ký tự đặc biệt"*)	Khớp với mật khẩu đã băm trong CSDL. Form đăng nhập không chặn mật khẩu yếu khi gõ, chỉ kiểm tra tính xác thực. SRS không nêu cận trên $\to$ Áp dụng chuẩn an toàn OWASP ASVS: tối đa 255 ký tự để chống tấn công DoS tính toán hash. Mật khẩu mẫu: `Test1234!`.
VAR-02-03	failed_attempts	Integer	Hệ thống	1 lần	Trích dẫn SRS FR-02: *"Sau mỗi lần đăng nhập sai, hệ thống tăng bộ đếm lên đúng 1 đơn vị."* và *"Nếu đăng nhập sai từ 3 lần trở lên liên tiếp, tài khoản bị tạm khóa 30 giây"*	Biến trạng thái đếm số lần sai liên tiếp theo từng tài khoản (`email`). Đăng nhập thành công trước khi đạt ngưỡng 3 lần sẽ reset bộ đếm về 0.
VAR-02-04	lockout_time	Seconds	Hệ thống	1 giây	Trích dẫn SRS FR-02: *"tài khoản bị tạm khóa 30 giây (môi trường demo). Hệ thống trả về thông báo lỗi phù hợp; không để lộ chi tiết nguyên nhân."*	Thời gian phạt tính từ lúc đăng nhập sai lần thứ 3 ($T = 30s$). Trong thời gian này, mọi thử nghiệm đăng nhập đều bị từ chối; sau 30s tài khoản tự động mở khóa. Thông báo lỗi phải trung tính để bảo mật.

2.2 Bước 2: Phân vùng tương đương (Step 2 Deliverables)

> **Nguyên tắc Giả định Đơn lỗi (Single Fault Assumption - ISTQB FL §4.2.1):** Khi kiểm thử một phân vùng không hợp lệ (IEP), chỉ duy nhất 1 điều kiện hoặc biến đầu vào mang giá trị lỗi; toàn bộ các tham số còn lại phải được giữ ở giá trị hợp lệ cơ sở (Baseline) nhằm cô lập nguyên nhân từ chối và ngăn chặn hiện tượng che giấu lỗi (error masking).

Class ID	Variable / Constraint	Domain / Sub-condition	Type (VEP / IEP)	Representative Value	Single-Fault Baseline / Rationale
EP-02-01	email	Định dạng hợp lệ và tài khoản tồn tại trong CSDL	VEP	`test@eshop.com`	Baseline hợp lệ chuẩn; kết hợp với mật khẩu hợp lệ để đăng nhập thành công.
EP-02-02	email	Định dạng không hợp lệ theo HTML5/RFC (thiếu `@`, ký tự cấm)	IEP	`testeshop.com`	`password = "Test1234!"` (hợp lệ). Cô lập lỗi do cú pháp email; HTML5/UI validate chặn submit trước khi gửi API.
EP-02-03	email	Chuỗi rỗng / để trống trường bắt buộc	IEP	`""`	`password = "Test1234!"` (hợp lệ). Cô lập lỗi thiếu trường bắt buộc email.
EP-02-04	email	Chuỗi chỉ chứa khoảng trắng (Whitespace-only)	IEP	`" "`	`password = "Test1234!"` (hợp lệ). Cô lập lỗi chuỗi rỗng nguy trang; HTML5 trim/pattern hoặc backend từ chối.
EP-02-04b	email	Độ dài vượt quá giới hạn tối đa RFC 5321 ($Len > 254$)	IEP	Email dài 255 ký tự	`password = "Test1234!"` (hợp lệ). Cô lập lỗi tràn trường email; hệ thống từ chối hoặc báo lỗi định dạng.
EP-02-05	email	Định dạng hợp lệ nhưng tài khoản không tồn tại	IEP	`notfound@eshop.com`	`password = "Test1234!"` (hợp lệ). Kiểm tra phản hồi bảo mật chung, không làm lộ thông tin tài khoản chưa tồn tại.
EP-02-06	password	Mật khẩu khớp chính xác với tài khoản	VEP	`Test1234!`	`email = "test@eshop.com"`. Đáp ứng đầy đủ thông tin xác thực, cấp JWT token.
EP-02-07	password	Mật khẩu sai / không khớp CSDL	IEP	`WrongPass1!`	`email = "test@eshop.com"` (hợp lệ). Cô lập lỗi sai mật khẩu; hệ thống tăng bộ đếm thất bại đúng 1 đơn vị.
EP-02-08	password	Mật khẩu để trống	IEP	`""`	`email = "test@eshop.com"` (hợp lệ). Cô lập lỗi thiếu trường bắt buộc mật khẩu.
EP-02-09	password	Mật khẩu chỉ chứa khoảng trắng (Whitespace-only)	IEP	`" "`	`email = "test@eshop.com"` (hợp lệ). Cô lập lỗi chuỗi rỗng nguy trang cho password; hệ thống từ chối xác thực.
EP-02-09b	password	Độ dài vượt quá giới hạn an toàn OWASP ($Len > 255$)	IEP	Mật khẩu dài 256 ký tự	`email = "test@eshop.com"` (hợp lệ). Cô lập lỗi tràn bộ đệm tính toán hash; hệ thống xử lý an toàn không treo máy.
EP-02-10	failed_attempts	Số lần đăng nhập sai dưới ngưỡng ($1 \le N \le 2$), sau đó nhập đúng	VEP	$N = 1$ hoặc $N = 2 \to$ nhập đúng `Test1234!`	Tài khoản `test@eshop.com`. Kiểm tra sequence: đăng nhập đúng sau 1 hoặc 2 lần sai phải thành công và reset $N \to 0$.
EP-02-11	failed_attempts	Số lần sai đạt hoặc vượt ngưỡng lần đầu ($N \ge 3$)	IEP	$N = 3$	Nhập sai liên tiếp 3 lần trên cùng tài khoản `test@eshop.com`. Kích hoạt tạm khóa 30 giây lần đầu.
EP-02-12	failed_attempts	Chu kỳ khóa lặp lại (Second lockout cycle sau khi hết lần 1)	IEP	Hết khóa 1 $\to$ sai tiếp 3 lần	Tài khoản `test@eshop.com`. Kiểm tra thời gian phạt lần 2 có bị lũy tiến (escalate/backoff) hay giữ nguyên 30s theo SRS.
EP-02-13a	lockout_time	Đang trong thời gian tạm khóa ($0 \le t < 30s$) – Nhập đúng pass	IEP	$t = 15s$ / pass đúng `Test1234!`	Tài khoản đang bị khóa. Dù người dùng nhập đúng mật khẩu vẫn bị từ chối (HTTP 403); không mở khóa sớm.
EP-02-13b	lockout_time	Đang trong thời gian tạm khóa ($0 \le t < 30s$) – Nhập sai pass	IEP	$t = 15s$ / pass sai `WrongPass!`	Tài khoản đang bị khóa. Từ chối đăng nhập; kiểm tra xem lần thử sai này có reset lại bộ đếm 30s (timer reset) hay không.
EP-02-14	lockout_time	Đã hết thời gian tạm khóa ($t \ge 30s$)	VEP	$t = 31s$	Tài khoản tự động giải tỏa tạm khóa; nhập đúng mật khẩu `Test1234!` đăng nhập thành công.

> **Xác nhận về Trạng thái Tài khoản (Account Status: Banned / Inactive):**
>
> - **Đối chiếu Đặc tả SRS:** Tài liệu SRS (`specs/system_requirements_specification.md`) và CSDL SUT (`users` table: `id, name, email, password, role, login_attempts, locked_until, reset_token, shipping_address, phone`) **hoàn toàn không hỗ trợ** trạng thái tài khoản bị cấm (`banned`) hay vô hiệu hóa (`inactive`).
> - **Kết luận phân loại:** Trạng thái duy nhất điều khiển quyền truy cập tài khoản trong FR-02 là **Tạm khóa do đăng nhập sai (`locked_until`)**. Tình huống tài khoản bị cấm/vô hiệu hóa sẽ được ghi nhận vào phần **Khoảng trống Đặc tả & Giả thuyết Khám phá (Step 6 Gap Analysis)** thay vì lớp tương đương nghiệp vụ bắt buộc.

2.3 Bước 3: Phân tích giá trị biên (Step 3 Deliverables)

> **Phương pháp tiếp cận Phân tích Giá trị Biên (ISTQB FL §4.2.2):** Áp dụng kỹ thuật **3-Value BVA** cho các miền có thứ tự (số nguyên, thời gian, độ dài chuỗi) với bước vi sai nhỏ nhất (ϵ).
>
> - **On-point:** Giá trị nằm chính xác trên ranh giới phân hoạch.
> - **Off-point:** Giá trị nằm sát biên nhất ở phía đối diện (ngoài vùng nếu On nằm trong; hoặc trong vùng nếu On nằm ngoài).
> - **In-point:** Giá trị nằm liền kề bên trong phân vùng cần kiểm thử.

Boundary ID	Variable / Field	Boundary Rule	BVA Method	On-point (Value/Type)	Off-point (Value/Type)	In-point (Value/Type)
`BVA-02-01`	`failed_attempts`	$N \ge 3$ (Ngưỡng khóa tài khoản do nhập sai liên tiếp)	3-Value ($\epsilon=1\text{ lần}$)	$N = 3$ [Invalid - Khóa] Nhập sai đúng lần thứ 3 $\to$ Kích hoạt tạm khóa 30s.	$N = 2$ [Valid - Chưa khóa] Nhập sai 2 lần ($3 - 1$) $\to$ Cảnh báo lỗi, chưa bị khóa.	$N = 4$ [Invalid - Đang khóa] Cố tình thử tiếp khi đã đạt ngưỡng $\to$ Giữ trạng thái khóa.
`BVA-02-02`	`lockout_time`	$t \ge 30s$ (Ngưỡng tự động giải tỏa tạm khóa tài khoản)	3-Value ($\epsilon=1\text{ giây}$)	$t = 30s$ [Valid - Mở khóa] Đúng mốc 30 giây $\to$ Biên chuyển đổi từ khóa sang mở.	$t = 29s$ [Invalid - Vẫn khóa] Sát biên mở 1s ($30 - 1$) $\to$ Vẫn đang bị khóa, từ chối login.	$t = 31s$ [Valid - Mở khóa] Sau mở khóa 1s ($30 + 1$) $\to$ Đăng nhập đúng thành công, cấp JWT.
`BVA-02-03`	`password.length`	$Len \ge 8$ (Độ dài mật khẩu tối thiểu - đối chiếu FR-01)	3-Value ($\epsilon=1\text{ char}$)	$Len = 8$ [Valid] `"Pass123!"` (đúng 8 ký tự).	$Len = 7$ [Invalid] `"Pass12!"` ($8 - 1$ ký tự, dưới ngưỡng tối thiểu).	$Len = 9$ [Valid] `"Pass1234!"` ($8 + 1$ ký tự, trên ngưỡng tối thiểu).
`BVA-02-04`	`login_sequence`	$N = 1, 2$ (Ngưỡng reset bộ đếm trước khi chạm mốc khóa)	3-Value Transition	$N = 2 \to$ Đúng [Valid] Sai 2 lần rồi nhập đúng $\to$ Login thành công, reset $N \to 0$.	$N = 3 \to$ Sai [Invalid] Sai tiếp lần 3 $\to$ Vượt biên chịu lỗi, tài khoản bị tạm khóa 30s.	$N = 1 \to$ Đúng [Valid] Sai 1 lần rồi nhập đúng $\to$ Login thành công, reset $N \to 0$.
`BVA-02-05`	`email.length`	$Len \le 254$ ký tự (Cận trên độ dài theo chuẩn RFC 5321 / RFC 3696)	3-Value ($\epsilon=1\text{ char}$)	$Len = 254$ [Valid] Email đúng chuẩn dài đúng 254 ký tự.	$Len = 255$ [Invalid] Email dài 255 ký tự ($254 + 1$, vượt chuẩn RFC).	$Len = 253$ [Valid] Email dài 253 ký tự ($254 - 1$, cận kề trong vùng hợp lệ).
`BVA-02-06`	`password.length`	$Len \le 255$ ký tự (Cận trên độ dài an toàn theo OWASP ASVS)	3-Value ($\epsilon=1\text{ char}$)	$Len = 255$ [Valid] Mật khẩu dài đúng 255 ký tự.	$Len = 256$ [Invalid] Mật khẩu dài 256 ký tự ($255 + 1$, vượt ngưỡng an toàn).	$Len = 254$ [Valid] Mật khẩu dài 254 ký tự ($255 - 1$, cận kề trong vùng hợp lệ).

2.4 Bước 4: Thiết kế kịch bản kiểm thử (Step 4 Deliverables)

> **Phương pháp luận Thiết kế Kịch bản Kiểm thử (Theo `Comprehensive Guide: How to Write Test Cases`):**
>
> 1. **Cú pháp Định danh Mục tiêu (Title / Objective Formula):** Mọi kịch bản kiểm thử đều tuân thủ công thức chuẩn: $$\text{Action (Hành động)} + \text{Function (Chức năng)} + \text{Operating Condition (Điều kiện vận hành)}$$
> 2. **Nguyên tắc Giả định Đơn lỗi (Single Fault Isolation):** Mỗi kịch bản Negative chỉ kích hoạt đúng 1 điều kiện không hợp lệ; tất cả các biến khác giữ nguyên giá trị hợp lệ cơ sở (Baseline).
> 3. **Tính Tự dọn dẹp (Self-Cleaning / Teardown):** Mỗi kịch bản có trạng thái (Stateful) đều bao gồm bước hoàn trả trạng thái CSDL SQLite để đảm bảo tính độc lập tuyệt đối giữa các lượt kiểm thử.
> 4. **Điểm kiểm chứng đa tầng (Multi-tier Validation Points):** Kết quả mong đợi phân tách rõ ràng 4 tầng: Giao diện Web (UI), Giao diện Lập trình (API), Trạng thái CSDL (Database), và Tuân thủ An toàn Thông tin (Security).

2.4.1 Bảng Tổng Hợp Kịch Bản Kiểm Thử & Ma Trận Truy Vết (Overview Traceability Matrix)

TC ID	Test Case Objective ($\text{Action} + \text{Function} + \text{Condition}$)	Phân loại	Mã EP bao phủ	Mã BVA bao phủ	Dữ liệu đầu vào & Baseline	Kết quả cốt lõi (Core Expected)	Trạng thái
`TC_FR02_01`	Verify login authentication with valid registered credentials on Web & API	Positive (VEP)	`EP-02-01`, `EP-02-06`	`BVA-02-03-IN`	`test@eshop.com` / `Test1234!` ($Len=9$)	HTTP 200, JWT token, redirect `/`	**Pass**
`TC_FR02_02`	Verify login authentication with boundary password length exactly 8 chars	Positive (BVA)	`EP-02-01`, `EP-02-06`	`BVA-02-03-ON`	`test@eshop.com` / `Pass123!` ($Len=8$)	HTTP 200, xác thực thành công	**Pass**

TC ID	Test Case Objective ($\text{Action} + \text{Function} + \text{Condition}$)	Phân loại	Mã EP bao phủ	Mã BVA bao phủ	Dữ liệu đầu vào & Baseline	Kết quả cốt lõi (Core Expected)	Trạng thái
`TC_FR02_03`	Validate consecutive failure counter increment on attempt N=1 (Below threshold)	Negative (IEP)	`EP-02-07`	`BVA-02-04-IN`	`test@eshop.com` / `WrongPass1!`	HTTP 401, `login_attempts = 1`, chưa khóa	**Fail (BUG-FR02-01)**
`TC_FR02_04`	Validate consecutive failure counter increment on attempt N=2 (Boundary Off-point)	Negative (BVA)	`EP-02-07`	`BVA-02-01-OFF`, `BVA-02-04-ON`	`test@eshop.com` / `WrongPass2!`	HTTP 401, `login_attempts = 2`, chưa khóa	**Fail (BUG-FR02-02)**
`TC_FR02_05`	Verify failure counter reset to 0 upon submitting correct password after N=2 failures	Positive (Seq)	`EP-02-06`, `EP-02-10`	`BVA-02-04-ON`	`test@eshop.com` / `Test1234!`	HTTP 200, reset `login_attempts = 0`	**Pass**
`TC_FR02_06`	Validate account lockout activation upon reaching threshold N=3 consecutive failures	Negative (BVA)	`EP-02-07`, `EP-02-11`	`BVA-02-01-ON`, `BVA-02-04-OFF`	`test@eshop.com` / `WrongPass3!`	HTTP 401/403, khóa đúng 30 giây	**Fail (BUG-FR02-03)**
`TC_FR02_07`	Validate login rejection when submitting correct password during lockout at t=15s	Negative (IEP)	`EP-02-13a`	`BVA-02-02-OFF`	`test@eshop.com` / `Test1234!` ($t=15s$)	HTTP 403 Forbidden, từ chối mở sớm	**Fail (BUG-FR02-04)**
`TC_FR02_08`	Validate lockout timer non-reset behavior when submitting wrong password at t=15s	Negative (IEP)	`EP-02-13b`	N/A (State Probe)	`test@eshop.com` / `WrongPassMid!` ($t=15s$)	HTTP 403, timer 30s KHÔNG bị reset	**Pass**
`TC_FR02_09`	Validate lockout rejection enforcement at boundary offset t=29s (1s before unlock)	Negative (BVA)	`EP-02-13a`	`BVA-02-02-OFF` ($t=29s$)	`test@eshop.com` / `Test1234!` ($t=29s$)	HTTP 403 Forbidden, vẫn đang bị khóa	**Pass**
`TC_FR02_10`	Verify automatic account unlock transition at exact boundary expiration t=30s	Positive (BVA)	`EP-02-14`	`BVA-02-02-ON` ($t=30s$)	`test@eshop.com` / `Test1234!` ($t=30s$)	HTTP 200 OK, mở khóa, cấp JWT	**Pass**
`TC_FR02_11`	Verify login authentication success at boundary offset t=31s (1s after unlock)	Positive (BVA)	`EP-02-14`	`BVA-02-02-IN` ($t=31s$)	`test@eshop.com` / `Test1234!` ($t=31s$)	HTTP 200 OK, đăng nhập thành công	**Pass**
`TC_FR02_12`	Validate lockout duration non-escalation behavior on repeated second lockout cycle	Negative (IEP)	`EP-02-12`	`BVA-02-01-ON`	3 lần thử sai liên tiếp lần 2	Khóa lần 2 vẫn đúng 30s (không backoff)	**Fail (BUG-FR02-02)**
`TC_FR02_13`	Validate rejection of additional login attempt N=4 while already in lockout	Negative (BVA)	`EP-02-11`	`BVA-02-01-IN` ($N=4$)	`test@eshop.com` / `WrongPass4!`	HTTP 403 Forbidden, giữ nguyên khóa	**Pass**
`TC_FR02_14`	Validate rejection of invalid email format lacking '@' symbol (Single Fault)	Negative (IEP)	`EP-02-02`	N/A (Format)	`testeshop.com` / `Test1234!`	HTML5 validation tooltip / HTTP 400	**Fail (BUG-FR02-05)**
`TC_FR02_15`	Validate rejection of empty email field on Web Client & API (Single Fault)	Negative (IEP)	`EP-02-03`	N/A (Required)	`""` / `Test1234!`	HTML5 `required` tooltip / HTTP 400	**Pass**
`TC_FR02_16`	Validate rejection of email containing only whitespace characters (Single Fault)	Negative (IEP)	`EP-02-04`	N/A (Whitespace)	`" "` / `Test1234!`	Báo lỗi định dạng email / HTTP 400	**Fail (BUG-FR02-05)**
`TC_FR02_17`	Validate rejection of email exceeding RFC 5321 max length 254 chars (Single Fault)	Negative (BVA)	`EP-02-04b`	`BVA-02-05-OFF` ($Len=255$)	Email 255 chars / `Test1234!`	Báo lỗi chuỗi quá dài / HTTP 400	**Fail (BUG-FR02-05)**
`TC_FR02_18`	Verify acceptance of email with maximum allowable RFC 5321 length of 254 chars	Positive (BVA)	`EP-02-01`	`BVA-02-05-ON` ($Len=254$)	Email 254 chars / `Test1234!`	HTTP 200 OK, xác thực thành công	**Pass**
`TC_FR02_19`	Validate generic error response when submitting non-existent email (Single Fault)	Negative (IEP)	`EP-02-05`	N/A (Account)	`notfound@eshop.com` / `Test1234!`	HTTP 401, không lộ email chưa tồn tại	**Pass**
`TC_FR02_20`	Validate rejection of empty password field on Web Client & API (Single Fault)	Negative (IEP)	`EP-02-08`	N/A (Required)	`test@eshop.com` / `""`	HTML5 `required` tooltip / HTTP 400	**Pass**
`TC_FR02_21`	Validate rejection of password containing only whitespace characters (Single Fault)	Negative (IEP)	`EP-02-09`	N/A (Whitespace)	`test@eshop.com` / `" "`	HTTP 401 Unauthorized, tăng đếm sai	**Pass**
`TC_FR02_22`	Validate rejection of password below minimum 8 chars length at Len=7 (Single Fault)	Negative (BVA)	`EP-02-07`	`BVA-02-03-OFF` ($Len=7$)	`test@eshop.com` / `Pass12!` ($Len=7$)	HTTP 401 Unauthorized, tăng đếm sai	**Pass**
`TC_FR02_23`	Validate handling of password exceeding OWASP max length at Len=256 (Single Fault)	Negative (BVA)	`EP-02-09b`	`BVA-02-06-OFF` ($Len=256$)	`test@eshop.com` / Pass 256 chars	HTTP 401/400, không bị treo CPU DoS	**Pass**
`TC_FR02_24`	Verify acceptance of password with maximum safe OWASP length of 255 chars	Positive (BVA)	`EP-02-06`	`BVA-02-06-ON` ($Len=255$)	`test@eshop.com` / Pass 255 chars	HTTP 200 OK, băm và khớp thành công	**Pass**

2.4.2 Hồ Sơ Đặc Tả Chi Tiết Từng Kịch Bản Kiểm Thử (Detailed Test Specification Cards for All 24 Test Cases)

Tuân thủ nghiêm ngặt chuẩn mực **Comprehensive Guide: How to Write Test Cases** và nguyên tắc **100% Card Coverage** của quy trình Kiểm thử Miền (Domain Testing), dưới đây là hồ sơ đặc tả chi tiết 6 phân đoạn vận hành (Tracking Information, Test Objective, Pre-conditions & Environment, Test Data, Step-by-Step Procedure, Multi-tier Validation Points, Teardown & Clean-up) cho toàn bộ 24 kịch bản kiểm thử độc lập:

Phân Nhóm 1: Xác Thực Cơ Sở & Kiểm Chứng Biên Độ Dài Hợp Lệ (Baseline & Valid Length Boundaries)

Test Case `TC_FR02_01`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR02_01` | **Priority:** High (P1) | **Target Component:** Web Client (`:5173`) & Backend API (`POST /api/login`)
  - **Traceability:** SRS FR-02 (Item 1, 4), API Spec §1.2 | **Coverage IDs:** `EP-02-01`, `EP-02-06`, `BVA-02-03-IN`
  - **Single-Fault Isolation:** N/A (Positive baseline test case).
- **2. Test Objective:** Verify user login authentication with valid registered credentials on Web Client & REST API.

- **3. Pre-conditions & Environment:**
  - Hệ thống EShop đang chạy bình thường (`backend: 3000` , `frontend-web: 5173`).
  - Tài khoản `test@eshop.com` tồn tại trong CSDL SQLite với mật khẩu hợp lệ `Test1234!` .
  - Trạng thái CSDL ban đầu: `login_attempts = 0` , `locked_until = NULL` .
  - Client chưa lưu token đăng nhập (`localStorage.getItem('token') === null`).
- **4. Test Data:** `email = "test@eshop.com"` , `password = "Test1234!"`
- **5. Step-by-Step Procedure:**
  1. Mở trình duyệt và truy cập trang đăng nhập `http://localhost:5173/login` .
  2. Nhập `test@eshop.com` vào trường "Email".
  3. Nhập `Test1234!` vào trường "Mật khẩu".
  4. Nhấn nút "Sign In" (gửi yêu cầu `POST /api/login`).
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Client:* Trình duyệt tự động chuyển hướng về trang chủ `http://localhost:5173/` . Header hiển thị thông tin người dùng và nút "Đăng xuất". Không xuất hiện thông báo lỗi.
  - *API / Network:* Endpoint `POST /api/login` trả về mã trạng thái `200 OK` . Response JSON chứa: `message: "Login successful"` , chuỗi `token` định dạng JWT hợp lệ, và đối tượng `user` (`email: "test@eshop.com"` , `role: "user"`).
  - *Client Storage:* `localStorage.getItem('token')` được gán chuỗi JWT không rỗng.
  - *Database State:* Trường `login_attempts` giữ nguyên giá trị `0` , `locked_until` là `NULL` .
- **7. Teardown & Clean-up:** Xóa token khỏi `localStorage` (`localStorage.removeItem('token')`) trên trình duyệt để đưa phiên làm việc về trạng thái ẩn danh.

Test Case `TC_FR02_02`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR02_02` | **Priority:** High (P1) | **Target Component:** Web Client & Backend API
  - **Traceability:** SRS FR-01/02 (Ràng buộc độ dài mật khẩu tối thiểu 8 ký tự) | **Coverage IDs:** `EP-02-01` , `EP-02-06` , `BVA-02-03-ON` ($Len=8$)
  - **Single-Fault Isolation:** N/A (Positive boundary test case).
- **2. Test Objective:** Verify user login authentication with boundary password length exactly 8 chars on Web Client & REST API.
- **3. Pre-conditions & Environment:**
  - Tài khoản `test8char@eshop.com` đã được đăng ký trong CSDL SQLite với mật khẩu đúng 8 ký tự `Pass123!` .
  - Trạng thái CSDL ban đầu: `login_attempts = 0` , `locked_until = NULL` .
- **4. Test Data:** `email = "test8char@eshop.com"` , `password = "Pass123!"` ($Len = 8$)
- **5. Step-by-Step Procedure:**
  1. Truy cập `http://localhost:5173/login` .
  2. Nhập `test8char@eshop.com` vào trường Email.
  3. Nhập `Pass123!` vào trường Mật khẩu.
  4. Nhấn nút "Sign In".
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Client:* Đăng nhập thành công, tự động chuyển hướng về trang chủ `/` .
  - *API / Network:* HTTP Status trả về `200 OK` , response JSON trả về JWT token và thông tin user.
  - *Client Storage:* `localStorage.getItem('token')` lưu trữ token JWT hợp lệ.
  - *Database State:* `login_attempts` vẫn bằng `0` , `locked_until` là `NULL` .
- **7. Teardown & Clean-up:** Xóa token khỏi `localStorage` .

Phân Nhóm 2: Thử Sai Dưới Ngưỡng, Đơn Lỗi & Phục Hồi Bộ Đếm (Failure Counter & Reset Sequence)

Test Case `TC_FR02_03`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR02_03` | **Priority:** High (P1) | **Target Component:** Backend Failure Counter & Web Client
  - **Traceability:** SRS FR-02 (Item 2: *"Sau mỗi lần đăng nhập sai, hệ thống tăng bộ đếm lên đúng 1 đơn vị."*) | **Coverage IDs:** `EP-02-07` , `BVA-02-04-IN` ($N=1$)
  - **Single-Fault Isolation:** Email hợp lệ chuẩn `test@eshop.com` ; mật khẩu sai lần thứ 1 ($N=1$).
- **2. Test Objective:** Validate consecutive failure counter increment on attempt N=1 (Below lockout threshold).
- **3. Pre-conditions & Environment:**
  - Tài khoản `test@eshop.com` ở trạng thái ban đầu (`login_attempts = 0` , `locked_until = NULL`).
- **4. Test Data:** `email = "test@eshop.com"` , `password = "WrongPass1!"`
- **5. Step-by-Step Procedure:**
  1. Truy cập `http://localhost:5173/login` .
  2. Nhập `test@eshop.com` vào trường Email.
  3. Nhập mật khẩu sai lần 1: `WrongPass1!` .
  4. Nhấn nút "Sign In".
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Client:* Hiển thị thông báo lỗi đăng nhập thất bại. Form vẫn mở bình thường, không hiện thông báo tài khoản bị khóa.
  - *API / Network:* HTTP Status code trả về `401 Unauthorized` . Body JSON: `{"error": "Invalid email or password"}` (hoặc tương đương). Không cấp JWT token.
  - *Database State:* `SELECT login_attempts, locked_until FROM users WHERE email='test@eshop.com'` : `login_attempts` tăng từ 0 lên **đúng 1** đơn vị ($N=1$); `locked_until` vẫn là `NULL` .
  - *Security:* Thông báo lỗi trung tính, không tiết lộ mật khẩu sai hay email sai.
- **7. Teardown & Clean-up:** Giữ nguyên trạng thái $N=1$ nếu chuẩn bị thực thi `TC_FR02_04` , hoặc reset về 0 bằng SQL: `UPDATE users SET login_attempts = 0 WHERE email = 'test@eshop.com'` .

Test Case `TC_FR02_04`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR02_04` | **Priority:** High (P1) | **Target Component:** Backend API & Web Client
  - **Traceability:** SRS FR-02 (Item 2, 3) | **Coverage IDs:** `EP-02-07`, `BVA-02-01-OFF` ($N=2$), `BVA-02-04-ON`
  - **Single-Fault Isolation:** Email hợp lệ chuẩn `test@eshop.com`; biến lỗi duy nhất là mật khẩu sai lần thứ 2 ($N=2$).
- **2. Test Objective:** Validate consecutive failure counter increment on attempt N=2 (Boundary Off-point before lockout).
- **3. Pre-conditions & Environment:**
  - Tài khoản `test@eshop.com` vừa hoàn thành bước thử sai lần 1 (`login_attempts = 1`, `locked_until = NULL`).
- **4. Test Data:** `email = "test@eshop.com"`, `password = "WrongPass2!"`
- **5. Step-by-Step Procedure:**
  1. Giữ nguyên email `test@eshop.com`.
  2. Nhập mật khẩu sai lần 2: `WrongPass2!`.
  3. Nhấn "Sign In" (gửi request `POST /api/login`).
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Client:* Hiển thị thông báo lỗi cảnh báo đăng nhập thất bại. Form vẫn cho phép tiếp tục nhập, chưa xuất hiện thông báo tài khoản bị khóa.
  - *API / Network:* HTTP Status code trả về `401 Unauthorized`. Body JSON: `{"error": "Invalid email or password"}` (hoặc tương đương).
  - *Database State:* Truy vấn CSDL `SELECT login_attempts, locked_until FROM users WHERE email='test@eshop.com'` : `login_attempts` tăng từ 1 lên **đúng 2** đơn vị ($N=2$); `locked_until` vẫn là `NULL`.
  - *Security:* Phản hồi lỗi trung tính, không tiết lộ số lần thử sai còn lại hay chi tiết nội bộ.
- **7. Teardown & Clean-up:** Giữ nguyên trạng thái $N=2$ nếu chuẩn bị thực thi `TC_FR02_05` hoặc `TC_FR02_06`.

Test Case `TC_FR02_05`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR02_05` | **Priority:** High (P1) | **Target Component:** Backend API & Database State
  - **Traceability:** SRS FR-02 (Quy tắc "liên tiếp" & Reset bộ đếm khi đăng nhập đúng) | **Coverage IDs:** `EP-02-06`, `EP-02-10`, `BVA-02-04-ON`
  - **Single-Fault Isolation:** N/A (Positive recovery sequence).
- **2. Test Objective:** Verify failure counter reset to 0 upon submitting correct password after N=2 consecutive failures.
- **3. Pre-conditions & Environment:**
  - Tài khoản `test@eshop.com` đang tích lũy đúng 2 lần thử sai ($N=2$ từ `TC_FR02_04`), chưa bị khóa (`locked_until = NULL`).
- **4. Test Data:** `email = "test@eshop.com"`, `password = "Test1234!"` (mật khẩu chính xác)
- **5. Step-by-Step Procedure:**
  1. Tại ô mật khẩu, xóa chuỗi sai và nhập chính xác: `Test1234!`.
  2. Nhấn "Sign In" để gửi request đăng nhập.
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Client:* Đăng nhập thành công, chuyển hướng về `/`.
  - *API / Network:* HTTP Status `200 OK`, cấp JWT token hợp lệ.
  - *Database State:* Bộ đếm thất bại được reset hoàn toàn về **0** (`UPDATE users SET login_attempts = 0, locked_until = NULL`). Xác nhận qua câu lệnh: `SELECT login_attempts FROM users WHERE email='test@eshop.com'` trả về `0`.
- **7. Teardown & Clean-up:** Đăng xuất tài khoản, xóa token khỏi `localStorage`.

Phân Nhóm 3: Kích Hoạt Khóa, Thăm Dò & Ranh Giới Thời Gian Khóa 30 Giây (Lockout Cycle & Timing Boundaries)

Test Case `TC_FR02_06`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR02_06` | **Priority:** Critical (P0) | **Target Component:** Backend Security & Lockout Guard
  - **Traceability:** SRS FR-02 (Item 2, 3: *"Nếu đăng nhập sai từ 3 lần trở lên liên tiếp, tài khoản bị tạm khóa 30 giây"*) | **Coverage IDs:** `EP-02-07`, `EP-02-11`, `BVA-02-01-ON` ($N=3$), `BVA-02-04-OFF`
  - **Single-Fault Isolation:** Email chuẩn `test@eshop.com`; mật khẩu sai liên tiếp lần thứ 3 ($N=3$).
- **2. Test Objective:** Validate account lockout activation upon reaching threshold N=3 consecutive failures (Boundary On-point).
- **3. Pre-conditions & Environment:**
  - Tài khoản `test@eshop.com` đang ở mốc $N=2$ lần sai liên tiếp (`login_attempts = 2`).
- **4. Test Data:** `email = "test@eshop.com"`, `password = "WrongPass3!"`
- **5. Step-by-Step Procedure:**
  1. Nhập mật khẩu sai lần thứ 3: `WrongPass3!`.
  2. Ghi nhận thời điểm gửi request: $T_{trigger} = ext{Date.now()}$.
  3. Nhấn "Sign In" (gửi `POST /api/login`).
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Client:* Hiển thị thông báo tài khoản bị tạm khóa (ví dụ: *"Tài khoản đã bị tạm khóa do nhập sai nhiều lần"*).
  - *API / Network:* Trả về HTTP `401 Unauthorized` hoặc `403 Forbidden`. Không trả về JWT token.
  - *Database State:* `login_attempts` đạt mốc **3** ($N=3$); trường `locked_until` được gán giá trị thời gian bằng đúng **$T_{trigger} + 30 ext{ giây}$** (sai số cho phép ≤ 1 giây của môi trường demo, trích dẫn SRS: *"tài khoản bị tạm khóa 30 giây"*).
  - *Security:* Thông báo lỗi không để lộ chi tiết cấu hình nội bộ hoặc thời gian khóa chính xác nếu hệ thống áp dụng thông báo an toàn.
- **7. Teardown & Clean-up:** Duy trì mốc khóa để tiếp tục thực hiện chuỗi test `TC_FR02_07` $ o$ `TC_FR02_11`.

Test Case `TC_FR02_07`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR02_07` | **Priority:** High (P1) | **Target Component:** Lockout Guard Middleware
  - **Traceability:** SRS FR-02 (Item 3) | **Coverage IDs:** `EP-02-13a`, `BVA-02-02-OFF` (Active window $0 \le t < 30s$)
  - **Single-Fault Isolation:** Trạng thái tài khoản đang bị tạm khóa; thử đăng nhập bằng mật khẩu đúng.

- **2. Test Objective:** Validate login rejection when submitting correct password during lockout period at elapsed time t=15s.
- **3. Pre-conditions & Environment:**
  - Tài khoản `test@eshop.com` đang trong thời gian bị tạm khóa từ `TC_FR02_06`.
  - Thời gian trôi qua kể từ lúc khóa: chính xác 15 giây ($t = 15s < 30s$).
- **4. Test Data:** `email = "test@eshop.com"`, `password = "Test1234!"` (mật khẩu hoàn toàn chính xác)
- **5. Step-by-Step Procedure:**
  1. Đợi đồng hồ đếm đến giây thứ 15 sau khi khóa ($t = 15s$).
  2. Nhập mật khẩu đúng: `Test1234!`.
  3. Nhấn "Sign In".
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Client:* Giao diện tiếp tục báo lỗi tài khoản đang bị khóa; tuyệt đối không cho phép đăng nhập hoặc chuyển hướng.
  - *API / Network:* HTTP Status code trả về **`403 Forbidden`**. JSON Error chứa thông báo về việc tài khoản đang bị tạm khóa.
  - *Database State:* CSDL giữ nguyên mốc `locked_until` ban đầu; tài khoản không được mở khóa sớm dù nhập đúng mật khẩu.
- **7. Teardown & Clean-up:** Tiếp tục giữ khóa để thực thi test `TC_FR02_08`.

Test Case `TC_FR02_08`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR02_08` | **Priority:** High (P1) | **Target Component:** Lockout Timer Integrity Probe
  - **Traceability:** SRS FR-02 | **Coverage IDs:** `EP-02-13b` (State Probe theo yêu cầu kiểm thử miền)
  - **Single-Fault Isolation:** Mật khẩu sai trong khi tài khoản đang trong trạng thái tạm khóa.
- **2. Test Objective:** Validate lockout timer non-reset behavior when submitting incorrect password at elapsed time t=15s.
- **3. Pre-conditions & Environment:**
  - Tài khoản `test@eshop.com` đang bị khóa; thời gian trôi qua: $t = 15s$.
- **4. Test Data:** `email = "test@eshop.com"`, `password = "WrongPassMidLock!"` (mật khẩu không chính xác)
- **5. Step-by-Step Procedure:**
  1. Tại mốc $t = 15s$, gửi request đăng nhập với mật khẩu sai `WrongPassMidLock!`.
  2. Ghi nhận phản hồi và kiểm tra giá trị cột `locked_until` trong SQLite.
- **6. Multi-tier Validation Points (Expected Results):**
  - *API / Network:* Trả về HTTP `403 Forbidden`.
  - *Database State:* **Không được phép cập nhật lại `locked_until` thành $t + 30s$ mới!** Mốc hết hạn khóa `locked_until` phải giữ nguyên mốc $T_{trigger} + 30s$ ban đầu. (Hệ thống không được phạt cộng dồn đè lên timer hiện có trừ khi có đặc tả rõ ràng).
- **7. Teardown & Clean-up:** Giữ nguyên để kiểm tra điểm biên hết hạn 30s ở các TC tiếp theo.

Test Case `TC_FR02_09`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR02_09` | **Priority:** High (P1) | **Target Component:** Lockout Boundary Precision Check
  - **Traceability:** SRS FR-02 (Thời gian khóa 30 giây) | **Coverage IDs:** `EP-02-13a`, `BVA-02-02-OFF` ($t = 29s$)
  - **Single-Fault Isolation:** Thời điểm thử đăng nhập nằm sát biên mở khóa 1 giây ($30 - 1 = 29s$).
- **2. Test Objective:** Validate lockout rejection enforcement at boundary offset t=29s (1s before unlock threshold).
- **3. Pre-conditions & Environment:**
  - Tài khoản `test@eshop.com` bị khóa tại thời điểm T_0.
  - Đợi đồng hồ đếm đúng 29 giây ($t = 29s$).
- **4. Test Data:** `email = "test@eshop.com"`, `password = "Test1234!"`
- **5. Step-by-Step Procedure:**
  1. Tại đúng mốc $t = 29s$ (sai số $< 200 ext{ms}$ bằng automated script hoặc quan sát timestamp), gửi request đăng nhập với mật khẩu đúng `Test1234!`.
- **6. Multi-tier Validation Points (Expected Results):**
  - *API / Network:* HTTP Status code trả về **`403 Forbidden`**.
  - *Database State:* `Date.now() < new Date(user.locked_until)` vẫn thỏa mãn $ o$ Hệ thống chặn yêu cầu xác thực.
- **7. Teardown & Clean-up:** Chờ tiếp 1 giây để thực hiện `TC_FR02_10`.

Test Case `TC_FR02_10`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR02_10` | **Priority:** Critical (P0) | **Target Component:** Lockout Boundary On-point Transition
  - **Traceability:** SRS FR-02 (Thời gian khóa 30 giây) | **Coverage IDs:** `EP-02-14`, `BVA-02-02-ON` ($t = 30s$)
  - **Single-Fault Isolation:** N/A (Positive boundary transition).
- **2. Test Objective:** Verify automatic account unlock transition at exact boundary expiration t=30s.
- **3. Pre-conditions & Environment:**
  - Tài khoản `test@eshop.com` bị khóa tại T_0.
  - Đồng hồ đếm thời gian chạm mốc đúng 30 giây ($t = 30s$).
- **4. Test Data:** `email = "test@eshop.com"`, `password = "Test1234!"`
- **5. Step-by-Step Procedure:**
  1. Tại mốc $t = 30s$, gửi request đăng nhập với mật khẩu đúng `Test1234!`.
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Client:* Đăng nhập thành công, tự động chuyển trang về `/`.
  - *API / Network:* HTTP Status code trả về **`200 OK`**, cấp JWT token mới.
  - *Database State:* Khóa tự động hết hiệu lực (`Date.now() >= new Date(user.locked_until)`), `login_attempts` được reset về `0`, `locked_until` được giải phóng.
- **7. Teardown & Clean-up:** Xóa token phiên làm việc.

Test Case `TC_FR02_11`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR02_11` | **Priority:** Normal (P2) | **Target Component:** Post-lockout Stability
  - **Traceability:** SRS FR-02 | **Coverage IDs:** `EP-02-14`, `BVA-02-02-IN` ($t = 31s$)
  - **Single-Fault Isolation:** N/A (Positive post-boundary test case).
- **2. Test Objective:** Verify login authentication success at boundary offset t=31s (1s after unlock threshold).
- **3. Pre-conditions & Environment:**
  - Đã trôi qua 31 giây kể từ thời điểm tài khoản bị khóa ($t = 31s > 30s$).
- **4. Test Data:** `email = "test@eshop.com"`, `password = "Test1234!"`
- **5. Step-by-Step Procedure:**
  1. Gửi request đăng nhập tại mốc $t = 31s$.
- **6. Multi-tier Validation Points (Expected Results):**
  - *API / Network:* HTTP Status `200 OK`, cấp JWT token bình thường.
  - *Database State:* Hệ thống hoạt động ở trạng thái bình thường ổn định.
- **7. Teardown & Clean-up:** Đăng xuất tài khoản.

Test Case `TC_FR02_12`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR02_12` | **Priority:** Medium (P2) | **Target Component:** Lockout Policy Verification
  - **Traceability:** SRS FR-02 (Chu kỳ khóa lặp lại) | **Coverage IDs:** `EP-02-12`, `BVA-02-01-ON`
  - **Single-Fault Isolation:** Thử sai liên tiếp 3 lần trong chu kỳ khóa thứ 2.
- **2. Test Objective:** Validate lockout duration non-escalation behavior on repeated second lockout cycle.
- **3. Pre-conditions & Environment:**
  - Tài khoản `test@eshop.com` vừa kết thúc chu kỳ khóa thứ nhất và đã được giải tỏa hoàn toàn.
- **4. Test Data:** Gửi 3 lần mật khẩu sai liên tiếp: `Wrong1!`, `Wrong2!`, `Wrong3!`.
- **5. Step-by-Step Procedure:**
  1. Thực hiện 3 lần đăng nhập sai liên tiếp trên cùng tài khoản.
  2. Ghi nhận thời điểm khóa lần 2 và kiểm tra thời hạn `locked_until`.
- **6. Multi-tier Validation Points (Expected Results):**
  - *Hành vi theo SRS:* SRS FR-02 không định nghĩa cơ chế phạt lũy tiến (Exponential Backoff). Do đó, thời gian tạm khóa lần 2 vẫn phải là **đúng 30 giây** (`locked_until = now + 30s`).
  - *Ghi chú đánh giá:* Nếu SUT tự động tăng lên 60s hay 180s mà không có trong SRS, đây là một điểm sai lệch cần ghi nhận vào Bug Report.
- **7. Teardown & Clean-up:** Chạy script khôi phục CSDL SQLite: `UPDATE users SET login_attempts = 0, locked_until = NULL WHERE email = 'test@eshop.com'`.

Test Case `TC_FR02_13`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR02_13` | **Priority:** Medium (P2) | **Target Component:** Lockout State Guard
  - **Traceability:** SRS FR-02 (Item 3) | **Coverage IDs:** `EP-02-11`, `BVA-02-01-IN` ($N=4$)
  - **Single-Fault Isolation:** Tài khoản đang trong trạng thái bị tạm khóa; gửi tiếp lần thử sai thứ 4 ($N=4$).
- **2. Test Objective:** Validate rejection of additional login attempt N=4 while already in lockout state.
- **3. Pre-conditions & Environment:**
  - Tài khoản `test@eshop.com` vừa đạt ngưỡng khóa 3 lần thử sai ($N=3$, `locked_until` đang hiệu lực).
- **4. Test Data:** `email = "test@eshop.com"`, `password = "WrongPass4!"`
- **5. Step-by-Step Procedure:**
  1. Tại giao diện đăng nhập, tiếp tục nhập mật khẩu sai: `WrongPass4!`.
  2. Nhấn nút "Sign In" (gửi `POST /api/login`).
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Client:* Hiển thị thông báo tài khoản đang bị khóa; form tiếp tục chặn truy cập.
  - *API / Network:* HTTP Status trả về `403 Forbidden` (hoặc `401 Unauthorized` có thông báo locked). Không sinh session/token.
  - *Database State:* Trạng thái khóa được giữ nguyên; thời gian hết hạn khóa `locked_until` không bị xáo trộn.
- **7. Teardown & Clean-up:** Chờ hết thời gian khóa hoặc reset CSDL bằng câu lệnh: `UPDATE users SET login_attempts = 0, locked_until = NULL WHERE email = 'test@eshop.com'`.

Phân Nhóm 4: Phân Vùng Đơn Lỗi & Ranh Giới Biên Trường Email (Email Single-Fault & RFC Boundaries)

Test Case `TC_FR02_14`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR02_14` | **Priority:** High (P1) | **Target Component:** Web Client HTML5 Validation & API Input Guard
  - **Traceability:** SRS FR-02 (Item 5: *"Trường email phải dùng `type="email"` (có validate HTML5 format)"*) | **Coverage IDs:** `EP-02-02` (Email cú pháp không hợp lệ)
  - **Single-Fault Isolation:** Mật khẩu hợp lệ chuẩn `Test1234!`; biến lỗi duy nhất là email thiếu ký tự `@`.
- **2. Test Objective:** Validate rejection of invalid email format lacking '@' symbol on Web Client and REST API.
- **3. Pre-conditions & Environment:** Form đăng nhập mở tại `http://localhost:5173/login`.
- **4. Test Data:** `email = "testeshop.com"`, `password = "Test1234!"`
- **5. Step-by-Step Procedure:**
  1. Nhập `testeshop.com` (thiếu `@`) vào ô Email.
  2. Nhập `Test1234!` vào ô Mật khẩu.
  3. Nhấn "Sign In".

4. (Bypass test): Gửi trực tiếp payload JSON `{email: "testeshop.com", password: "Test1234!"}` tới `POST /api/login`.

- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Client:* Trình duyệt hiển thị popup/tooltip HTML5 chuẩn: *"Please include an '@' in the email address..."*. Request HTTP không được phát đi từ form.
  - *API / Network (nếu bypass UI):* Backend trả về `400 Bad Request` hoặc `422 Unprocessable Entity` với thông báo định dạng email không hợp lệ.
  - *Database State:* Không có bản ghi nào bị thay đổi; bộ đếm thất bại của user không bị ảnh hưởng.
- **7. Teardown & Clean-up:** Xóa nội dung trong trường email.

Test Case `TC_FR02_15`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR02_15` | **Priority:** High (P1) | **Target Component:** Web Client Form & API Required Validation
  - **Traceability:** SRS FR-02 (Item 1, 5) | **Coverage IDs:** `EP-02-03` (Email để trống)
  - **Single-Fault Isolation:** Mật khẩu hợp lệ `Test1234!`; biến lỗi duy nhất là trường email rỗng.
- **2. Test Objective:** Validate rejection of empty email field on Web Client & REST API.
- **3. Pre-conditions & Environment:** Form đăng nhập mở tại `http://localhost:5173/login`.
- **4. Test Data:** `email = ""` (chuỗi rỗng), `password = "Test1234!"`
- **5. Step-by-Step Procedure:**
  1. Để trống trường Email.
  2. Nhập `Test1234!` vào trường Mật khẩu.
  3. Nhấn "Sign In".
  4. Gửi request trực tiếp qua Postman/cURL với `{email: "", password: "Test1234!"}` tới `POST /api/login`.
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Client:* Thuộc tính `required` của HTML5 kích hoạt cảnh báo *"Please fill out this field"*; nút submit bị chặn.
  - *API / Network:* Backend trả về HTTP `400 Bad Request` với message báo thiếu trường bắt buộc (`"Email is required"` hoặc tương đương).
  - *Database State:* Không có truy vấn SQL ghi nào được thực thi.
- **7. Teardown & Clean-up:** Không yêu cầu hành động teardown.

Test Case `TC_FR02_16`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR02_16` | **Priority:** Medium (P2) | **Target Component:** Client & Server Input Validation
  - **Traceability:** SRS FR-02 (Item 5: `type="email"` HTML5) | **Coverage IDs:** `EP-02-04` (Whitespace-only)
  - **Single-Fault Isolation:** Mật khẩu hợp lệ `Test1234!`; biến lỗi duy nhất là email chỉ gồm khoảng trắng.
- **2. Test Objective:** Validate rejection of email input containing only whitespace characters on Web Client & API.
- **3. Pre-conditions & Environment:** Form đăng nhập mở tại `http://localhost:5173/login`.
- **4. Test Data:** `email = " "`, `password = "Test1234!"`
- **5. Step-by-Step Procedure:**
  1. Nhập 5 dấu cách `" "` vào ô Email.
  2. Nhập mật khẩu đúng `Test1234!`.
  3. Bấm "Sign In".
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Client:* Trình duyệt kích hoạt HTML5 validation yêu cầu điền trường hợp lệ hoặc form tự trim khoảng trắng và báo lỗi định dạng email. Không cho phép gửi request hợp lệ.
  - *API / Network (nếu bypass UI):* Backend trả về HTTP `400 Bad Request` hoặc `401 Unauthorized`.
  - *Database State:* Không làm biến đổi CSDL.
- **7. Teardown & Clean-up:** Xóa trắng ô input.

Test Case `TC_FR02_17`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR02_17` | **Priority:** Medium (P2) | **Target Component:** Backend RFC Parser & Input Sanity
  - **Traceability:** RFC 5321 §4.5.3.1.3 & SRS FR-02 | **Coverage IDs:** `EP-02-04b`, `BVA-02-05-OFF` ($Len=255$)
  - **Single-Fault Isolation:** Mật khẩu chuẩn `Test1234!`; biến lỗi duy nhất là email vượt cận trên RFC 5321 (255 ký tự).
- **2. Test Objective:** Validate boundary rejection of email input exceeding RFC 5321 max length at Len=255.
- **3. Pre-conditions & Environment:** Mở form đăng nhập hoặc công cụ API client (cURL/Postman).
- **4. Test Data:** `email = "a"*244 + "@eshop.com"` ($Len = 255$ chars), `password = "Test1234!"`
- **5. Step-by-Step Procedure:**
  1. Tạo chuỗi email dài đúng 255 ký tự: 244 ký tự `'a'` nối với `'@eshop.com'`.
  2. Nhập email vào form đăng nhập hoặc gửi payload qua `POST /api/login`.
  3. Quan sát mã phản hồi HTTP và hành vi của server.
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Client:* Input tự động giới hạn độ dài (nếu có `maxlength`) hoặc báo lỗi chuỗi email không hợp lệ.
  - *API / Network:* HTTP Status code trả về `400 Bad Request` hoặc `422 Unprocessable Entity`; **tuyệt đối không gây sập hệ thống (500 Internal Server Error)**.
  - *Database State:* Không ghi nhận chuỗi quá dài gây lỗi tràn bộ đệm vào CSDL.
- **7. Teardown & Clean-up:** Xóa dữ liệu test.

Test Case `TC_FR02_18`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR02_18` | **Priority:** Medium (P2) | **Target Component:** Backend RFC Compatibility & Web Client
  - **Traceability:** RFC 5321 §4.5.3.1.3 & SRS FR-02 | **Coverage IDs:** `EP-02-01`, `BVA-02-05-ON` ($Len=254$)
  - **Single-Fault Isolation:** N/A (Positive boundary test case).
- **2. Test Objective:** Verify acceptance of email with maximum allowable RFC 5321 length of exactly 254 chars.

- **3. Pre-conditions & Environment:**
  - Chuỗi email 254 ký tự: `"a"*243 + "@eshop.com"`.
  - Tài khoản với email trên đã được tạo trong CSDL SQLite hoặc hệ thống cho phép xử lý xác thực bình thường.
- **4. Test Data:** `email = "a"*243 + "@eshop.com"` ($Len = 254$ chars), `password = "Test1234!"`
- **5. Step-by-Step Procedure:**
  1. Nhập email 254 ký tự vào trường Email.
  2. Nhập mật khẩu đúng `Test1234!`.
  3. Nhấn "Sign In" (gửi `POST /api/login`).
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Client:* Giao diện tiếp nhận chuỗi 254 ký tự bình thường, không bị cắt cụt hay vỡ layout.
  - *API / Network:* Backend chấp nhận chuỗi định dạng RFC 5321; trả về HTTP `200 OK` (nếu đã seed tài khoản) hoặc `401 Unauthorized` (nếu chưa có trong DB nhưng cú pháp hợp lệ). Không trả về `400 Bad Request` do lỗi độ dài.
  - *Database State:* Xử lý an toàn trên trường dữ liệu chuỗi của CSDL.
- **7. Teardown & Clean-up:** Xóa tài khoản test 254 ký tự nếu đã tạo trong CSDL.

Test Case `TC_FR02_19`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR02_19` | **Priority:** High (P1) | **Target Component:** Backend Authentication & User Enumeration Defense
  - **Traceability:** SRS FR-02 (Item 3: *"Hệ thống trả về thông báo lỗi phù hợp; không để lộ chi tiết nguyên nhân."*) | **Coverage IDs:** `EP-02-05` (Email không tồn tại trong DB)
  - **Single-Fault Isolation:** Mật khẩu chuẩn `Test1234!`; biến lỗi duy nhất là email chưa từng được đăng ký trong CSDL.
- **2. Test Objective:** Validate generic error response when submitting non-existent email to prevent user enumeration.
- **3. Pre-conditions & Environment:** Tài khoản `nonexistent_user_99@eshop.com` hoàn toàn không tồn tại trong CSDL SQLite.
- **4. Test Data:** `email = "nonexistent_user_99@eshop.com"`, `password = "Test1234!"`
- **5. Step-by-Step Procedure:**
  1. Mở trang đăng nhập `http://localhost:5173/login`.
  2. Nhập `nonexistent_user_99@eshop.com` vào trường Email.
  3. Nhập `Test1234!` vào trường Mật khẩu.
  4. Nhấn "Sign In".
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Client:* Hiển thị thông báo lỗi chung (ví dụ: *"Invalid email or password"* hoặc *"Sai email hoặc mật khẩu"*). Không hiển thị thông báo phân biệt như *"Email không tồn tại"*.
  - *API / Network:* HTTP Status code trả về `401 Unauthorized`. Body phản hồi sử dụng cùng một thông điệp lỗi như khi nhập sai mật khẩu của tài khoản tồn tại.
  - *Security:* Chống lộ thông tin nhận diện tài khoản (User Enumeration Defense) theo chuẩn OWASP ASVS và SRS FR-02.
  - *Database State:* Không thêm hoặc sửa đổi bản ghi nào trong bảng `users`.
- **7. Teardown & Clean-up:** Không cần dọn dẹp.

Phân Nhóm 5: Phân Vùng Đơn Lỗi & Ranh Giới Biên Trường Mật Khẩu (Password Single-Fault & OWASP Boundaries)

Test Case `TC_FR02_20`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR02_20` | **Priority:** High (P1) | **Target Component:** Web Client Form & API Required Validation
  - **Traceability:** SRS FR-02 (Item 1) | **Coverage IDs:** `EP-02-08` (Mật khẩu để trống)
  - **Single-Fault Isolation:** Email hợp lệ chuẩn `test@eshop.com`; biến lỗi duy nhất là để trống mật khẩu.
- **2. Test Objective:** Validate rejection of empty password field on Web Client & REST API.
- **3. Pre-conditions & Environment:** Form đăng nhập mở tại `http://localhost:5173/login`.
- **4. Test Data:** `email = "test@eshop.com"`, `password = ""` (chuỗi rỗng)
- **5. Step-by-Step Procedure:**
  1. Nhập email hợp lệ: `test@eshop.com`.
  2. Để trống ô Mật khẩu (`""`).
  3. Nhấn nút "Sign In".
  4. Gửi request trực tiếp: `POST /api/login` với body `{email: "test@eshop.com", password: ""}`.
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Client:* Thuộc tính `required` của trường password kích hoạt cảnh báo *"Please fill out this field"*; chặn không gửi request.
  - *API / Network:* Backend trả về HTTP `400 Bad Request` với message `"Password is required"`.
  - *Database State:* CSDL không thay đổi; `login_attempts` không tăng.
- **7. Teardown & Clean-up:** Xóa nội dung trên form.

Test Case `TC_FR02_21`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR02_21` | **Priority:** Medium (P2) | **Target Component:** Backend Password Validation
  - **Traceability:** SRS FR-02 | **Coverage IDs:** `EP-02-09` (Mật khẩu toàn khoảng trắng)
  - **Single-Fault Isolation:** Email hợp lệ chuẩn `test@eshop.com`; mật khẩu chỉ chứa dấu cách.
- **2. Test Objective:** Validate rejection of password containing only whitespace characters on Web Client & API.
- **3. Pre-conditions & Environment:** Tài khoản `test@eshop.com` có mật khẩu thực tế là `Test1234!`.
- **4. Test Data:** `email = "test@eshop.com"`, `password = "  "` (4 dấu cách)
- **5. Step-by-Step Procedure:**
  1. Nhập `test@eshop.com` vào trường Email.
  2. Nhập 4 khoảng trắng `"  "` vào trường Mật khẩu.

3. Nhấn "Sign In".

- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Client:* Hiển thị thông báo đăng nhập thất bại.
  - *API / Network:* HTTP Status trả về `401 Unauthorized` (không khớp băm mật khẩu); không cấp JWT token.
  - *Database State:* `login_attempts` tăng thêm 1 đơn vị do xác thực thất bại.
- **7. Teardown & Clean-up:** Reset `login_attempts = 0` nếu cần.

Test Case `TC_FR02_22`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR02_22` | **Priority:** High (P1) | **Target Component:** Backend Authentication & Password Boundary
  - **Traceability:** SRS FR-01/02 (Ràng buộc tối thiểu 8 ký tự) | **Coverage IDs:** `EP-02-07`, `BVA-02-03-OFF` ($Len=7$)
  - **Single-Fault Isolation:** Email chuẩn `test@eshop.com`; mật khẩu sai do thiếu 1 ký tự dưới ngưỡng biên ($8 - 1 = 7$).
- **2. Test Objective:** Validate rejection of password below minimum 8 chars length at boundary offset Len=7.
- **3. Pre-conditions & Environment:** Tài khoản `test@eshop.com` yêu cầu mật khẩu chuẩn `Test1234!`.
- **4. Test Data:** `email = "test@eshop.com"`, `password = "Pass12!"` ($Len = 7$)
- **5. Step-by-Step Procedure:**
  1. Nhập `test@eshop.com` vào ô Email.
  2. Nhập mật khẩu 7 ký tự: `Pass12!`.
  3. Nhấn "Sign In".
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Client:* Báo lỗi đăng nhập không thành công.
  - *API / Network:* HTTP Status code trả về `401 Unauthorized`. Body thông báo sai thông tin xác thực.
  - *Database State:* `login_attempts` tăng thêm 1 đơn vị.
- **7. Teardown & Clean-up:** Reset `login_attempts = 0` nếu cần thiết.

Test Case `TC_FR02_23`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR02_23` | **Priority:** High (P1) | **Target Component:** Backend Password Hasher & DoS Resilience
  - **Traceability:** OWASP ASVS §2.1.7 & SRS FR-02 | **Coverage IDs:** `EP-02-09b`, `BVA-02-06-OFF` ($Len=256$)
  - **Single-Fault Isolation:** Email chuẩn `test@eshop.com`; mật khẩu vượt quá ngưỡng an toàn tối đa 255 ký tự ($255 + 1 = 256$).
- **2. Test Objective:** Validate graceful handling of password exceeding OWASP max length at boundary offset Len=256 without CPU DoS.
- **3. Pre-conditions & Environment:** Hệ thống backend đang chạy bình thường; mở client API (cURL hoặc Postman).
- **4. Test Data:** `email = "test@eshop.com"`, `password = "P"*254 + "1!"` ($Len = 256$ chars)
- **5. Step-by-Step Procedure:**
  1. Tạo chuỗi mật khẩu 256 ký tự.
  2. Gửi request `POST /api/login` với body `{email: "test@eshop.com", password: "<256_chars>"}`.
  3. Đo thời gian phản hồi của backend.
- **6. Multi-tier Validation Points (Expected Results):**
  - *API / Network:* Backend xử lý an toàn (thời gian phản hồi ≤ 1.5 giây); trả về HTTP `400 Bad Request` (chuỗi vượt độ dài tối đa) hoặc `401 Unauthorized`.
  - *Security / Resilience:* Tuyệt đối không gây nghẽn luồng xử lý Node.js (Hash DoS Attack mitigation); không làm crash server (`500 Internal Server Error`).
  - *Database State:* Không ghi nhận chuỗi bất thường vào SQLite.
- **7. Teardown & Clean-up:** Không cần teardown.

Test Case `TC_FR02_24`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR02_24` | **Priority:** Medium (P2) | **Target Component:** Backend Authentication & Boundary Capacity
  - **Traceability:** OWASP ASVS §2.1.7 & SRS FR-02 | **Coverage IDs:** `EP-02-06`, `BVA-02-06-ON` ($Len=255$)
  - **Single-Fault Isolation:** N/A (Positive boundary test case).
- **2. Test Objective:** Verify acceptance of password with maximum safe OWASP length of exactly 255 chars.
- **3. Pre-conditions & Environment:**
  - Tài khoản `testmaxpass@eshop.com` được cấu hình trong CSDL SQLite với mật khẩu đúng 255 ký tự: `"P"*253 + "1!"`.
  - Trạng thái tài khoản bình thường (`login_attempts = 0`, `locked_until = NULL`).
- **4. Test Data:** `email = "testmaxpass@eshop.com"`, `password = "P"*253 + "1!"` ($Len = 255$ chars)
- **5. Step-by-Step Procedure:**
  1. Truy cập `http://localhost:5173/login`.
  2. Nhập `testmaxpass@eshop.com` vào trường Email.
  3. Nhập mật khẩu 255 ký tự vào trường Mật khẩu.
  4. Nhấn nút "Sign In".
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Client:* Đăng nhập thành công, chuyển hướng về trang chủ `/`.
  - *API / Network:* HTTP Status code trả về `200 OK`, cấp chuỗi JWT token hợp lệ.
  - *Database State:* Hệ thống băm mật khẩu 255 ký tự và so khớp với chuỗi hash trong CSDL thành công. `login_attempts` giữ nguyên `0`.
- **7. Teardown & Clean-up:** Xóa token khỏi `localStorage`.

2.5 Bước 5: Ma trận truy vết & Kiểm tra độ bao phủ (Step 5 Deliverables)

> **Mục tiêu kiểm định:** Xác minh tính toàn diện của bộ kiểm thử bằng cách ánh xạ 100% các phân vùng tương đương (VEP / IEP) và các giá trị biên (On / Off / In) tới ít nhất một kịch bản kiểm thử cụ thể.

5.1. Bảng Độ Bao Phủ Phân Vùng Tương Đương (Equivalence Partition Coverage)

Class ID	Variable / Field	Type	Mô tả phân vùng tương đương	Covered by TC ID(s)	Trạng thái (Status)
EP-02-01	email	VEP	Email đúng định dạng và tài khoản tồn tại trong CSDL	TC_FR02_01, TC_FR02_02, TC_FR02_18	Covered
EP-02-02	email	IEP	Email không đúng định dạng cú pháp HTML5/RFC (thiếu @)	TC_FR02_14	Covered
EP-02-03	email	IEP	Email để trống / chuỗi rỗng ("")	TC_FR02_15	Covered
EP-02-04	email	IEP	Email chỉ chứa toàn khoảng trắng (" ")	TC_FR02_16	Covered
EP-02-04b	email	IEP	Email vượt độ dài tối đa chuẩn RFC 5321 ($Len > 254$)	TC_FR02_17	Covered
EP-02-05	email	IEP	Email đúng cú pháp nhưng chưa đăng ký trong CSDL	TC_FR02_19	Covered
EP-02-06	password	VEP	Mật khẩu khớp chính xác với tài khoản trong CSDL	TC_FR02_01, TC_FR02_02, TC_FR02_05, TC_FR02_24	Covered
EP-02-07	password	IEP	Mật khẩu sai / không khớp CSDL	TC_FR02_03, TC_FR02_04, TC_FR02_06, TC_FR02_22	Covered
EP-02-08	password	IEP	Mật khẩu để trống / chuỗi rỗng ("")	TC_FR02_20	Covered
EP-02-09	password	IEP	Mật khẩu chỉ chứa toàn khoảng trắng (" ")	TC_FR02_21	Covered
EP-02-09b	password	IEP	Mật khẩu vượt độ dài an toàn OWASP ASVS ($Len > 255$)	TC_FR02_23	Covered
EP-02-10	failed_attempts	VEP	Thử sai dưới ngưỡng ($1 \le N \le 2$), nhập đúng $\to$ reset về 0	TC_FR02_05	Covered
EP-02-11	failed_attempts	IEP	Số lần sai đạt/vượt ngưỡng ($N \ge 3$) $\to$ kích hoạt khóa	TC_FR02_06, TC_FR02_13	Covered
EP-02-12	failed_attempts	IEP	Chu kỳ khóa lặp lại lần 2 (Second lockout cycle)	TC_FR02_12	Covered
EP-02-13a	lockout_time	IEP	Trong thời gian tạm khóa ($0 \le t < 30s$) – Thử pass đúng	TC_FR02_07, TC_FR02_09	Covered
EP-02-13b	lockout_time	IEP	Trong thời gian tạm khóa ($0 \le t < 30s$) – Thử pass sai (timer probe)	TC_FR02_08	Covered
EP-02-14	lockout_time	VEP	Hết thời gian tạm khóa ($t \ge 30s$) – Tự động mở khóa	TC_FR02_10, TC_FR02_11	Covered

5.2. Bảng Độ Bao Phủ Phân Tích Giá Trị Biên (Boundary Value Coverage)

Boundary ID	Biến / Điều kiện	Điểm biên (Point Type)	Giá trị kiểm thử	Covered by TC ID(s)	Trạng thái (Status)
BVA-02-01	Ngưỡng số lần thử sai ($N \ge 3$)	**On-point**	$N = 3$ (Kích hoạt khóa)	TC_FR02_06, TC_FR02_12	Covered
		Off-point	$N = 2$ (Chưa bị khóa)	TC_FR02_04	Covered
		In-point	$N = 4$ (Đang bị khóa)	TC_FR02_13	Covered
BVA-02-02	Thời gian tạm khóa ($t \ge 30s$)	**On-point**	$t = 30s$ (Ranh giới mở khóa)	TC_FR02_10	Covered
		Off-point	$t = 29s$ (Vẫn bị khóa)	TC_FR02_07, TC_FR02_09	Covered
		In-point	$t = 31s$ (Đã mở khóa)	TC_FR02_11	Covered
BVA-02-03	Độ dài mật khẩu min ($Len \ge 8$)	**On-point**	$Len = 8$ (Pass123!)	TC_FR02_02	Covered
		Off-point	$Len = 7$ (Pass12!)	TC_FR02_22	Covered
		In-point	$Len = 9$ (Test1234!)	TC_FR02_01	Covered
BVA-02-04	Sequence reset ($N = 1, 2$)	**On-point**	$N = 2 \to$ Đúng (Reset 0)	TC_FR02_04, TC_FR02_05	Covered
		Off-point	$N = 3 \to$ Sai (Khóa 30s)	TC_FR02_06	Covered
		In-point	$N = 1 \to$ Sai ($N tăng 1)	TC_FR02_03	Covered
BVA-02-05	Email max RFC ($Len \le 254$)	**On-point**	$Len = 254$ (Max hợp lệ)	TC_FR02_18	Covered
		Off-point	$Len = 255$ (Vượt chuẩn RFC)	TC_FR02_17	Covered
		In-point	$Len = 253$ (Cận biên trong)	TC_FR02_01	Covered
BVA-02-06	Pass max OWASP ($Len \le 255$)	**On-point**	$Len = 255$ (Max an toàn)	TC_FR02_24	Covered
		Off-point	$Len = 256$ (Vượt ngưỡng an toàn)	TC_FR02_23	Covered
		In-point	$Len = 254$ (Cận biên trong)	TC_FR02_01	Covered

5.3. Bảng Tổng Hợp Chỉ Số Độ Bao Phủ (Coverage Summary)

Chỉ số kiểm thử (Metric)	Tổng số nhận diện	Đã bao phủ (Covered)	Chưa bao phủ / Bị chặn	Tỷ lệ bao phủ (%)
Phân vùng hợp lệ (VEP)	4	4	0	**100.0%**
Phân vùng không hợp lệ (IEP)	13	13	0	**100.0%**
Giá trị biên (On / Off / In Points)	18	18	0	**100.0%**
Tổng số kịch bản kiểm thử (Test Cases)	24	24	0	**100.0%**
Bảo đảm Giả định Đơn lỗi (Single-Fault Isolation)	13 / 13 IEPs	13	0	**100.0%**

2.6 Bước 6: AI Gap Analysis & Giả thuyết lỗi SUT (Step 6 Deliverables)

> **Mục đích:** Cung cấp minh chứng cho **Mục 6 Task 3 (AI Gap Analysis)** và làm luận cứ cho **Mục 10 (AI Critique)** của bài tập HW02, phân tách rạch ròi giữa kịch bản kiểm thử theo đặc tả và các giả thuyết khiếm khuyết nội bộ hệ thống.

6.1. Bảng Phân Tích Khoảng Trống Kiểm Thử Của AI (AI Gap Analysis)

Hạng mục / Điều kiện kiểm thử	Nguyên nhân AI bỏ sót hoặc gặp khó khăn	Hiệu chỉnh của con người (Human Fix Applied)	Căn cứ Tiêu chuẩn & Kỹ thuật Kiểm thử
Chuỗi khoảng trắng (Whitespace-only Strings)	Mô hình AI mặc định coi chuỗi rỗng (`""`) là đủ cho phân vùng không hợp lệ; bỏ qua trường hợp chuỗi chỉ chứa dấu cách (`" "`) có thể ngụy trang vượt qua validation lỏng lẻo.	Bổ sung phân vùng `EP-02-04` và `EP-02-09` kèm các test case kiểm tra cơ chế trim/reject (`TC_FR02_16` , `TC_FR02_21`).	**ISTQB FL §4.2.1:** Phân chia triệt để các lớp tương đương của dữ liệu chuỗi (Null vs Empty vs Whitespace).
Tuần tự chuyển trạng thái & Reset bộ đếm ($N \to 0$)	AI thiết kế kiểm thử theo tư duy tĩnh (Stateless input mapping), chỉ xét các giá trị riêng lẻ ($N=1, 2, 3$) mà bỏ quên kiểm chứng chuỗi tương tác thời gian (Temporal sequence).	Thiết lập kịch bản `TC_FR02_05` (`BVA-02-04-ON`): đăng nhập đúng sau 2 lần sai để xác nhận biến trạng thái `login_attempts` phải được reset về 0.	**ISTQB FL §4.2.4 & §4.2.1:** Kiểm thử chuyển đổi trạng thái kết hợp phân vùng điều kiện tuần tự.
Thăm dò Reset Timer trong thời gian khóa ($0 \le t < 30s$)	Do SRS chỉ nêu thời gian khóa 30 giây mà không mô tả hành vi khi có request sai trong lúc khóa, AI mặc định chỉ trả về 403 mà không đánh giá nguy cơ timer bị reset kéo dài vô cớ.	Bổ sung kịch bản thăm dò `TC_FR02_08` (`EP-02-13b`) để xác nhận lần thử sai trong lúc đang khóa không được phép reset lại đồng hồ đếm 30s ban đầu.	**Exploratory State Testing:** Kiểm tra tính toàn vẹn của máy trạng thái và bộ đếm thời gian phạt.
Cận trên độ dài chuỗi (Email & Password Max Length)	Đặc tả SRS không ghi rõ độ dài tối đa cho Email/Password; AI có xu hướng dừng lại ở việc không test cận trên (Open-ended boundary).	Con người chủ động truy nguyên chuẩn RFC 5321 (Email ≤ 254 ký tự) và chuẩn an toàn OWASP ASVS (Password ≤ 255 ký tự để chống DoS băm mật khẩu); bổ sung `BVA-02-05` và `BVA-02-06` .	**ISTQB FL §4.2.2 & RFC 5321 / OWASP ASVS:** Bổ sung các giá trị biên kỹ thuật dựa trên tiêu chuẩn quốc tế khi đặc tả ngầm định.
Trạng thái tài khoản Bị cấm / Vô hiệu hóa (Banned/Inactive)	Khi người dùng đề xuất kiểm thử trạng thái banned/inactive, AI cần kiểm chứng chéo với tài liệu SRS và cấu trúc CSDL thực tế thay vì tự tiện bịa đặt kịch bản nghiệp vụ.	Con người và AI rà soát mã nguồn CSDL SQLite (`backend/database.js`), xác nhận bảng `users` không có trường `status` / `banned` ; chuyển trạng thái này sang mục Khoảng trống Đặc tả.	**Nguyên tắc "Đặc tả là căn cứ duy nhất":** Không tự ý bịa đặt trạng thái hệ thống khi SRS không hỗ trợ.

6.2. Danh Sách Giả Thuyết Khiếm Khuyết Thăm Dò Trên SUT (Candidate Defect Hypotheses)

> *Ghi chú: Toàn bộ các giả thuyết dưới đây được gắn nhãn `[Exploratory - Unconfirmed Hypothesis - Requires SUT execution]` và sẽ được kiểm chứng thực tế trong Giai đoạn 3 trên hệ thống EShop đang chạy.*

Hypothesis ID	Khu vực kiểm thử (Target Area)	Mô tả giả thuyết lỗi (Description)	Kỳ vọng theo đặc tả (Spec Expected)	Dự đoán hành vi thực tế (Hypothesized Actual)	Bước kiểm chứng trên SUT (Verification Step)
`HYP-02-01`	Backend API (`server.js:54`)	Bộ đếm số lần sai tăng sai bước nhảy ($+2$ thay vì $+1$)	SRS FR-02: *"Sau mỗi lần đăng nhập sai, hệ thống tăng bộ đếm lên đúng 1 đơn vị."*	Backend thực hiện phép tính `newAttempts = user.login_attempts + 2` $\to$ Nhập sai lần 1 đã nhảy lên 2, sai lần 2 đã chạm mốc 4 và khóa ngay lập tức!	Thực thi `TC_FR02_03` & `TC_FR02_04` , kiểm tra trực tiếp cột `login_attempts` trong file `database.sqlite` sau lần sai thứ 1.
`HYP-02-02`	Backend Lockout (`server.js:57`)	Thời gian khóa thực tế sai lệch nghiêm trọng so với đặc tả demo	SRS FR-02: *"Nếu đăng nhập sai từ 3 lần trở lên liên tiếp, tài khoản bị tạm khóa 30 giây (môi trường demo)."*	Mã nguồn backend gán `Date.now() + 180000` (180 giây = 3 phút)! Người dùng bị khóa tới 3 phút thay vì 30 giây.	Thực thi `TC_FR02_10` tại mốc $t = 30s$; nếu backend vẫn trả về HTTP 403 đến phút thứ 3 thì giả thuyết được xác nhận.
`HYP-02-03`	Frontend UI (`Login.jsx:30, 40`)	Sai thuộc tính `type` của ô input Email và Password	SRS FR-02: *"Trường email phải dùng `type="email"` (có validate HTML5 format)."* Mật khẩu phải ẩn ký tự (`type="password"`).	Input email dùng `type="text"` (mất validate HTML5); input password dùng `type="text"` làm lộ mật khẩu dạng bản rõ (plaintext) trên màn hình!	Mở DevTools Elements trên `http://localhost:5173/login` , inspect thẻ `<input>` của trường Email và Password.
`HYP-02-04`	Frontend UI (`Login.jsx:24`)	Tiêu đề giao diện Đăng nhập bị hiển thị sai thành "Đăng Ký"	Giao diện đăng nhập phải hiển thị tiêu đề nhất quán: "Đăng Nhập" hoặc "Sign In".	Mã nguồn React hardcode: `<h2 ...>Đăng Ký</h2>` tại trang Login gây hoang mang cho người dùng.	Mở `http://localhost:5173/login` trên trình duyệt và quan sát thẻ `<h2>` đầu trang.
`HYP-02-05`	Backend State (`server.js:48, 54`)	Bộ đếm thử sai không được tự động giải phóng sau khi hết hạn 30s	Sau khi hết 30s khóa, nếu người dùng thử lại sai 1 lần thì phải tính là lần sai thứ 1 ($N=1$).	CSDL không tự reset `login_attempts` khi hết hạn. Nếu người dùng nhập sai tiếp, `newAttempts = 3 + 2 = 5` và lập tức bị khóa tiếp lần 2 chỉ sau đúng 1 lần thử sai!	Thực thi `TC_FR02_12` sau khi hết thời gian khóa lần 1; quan sát số lần thử cần thiết để kích hoạt khóa lần 2.

---

3. TÍNH NĂNG B (POOL B): FR-07 – GIỎ HÀNG (SHOPPING CART)

3.1 Bước 1: Nhận diện biến đầu vào & Ràng buộc (Step 1 Deliverables)

Trích dẫn đặc tả SRS FR-07 (Giỏ hàng):

- Hiển thị danh sách sản phẩm với các cột: **Sản phẩm**, **Đơn giá**, **Số lượng** (có nút +/- để chỉnh), **Thành tiền**, **Thao tác**.
- Thêm cùng một sản phẩm vào giỏ sẽ tăng số lượng, không tạo dòng mới.
- Nút **Xóa sản phẩm** phải có dialog xác nhận trước khi thực hiện.
- Có nút **Tiếp tục mua sắm** để quay về trang chủ.
- Tổng tiền hiển thị nhãn chính xác: **"Tổng cộng"** (không phải "Tổng tạm tính").
- Giỏ hàng trống phải có hình minh họa và thông báo rõ ràng.

(Đối chiếu SRS FR-06: "Có ô nhập Số lượng (chỉ nhận số nguyên dương, tối thiểu là 1). Nút Thêm vào giỏ hàng — sau khi bấm hiển thị phản hồi trực quan"; SRS FR-08: "Tổng tiền thanh toán được tính tự động từ giỏ hàng và không cho phép người dùng chỉnh sửa trực tiếp.")

Variable ID	Variable Name	Data Type	Mandatory?	Precision (ϵ)	Specification Constraint (Quote)	Assumptions / Ambiguities
`VAR-07-01`	`product_id` (hoặc sản phẩm trong giỏ)	Integer	Có	1	Trích dẫn SRS FR-07: *"Hiển thị danh sách sản phẩm với các cột: Sản phẩm..."* và *"Thêm cùng một sản phẩm vào giỏ sẽ tăng số lượng, không tạo dòng mới."*	Sản phẩm phải tồn tại trong CSDL (`products` table). Phân biệt 2 tình huống: thêm sản phẩm mới (chưa có trong giỏ $\to$ tạo dòng mới) và thêm sản phẩm trùng lặp (đã có trong giỏ $\to$ cộng dồn số lượng vào dòng hiện tại).
`VAR-07-02`	`quantity` (Số lượng sản phẩm)	Integer	Có	1 đơn vị	Trích dẫn SRS FR-07: *"Số lượng (có nút +/- để chỉnh)"* (Đối chiếu FR-06: *"Có ô nhập Số lượng (chỉ nhận số nguyên dương, tối thiểu là 1)"*)	Số nguyên dương $quantity \ge 1$. Đặc tả SRS không định nghĩa số lượng tối đa hay ràng buộc tồn kho (`stock_quantity`) trong CSDL (bảng `products` thực tế không có cột tồn kho). Cần kiểm tra cận biên số học, nhập số 0, số âm, số thập phân hoặc chuỗi phi số.
`VAR-07-03`	`unit_price` (Đơn giá sản phẩm)	Integer / ₫	Hệ thống	1 ₫	Trích dẫn SRS FR-07: *"cột: Sản phẩm, Đơn giá..."*	Lấy trực tiếp từ bảng `products.price` trong CSDL; là số nguyên dương > 0. Người dùng không được phép chỉnh sửa đơn giá trên giao diện giỏ hàng.
`VAR-07-04`	`subtotal` (Thành tiền từng dòng)	Integer / ₫	Hệ thống	1 ₫	Trích dẫn SRS FR-07: *"cột: ... Thành tiền"*	Công thức tính toán: $Subtotal = unit_price \times quantity$. Cập nhật tức thời (reactive) khi thay đổi số lượng.
`VAR-07-05`	`total_amount` & `total_label` (Tổng tiền & Nhãn hiển thị)	Compound (Integer & String)	Hệ thống	1 ₫ / Text	Trích dẫn SRS FR-07: *"Tổng tiền hiển thị nhãn chính xác: 'Tổng cộng' (không phải 'Tổng tạm tính')."*	Tổng tiền bằng tổng các thành tiền: $Total = \sum Subtotal$. Nhãn văn bản hiển thị cạnh số tiền tổng bắt buộc phải là `"Tổng cộng"`; bất kỳ nhãn nào như `"Tổng tạm tính"` hay `"Subtotal"` đều bị coi là lỗi vi phạm đặc tả.
`VAR-07-06`	`cart_state` (Trạng thái giỏ hàng)	Categorical	Hệ thống	N/A	Trích dẫn SRS FR-07: *"Giỏ hàng trống phải có hình minh họa và thông báo rõ ràng."*	Trạng thái hiển thị gồm 2 miền: Trống ($Items = 0$) và Có sản phẩm ($Items \ge 1$). Khi giỏ trống, hệ thống bắt buộc phải hiển thị đồng thời cả hình minh họa (illustration image) và câu thông báo rõ ràng.
`VAR-07-07`	`button_decrement_action` (Thao tác nút `-` tại biên số lượng)	Interaction / Action	Tương tác	N/A	Trích dẫn SRS FR-07: *"Số lượng (có nút +/- để chỉnh)"* và *"Nút Xóa sản phẩm phải có dialog xác nhận trước khi thực hiện."*	Khi $quantity > 1$, bấm `-` giảm 1 đơn vị. Khi $quantity = 1$, bấm `-` là điểm biên chuyển đổi trạng thái: hệ thống cần disable nút hoặc nếu xem là hành vi xóa sản phẩm thì bắt buộc phải kích hoạt dialog xác nhận xóa, không được tự ý xóa mà không hỏi hoặc giảm về 0 / số âm.
`VAR-07-08`	`button_delete_action` & `dialog_confirmation` (Nút Xóa & Dialog xác nhận)	Boolean (Confirm / Cancel)	Tương tác	N/A	Trích dẫn SRS FR-07: *"Nút Xóa sản phẩm phải có dialog xác nhận trước khi thực hiện."*	Bấm nút "Xóa" bắt buộc phải hiển thị dialog xác nhận. Nếu người dùng chọn Hủy (`Cancel`), giữ nguyên sản phẩm trong giỏ; chỉ khi chọn Đồng ý (`Confirm`), sản phẩm mới bị xóa khỏi giỏ và tính lại tổng tiền.
`VAR-07-09`	`continue_shopping_navigation` (Nút Tiếp tục mua sắm)	Action / Route	Tương tác	N/A	Trích dẫn SRS FR-07: *"Có nút Tiếp tục mua sắm để quay về trang chủ."*	Nút bấm hoặc liên kết điều hướng phải đưa người dùng quay trở lại trang chủ (`/`) một cách an toàn mà không làm mất trạng thái giỏ hàng.

3.2 Bước 2: Phân vùng tương đương (Step 2 Deliverables)

Nguyên tắc Giả định Đơn lỗi (Single Fault Assumption - ISTQB FL §4.2.1) & Tách bạch Phạm vi Kiểm thử:

- **Phân tách rõ ràng 3 lớp khái niệm:** Để đảm bảo số liệu đo lường độ bao phủ EP mang ý nghĩa chuẩn xác và không bị thổi phồng, thiết kế phân tách rõ giữa: (1) Các phân vùng tương đương biến đầu vào thực thụ (Input EPs); (2) Các quy tắc nghiệp vụ, chuyển trạng thái và kịch bản tương tác (State & Interaction Rules); và (3) Các điểm kiểm chứng giao diện / giả thuyết khiếm khuyết (Assertions & Defect Probes).
- **Nguyên tắc Single-Fault Isolation:** Khi kiểm thử một phân vùng không hợp lệ (IEP), chỉ duy nhất 1 điều kiện hoặc biến mang giá trị sai; toàn bộ các tham số còn lại bắt buộc phải giữ ở giá trị hợp lệ cơ sở (Baseline).
- **Căn cứ đặc tả:** Căn cứ duy nhất là SRS FR-07 và FR-06. Số lượng Q là số nguyên dương $Q \ge 1$ (không tự ý đặt cận trên 99 khi SRS không quy định).

3.2.1 Bảng Phân Vùng Tương Đương Biến Đầu Vào (Pure Input Equivalence Partitions)

Class ID	Variable / Input	Domain / Sub-condition	Type (VEP / IEP)	Representative Value	Single-Fault Baseline / Rationale
`EP-07-01`	`product_id`	Mã sản phẩm hợp lệ và tồn tại trong CSDL	VEP	`id = 1` (iPhone 15 Pro Max)	Giỏ hàng ban đầu chưa có sản phẩm ID 1. Hệ thống tạo thêm đúng 1 dòng mới trong bảng giỏ hàng với số lượng ban đầu.
`EP-07-02`	`product_id`	Mã sản phẩm đúng định dạng số nhưng không tồn tại trong CSDL	IEP	`id = 9999`	Gửi request thêm sản phẩm với ID không tồn tại. Cô lập lỗi ID không có trong CSDL; hệ thống từ chối hoặc trả về lỗi 404/400.
`EP-07-03`	`product_id`	Mã sản phẩm sai định dạng / sai kiểu dữ liệu (Malformed ID)	IEP	`id = "abc"` (hoặc `-1`)	Request gửi qua API `POST /api/cart` hoặc route. Cô lập lỗi sai kiểu dữ liệu (String/Negative thay vì positive integer); hệ thống từ chối dữ liệu rác.
`EP-07-04`	`quantity`	Số lượng nguyên dương hợp lệ ($Q \in \mathbb{Z}, Q \ge 1$)	VEP	$Q = 2$	Sản phẩm hợp lệ trong giỏ. Đặc tả SRS FR-06/FR-07 quy định: *"chỉ nhận số nguyên dương, tối thiểu là 1"*, không đặt cận trên 99 hay giới hạn tồn kho. Cập nhật số lượng và tính lại thành tiền.
`EP-07-05`	`quantity`	Số lượng bằng 0 ($Q = 0$)	IEP	$Q = 0$	Sản phẩm hợp lệ trong giỏ. Cô lập vi phạm quy tắc số nguyên dương tối thiểu là 1; hệ thống từ chối hoặc yêu cầu xác nhận xóa, không được lưu $Q=0$.
`EP-07-06`	`quantity`	Số lượng là số nguyên âm ($Q < 0$)	IEP	$Q = -1$ (hoặc -5)	Sản phẩm hợp lệ trong giỏ. Cô lập lỗi số âm; hệ thống từ chối hoặc tự động phục hồi về 1, không làm âm tổng tiền giỏ hàng.
`EP-07-07`	`quantity`	Chuỗi rỗng / để trống ô số lượng	IEP	$Q = \text{""}$	Sản phẩm hợp lệ trong giỏ. Cô lập lỗi thiếu giá trị bắt buộc; hệ thống chặn submit hoặc khôi phục giá trị hợp lệ trước đó.
`EP-07-08`	`quantity`	Chuỗi chỉ chứa khoảng trắng (Whitespace-only)	IEP	$Q = \text{" "}$	Sản phẩm hợp lệ trong giỏ. Cô lập lỗi chuỗi rỗng nguy trang; hệ thống chặn submit hoặc trim và báo lỗi.
`EP-07-09`	`quantity`	Ký tự phi số (Chữ cái, ký hiệu đặc biệt)	IEP	$Q = \text{"abc"}$ hoặc $\text{"@#$"}$	Sản phẩm hợp lệ trong giỏ. Input `type="number"` hoặc validator chặn ký tự chữ, không để xảy ra lỗi tính toán `NaN`.
`EP-07-10`	`quantity`	Số thập phân / số thực không nguyên	IEP	$Q = 1.5$ (hoặc 2.5)	Sản phẩm hợp lệ trong giỏ. Cô lập lỗi vi phạm quy tắc "chỉ nhận số nguyên dương" (FR-06/FR-07); từ chối dấu chấm/phẩy thập phân.
`EP-07-11`	`quantity`	Bị bỏ khuyết hoặc giá trị `null` trong payload API	IEP	`quantity: null` hoặc omitted	Gửi request `POST /api/cart` không truyền `quantity` hoặc truyền `null`. Cô lập lỗi khuyết trường bắt buộc; backend trả về lỗi 400.
`EP-07-12`	`quantity`	Số nguyên cực lớn kiểm thử độ bền số học (Robustness / Stress)	IEP (Stress)	$Q = 10^9$ (1 tỷ đơn vị)	Sản phẩm hợp lệ trong giỏ. Kiểm tra độ bền hệ thống, không để tràn số nguyên JavaScript hoặc biến thành `Infinity`.

3.2.2 Bảng Phân Tích Quy Tắc Nghiệp Vụ, Trạng Thái & Tương Tác (Business Rules, State & Interaction Scenarios)

Rule / State ID	Hạng mục	Quy tắc nghiệp vụ & Hành vi tương tác	Phân loại	Dữ liệu / Kịch bản đại diện	Kết quả mong đợi theo Đặc tả SRS
`RULE-07-01`	Trùng sản phẩm	Thêm sản phẩm đã tồn tại sẵn trong giỏ hàng	Business Rule	Giỏ đã có SP1 ($Q=1$). Thêm tiếp SP1 với $Q_{add}=1$.	Tăng số lượng thành $Q = 1 + 1 = 2$ trên cùng dòng sản phẩm; tuyệt đối **không tạo dòng mới** (SRS FR-07).
`RULE-07-02`	Thành tiền dòng	Tính toán Thành tiền cho từng dòng sản phẩm	Calculation Rule	SP1: Đơn giá $30.000.000\text{ đ}$, Số lượng 2	Công thức: $Subtotal = unit_price \times quantity = 60.000.000\text{ đ}$. Cập nhật tức thời khi đổi số lượng.
`RULE-07-03`	Tổng tiền giỏ	Tính toán Tổng tiền toàn bộ giỏ hàng	Calculation Rule	SP1 ($Subtotal = 60.000.000\text{ đ}$) + SP2 ($Subtotal = 6.000.000\text{ đ}$)	Công thức: $Total = \sum Subtotal = 66.000.000\text{ đ}$.
`STATE-07-01`	Trạng thái giỏ	Giỏ hàng có sản phẩm ($Items \ge 1$)	Valid State	Giỏ có 1 hoặc nhiều mặt hàng	Hiển thị bảng chi tiết gồm đủ 5 cột: Sản phẩm, Đơn giá, Số lượng, Thành tiền, Thao tác; cùng tổng tiền và nút thanh toán.
`STATE-07-02`	Trạng thái giỏ	Giỏ hàng trống hoàn toàn ($Items = 0$)	Valid State	Giỏ rỗng (0 sản phẩm)	Ẩn bảng sản phẩm; bắt buộc hiển thị đồng thời cả **hình minh họa** (illustration) và thông báo giỏ hàng trống rõ ràng.
`ACTION-07-01`	Nút `-`	Bấm nút `-` khi số lượng lớn hơn 1 ($Q > 1$)	Interaction	Đang có $Q=2$, click nút `-`	Số lượng giảm đúng 1 đơn vị ($Q = 1$), thành tiền và tổng tiền lập tức cập nhật lại.
`ACTION-07-02`	Nút `-`	Bấm nút `-` khi số lượng chạm biên tối thiểu ($Q = 1$)	State Transition / Ambiguity	Đang có $Q=1$, click nút `-`	**Ghi nhận độ mơ hồ đặc tả:** SRS FR-07 không nêu rõ nút `-` sẽ bị vô hiệu hóa (disabled) để giữ $Q \ge 1$ hay sẽ kích hoạt dialog xác nhận xóa sản phẩm. Hệ thống tuyệt đối không được tự ý xóa không hỏi hoặc giảm âm.
`ACTION-07-03`	Nút "Xóa"	Bấm nút "Xóa" $\to$ Xác nhận Đồng ý (Confirm)	Interaction / Modal	Click Xóa $\to$ Click "OK / Confirm" trên dialog	Hiển thị dialog xác nhận; khi bấm OK, dòng sản phẩm bị xóa khỏi giỏ và cập nhật lại tổng tiền.
`ACTION-07-04`	Nút "Xóa"	Bấm nút "Xóa" $\to$ Hủy bỏ thao tác (Cancel)	Interaction / Modal	Click Xóa $\to$ Click "Cancel / Hủy" trên dialog	Hiển thị dialog xác nhận; khi bấm Cancel, sản phẩm vẫn được bảo toàn nguyên vẹn trong giỏ hàng.
`ACTION-07-05`	Điều hướng	Bấm nút "Tiếp tục mua sắm"	Navigation	Click link/button "Tiếp tục mua sắm"	Trình duyệt chuyển hướng an toàn về trang chủ (`/`), giỏ hàng được bảo toàn.

3.2.3 Bảng Điểm Kiểm Chứng Giao Diện & Giả Thuyết Khiếm Khuyết SUT (UI Assertions & Defect Probes)

Assertion ID	Hạng mục kiểm chứng	Yêu cầu bắt buộc của SRS FR-07	Dấu hiệu vi phạm (Potential SUT Defect)	Phương pháp kiểm chứng
`ASSERT-07-01`	Nhãn Tổng tiền	Tổng tiền hiển thị nhãn chính xác: **"Tổng cộng"** (không phải "Tổng tạm tính").	Giao diện hiển thị nhãn `"Tổng tạm tính:"` hoặc `"Subtotal"`.	Quan sát văn bản hiển thị cạnh số tiền tổng ở footer giỏ hàng.

Assertion ID	Hạng mục kiểm chứng	Yêu cầu bắt buộc của SRS FR-07	Dấu hiệu vi phạm (Potential SUT Defect)	Phương pháp kiểm chứng
`ASSERT-07-02`	Dialog xác nhận xóa	Nút Xóa sản phẩm phải có **dialog xác nhận** trước khi thực hiện.	Bấm nút Xóa thì sản phẩm biến mất ngay lập tức mà không xuất hiện bất kỳ dialog xác nhận nào.	Click nút Xóa tại dòng sản phẩm và kiểm tra sự xuất hiện của popup/dialog xác nhận.
`ASSERT-07-03`	Hình minh họa giỏ trống	Giỏ hàng trống phải có **hình minh họa** và thông báo rõ ràng.	Giao diện giỏ hàng trống chỉ có dòng chữ thông báo mà hoàn toàn không có thẻ hình ảnh (`<img>` /SVG minh họa).	Xóa hết sản phẩm trong giỏ và kiểm tra cây DOM xem có thẻ ảnh minh họa hay không.

3.3 Bước 3: Phân tích giá trị biên (Step 3 Deliverables - BVA)

> **Phương pháp tiếp cận Phân tích Giá trị Biên (ISTQB FL §4.2.2):**
>
> - **Lựa chọn phương pháp:** Áp dụng kỹ thuật **3-Value BVA** (On-point, Off-point, In-point) cho các miền giá trị có thứ tự và ranh giới chuyển tiếp trạng thái với bước vi sai nhỏ nhất (ϵ).
> - **Định nghĩa các điểm biên:**
>   - **On-point:** Giá trị nằm chính xác trên đường phân định ranh giới.
>   - **Off-point:** Giá trị nằm sát biên nhất ở phía đối diện (ngay ngoài biên nếu On nằm trong; hoặc ngay trong nếu On nằm ngoài).
>   - **In-point:** Giá trị nằm liền kề bên trong phân vùng cần kiểm thử.
> - **Ghi chú về Cận trên Số lượng (Upper Bound Confirmation):** Đối chiếu đặc tả SRS FR-06 và FR-07, hệ thống **không quy định giới hạn tồn kho (`stock_quantity`) hay cận trên số lượng tối đa**. Do đó, miền số lượng có biên dưới đóng $[1$ và biên trên mở $+\infty)$. Thiết kế tập trung vào biên dưới hợp lệ, ranh giới chuyển tiếp số lượng, ranh giới trạng thái số mặt hàng trong giỏ, và kiểm thử độ bền số học cực hạn.

Boundary ID	Variable / Field	Boundary Rule	BVA Method	On-point (Value/Type)	Off-point (Value/Type)	In-point (Value/Type)
`BVA-07-01`	`quantity`	$Q \ge 1$ (Ngưỡng số lượng nguyên dương tối thiểu theo SRS FR-06/FR-07)	3-Value ($\epsilon=1\text{ đơn vị}$)	$Q = 1$ [Valid - On-point] Số lượng tối thiểu hợp lệ; hiển thị đúng thành tiền.	$Q = 0$ [Invalid - Off-point] $1 - 1 = 0$, vi phạm điều kiện $Q \ge 1$; chặn không cho lưu hoặc yêu cầu xác nhận.	$Q = 2$ [Valid - In-point] $1 + 1 = 2$, nằm liền kề bên trong vùng hợp lệ.
`BVA-07-02`	`cart_items_count`	$Items \ge 1 \leftrightarrow Items = 0$ (Ranh giới chuyển tiếp trạng thái Giỏ rỗng vs Giỏ có hàng)	3-Value State Transition ($\epsilon=1\text{ SP}$)	$Items = 1$ [Valid Non-Empty] Còn đúng 1 SP trong giỏ $\to$ Hiển thị bảng chi tiết, thành tiền, tổng tiền.	$Items = 0$ [Valid Empty] $1 - 1 = 0$ (sau khi xóa SP cuối) $\to$ Ẩn bảng; hiển thị hình minh họa và thông báo giỏ rỗng.	$Items = 2$ [Valid Non-Empty] $1 + 1 = 2$ SP trong giỏ $\to$ Hiển thị nhiều dòng và cộng dồn tổng tiền.
`BVA-07-03`	`button_decrement_action`	$Q = 1 \xrightarrow{\text{click } -} \text{Chuyển tiếp}$ (Ranh giới thao tác nút `-` tại mốc tối thiểu)	3-Value Transition	Bấm nút `-` tại $Q = 1$ [Boundary Event] Hệ thống disable nút `-` hoặc kích hoạt dialog xác nhận xóa.	Giảm âm $Q \le 0$ [Invalid Behavior] Nếu bấm `-` mà số lượng giảm về 0 hoặc số âm mà không hỏi, vi phạm SRS.	Bấm nút `-` tại $Q = 2$ [Valid Event] $2 - 1 = 1$, số lượng giảm về 1 bình thường.
`BVA-07-04`	`quantity.format`	$Q \in \mathbb{Z}^+$ (Ranh giới số nguyên vs số thập phân kề cận mốc tối thiểu)	3-Value Real ($\epsilon=0.1$)	$Q = 1.0$ [Valid - On-point] Số nguyên chuẩn xác $Q = 1$.	$Q = 0.9$ hoặc $Q = 1.1$ [Invalid - Off-point] Số thập phân kề cận; hệ thống từ chối hoặc không nhận dấu chấm/phẩy.	$Q = 2.0$ [Valid - In-point] Số nguyên hợp lệ tiếp theo.
`BVA-07-05`	`quantity.robustness`	$Q \le 2^{53}-1$ (Ngưỡng an toàn số nguyên JavaScript `MAX_SAFE_INTEGER`)	Robustness Boundary ($\epsilon=1$)	$Q = 9.007.199.254.740.991$ [On-point] Giá trị nguyên lớn nhất an toàn trong JS; không gây lỗi tràn số.	$Q = 9.007.199.254.740.992$ [Off-point] Vượt ngưỡng an toàn JS; kiểm tra hệ thống không bị lỗi crash hoặc `Infinity` .	$Q = 9.007.199.254.740.990$ [In-point] Giá trị kề cận trong ngưỡng an toàn.

3.4 Bước 4: Thiết kế kịch bản kiểm thử (Step 4 Deliverables)

> **Phương pháp luận Thiết kế Kịch bản Kiểm thử (Theo `Comprehensive Guide: How to Write Test Cases`):**
>
> 1. **Cú pháp Định danh Mục tiêu (Title / Objective Formula):** Tuân thủ công thức: $$\text{Action (Hành động)} + \text{Function (Chức năng)} + \text{Operating Condition (Điều kiện vận hành)}$$
> 2. **Nguyên tắc Giả định Đơn lỗi (Single Fault Isolation):** Mọi kịch bản Negative (IEP hoặc Off-point) chỉ kích hoạt đúng 1 điều kiện sai; các biến còn lại giữ nguyên giá trị hợp lệ cơ sở (Baseline).
> 3. **Tính Tự dọn dẹp (Self-Cleaning / Teardown):** Khôi phục trạng thái giỏ hàng rỗng sau mỗi kịch bản để đảm bảo tính độc lập tuyệt đối giữa các lượt test.
> 4. **Điểm kiểm chứng đa tầng (Multi-tier Validation Points):** Phân tách rõ kỳ vọng ở các tầng: Giao diện Web (UI), Giao diện Lập trình (API), Trạng thái CSDL / Context (Data State), và Tuân thủ Đặc tả SRS (Specification Adherence).

3.4.1 Bảng Tổng Hợp Kịch Bản Kiểm Thử & Ma Trận Truy Vết (Overview Traceability Matrix)

TC ID	Test Case Objective ($\text{Action} + \text{Function} + \text{Condition}$)	Phân loại	Mã EP / Rule bao phủ	Mã BVA bao phủ	Dữ liệu đầu vào & Baseline	Kết quả cốt lõi (Core Expected)	Trạng thái
`TC_FR07_01`	Verify adding a new valid product to empty cart on Web & API	Positive (VEP)	`EP-07-01` , `STATE-07-01`	`BVA-07-01-ON` , `BVA-07-02-ON`	Giỏ rỗng; thêm `id = 1` (iPhone 15, giá 30tr), $Q = 1$	Tạo đúng 1 dòng trong giỏ, hiển thị đủ 5 cột, Thành tiền 30tr, Tổng 30tr.	**Fail (BUG-FR07-01)**

TC ID	Test Case Objective ($\text{Action} + \text{Function} + \text{Condition}$)	Phân loại	Mã EP / Rule bao phủ	Mã BVA bao phủ	Dữ liệu đầu vào & Baseline	Kết quả cốt lõi (Core Expected)	Trạng thái
TC_FR07_02	Verify adding duplicate product increments quantity without creating new row	Positive (Rule)	RULE-07-01, EP-07-04	BVA-07-01-IN ($Q=2$)	Giỏ đã có id = 1 ($Q=1$); thêm tiếp id = 1 ($Q_{add}=1$)	Số lượng tăng $1 \to 2$ trên dòng hiện tại, vẫn duy trì đúng 1 dòng; Thành tiền 60tr.	**Fail (BUG-FR07-02)**
TC_FR07_03	Validate rejection when adding well-formed non-existent product ID (Single Fault)	Negative (IEP)	EP-07-02	N/A (Product DB)	id = 9999, $Q = 1$	Báo lỗi không tìm thấy sản phẩm; không thêm bản ghi rác vào giỏ.	**Fail (BUG-FR07-03)**
TC_FR07_04	Validate rejection when submitting malformed product ID via API (Single Fault)	Negative (IEP)	EP-07-03	N/A (Format/Type)	id = "abc" (hoặc -1), $Q = 1$ gửi qua POST /api/cart	HTTP 400 Bad Request; từ chối xử lý dữ liệu sai kiểu.	**Fail (BUG-FR07-04)**
TC_FR07_05	Validate rejection of quantity zero input on Cart item (Boundary Off-point)	Negative (BVA)	EP-07-05	BVA-07-01-OFF ($Q=0$)	Giỏ có SP1; sửa ô số lượng thành $Q = 0$	Chặn không cho nhập 0, hoặc tự khôi phục về 1, hoặc hiện dialog hỏi xóa; không lưu $Q=0$.	**Fail (BUG-FR07-05)**
TC_FR07_06	Validate rejection of negative integer quantity input (Single Fault)	Negative (IEP)	EP-07-06	N/A (Negative Range)	Giỏ có SP1; nhập $Q = -1$ (hoặc -5)	Chặn ký tự - hoặc tự khôi phục về 1; không để thành tiền âm.	**Fail (BUG-FR07-06)**
TC_FR07_07	Validate rejection of empty or whitespace quantity input (Single Fault)	Negative (IEP)	EP-07-07, EP-07-08	N/A (Required)	Giỏ có SP1; xóa trắng ô số lượng ($Q = \text{""}$ hoặc " ")	Chặn submit hoặc tự động khôi phục về giá trị hợp lệ trước đó.	**Fail (BUG-FR07-07)**
TC_FR07_08	Validate rejection of non-numeric string characters in quantity field (Single Fault)	Negative (IEP)	EP-07-09	N/A (Format)	Giỏ có SP1; gõ ký tự chữ $Q = \text{"abc"}$	Input type="number" chặn ký tự chữ; không làm xuất hiện lỗi NaN.	**Pass**
TC_FR07_09	Validate rejection of decimal quantity near lower boundary (Boundary Off-point)	Negative (BVA)	EP-07-10	BVA-07-04-OFF ($Q=1.5$)	Giỏ có SP1; nhập số thập phân $Q = 1.5$ (hoặc $0.9 / 1.1$)	Chặn ký tự dấu . / , ; chỉ chấp nhận số nguyên dương $Q \in \mathbb{Z}^+$.	**Fail (BUG-FR07-08)**
TC_FR07_10	Validate API rejection when quantity is omitted or null in payload (Single Fault)	Negative (IEP)	EP-07-11	N/A (Payload)	POST /api/cart với payload thiếu quantity hoặc quantity: null	HTTP 400 Bad Request; backend bắt buộc trường số lượng nguyên dương.	**Fail (BUG-FR07-09)**
TC_FR07_11	Validate system robustness against extremely large integer quantity (Robustness)	Robustness	EP-07-12	BVA-07-05 ($Q=10^9$)	Giỏ có SP1; nhập $Q = 10^9$	Hệ thống xử lý an toàn không treo máy, không gây tràn số JS hay biến thành Infinity.	**Pass**
TC_FR07_12	Verify arithmetic calculation of line subtotals and cart total for multiple items	Positive (Calc)	RULE-07-02, RULE-07-03	BVA-07-02-IN ($Items=2$)	SP1 (30tr, $Q=2$) + SP2 (AirPods 6tr, $Q=1$)	Thành tiền SP1: 60tr; SP2: 6tr; Tổng tiền: 66.000.000 ₫ (khớp công thức $\sum Subtotal$).	**Pass**
TC_FR07_13	Verify quantity decrement via '-' button when quantity exceeds minimum (Normal)	Positive (Action)	ACTION-07-01	BVA-07-03-IN ($Q=2 \to 1$)	Đang có SP1 với $Q = 2$; bấm nút -	Số lượng giảm về $Q = 1$; Thành tiền giảm từ 60tr về 30tr; Tổng tiền cập nhật tương ứng.	**Fail (BUG-FR07-10)**
TC_FR07_14	Validate boundary decrement behavior via '-' button at minimum quantity Q=1 (Probe)	Boundary / Probe	ACTION-07-02	BVA-07-03-ON ($Q=1$)	Đang có SP1 với $Q = 1$; bấm nút -	Nút - bị disabled HOẶC bật dialog xác nhận xóa; tuyệt đối KHÔNG tự ý giảm âm $Q \le 0$.	**Fail (BUG-FR07-10)**
TC_FR07_15	Verify item removal upon confirming delete dialog (Interaction Flow)	Positive (Action)	ACTION-07-03	N/A (Delete Flow)	Giỏ có 2 SP; bấm "Xóa" tại SP1 $\to$ Bấm "OK / Đồng ý" trên dialog	Dialog xác nhận xuất hiện; sau khi xác nhận, SP1 bị xóa khỏi giỏ, tổng tiền cập nhật lại.	**Fail (BUG-FR07-11)**
TC_FR07_16	Verify item retention upon cancelling delete dialog (Interaction Flow)	Positive (Action)	ACTION-07-04	N/A (Cancel Flow)	Giỏ có 1 SP; bấm "Xóa" $\to$ Bấm "Cancel / Hủy" trên dialog	Dialog xác nhận xuất hiện; sau khi bấm Cancel, SP vẫn giữ nguyên vẹn trong giỏ hàng.	**Fail (BUG-FR07-11)**
TC_FR07_17	Verify empty cart state display upon deleting last remaining item (State Boundary)	Positive (State)	STATE-07-02	BVA-07-02-OFF ($Items=0$)	Giỏ còn đúng 1 SP ($Items=1$); xóa SP đó khỏi giỏ $\to Items=0$	Ẩn bảng sản phẩm; bắt buộc hiển thị đồng thời cả **hình minh họa** và thông báo giỏ rỗng.	**Fail (BUG-FR07-12)**
TC_FR07_18	Validate total amount UI label matches required "Tổng cộng" text (UI Assertion)	Negative / Probe	ASSERT-07-01	N/A (UI String)	Giỏ có sản phẩm; quan sát nhãn hiển thị cạnh số tiền tổng	Nhãn hiển thị chính xác là "Tổng cộng"; cấm hiển thị "Tổng tạm tính" (SRS FR-07).	**Fail (BUG-FR07-13)**
TC_FR07_19	Validate enforcement of confirmation dialog before product deletion (Safety Probe)	Negative / Probe	ASSERT-07-02	N/A (Safety Check)	Giỏ có sản phẩm; bấm nút "Xóa" tại dòng sản phẩm	Bắt buộc phải bật dialog xác nhận; nếu SP bị xóa ngay không hỏi $\to$ Defect vi phạm SRS.	**Fail (BUG-FR07-11)**
TC_FR07_20	Verify navigation back to home page via "Tiếp tục mua sắm" button (Navigation Flow)	Positive (Nav)	ACTION-07-05	N/A (Routing)	Tại trang giỏ hàng (/cart), bấm nút "Tiếp tục mua sắm"	Trình duyệt chuyển hướng về trang chủ (/), giỏ hàng vẫn được bảo toàn nguyên trạng.	**Pass**

3.4.2 Hồ Sơ Đặc Tả Chi Tiết 100% Các Kịch Bản Kiểm Thử (Detailed Test Specification Cards for All 20 TCs)

Dưới đây là hồ sơ đặc tả chi tiết 7 phân đoạn chuẩn (*Tracking Information, Objective, Pre-conditions & Environment, Test Data, Procedure, Multi-tier Validation Points, Teardown & Clean-up*) cho toàn bộ 20 kịch bản kiểm thử trong ma trận:

Nhóm 1: Thao Tác Thêm Sản Phẩm & Quy Tắc Dòng Giỏ Hàng (Product Addition & Line Item Rules)

Test Case TC_FR07_01

- **1. Tracking Information:**

  - **TC ID:** `TC_FR07_01` | **Priority:** High (P1) | **Target:** Web Frontend (`/cart`) & Backend API (`/api/cart`)
  - **Traceability:** SRS FR-07 (Item 1), API Spec §4.1, §4.2 | **Coverage IDs:** `EP-07-01` , `STATE-07-01` , `BVA-07-01-ON` , `BVA-07-02-ON`
- **2. Test Objective:** Verify adding a new valid product to empty cart on Web & API with exact 5 columns rendering.
- **3. Pre-conditions & Environment:**
  - Hệ thống EShop đang chạy (Backend `:3000` , Web `:5173`).
  - Giỏ hàng của người dùng đang trống hoàn toàn (`cart.length === 0`).
  - CSDL có sẵn sản phẩm ID 1 (`iPhone 15 Pro Max` , đơn giá `30.000.000 ₫`).
- **4. Test Data:** `id = 1` , `quantity = 1`
- **5. Step-by-Step Procedure:**
  1. Mở trình duyệt, truy cập trang chi tiết sản phẩm: `http://localhost:5173/products/1` .
  2. Quan sát ô nhập số lượng (mặc định là 1).
  3. Nhấn nút "Thêm vào giỏ hàng".
  4. Điều hướng vào trang giỏ hàng `http://localhost:5173/cart` .
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Table Structure:* Trang `/cart` hiển thị bảng danh sách sản phẩm với đầy đủ **đúng 5 cột**:
    1. **Sản phẩm:** Tên sản phẩm "iPhone 15 Pro Max" kèm ảnh thu nhỏ (thumbnail).
    2. **Đơn giá:** Hiển thị `30.000.000 ₫` .
    3. **Số lượng:** Hiển thị số `1` kèm hai nút tương tác `+` và `-` để điều chỉnh.
    4. **Thành tiền:** Hiển thị `30.000.000 ₫` ($30tr \times 1$).
    5. **Thao tác:** Hiển thị nút "Xóa" (hoặc icon thùng rác).
  - *UI / Summary:* Footer hiển thị nhãn "Tổng cộng" kèm số tiền `30.000.000 ₫` . Nút "Tiến hành thanh toán" và "Tiếp tục mua sắm" sẵn sàng.
  - *Context State:* Mảng giỏ hàng chứa đúng 1 phần tử `{ id: 1, name: "iPhone 15 Pro Max", price: 30000000, quantity: 1 }` .
- **7. Teardown & Clean-up:** Xóa sản phẩm khỏi giỏ hàng hoặc refresh state để đưa giỏ về rỗng.

Test Case `TC_FR07_02`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR07_02` | **Priority:** High (P1) | **Target:** Web Frontend & Context State
  - **Traceability:** SRS FR-07 (Item 2: *"Thêm cùng một sản phẩm vào giỏ sẽ tăng số lượng, không tạo dòng mới"*)
  - **Coverage IDs:** `RULE-07-01` , `EP-07-04` , `BVA-07-01-IN` ($Q=2$)
- **2. Test Objective:** Verify adding duplicate product increments quantity without creating new row.
- **3. Pre-conditions & Environment:**
  - Giỏ hàng đã có sẵn sản phẩm ID 1 (`iPhone 15 Pro Max`) với số lượng $Q = 1$.
- **4. Test Data:** `id = 1` , `quantity = 1`
- **5. Step-by-Step Procedure:**
  1. Từ trang chủ hoặc danh mục, truy cập lại trang chi tiết `http://localhost:5173/products/1` .
  2. Giữ nguyên số lượng là 1, nhấn nút "Thêm vào giỏ hàng" lần thứ hai.
  3. Mở trang giỏ hàng `http://localhost:5173/cart` .
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI Table:* Bảng giỏ hàng vẫn duy trì **chính xác 1 dòng sản phẩm duy nhất** (không xuất hiện dòng thứ 2 cho cùng iPhone 15 Pro Max).
  - *Quantity & Price:*
    - Cột Số lượng hiển thị giá trị tăng lên: $Q = 1 + 1 = 2$.
    - Cột Thành tiền tự động nhân đôi: $30.000.000 \times 2 = 60.000.000\text{ ₫}$.
    - Tổng tiền giỏ hàng hiển thị `60.000.000 ₫` .
  - *SUT Verification Alert:* Nếu giỏ hàng tạo ra 2 dòng riêng biệt cho cùng sản phẩm ID 1, đây là lỗi vi phạm trực tiếp SRS FR-07 (Defect trong `CartContext.jsx`).
- **7. Teardown & Clean-up:** Xóa toàn bộ sản phẩm để dọn dẹp state.

Test Case `TC_FR07_03`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR07_03` | **Priority:** Medium (P2) | **Target:** Web Client & Backend API
  - **Traceability:** SRS FR-07 (Item 1), API Spec §4.2 | **Coverage IDs:** `EP-07-02`
  - **Single-Fault Isolation:** `quantity = 1` (hợp lệ); biến sai duy nhất là `product_id = 9999` không tồn tại trong CSDL.
- **2. Test Objective:** Validate rejection when adding well-formed non-existent product ID.
- **3. Pre-conditions & Environment:**
  - CSDL SQLite không chứa sản phẩm có `id = 9999` .
- **4. Test Data:** `id = 9999` , `quantity = 1`
- **5. Step-by-Step Procedure:**
  1. Mở trình duyệt truy cập route sản phẩm giả định: `http://localhost:5173/products/9999` .
  2. Hoặc gửi HTTP `POST /api/cart` với body JSON: `{"id": 9999, "price": 100000, "quantity": 1}` kèm Authorization header.
  3. Quan sát phản hồi trên giao diện và trong Network tab.
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI:* Giao diện hiển thị thông báo "Sản phẩm không tồn tại", nút "Thêm vào giỏ hàng" không hoạt động hoặc bị ẩn.
  - *API:* Endpoint trả về HTTP `404 Not Found` hoặc `400 Bad Request` .
  - *Cart State:* Giỏ hàng không bị thêm bản ghi rác mang ID 9999.
- **7. Teardown & Clean-up:** Không cần dọn dẹp nếu không có dữ liệu chèn vào.

Test Case `TC_FR07_04`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR07_04` | **Priority:** Medium (P2) | **Target:** Backend API (`POST /api/cart`)
  - **Traceability:** API Spec §4.2 | **Coverage IDs:** `EP-07-03`

    - **Single-Fault Isolation:** `quantity = 1` (hợp lệ); biến sai duy nhất là `id = "abc"` sai kiểu dữ liệu.
- **2. Test Objective:** Validate rejection when submitting malformed product ID via API.
- **3. Pre-conditions & Environment:**
    - Backend EShop đang chạy tại cổng 3000. Có JWT token của người dùng hợp lệ.
- **4. Test Data:** `id = "abc"` (hoặc `-1`), `price = 100000`, `quantity = 1`
- **5. Step-by-Step Procedure:**
    1. Gửi request `POST http://localhost:3000/api/cart` kèm Header `Authorization: Bearer <token>`.
    2. Body JSON truyền: `{"id": "abc", "price": 100000, "quantity": 1}`.
- **6. Multi-tier Validation Points (Expected Results):**
    - *API:* Endpoint trả về mã trạng thái `400 Bad Request` với thông báo lỗi rõ ràng về định dạng ID.
    - *Backend State:* Mảng `userCarts` không lưu trữ đối tượng chứa ID sai kiểu dữ liệu.
- **7. Teardown & Clean-up:** Reset giỏ hàng qua `GET /api/cart` nếu cần.

Nhóm 2: Kiểm Thử Miền Dữ Liệu & Giá Trị Biên Của Số Lượng (Quantity EP & BVA Negative Testing)

Test Case `TC_FR07_05`

- **1. Tracking Information:**
    - **TC ID:** `TC_FR07_05` | **Priority:** High (P1) | **Target:** Web Frontend (`Cart.jsx`)
    - **Traceability:** SRS FR-06, FR-07 (Tối thiểu là 1) | **Coverage IDs:** `EP-07-05`, `BVA-07-01-OFF` ($Q=0$)
    - **Single-Fault Isolation:** Sản phẩm ID 1 hợp lệ; biến sai duy nhất là số lượng $Q = 0$.
- **2. Test Objective:** Validate rejection of quantity zero input on Cart item (Boundary Off-point).
- **3. Pre-conditions & Environment:**
    - Giỏ hàng đang có 1 sản phẩm ID 1 với $Q = 1$.
- **4. Test Data:** `quantity = 0`
- **5. Step-by-Step Procedure:**
    1. Truy cập trang giỏ hàng `http://localhost:5173/cart`.
    2. Nhấp vào ô số lượng của sản phẩm ID 1.
    3. Xóa số `1` và gõ `0`.
    4. Nhấn Enter hoặc nhấp chuột ra ngoài ô nhập (blur).
- **6. Multi-tier Validation Points (Expected Results):**
    - *UI:* Hệ thống từ chối lưu giá trị 0; tự động phục hồi về 1 HOẶC hiển thị dialog hỏi "Bạn có muốn xóa sản phẩm này?".
    - *Cart State:* Tuyệt đối không lưu dòng sản phẩm có $Q = 0$ và thành tiền = 0 ₫ trong bảng.
- **7. Teardown & Clean-up:** Khôi phục giỏ hàng rỗng.

Test Case `TC_FR07_06`

- **1. Tracking Information:**
    - **TC ID:** `TC_FR07_06` | **Priority:** High (P1) | **Target:** Web Frontend & Backend API
    - **Traceability:** SRS FR-06, FR-07 (*"chỉ nhận số nguyên dương, tối thiểu là 1"*) | **Coverage IDs:** `EP-07-06`
    - **Single-Fault Isolation:** Sản phẩm ID 1 hợp lệ; biến sai duy nhất là số lượng âm $Q = -1$.
- **2. Test Objective:** Validate rejection of negative integer quantity input (Single Fault).
- **3. Pre-conditions & Environment:**
    - Giỏ hàng có 1 sản phẩm ID 1.
- **4. Test Data:** `quantity = -1` (hoặc `-5`)
- **5. Step-by-Step Procedure:**
    1. Tại ô số lượng của sản phẩm, gõ `-1`.
    2. Hoặc gửi API `POST /api/cart` với `quantity: -1`.
- **6. Multi-tier Validation Points (Expected Results):**
    - *UI:* Input chặn ký tự dấu trừ `-` hoặc tự động reset về 1; hiển thị cảnh báo số lượng phải ≥ 1.
    - *Arithmetic Check:* Thành tiền và Tổng tiền giỏ hàng tuyệt đối không bị tính ra số âm.
- **7. Teardown & Clean-up:** Dọn dẹp giỏ hàng.

Test Case `TC_FR07_07`

- **1. Tracking Information:**
    - **TC ID:** `TC_FR07_07` | **Priority:** Medium (P2) | **Target:** Web Frontend (`Cart.jsx`)
    - **Traceability:** SRS FR-07 (Trường số lượng bắt buộc) | **Coverage IDs:** `EP-07-07`, `EP-07-08`
    - **Single-Fault Isolation:** Sản phẩm hợp lệ; ô số lượng bị xóa trắng thành chuỗi rỗng `""` hoặc chỉ có khoảng trắng `" "`.
- **2. Test Objective:** Validate rejection of empty or whitespace quantity input.
- **3. Pre-conditions & Environment:**
    - Giỏ hàng có 1 sản phẩm ID 1 ($Q = 1$).
- **4. Test Data:** `quantity = ""` (chuỗi rỗng) hoặc `" "` (khoảng trắng)
- **5. Step-by-Step Procedure:**
    1. Mở `/cart`.
    2. Bôi đen toàn bộ số trong ô số lượng và nhấn Backspace để xóa sạch.
    3. Nhấp chuột ra ngoài ô nhập (onBlur).
- **6. Multi-tier Validation Points (Expected Results):**
    - *UI:* Ô số lượng tự động khôi phục về giá trị hợp lệ gần nhất ($Q = 1$).
    - *Calculation Safety:* Thành tiền và Tổng tiền không bị biến thành `NaN ₫` hay `0 ₫`.
- **7. Teardown & Clean-up:** Dọn dẹp giỏ hàng.

Test Case `TC_FR07_08`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR07_08` | **Priority:** Medium (P2) | **Target:** Web Frontend Input Control
  - **Traceability:** SRS FR-06 (*"chỉ nhận số nguyên dương"*) | **Coverage IDs:** `EP-07-09`
  - **Single-Fault Isolation:** Sản phẩm hợp lệ; nhập ký tự chữ/ký hiệu đặc biệt vào ô số lượng.
- **2. Test Objective:** Validate rejection of non-numeric string characters in quantity field.
- **3. Pre-conditions & Environment:**
  - Giỏ hàng có 1 sản phẩm ID 1.
- **4. Test Data:** `quantity = "abc"` hoặc `"@#$"`
- **5. Step-by-Step Procedure:**
  1. Nhấp chuột vào ô số lượng của sản phẩm.
  2. Cố tình gõ chuỗi chữ `abc` hoặc paste ký tự đặc biệt `@#$`.
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI:* Ô nhập liệu không hiển thị các ký tự chữ cái (do có `type="number"` hoặc input filter).
  - *Safety:* Giá trị số lượng trong state vẫn là số nguyên hợp lệ; không gây crash ứng dụng.
- **7. Teardown & Clean-up:** Dọn dẹp giỏ hàng.

Test Case `TC_FR07_09`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR07_09` | **Priority:** High (P1) | **Target:** Web Frontend & Validation Rule
  - **Traceability:** SRS FR-06, FR-07 (*"chỉ nhận số nguyên dương"*) | **Coverage IDs:** `EP-07-10`, `BVA-07-04-OFF` ($Q=1.5$)
  - **Single-Fault Isolation:** Sản phẩm hợp lệ; số lượng là số thập phân $Q = 1.5$.
- **2. Test Objective:** Validate rejection of decimal quantity near lower boundary (Boundary Off-point).
- **3. Pre-conditions & Environment:**
  - Giỏ hàng có 1 sản phẩm ID 1.
- **4. Test Data:** `quantity = 1.5` (hoặc `0.9`, `1.1`)
- **5. Step-by-Step Procedure:**
  1. Nhập `1.5` (hoặc `1,5`) vào ô số lượng.
  2. Nhấn Enter hoặc chuyển tab.
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI:* Input chặn ký tự dấu chấm/phẩy hoặc hiển thị thông báo lỗi "Số lượng phải là số nguyên".
  - *Safety:* Thành tiền không xuất hiện phần lẻ thập phân gây sai lệch tiền tệ VND (ví dụ không cho phép $30tr \times 1.5 = 45tr$ khi bán mặt hàng nguyên chiếc).
- **7. Teardown & Clean-up:** Dọn dẹp giỏ hàng.

Test Case `TC_FR07_10`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR07_10` | **Priority:** Medium (P2) | **Target:** Backend REST API (`POST /api/cart`)
  - **Traceability:** API Spec §4.2 | **Coverage IDs:** `EP-07-11`
  - **Single-Fault Isolation:** `id` và `price` hợp lệ; trường `quantity` bị null hoặc khuyết hoàn toàn trong payload.
- **2. Test Objective:** Validate API rejection when quantity is omitted or null in payload.
- **3. Pre-conditions & Environment:**
  - Backend `:3000` đang hoạt động, có token xác thực.
- **4. Test Data:** Payload JSON: `{"id": 1, "price": 30000000, "quantity": null}` hoặc `{"id": 1, "price": 30000000}`
- **5. Step-by-Step Procedure:**
  1. Dùng Postman hoặc cURL gửi `POST /api/cart` với payload trên.
- **6. Multi-tier Validation Points (Expected Results):**
  - *API:* HTTP Status `400 Bad Request`. Body JSON thông báo trường `quantity` là bắt buộc và phải là số nguyên dương.
  - *State:* Không đẩy bản ghi không có số lượng vào giỏ hàng.
- **7. Teardown & Clean-up:** Không có dữ liệu rác cần dọn.

Test Case `TC_FR07_11`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR07_11` | **Priority:** Low (P3) | **Target:** System Robustness & JS Number Range
  - **Traceability:** Tiêu chuẩn ổn định số học (Robustness) | **Coverage IDs:** `EP-07-12`, `BVA-07-05`
  - **Single-Fault Isolation:** Số lượng cực lớn $Q = 10^9$ (1 tỷ đơn vị).
- **2. Test Objective:** Validate system robustness against extremely large integer quantity.
- **3. Pre-conditions & Environment:**
  - Giỏ hàng có 1 sản phẩm ID 1.
- **4. Test Data:** `quantity = 1000000000` (10^9)
- **5. Step-by-Step Procedure:**
  1. Nhập `1000000000` vào ô số lượng của sản phẩm.
  2. Quan sát khả năng phản hồi của giao diện.
- **6. Multi-tier Validation Points (Expected Results):**
  - *System:* Ứng dụng không bị đứng hình, không tràn bộ nhớ CPU/RAM.
  - *Display:* Thành tiền không hiển thị `Infinity` hoặc `NaN`; hiển thị cảnh báo vượt giới hạn giao dịch nếu có.
- **7. Teardown & Clean-up:** Đưa giỏ hàng về rỗng.

Nhóm 3: Kiểm Chứng Tính Toán Số Học Thành Tiền & Tổng Cộng (Subtotal & Total Calculation Verification)

Test Case `TC_FR07_12`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR07_12` | **Priority:** High (P1) | **Target:** Business Logic & Arithmetic Calculation
  - **Traceability:** SRS FR-07 (Cột Thành tiền & Nhãn Tổng cộng) | **Coverage IDs:** `RULE-07-02`, `RULE-07-03`, `BVA-07-02-IN` ($Items=2$)
- **2. Test Objective:** Verify arithmetic calculation of line subtotals and cart total for multiple items.
- **3. Pre-conditions & Environment:**
  - Giỏ hàng rỗng ban đầu.
  - Có 2 sản phẩm trong CSDL: SP1 (iPhone 15, giá `30.000.000 ₫`) và SP2 (AirPods Pro, giá `6.000.000 ₫`).
- **4. Test Data:** SP1 với $Q_1 = 2$; SP2 với $Q_2 = 1$
- **5. Step-by-Step Procedure:**
  1. Thêm SP1 vào giỏ với số lượng 2.
  2. Thêm SP2 vào giỏ với số lượng 1.
  3. Truy cập vào trang giỏ hàng `http://localhost:5173/cart`.
- **6. Multi-tier Validation Points (Expected Results):**
  - *Thành tiền dòng 1:* $30.000.000 \times 2 = 60.000.000\text{ ₫}$.
  - *Thành tiền dòng 2:* $6.000.000 \times 1 = 6.000.000\text{ ₫}$.
  - *Tổng tiền giỏ hàng:* $60.000.000 + 6.000.000 = 66.000.000\text{ ₫}$ (khớp chính xác công thức $Total = \sum Subtotal$).
- **7. Teardown & Clean-up:** Xóa cả 2 sản phẩm khỏi giỏ.

Nhóm 4: Nút Bấm Điều Chỉnh Số Lượng & Ranh Giới Chuyển Tiếp (Quantity Adjustment & Decrement Transition)

Test Case `TC_FR07_13`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR07_13` | **Priority:** High (P1) | **Target:** Web Frontend Interactive Controls
  - **Traceability:** SRS FR-07 (*"Số lượng (có nút +/- để chỉnh)"*) | **Coverage IDs:** `ACTION-07-01`, `BVA-07-03-IN` ($Q=2 \to 1$)
- **2. Test Objective:** Verify quantity decrement via '-' button when quantity exceeds minimum (Normal Action).
- **3. Pre-conditions & Environment:**
  - Giỏ hàng có sản phẩm ID 1 đang mang số lượng $Q = 2$.
- **4. Test Data:** Thao tác click nút `-` khi $Q = 2$
- **5. Step-by-Step Procedure:**
  1. Vào `/cart`.
  2. Tìm nút `-` cạnh ô số lượng của sản phẩm ID 1.
  3. Nhấn nút `-` một lần duy nhất.
- **6. Multi-tier Validation Points (Expected Results):**
  - *Số lượng:* Giảm từ 2 xuống đúng `1`.
  - *Thành tiền:* Tự động giảm từ 60tr xuống `30.000.000 ₫`.
  - *Tổng tiền:* Tự động giảm từ 60tr xuống `30.000.000 ₫`.
- **7. Teardown & Clean-up:** Dọn dẹp giỏ hàng.

Test Case `TC_FR07_14`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR07_14` | **Priority:** Medium (P2) | **Target:** Web Frontend Interactive Controls
  - **Traceability:** SRS FR-07 (Item 1, 3: Nút +/- và Dialog xác nhận xóa) | **Coverage IDs:** `ACTION-07-02`, `BVA-07-03-ON` ($Q=1$)
- **2. Test Objective:** Validate boundary decrement behavior via '-' button at minimum quantity Q=1 (State Transition Probe).
- **3. Pre-conditions & Environment:**
  - Giỏ hàng có 1 sản phẩm ID 1 với số lượng đúng bằng $Q = 1$.
- **4. Test Data:** Click nút `-` khi $Q = 1$
- **5. Step-by-Step Procedure:**
  1. Vào `/cart`.
  2. Nhấn nút `-` tại dòng sản phẩm đang có số lượng 1.
- **6. Multi-tier Validation Points (Expected Results):**
  - *Chuẩn hành vi chấp nhận được:*
    - **Phương án A (Disable):** Nút `-` bị vô hiệu hóa (`disabled`), không cho phép giảm dưới 1.
    - **Phương án B (Confirm Delete):** Nút `-` kích hoạt dialog xác nhận xóa sản phẩm tương tự nút Xóa.
  - *Hành vi vi phạm nghiêm trọng (Defect):*
    - Giảm số lượng về $Q = 0$ hoặc số âm (-1) làm thành tiền âm.
    - Tự động xóa sản phẩm ngay lập tức mà không có bất kỳ dialog xác nhận nào.
- **7. Teardown & Clean-up:** Dọn dẹp giỏ hàng.

Nhóm 5: Thao Tác Xóa Sản Phẩm, Hộp Thoại Xác Nhận & Giỏ Hàng Trống (Delete Flow, Confirmation Dialog & Empty Cart State)

Test Case `TC_FR07_15`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR07_15` | **Priority:** High (P1) | **Target:** Web Frontend Modal & State
  - **Traceability:** SRS FR-07 (Item 3: *"Nút Xóa sản phẩm phải có dialog xác nhận trước khi thực hiện"*) | **Coverage IDs:** `ACTION-07-03`
- **2. Test Objective:** Verify item removal upon confirming delete dialog (Interaction Flow).
- **3. Pre-conditions & Environment:**

    - Giỏ hàng có 2 sản phẩm (SP1 giá 30tr, SP2 giá 6tr; Tổng: 36tr).
- **4. Test Data:** Thao tác click "Xóa" tại SP1 $\to$ Click "OK / Confirm"
- **5. Step-by-Step Procedure:**
    1. Mở `/cart`.
    2. Tại dòng SP1, nhấn nút "Xóa".
    3. Khi hộp thoại xác nhận hiện lên, nhấn nút "OK" (hoặc "Xóa / Confirm").
- **6. Multi-tier Validation Points (Expected Results):**
    - *Dialog:* Hộp thoại xác nhận hiển thị rõ ràng trước khi xóa.
    - *Table:* Dòng SP1 biến mất khỏi bảng giỏ hàng; SP2 vẫn giữ nguyên.
    - *Tổng tiền:* Cập nhật tức thời còn `6.000.000 ₫`.
- **7. Teardown & Clean-up:** Xóa nốt SP2 để đưa giỏ về rỗng.

Test Case `TC_FR07_16`

- **1. Tracking Information:**
    - **TC ID:** `TC_FR07_16` | **Priority:** High (P1) | **Target:** Web Frontend Modal Interaction
    - **Traceability:** SRS FR-07 (Item 3) | **Coverage IDs:** `ACTION-07-04`
- **2. Test Objective:** Verify item retention upon cancelling delete dialog (Interaction Flow).
- **3. Pre-conditions & Environment:**
    - Giỏ hàng có 1 sản phẩm SP1 ($Q = 1$).
- **4. Test Data:** Click "Xóa" $\to$ Click "Cancel / Hủy"
- **5. Step-by-Step Procedure:**
    1. Mở `/cart`.
    2. Nhấn nút "Xóa" tại dòng SP1.
    3. Khi hộp thoại xác nhận hiện lên, nhấn nút "Cancel" (hoặc "Hủy").
- **6. Multi-tier Validation Points (Expected Results):**
    - *Dialog:* Hộp thoại đóng lại.
    - *Table:* Dòng sản phẩm SP1 vẫn hiển thị nguyên vẹn, số lượng vẫn là 1, tổng tiền không đổi.
- **7. Teardown & Clean-up:** Dọn dẹp giỏ hàng.

Test Case `TC_FR07_17`

- **1. Tracking Information:**
    - **TC ID:** `TC_FR07_17` | **Priority:** High (P1) | **Target:** Web Frontend State Transition (`Cart.jsx`)
    - **Traceability:** SRS FR-07 (Item 6: *"Giỏ hàng trống phải có hình minh họa và thông báo rõ ràng"*) | **Coverage IDs:** `STATE-07-02`, `BVA-07-02-OFF` ($Items=0$), `ASSERT-07-03`
- **2. Test Objective:** Verify empty cart state display upon deleting last remaining item with both illustration and message.
- **3. Pre-conditions & Environment:**
    - Giỏ hàng còn đúng 1 sản phẩm duy nhất ($Items = 1$).
- **4. Test Data:** Xóa sản phẩm cuối cùng $\to Items = 0$
- **5. Step-by-Step Procedure:**
    1. Mở `/cart`.
    2. Nhấn "Xóa" sản phẩm duy nhất và xác nhận xóa.
    3. Quan sát giao diện giỏ hàng sau khi xóa.
- **6. Multi-tier Validation Points (Expected Results):**
    - *Bảng danh mục:* Ẩn bảng sản phẩm và tổng tiền.
    - *Thông báo văn bản:* Hiển thị câu thông báo rõ ràng (ví dụ: *"Giỏ hàng của bạn đang trống"*).
    - *Hình minh họa:* **Bắt buộc phải có thẻ hình ảnh minh họa** (thẻ `<img>` hoặc SVG minh họa giỏ hàng rỗng). Nếu chỉ có dòng chữ đơn điệu $\to$ Defect vi phạm SRS FR-07.
    - *Liên kết:* Có nút/link "Tiếp tục mua sắm" dẫn về trang chủ.
- **7. Teardown & Clean-up:** Không cần dọn dẹp.

Nhóm 6: Điểm Kiểm Chứng Giao Diện & Điều Hướng Hệ Thống (UI Assertions, Safety Probes & Navigation)

Test Case `TC_FR07_18`

- **1. Tracking Information:**
    - **TC ID:** `TC_FR07_18` | **Priority:** Medium (P2) | **Target:** Web Frontend UI Text (`Cart.jsx`)
    - **Traceability:** SRS FR-07 (Item 5: *"Tổng tiền hiển thị nhãn chính xác: 'Tổng cộng' (không phải 'Tổng tạm tính')"*) | **Coverage IDs:** `ASSERT-07-01`
- **2. Test Objective:** Validate total amount UI label matches required "Tổng cộng" text (UI Assertion).
- **3. Pre-conditions & Environment:**
    - Giỏ hàng có ít nhất 1 sản phẩm hợp lệ.
- **4. Test Data:** Footer giỏ hàng
- **5. Step-by-Step Procedure:**
    1. Thêm 1 sản phẩm vào giỏ và mở `/cart`.
    2. Quan sát chuỗi ký tự hiển thị cạnh số tiền tổng ở cuối bảng.
- **6. Multi-tier Validation Points (Expected Results):**
    - *Nhãn văn bản:* Bắt buộc phải là **"Tổng cộng"** (hoặc `"Tổng cộng:"`).
    - *Defect Detection:* Nếu giao diện hiển thị nhãn là **"Tổng tạm tính"** hoặc **"Subtotal"** $\to$ Ghi nhận Defect vi phạm trực tiếp yêu cầu SRS FR-07.
- **7. Teardown & Clean-up:** Dọn dẹp giỏ hàng.

Test Case `TC_FR07_19`

- **1. Tracking Information:**

  - **TC ID:** `TC_FR07_19` | **Priority:** High (P1) | **Target:** Web Frontend Deletion Safety
  - **Traceability:** SRS FR-07 (Item 3: *"Nút Xóa sản phẩm phải có dialog xác nhận trước khi thực hiện"*) | **Coverage IDs:** `ASSERT-07-02`
- **2. Test Objective:** Validate enforcement of confirmation dialog before product deletion (Safety Probe).
- **3. Pre-conditions & Environment:**
  - Giỏ hàng có sản phẩm.
- **4. Test Data:** Thao tác click nút "Xóa"
- **5. Step-by-Step Procedure:**
  1. Vào `/cart`.
  2. Nhấn nút "Xóa" tại dòng sản phẩm và quan sát màn hình.
- **6. Multi-tier Validation Points (Expected Results):**
  - *Hộp thoại xác nhận:* Bắt buộc phải xuất hiện dialog xác nhận trước khi xóa.
  - *Defect Detection:* Nếu sản phẩm bị xóa lập tức mà không có bất kỳ dialog nào hỏi ý kiến người dùng $\to$ Ghi nhận Defect vi phạm an toàn thao tác SRS FR-07.
- **7. Teardown & Clean-up:** Dọn dẹp giỏ hàng.

Test Case `TC_FR07_20`

- **1. Tracking Information:**
  - **TC ID:** `TC_FR07_20` | **Priority:** Medium (P2) | **Target:** Web Frontend Routing & Navigation
  - **Traceability:** SRS FR-07 (Item 4: *"Có nút Tiếp tục mua sắm để quay về trang chủ"*) | **Coverage IDs:** `ACTION-07-05`
- **2. Test Objective:** Verify navigation back to home page via "Tiếp tục mua sắm" button (Navigation Flow).
- **3. Pre-conditions & Environment:**
  - Người dùng đang đứng tại trang giỏ hàng `http://localhost:5173/cart`.
  - Giỏ hàng có 1 sản phẩm ID 1.
- **4. Test Data:** Click nút "Tiếp tục mua sắm" (hoặc "← Mua tiếp")
- **5. Step-by-Step Procedure:**
  1. Tại trang giỏ hàng `/cart`, tìm nút hoặc liên kết "Tiếp tục mua sắm".
  2. Nhấp chuột vào nút/liên kết này.
- **6. Multi-tier Validation Points (Expected Results):**
  - *Routing:* URL trình duyệt chuyển hướng an toàn về trang chủ `http://localhost:5173/`.
  - *State Preservation:* Dữ liệu sản phẩm trong giỏ hàng vẫn được lưu giữ nguyên vẹn, không bị mất khi quay lại `/cart`.
- **7. Teardown & Clean-up:** Dọn dẹp giỏ hàng.

3.5 Bước 5: Ma trận truy vết & Kiểm tra độ bao phủ (Step 5 Deliverables)

> **Mục tiêu Kiểm chứng Độ bao phủ (ISTQB FL §4.2.1 & §4.2.2):** Xác nhận toàn bộ 100% các phân vùng biến đầu vào (VEP/IEP), các quy tắc nghiệp vụ/chuyển trạng thái, các điểm kiểm chứng khiếm khuyết giao diện, và các điểm biên On/Off/In đều được kiểm thử trực tiếp bởi ít nhất một kịch bản trong ma trận 20 Test Cases.

3.5.1 Bảng Độ Bao Phủ Phân Vùng Tương Đương Biến Đầu Vào (Pure Input EP Coverage)

Class ID	Variable / Input	Type	Mô tả phân vùng tương đương	Covered by TC ID(s)	Trạng thái (Status)
`EP-07-01`	`product_id`	**VEP**	Mã sản phẩm hợp lệ và tồn tại trong CSDL	`TC_FR07_01`	**Covered**
`EP-07-02`	`product_id`	**IEP**	Mã sản phẩm đúng định dạng số nhưng không tồn tại trong CSDL (`id = 9999`)	`TC_FR07_03`	**Covered**
`EP-07-03`	`product_id`	**IEP**	Mã sản phẩm sai định dạng / sai kiểu dữ liệu (`id = "abc"` hoặc `-1`)	`TC_FR07_04`	**Covered**
`EP-07-04`	`quantity`	**VEP**	Số lượng nguyên dương hợp lệ ($Q \in \mathbb{Z}, Q \ge 1$)	`TC_FR07_01` , `TC_FR07_02` , `TC_FR07_12`	**Covered**
`EP-07-05`	`quantity`	**IEP**	Số lượng bằng 0 ($Q = 0$)	`TC_FR07_05`	**Covered**
`EP-07-06`	`quantity`	**IEP**	Số lượng là số nguyên âm ($Q < 0$)	`TC_FR07_06`	**Covered**
`EP-07-07`	`quantity`	**IEP**	Chuỗi rỗng / để trống trường số lượng (`""`)	`TC_FR07_07`	**Covered**
`EP-07-08`	`quantity`	**IEP**	Chuỗi chỉ chứa toàn khoảng trắng (`" "`)	`TC_FR07_07`	**Covered**
`EP-07-09`	`quantity`	**IEP**	Ký tự phi số (Chữ cái, ký hiệu đặc biệt)	`TC_FR07_08`	**Covered**
`EP-07-10`	`quantity`	**IEP**	Số thập phân / số thực không nguyên ($Q = 1.5$)	`TC_FR07_09`	**Covered**
`EP-07-11`	`quantity`	**IEP**	Bị bỏ khuyết hoặc giá trị `null` trong payload API	`TC_FR07_10`	**Covered**
`EP-07-12`	`quantity`	**IEP**	Số nguyên cực lớn kiểm thử độ bền số học ($Q = 10^9$)	`TC_FR07_11`	**Covered**

3.5.2 Bảng Độ Bao Phủ Quy Tắc Nghiệp Vụ, Trạng Thái & Tương Tác (State, Rule & Interaction Coverage)

Rule / State ID	Hạng mục	Mô tả quy tắc & kịch bản tương tác	Covered by TC ID(s)	Trạng thái (Status)
`RULE-07-01`	Trùng sản phẩm	Thêm SP trùng lặp phải cộng dồn số lượng, không tạo dòng mới	`TC_FR07_02`	**Covered**
`RULE-07-02`	Thành tiền dòng	Công thức số học $Subtotal = unit_price \times quantity$	`TC_FR07_01` , `TC_FR07_02` , `TC_FR07_12`	**Covered**
`RULE-07-03`	Tổng tiền giỏ	Công thức tổng tiền toàn giỏ $Total = \sum Subtotal$	`TC_FR07_01` , `TC_FR07_12`	**Covered**
`STATE-07-01`	Trạng thái giỏ	Giỏ hàng có sản phẩm ($Items \ge 1$), hiển thị đủ 5 cột	`TC_FR07_01` , `TC_FR07_12`	**Covered**

Rule / State ID	Hạng mục	Mô tả quy tắc & kịch bản tương tác	Covered by TC ID(s)	Trạng thái (Status)
STATE-07-02	Trạng thái giỏ	Giỏ hàng trống ($Items = 0$), hiển thị cả thông báo và hình minh họa	TC_FR07_17	**Covered**
ACTION-07-01	Nút -	Bấm nút - khi $Q > 1 \to$ giảm số lượng 1 đơn vị, tính lại tiền	TC_FR07_13	**Covered**
ACTION-07-02	Nút -	Bấm nút - tại ranh giới $Q = 1 \to$ disable hoặc dialog xác nhận xóa	TC_FR07_14	**Covered**
ACTION-07-03	Nút "Xóa"	Bấm Xóa $\to$ Xác nhận OK $\to$ Xóa sản phẩm khỏi giỏ, tính lại tiền	TC_FR07_15	**Covered**
ACTION-07-04	Nút "Xóa"	Bấm Xóa $\to$ Xác nhận Cancel $\to$ Giữ nguyên vẹn sản phẩm	TC_FR07_16	**Covered**
ACTION-07-05	Điều hướng	Bấm nút "Tiếp tục mua sắm" $\to$ Chuyển về trang chủ (/), bảo toàn giỏ	TC_FR07_20	**Covered**

3.5.3 Bảng Độ Bao Phủ Điểm Kiểm Chứng Giao Diện & Giả Thuyết Khiếm Khuyết (UI Assertions Coverage)

Assertion ID	Hạng mục kiểm chứng	Yêu cầu bắt buộc của SRS FR-07	Covered by TC ID(s)	Trạng thái (Status)
ASSERT-07-01	Nhãn Tổng tiền	Tổng tiền hiển thị nhãn chính xác: **"Tổng cộng"** (không phải "Tổng tạm tính")	TC_FR07_18	**Covered**
ASSERT-07-02	Dialog xác nhận xóa	Nút Xóa sản phẩm phải có **dialog xác nhận** trước khi thực hiện	TC_FR07_19	**Covered**
ASSERT-07-03	Hình minh họa giỏ trống	Giỏ hàng trống phải có **hình minh họa** và thông báo rõ ràng	TC_FR07_17	**Covered**

3.5.4 Bảng Độ Bao Phủ Phân Tích Giá Trị Biên (Boundary Value Coverage - On/Off/In Points)

Boundary ID	Biến / Điều kiện	Điểm biên (Point Type)	Giá trị kiểm thử	Covered by TC ID(s)	Trạng thái (Status)
BVA-07-01	Biên dưới số lượng ($Q \ge 1$)	**On-point**	$Q = 1$ (Số lượng min hợp lệ)	TC_FR07_01, TC_FR07_13, TC_FR07_14	**Covered**
		Off-point	$Q = 0$ (Không hợp lệ, vi phạm $Q \ge 1$)	TC_FR07_05	**Covered**
		In-point	$Q = 2$ (Liền kề trong vùng hợp lệ)	TC_FR07_02, TC_FR07_12	**Covered**
BVA-07-02	Ranh giới số lượng SP trong giỏ ($Items \ge 1 \leftrightarrow 0$)	**On-point**	$Items = 1$ (Còn đúng 1 SP, giỏ có hàng)	TC_FR07_01, TC_FR07_16	**Covered**
		Off-point	$Items = 0$ (Giỏ rỗng, chuyển giao diện)	TC_FR07_17	**Covered**
		In-point	$Items = 2$ (Nhiều SP, cộng dồn tổng tiền)	TC_FR07_12, TC_FR07_15	**Covered**
BVA-07-03	Ranh giới chuyển tiếp nút - ($Q = 1 \to 0$)	**On-point**	Bấm nút - tại $Q = 1$ (Điểm chuyển tiếp)	TC_FR07_14	**Covered**
		Off-point	Giảm âm $Q \le 0$ (Hành vi sai, vi phạm SRS)	TC_FR07_14	**Covered**
		In-point	Bấm nút - tại $Q = 2 \to 1$ (Giảm hợp lệ)	TC_FR07_13	**Covered**
BVA-07-04	Ranh giới số nguyên vs số thực kề cận ($Q \in \mathbb{Z}^+$)	**On-point**	$Q = 1.0$ (Số nguyên chuẩn xác)	TC_FR07_01	**Covered**
		Off-point	$Q = 1.5$ (hoặc $0.9/1.1$, số thập phân bị từ chối)	TC_FR07_09	**Covered**
		In-point	$Q = 2.0$ (Số nguyên hợp lệ tiếp theo)	TC_FR07_02	**Covered**
BVA-07-05	Giới hạn an toàn số nguyên JS (MAX_SAFE_INTEGER)	**On-point**	$Q = 9.007.199.254.740.991$	TC_FR07_11	**Covered**
		Off-point	$Q = 9.007.199.254.740.992$ (Vượt ngưỡng)	TC_FR07_11	**Covered**
		In-point	$Q = 10^9$ (Số cực lớn trong ngưỡng)	TC_FR07_11	**Covered**

3.5.5 Bảng Tổng Hợp Chỉ Số Độ Bao Phủ (Coverage Summary)

Chỉ số kiểm thử (Metric)	Tổng số nhận diện	Đã bao phủ (Covered)	Chưa bao phủ / Bị chặn	Tỷ lệ bao phủ (%)
Phân vùng hợp lệ (VEP)	2	2	0	**100.0%**
Phân vùng không hợp lệ (IEP)	10	10	0	**100.0%**
Tổng số phân vùng tương đương biến (Input EPs)	12	12	0	**100.0%**
Quy tắc nghiệp vụ & Trạng thái (Rules & States)	5	5	0	**100.0%**
Kịch bản tương tác & Điều hướng (Actions & Navigation)	5	5	0	**100.0%**
Điểm kiểm chứng giao diện & Khiếm khuyết (Assertions)	3	3	0	**100.0%**
Giá trị biên (On / Off / In Points)	15	15	0	**100.0%**
Tổng số kịch bản kiểm thử (Test Cases)	20	20	0	**100.0%**
Bảo đảm Giả định Đơn lỗi (Single-Fault Isolation)	10 / 10 IEPs	10	0	**100.0%**

3.6 Bước 6: AI Gap Analysis & Giả thuyết lỗi SUT (Step 6 Deliverables)

> **Mục đích:** Cung cấp minh chứng thực tế cho **Mục 6 Task 3 (AI Gap Analysis)** và làm cơ sở biện luận cho **Mục 10 (AI Critique)** của báo cáo HW02. Phân tách rạch ròi giữa các thiếu sót do suy luận AI và các khiếm khuyết nội bộ thực tế trong mã nguồn SUT EShop.

3.6.1 Bảng Phân Tích Khoảng Trống Kiểm Thử Của AI (AI Gap Analysis)

Hạng mục / Vấn đề phát sinh	Nguyên nhân AI bỏ sót hoặc gặp khó khăn	Hiệu chỉnh của con người (Human Fix Applied)	Căn cứ Tiêu chuẩn & Kỹ thuật Kiểm thử
Gộp lẫn các khái niệm kiểm thử vào một bảng EP duy nhất	AI có xu hướng gộp chung mọi khía cạnh (miền đầu vào, công thức tính toán `subtotal` / `total`, hành vi click nút `+/-`, lỗi giao diện "Tổng tạm tính", mất dialog xóa) vào bảng `EP-07-xx`, làm loãng và thổi phồng tỷ lệ bao phủ EP.	Con người yêu cầu bóc tách thành 3 bảng chuyên biệt: (1) Pure Input EPs; (2) State & Interaction Scenarios; (3) UI Assertions & Defect Probes.	**ISTQB FL §4.2.1:** Phân vùng tương đương chỉ áp dụng cho miền dữ liệu đầu vào/đầu ra, không áp dụng cho lỗi hệ thống hay kịch bản tương tác.
Tự ý áp đặt cận trên không có căn cứ ($Q \le 99$)	AI bị ảnh hưởng bởi thói quen thiết kế thương mại điện tử thông thường (thường giới hạn số lượng mua ≤ 99 hoặc theo tồn kho), tự ý bịa đặt điều kiện $1 \le Q \le 99$.	Con người rà soát đặc tả SRS FR-06 & FR-07 (chỉ yêu cầu $Q \ge 1$) và kiểm tra CSDL SQLite (`products` table không có cột `stock_quantity`); yêu cầu sửa thành $Q \in \mathbb{Z}, Q \ge 1$ (cận trên mở).	**Nguyên tắc "Đặc tả là căn cứ duy nhất":** Tuyệt đối không tự ý bịa đặt ràng buộc nghiệp vụ khi SRS không đề cập.
Gộp chung mã sản phẩm không tồn tại với ID sai định dạng	AI ban đầu gộp `id = 9999` (ID số không tồn tại trong DB) và `id = "abc"` (ID sai kiểu dữ liệu) vào cùng một lớp `EP-07-03`.	Tách thành 2 lớp riêng: `EP-07-02` (ID đúng định dạng nhưng không có trong DB $\to$ kiểm tra tầng Data/Service) và `EP-07-03` (ID sai định dạng $\to$ kiểm tra tầng Controller/Routing).	**ISTQB FL §4.2.1:** Phân tách các điều kiện lỗi có cơ chế xử lý và điểm thất bại độc lập để tránh che giấu lỗi.
Bỏ sót các phân vùng khuyết/null ở tầng API payload	AI tập trung vào giao diện Web UI với input số học mà bỏ qua kiểm thử trực tiếp endpoint REST API `POST /api/cart` khi payload bị khuyết trường hoặc truyền `quantity: null`.	Con người bổ sung phân vùng `EP-07-11` (Bỏ khuyết/null trường bắt buộc trong payload JSON API).	**API Contract Testing & Robustness:** Kiểm thử tính toàn vẹn dữ liệu ở cả tầng API Backend và Frontend Client.
Chưa làm rõ tính mơ hồ của nút `-` tại mốc tối thiểu $Q=1$	SRS FR-07 không quy định rõ khi $Q=1$ bấm nút `-` thì nút sẽ bị disable hay sẽ kích hoạt dialog xóa sản phẩm. AI có xu hướng tự chọn một hành vi duy nhất làm chuẩn.	Con người chỉ đạo ghi nhận đây là điểm mơ hồ đặc tả (Ambiguity Point), chấp nhận cả hai phương án an toàn (disable nút hoặc confirm delete) và định vị hành vi sai là giảm âm hoặc tự ý xóa không hỏi.	**ISTQB FL §1.4.3 & §4.2:** Phát hiện và tài liệu hóa các điểm mơ hồ trong đặc tả yêu cầu phần mềm.

3.6.2 Danh Sách Giả Thuyết Khiếm Khuyết Thăm Dò Trên SUT EShop (Candidate Defect Hypotheses)

> *Ghi chú: Toàn bộ các giả thuyết dưới đây được gắn nhãn `[Exploratory - Unconfirmed Hypothesis - Requires SUT execution]` và sẽ được kiểm chứng thực tế trong Giai đoạn 3 trên hệ thống EShop đang chạy.*

Hypothesis ID	Khu vực kiểm thử (Target Area)	Mô tả giả thuyết lỗi (Description)	Kỳ vọng theo đặc tả (Spec Expected)	Dự đoán hành vi thực tế (Hypothesized Actual)	Bước kiểm chứng trên SUT (Verification Step)
`HYP-07-01`	Frontend UI (`Cart.jsx:63`)	Nhãn Tổng tiền bị hardcode hiển thị sai thành "Tổng tạm tính:"	SRS FR-07 Item 5: *"Tổng tiền hiển thị nhãn chính xác: 'Tổng cộng' (không phải 'Tổng tạm tính')."*	Trong mã nguồn `Cart.jsx:63`, lập trình viên hardcode: `<div ...>Tổng tạm tính: ... </div>`. Giao diện hiển thị sai nhãn quy định.	Mở `http://localhost:5173/cart` khi giỏ có sản phẩm; quan sát nhãn văn bản cạnh số tiền tổng ở footer giỏ hàng.
`HYP-07-02`	Frontend Interaction (`Cart.jsx:50-55`)	Nút Xóa sản phẩm xóa trực tiếp không có Dialog xác nhận	SRS FR-07 Item 3: *"Nút Xóa sản phẩm phải có dialog xác nhận trước khi thực hiện."*	Trong `Cart.jsx:51`, sự kiện `onClick={() => removeFromCart(index)}` được gọi ngay lập tức, không có `window.confirm` hay Modal Dialog nào.	Tại trang `/cart`, nhấn nút "Xóa" tại một dòng sản phẩm; kiểm tra xem sản phẩm có bị xóa ngay mà không xuất hiện dialog nào hay không.
`HYP-07-03`	Frontend UI (`Cart.jsx:20-27`)	Màn hình Giỏ hàng trống hoàn toàn thiếu hình ảnh minh họa	SRS FR-07 Item 6: *"Giỏ hàng trống phải có hình minh họa và thông báo rõ ràng."*	Trong `Cart.jsx:20-27`, code chỉ có thẻ `<h2>` chứa thông báo chữ và thẻ `<Link>`, không hề có thẻ `<img>` hay SVG vector minh họa nào.	Xóa hết sản phẩm trong giỏ để đưa về rỗng; kiểm tra cây DOM xem có bất kỳ thẻ hình ảnh minh họa nào hay không.
`HYP-07-04`	State & Business Logic (`CartContext.jsx:8-10`, `server.js:293`)	Thêm sản phẩm trùng lặp tạo dòng mới thay vì cộng dồn số lượng	SRS FR-07 Item 2: *"Thêm cùng một sản phẩm vào giỏ sẽ tăng số lượng, không tạo dòng mới."*	Code `CartContext.jsx:9` dùng `setCart([...cart, { ...product, quantity }])` (push phần tử mới vào mảng mà không kiểm tra trùng ID); `server.js:293` cũng dùng `push(req.body)`.	Mở `/products/1`, nhấn "Thêm vào giỏ hàng" 2 lần; mở `/cart` và kiểm tra bảng hiển thị có bị tách thành 2 dòng riêng biệt hay không.
`HYP-07-05`	Missing Interactive Feature (`Cart.jsx:47`)	Cột Số lượng trong bảng giỏ hàng thiếu hoàn toàn nút `+` và `-`	SRS FR-07 Item 1: *"Số lượng (có nút +/- để chỉnh)"*.	Trong `Cart.jsx:47`, code chỉ hiển thị text thụ động: `<td>{item.quantity}</td>`, hoàn toàn không có nút bấm `+` hay `-` để người dùng điều chỉnh số lượng.	Truy cập `/cart` khi giỏ có hàng; kiểm tra cột Số lượng xem có 2 nút `+` và `-` tương tác hay không.
`HYP-07-06`	Frontend Interaction (`ProductDetail.jsx:21-25`)	Nút "Thêm vào giỏ hàng" bị vô hiệu hóa ở lần nhấp chuột đầu tiên (Seeded Bug)	SRS FR-06 / FR-07: Bấm "Thêm vào giỏ hàng" phải thêm sản phẩm vào giỏ và hiển thị phản hồi trực quan ngay từ lần bấm đầu tiên.	Trong `ProductDetail.jsx:22-25`, mã nguồn cố ý chặn lần bấm đầu: `if (clickCount === 0) { setClickCount(1); return; }` khiến lần bấm đầu tiên hoàn toàn không có tác dụng!	Truy cập `/products/1`, click "Thêm vào giỏ hàng" lần thứ 1; quan sát thấy badge giỏ hàng không thay đổi; chỉ khi click lần thứ 2 giỏ mới nhận hàng.

4. TÍNH NĂNG C (POOL C): FR-17 – QUẢN LÝ MÃ GIẢM GIÁ (COUPON CRUD - ADMIN)

4.1 Bước 1: Nhận diện biến đầu vào & Ràng buộc (Step 1 Deliverables)

> **Trích dẫn đặc tả SRS FR-17 (Quản lý Mã Giảm Giá - Coupon CRUD):**
>
> - Admin có thể Thêm / Xem / Xóa mã giảm giá.
> - Các trường bắt buộc: `code` (duy nhất), `type` (percent/fixed), `discount_value` (dương), `expired_at`, `min_order_amount` (>= 0), `max_uses_per_user` (>= 1).
>
> *(Đối chiếu Đặc tả Hệ thống & API:*
>
> - *SRS Mục 5.3 & SEC-03: "Tất cả các API Admin (`/api/admin/*`) và các API có tính ảnh hưởng dữ liệu (`POST/PUT/DELETE /api/products`, `/api/categories`, `/api/coupons`) đều phải yêu cầu: Phải có JWT Token hợp lệ... User phải có role là admin (`role = 'admin'`). Nếu user thông thường (`role = 'user'`) gọi API admin -> trả về HTTP 403 Forbidden."*
> - *SRS FR-09: "Mã phải có trong CSDL và đang hoạt động (`is_active = 1`); Ngày hiện tại phải trước `expired_at` ; Tổng đơn hàng >= `min_order_amount` ; Số lần đã dùng mã này của user < `max_uses_per_user` ." Công thức: percent thì `discount_amount = total * discount_value / 100` , fixed thì `discount_amount = discount_value` .*
> - *API Spec §6.4: `POST /api/admin/coupons` và `DELETE /api/admin/coupons/:id` .*
> - *API Spec §5.2: `GET /api/coupons` (Lấy danh sách mã giảm giá).*
> - *Frontend Admin UI: Form tạo mã gồm Mã coupon (chuyển chữ hoa), Loại (select percent/fixed), Giá trị, Đơn tối thiểu, Ngày hết hạn, Giới hạn/người. Bảng danh sách mã giảm giá có nút Xóa từng mã.)*

Variable ID	Variable Name	Data Type	Mandatory?	Precision (ϵ)	Specification Constraint (Quote)	Assumptions / Ambiguities
`VAR-17-01`	`code` (Mã giảm giá)	String	Có	1 char	Trích dẫn SRS FR-17: *"Các trường bắt buộc: `code` (duy nhất)..."* (Đối chiếu FR-09: *"Mã phải có trong CSDL và đang hoạt động (`is_active = 1`)"*; API Spec §6.4)	Chuỗi định danh voucher; bắt buộc, không được để trống `""` hoặc chỉ chứa khoảng trắng `" "` . Tính duy nhất (`UNIQUE`): nếu mã đã tồn tại trong CSDL, hệ thống phải từ chối (HTTP 400/409) và thông báo lỗi rõ ràng, không được làm sập server (500). Giao diện Admin tự động chuyển thành chữ in hoa (`toUpperCase()`). SRS không quy định độ dài tối đa $\to$ Giả định chuẩn độ dài hợp lệ từ 1 đến 50 ký tự; không chứa ký tự đặc biệt gây lỗi XSS hay SQL injection.
`VAR-17-02`	`type` (Loại giảm giá)	Enum / String	Có	N/A	Trích dẫn SRS FR-17: *"Các trường bắt buộc: ... `type` (percent/fixed)..."* (Đối chiếu FR-09: *"Loại `percent` ... Loại `fixed` ..."*)	Chỉ chấp nhận tập giá trị rời rạc gồm đúng 2 lựa chọn: `'percent'` (tính theo tỷ lệ phần trăm) hoặc `'fixed'` (tính theo số tiền cố định ₫). Các giá trị khác ngoài tập (như chuỗi rỗng `""` , `'free_shipping'` , `'PERCENT'` , số, `null`) đều không hợp lệ và phải bị từ chối từ cả UI lẫn API.
`VAR-17-03`	`discount_value` (Giá trị giảm giá)	Number	Có	$1% \text{ hoặc } 1\text{ ₫}$	Trích dẫn SRS FR-17: *"Các trường bắt buộc: ... `discount_value` (dương)..."* (Đối chiếu FR-09: *"Loại percent: discount_amount = total * discount_value / 100; Loại fixed: discount_amount = discount_value"*)	Ràng buộc bất đẳng thức: phải là số dương ($discount_value > 0$). Giá trị bằng 0 hoặc âm (< 0) đều vi phạm đặc tả. Ràng buộc phụ thuộc theo `type` : (1) Nếu `type = 'percent'` : miền giá trị hợp lệ là $(0, 100%]$, với bước nhảy $\epsilon = 1%$; vượt quá 100% (ví dụ $101%$) là không hợp lệ vì không thể giảm vượt quá giá trị đơn hàng; (2) Nếu `type = 'fixed'` : số tiền nguyên dương > 0 (đơn vị ₫, $\epsilon = 1\text{ ₫}$); không được phép âm hay bằng 0.
`VAR-17-04`	`min_order_amount` (Ngưỡng đơn hàng tối thiểu)	Number / Currency	Có (Mặc định 0)	1 ₫	Trích dẫn SRS FR-17: *"Các trường bắt buộc: ... `min_order_amount` (>= 0)..."* (Đối chiếu FR-09: *"Tổng đơn hàng >= (lớn hơn hoặc bằng) min_order_amount"*)	Ràng buộc bất đẳng thức: $min_order_amount \ge 0$. Điểm biên hợp lệ tối thiểu là 0 ₫ (mã áp dụng cho mọi đơn hàng không ràng buộc giá trị). Giá trị âm (< 0) vi phạm đặc tả và phải bị từ chối. Trên form Admin, giá trị mặc định được gợi ý là `0` .
`VAR-17-05`	`max_uses_per_user` (Số lượt dùng tối đa / người)	Integer	Có (Mặc định 1)	1 lượt	Trích dẫn SRS FR-17: *"Các trường bắt buộc: ... `max_uses_per_user` (>= 1)."* (Đối chiếu FR-09: *"Số lần đã dùng mã này của user < max_uses_per_user"*)	Số nguyên dương $max_uses_per_user \ge 1$. Điểm biên hợp lệ tối thiểu là `1` (mỗi người dùng được áp dụng đúng 1 lần). Giá trị bằng `0` , số nguyên âm, số thập phân hoặc chuỗi phi số là không hợp lệ và phải bị từ chối. Trên form Admin, trường có thuộc tính HTML `min="1"` và mặc định là `1` .
`VAR-17-06`	`expired_at` (Ngày hết hạn của mã)	Date / DateTime	Có	1 ngày	Trích dẫn SRS FR-17: *"Các trường bắt buộc: ... `expired_at` ..."* (Đối chiếu FR-09: *"Còn hạn sử dụng: Ngày hiện tại phải trước expired_at"*)	Định dạng ngày theo chuẩn ISO (`YYYY-MM-DD`). Khi tạo mã mới, thời hạn phải là một ngày trong tương lai ($expired_at > \text{now}$) để mã có ý nghĩa sử dụng. Nếu chọn ngày trong quá khứ ($expired_at \le \text{now}$), mã tạo ra sẽ hết hạn ngay lập tức (cần cảnh báo/chặn). Ranh giới hết hạn: thời điểm hết hạn tính đến cuối ngày (`23:59:59`) hay đầu ngày (`00:00:00`) của ngày `expired_at` .
`VAR-17-07`	`auth_token_and_role` (Xác thực & Quyền hạn truy cập Admin)	Compound (JWT & Role)	Bắt buộc (Bảo mật)	N/A	Trích dẫn SRS Mục 5.3 & SEC-03: *"Tất cả các API Admin... phải có JWT Token hợp lệ... User phải có role là admin (`role = 'admin'`). Nếu user thông thường (`role = 'user'`) gọi API admin -> trả về HTTP 403 Forbidden."*	Precondition bảo mật: Chỉ tài khoản có `role = 'admin'` mới được phép thực hiện các thao tác Thêm (`POST /api/admin/coupons`), Xem danh sách quản trị (`GET /api/coupons`), và Xóa (`DELETE /api/admin/coupons/:id`). Gọi API không có Token $\to$ HTTP 401; gọi API bằng Token của User thường (`role = 'user'`) $\to$ HTTP 403 Forbidden.

Variable ID	Variable Name	Data Type	Mandatory?	Precision (ϵ)	Specification Constraint (Quote)	Assumptions / Ambiguities
`VAR-17-08`	`target_coupon_id` (Định danh coupon cho thao tác Xóa)	Integer	Bắt buộc (Thao tác Xóa)	1 đơn vị	Trích dẫn SRS FR-17: *"Admin có thể Thêm / Xem / Xóa mã giảm giá."*; API Spec §6.4: *"Xóa mã: `DELETE /api/admin/coupons/:id`"*	Tham số đường dẫn `:id` trong API xóa. Phải là số nguyên dương tương ứng với một coupon đang tồn tại trong bảng `coupons`. Nếu gửi ID không tồn tại (ví dụ: `id = 9999`) hoặc đã bị xóa trước đó, hệ thống phải xử lý an toàn (trả về HTTP 404 hoặc thông báo lỗi phù hợp). Trên giao diện Admin, cần kiểm tra xem thao tác Xóa có dialog xác nhận hay không (đối chiếu FR-07/FR-24).

4.2 Bước 2: Phân vùng tương đương (Step 2 Deliverables - Equivalence Partitioning)

Nguyên tắc Giả định Đơn lỗi (Single Fault Assumption - ISTQB FL §4.2.1) & Phân Tách Phạm Vi:

- **Phân tách 5 tầng kiểm thử độc lập (Tránh thổi phồng EP đầu vào):**
  1. **4.2.1 Pure Input Equivalence Partitions:** Phân vùng miền dữ liệu các trường nhập liệu form tạo mã giảm giá.
  2. **4.2.2 Access-Control & Authorization Partitions:** Phân vùng điều kiện xác thực và kiểm soát quyền truy cập Admin (SEC-03).
  3. **4.2.3 Target-Resource Partitions (CRUD Lifecycle):** Phân vùng tham số tài nguyên `:id` trong thao tác Xóa (Delete).
  4. **4.2.4 Business Rules & Coupon Behavior Scenarios (FR-09):** 14 kịch bản kiểm thử quy tắc tính toán, ngưỡng đơn, hạn dùng và lượt sử dụng khi áp dụng coupon tại Checkout.
  5. **4.2.5 Security & Robustness Probes:** Kiểm thử an toàn thông tin (XSS, SQLi) và độ bền vững trước dữ liệu cực hạn.
- **Nguyên tắc Đơn lỗi (Single-Fault Isolation):** Mỗi phân vùng không hợp lệ (IEP) chỉ kích hoạt đúng 1 điều kiện lỗi; toàn bộ các tham số còn lại bắt buộc phải được giữ ở giá trị hợp lệ cơ sở (Baseline).
- **Xử lý ràng buộc có điều kiện (Conditional Domain Modeling):** Miền giá trị của `discount_value` phụ thuộc chặt chẽ vào `type`:
  - Khi `type = 'percent'`: miền hợp lệ đề xuất là $(0, 100%]$ (ràng buộc nghiệp vụ cần xác nhận).
  - Khi `type = 'fixed'`: miền hợp lệ là $V > 0$ (VNĐ nguyên dương).
- **Bộ Giá trị Hợp lệ Cơ sở (Standard Baseline for Coupon Creation):**
  - `code`: `"PROMO2026"` (chuỗi chữ số hợp lệ, chưa tồn tại trong CSDL)
  - `type`: `"percent"`
  - `discount_value`: `10` (10% $\in (0, 100%]$)
  - `min_order_amount`: `100000` (100.000 ₫ ≥ 0)
  - `max_uses_per_user`: `1` (≥ 1)
  - `expired_at`: `"2026-12-31"` (ngày trong tương lai)
  - `auth_token_and_role`: JWT Token hợp lệ của Quản trị viên (`role = 'admin'`).

4.2.1 Bảng Phân Vùng Tương Đương Biến Đầu Vào Thuần Túy (Pure Input Equivalence Partitions)

Class ID	Variable / Constraint	Precondition	Domain / Partition Definition	Type	Representative Value	Expected Result	Specification Source & Status	Single-Fault Baseline & Rationale
`EP-17-01`	`code`	Baseline các trường khác hợp lệ	Mã mới, chưa tồn tại trong CSDL, đúng định dạng Alphanumeric	**VEP**	`"PROMO2026"`	Hệ thống chấp nhận, tạo bản ghi coupon mới trong CSDL với `is_active = 1`	SRS FR-17 / API Spec §6.4 `[SRS Confirmed]`	Baseline chuẩn hợp lệ; kết hợp với các tham số hợp lệ để tạo coupon thành công.
`EP-17-02`	`code`	Baseline các trường khác hợp lệ	Mã đã tồn tại sẵn trong CSDL (Duplicate code)	**IEP**	`"SAVE10"`	Hệ thống từ chối tạo mới; trả về lỗi vi phạm tính duy nhất (HTTP 400 hoặc 409 theo API contract); không sập lỗi SQLite 500	SRS FR-17 (*"duy nhất"*); CSDL `UNIQUE(code)` `[SRS Confirmed]`	Cô lập lỗi trùng lặp mã; các trường khác giữ nguyên giá trị hợp lệ cơ sở.
`EP-17-03`	`code`	Baseline các trường khác hợp lệ	Mã để trống / chuỗi rỗng (`""`)	**IEP**	`""`	Giao diện hiển thị validation lỗi required; API từ chối với HTTP 400 Bad Request	SRS FR-17 (*"Các trường bắt buộc"*); HTML `required` `[SRS Confirmed]`	Cô lập lỗi thiếu trường bắt buộc `code`.
`EP-17-04`	`code`	Baseline các trường khác hợp lệ	Mã chỉ chứa toàn khoảng trắng (Whitespace-only)	**IEP**	`" "`	Hệ thống tự động trim khoảng trắng và từ chối mã rỗng (HTTP 400); không lưu mã chỉ gồm dấu cách	Quy tắc validation dự kiến `[Needs clarification on Trim]`	Cô lập lỗi chuỗi rỗng nguy trang bằng khoảng trắng.
`EP-17-05`	`code`	Baseline các trường khác hợp lệ	Thiếu trường `code` hoàn toàn trong payload JSON	**IEP**	Payload khuyết `code`	API từ chối xử lý, trả về HTTP 400 Bad Request	API Spec §6.4 Schema `[API Contract Confirmed]`	Cô lập lỗi khuyết key trong JSON payload.
`EP-17-06`	`code`	CSDL đã có `"SAVE10"`	Trùng mã chỉ khác chữ hoa/thường (Case-insensitive conflict)	**Conditional**	`"save10"`	Cần xác nhận hành vi: Nếu hệ thống case-insensitive thì từ chối trùng mã; nếu case-sensitive thì chấp nhận tạo mã riêng	Chưa quy định trong SRS `[Needs clarification]`	Kiểm tra quy tắc phân biệt chữ hoa/thường ở backend và CSDL.

Class ID	Variable / Constraint	Precondition	Domain / Partition Definition	Type	Representative Value	Expected Result	Specification Source & Status	Single-Fault Baseline & Rationale
EP-17-07	code	Baseline các trường khác hợp lệ	Mã có khoảng trắng ở đầu hoặc cuối chuỗi	**Conditional**	" SAVE10 "	Cần xác nhận: Hệ thống tự động trim thành "SAVE10" hay lưu nguyên chuỗi khoảng trắng	Chưa quy định trong SRS [Needs clarification on Normalization]	Kiểm tra cơ chế tiền xử lý chuỗi (Sanitization / Trimming).
EP-17-08	code	Baseline các trường khác hợp lệ	Độ dài chuỗi vượt quá giới hạn lưu trữ quy ước ($Len > 50$)	**Robustness**	Chuỗi dài 51 ký tự ("P"*51)	Cần xác nhận: Hệ thống từ chối chuỗi quá dài (HTTP 400) hay tự cắt ngắn hay chấp nhận lưu chuỗi lớn	Giả định thiết kế CSDL [Assumption-based]	Kiểm tra độ bền vững trước chuỗi đầu vào cực dài.
EP-17-09	type	Baseline các trường khác hợp lệ	Loại giảm giá theo phần trăm ('percent')	**VEP**	'percent'	Hệ thống chấp nhận; áp dụng công thức giảm giá theo tỷ lệ % đơn hàng	SRS FR-17 & FR-09 [SRS Confirmed]	Baseline chuẩn cho loại phần trăm.
EP-17-10	type	discount_value = 50000	Loại giảm giá theo số tiền cố định ('fixed')	**VEP**	'fixed'	Hệ thống chấp nhận; áp dụng công thức trừ tiền mặt cố định ₫	SRS FR-17 & FR-09 [SRS Confirmed]	Baseline chuẩn cho loại số tiền cố định.
EP-17-11	type	Baseline các trường khác hợp lệ	Giá trị không nằm trong danh mục enum cho phép	**IEP**	'free_shipping' (hoặc 'other')	API từ chối với HTTP 400 Bad Request (Invalid type enum)	SRS FR-17 (*"type (percent/fixed)"*) [SRS Confirmed]	Cô lập lỗi gửi giá trị type không được hỗ trợ qua API bypass.
EP-17-12	type	Baseline các trường khác hợp lệ	Loại là chuỗi rỗng (type = "")	**IEP**	""	API từ chối với HTTP 400 Bad Request	SRS FR-17 (*"Các trường bắt buộc"*); API Spec [API Contract Confirmed]	Cô lập lỗi gửi chuỗi rỗng cho trường enum bắt buộc.
EP-17-13	type	Baseline các trường khác hợp lệ	Loại mang giá trị null (type = null)	**IEP**	null	API từ chối với HTTP 400 Bad Request	API Spec §6.4 Schema [API Contract Confirmed]	Cô lập lỗi truyền giá trị null vào trường bắt buộc.
EP-17-14	type	Baseline các trường khác hợp lệ	Thiếu trường type hoàn toàn trong payload JSON	**IEP**	Payload khuyết type	API từ chối với HTTP 400 Bad Request (hoặc gán mặc định nếu CSDL cho phép DEFAULT 'percent')	SRS FR-17 (*"Các trường bắt buộc"*); SQLite DEFAULT 'percent' [Needs clarification]	Kiểm tra xem backend bắt buộc payload hay tự động fallback CSDL.
EP-17-15	discount_value	type = 'percent'	Giá trị % hợp lệ trong khoảng chuẩn ($0 < V \le 100$)	**VEP**	10 (tương ứng 10%)	Hệ thống chấp nhận; lưu giá trị và áp dụng giảm 10% tại Checkout	SRS FR-17 (*"dương"*); Quy tắc nghiệp vụ % [SRS Confirmed & Business Rule]	Baseline hợp lệ chuẩn cho loại percent.
EP-17-16	discount_value	type = 'fixed'	Giá trị tiền mặt hợp lệ ($V > 0$ VNĐ)	**VEP**	50000 (50.000 ₫)	Hệ thống chấp nhận; lưu giá trị và áp dụng trừ 50.000 ₫ tại Checkout	SRS FR-17 (*"dương"*); FR-09 (*"Loại fixed"*) [SRS Confirmed]	Baseline hợp lệ chuẩn cho loại fixed.
EP-17-17	discount_value	type = 'percent' hoặc 'fixed'	Giá trị bằng 0 ($V = 0$)	**IEP**	0	Hệ thống từ chối; báo lỗi giá trị phải là số dương (> 0)	SRS FR-17 (*"discount_value (dương)"*) [SRS Confirmed]	Cô lập lỗi vi phạm ràng buộc số dương (bằng 0).
EP-17-18	discount_value	type = 'percent' hoặc 'fixed'	Giá trị mang số âm ($V < 0$)	**IEP**	-10 (percent) hoặc -50000 (fixed)	Hệ thống từ chối; báo lỗi giá trị phải dương (HTTP 400), không để xảy ra nghịch lý tăng tiền đơn	SRS FR-17 (*"discount_value (dương)"*) [SRS Confirmed]	Cô lập lỗi số âm; tránh lỗi tính toán làm tăng giá trị đơn hàng.
EP-17-19	discount_value	type = 'percent'	Giá trị phần trăm vượt quá 100% ($V > 100$)	**Conditional IEP**	101 (101%) hoặc 150 (150%)	Cần xác nhận: Hệ thống từ chối tạo mã (HTTP 400) do tỷ lệ giảm không thể vượt quá 100% giá trị đơn hàng	Quy tắc nghiệp vụ suy luận từ logic giảm giá [Needs clarification / Business Rule]	Chỉ áp dụng cho type = percent. Không áp dụng cho type = fixed.
EP-17-20	discount_value	Baseline các trường khác hợp lệ	Giá trị là chuỗi rỗng (discount_value = "")	**IEP**	""	Báo lỗi trường bắt buộc không được để trống (UI required / HTTP 400)	SRS FR-17 (*"Các trường bắt buộc"*); HTML required [SRS Confirmed]	Cô lập lỗi gửi chuỗi rỗng cho trường số bắt buộc.
EP-17-21	discount_value	Baseline các trường khác hợp lệ	Giá trị mang giá trị null	**IEP**	null	API từ chối với HTTP 400 Bad Request	API Spec §6.4 Schema [API Contract Confirmed]	Cô lập lỗi truyền null cho trường số.
EP-17-22	discount_value	Baseline các trường khác hợp lệ	Thiếu trường discount_value hoàn toàn trong payload	**IEP**	Payload khuyết discount_value	API từ chối với HTTP 400 Bad Request	SRS FR-17 (*"Các trường bắt buộc"*); API Spec [SRS Confirmed]	Cô lập lỗi thiếu key trong JSON payload.

Class ID	Variable / Constraint	Precondition	Domain / Partition Definition	Type	Representative Value	Expected Result	Specification Source & Status	Single-Fault Baseline & Rationale
EP-17-23	discount_value	Baseline các trường khác hợp lệ	Giá trị chứa ký tự phi số (chữ cái / ký hiệu)	**IEP**	"abc" hoặc "10%"	Form chặn gõ chữ (type="number"); API trả về HTTP 400 Bad Request do sai kiểu dữ liệu số	Ràng buộc kiểu dữ liệu Number [API Contract Confirmed]	Cô lập lỗi sai kiểu dữ liệu số (Data type mismatch).
EP-17-24	discount_value	type = 'percent'	Giá trị phần trăm là số thập phân ($V \in \mathbb{R} \setminus \mathbb{Z}$)	**Conditional**	10.5 (tương ứng 10.5%)	Cần xác nhận: Hệ thống chấp nhận số thập phân hay chỉ cho phép số nguyên %	Chưa quy định trong SRS [Needs clarification on Precision]	Kiểm tra độ chính xác bước nhảy phần trăm.
EP-17-25	discount_value	type = 'fixed'	Giá trị tiền cố định là số thập phân (có số lẻ hào/xu)	**Conditional**	50000.5 ₫	Cần xác nhận: Tiền tệ VND không có đơn vị xu lẻ; hệ thống làm tròn, tự cắt hay từ chối	Tiền tệ VND chuẩn không có phần thập phân [Needs clarification]	Kiểm tra tính nhất quán tiền tệ VND theo FR-21.
EP-17-26	min_order_amount	Baseline các trường khác hợp lệ	Ngưỡng đơn hàng hợp lệ là số nguyên dương ($M > 0$)	**VEP**	100000 (100.000 ₫)	Hệ thống chấp nhận tạo mã; coupon chỉ được dùng khi đơn hàng ≥ 100.000 ₫	SRS FR-17 (*"min_order_amount (>= 0)"*) [SRS Confirmed]	Baseline chuẩn hợp lệ cho ngưỡng đơn có điều kiện.
EP-17-27	min_order_amount	Baseline các trường khác hợp lệ	Ngưỡng đơn hàng bằng đúng 0 ($M = 0$)	**VEP**	0 (0 ₫)	Hệ thống chấp nhận tạo mã; coupon áp dụng cho mọi đơn hàng không phụ thuộc giá trị	SRS FR-17 (*"min_order_amount (>= 0)"*) [SRS Confirmed]	Giá trị biên dưới hợp lệ đặc biệt (không ràng buộc ngưỡng).
EP-17-28	min_order_amount	Baseline các trường khác hợp lệ	Ngưỡng đơn hàng mang giá trị âm ($M < 0$)	**IEP**	-1 hoặc -10000 ₫	Hệ thống từ chối; báo lỗi ngưỡng đơn hàng phải lớn hơn hoặc bằng 0 (HTTP 400)	SRS FR-17 (*"min_order_amount (>= 0)"*) [SRS Confirmed]	Cô lập lỗi vi phạm bất đẳng thức toán học ≥ 0.
EP-17-29	min_order_amount	Baseline các trường khác hợp lệ	Thiếu trường min_order_amount hoặc truyền null	**Conditional**	Omitted hoặc null	Cần xác nhận: Form Admin mặc định là 0 , nhưng nếu gửi API khuyết trường thì backend tự gán 0 hay từ chối HTTP 400	CSDL DEFAULT 0 vs SRS *"bắt buộc"* [Needs clarification]	Xác minh sự khác biệt giữa UI default và API contract validation.
EP-17-30	min_order_amount	Baseline các trường khác hợp lệ	Ngưỡng đơn hàng chứa ký tự phi số	**IEP**	"invalid_amount"	Form chặn gõ chữ; API trả về HTTP 400 Bad Request	Ràng buộc kiểu dữ liệu Number [API Contract Confirmed]	Cô lập lỗi sai kiểu dữ liệu số.
EP-17-31	max_uses_per_user	Baseline các trường khác hợp lệ	Số lượt dùng hợp lệ là số nguyên ≥ 1	**VEP**	1 (mặc định) hoặc 5	Hệ thống chấp nhận tạo mã; mỗi user được dùng tối đa số lần này	SRS FR-17 (*"max_uses_per_user (>= 1)"*) [SRS Confirmed]	Baseline chuẩn hợp lệ cho giới hạn lượt dùng/người.
EP-17-32	max_uses_per_user	Baseline các trường khác hợp lệ	Số lượt dùng bằng 0 ($U = 0$)	**IEP**	0	Hệ thống từ chối; báo lỗi số lượt dùng phải ≥ 1 (nếu bằng 0 thì mã vô dụng)	SRS FR-17 (*"max_uses_per_user (>= 1)"*) [SRS Confirmed]	Cô lập lỗi vi phạm ràng buộc tối thiểu là 1.
EP-17-33	max_uses_per_user	Baseline các trường khác hợp lệ	Số lượt dùng mang giá trị âm ($U < 0$)	**IEP**	-1	Hệ thống từ chối; báo lỗi số lượt dùng phải là số nguyên ≥ 1	SRS FR-17 (*"max_uses_per_user (>= 1)"*) [SRS Confirmed]	Cô lập lỗi số nguyên âm.
EP-17-34	max_uses_per_user	Baseline các trường khác hợp lệ	Số lượt dùng là số thập phân ($U \in \mathbb{R} \setminus \mathbb{Z}$)	**IEP**	1.5	Hệ thống từ chối; lượt dùng phải là số nguyên đếm được ≥ 1	SRS FR-17; Ràng buộc số đếm nguyên [SRS Confirmed]	Cô lập lỗi số thập phân cho đơn vị đếm lượt.
EP-17-35	max_uses_per_user	Baseline các trường khác hợp lệ	Thiếu trường max_uses_per_user hoặc truyền null	**Conditional**	Omitted hoặc null	Cần xác nhận: Form Admin mặc định 1 , backend server.js có code `max_uses_per_user		1` $\to$ kiểm tra API có chấp nhận fallback 1 hay yêu cầu bắt buộc
EP-17-36	expired_at	Baseline các trường khác hợp lệ	Ngày hết hạn trong tương lai ($expired_at > \text{now}$)	**VEP**	"2026-12-31"	Hệ thống chấp nhận; coupon tạo ra có thời hạn sử dụng hợp lệ	SRS FR-17 & FR-09 (*"Còn hạn sử dụng"*) [SRS Confirmed]	Baseline chuẩn hợp lệ cho thời hạn coupon.
EP-17-37	expired_at	Baseline các trường khác hợp lệ	Ngày hết hạn trong quá khứ ($expired_at < \text{now}$)	**Conditional IEP**	"2020-01-01"	Cần xác nhận: Hệ thống từ chối tạo mã (HTTP 400) vì mã quá hạn không thể sử dụng; hay chấp nhận tạo nhưng gắn trạng thái hết hạn	SRS chỉ ghi C2 ở checkout; chưa cấm ở create form [Needs clarification]	Kiểm tra quy tắc nghiệp vụ khi tạo coupon với ngày quá khứ.
EP-17-38	expired_at	Baseline các trường khác hợp lệ	Ngày hết hạn chính là ngày hiện tại ($expired_at = \text{today}$)	**Conditional**	Ngày hôm nay	Cần xác nhận mốc thời gian cutoff: coupon hết hạn lúc 00:00:00 (đầu ngày) hay 23:59:59 (hết ngày)	Chưa quy định giờ trong SRS [Needs clarification]	Kiểm tra độ phân giải thời gian (Date vs DateTime).

Class ID	Variable / Constraint	Precondition	Domain / Partition Definition	Type	Representative Value	Expected Result	Specification Source & Status	Single-Fault Baseline & Rationale
`EP-17-39`	`expired_at`	Baseline các trường khác hợp lệ	Ngày hết hạn để trống / chuỗi rỗng (`""`)	**IEP**	`""`	Giao diện báo lỗi trường bắt buộc; API từ chối với HTTP 400 Bad Request	SRS FR-17 (*"Các trường bắt buộc"*); HTML `required` `[SRS Confirmed]`	Cô lập lỗi thiếu trường bắt buộc ngày hết hạn.
`EP-17-40`	`expired_at`	Baseline các trường khác hợp lệ	Ngày hết hạn sai định dạng ngày tháng (Malformed Date)	**IEP**	`"2026-13-45"` hoặc `"invalid-date"`	Hệ thống từ chối chuỗi ngày không hợp lệ; trả về HTTP 400 Bad Request	Ràng buộc định dạng chuẩn ISO Date `[API Contract Confirmed]`	Cô lập lỗi sai cú pháp thời gian.

4.2.2 Bảng Phân Vùng Điều Kiện Truy Cập & Phân Quyền (Access-Control & Authorization Partitions)

> **Lưu ý phương pháp luận:** Nhóm này kiểm tra các điều kiện tiên quyết (Preconditions) về xác thực danh tính và phân quyền truy cập Admin (SEC-02, SEC-03, SRS Mục 5.3), được quản lý độc lập với các phân vùng dữ liệu coupon để không làm sai lệch số liệu đo lường độ bao phủ EP của form nhập liệu.

Class ID	Điều kiện kiểm thử	Endpoint & Thao tác	Type	Dữ liệu kiểm thử / Header	Kết quả mong đợi theo Đặc tả & Hợp đồng API	Căn cứ đặc tả & Trạng thái	Single-Fault Baseline & Ghi chú
`AUTH-17-01`	Token hợp lệ có quyền Admin (`role = 'admin'`)	`POST /api/admin/coupons` , `DELETE /api/admin/coupons/:id` , `GET /api/coupons`	**VEP**	Header: `Authorization: Bearer <Admin_JWT>`	Hệ thống xác thực thành công; cho phép thực thi toàn bộ các thao tác CRUD quản lý coupon	SRS Mục 5.3; SEC-03; API Spec §6 `[SRS Confirmed]`	Dữ liệu coupon hoàn toàn hợp lệ; người dùng đăng nhập bằng tài khoản quản trị viên.
`AUTH-17-02`	Không gửi Token xác thực (Anonymous / Unauthenticated)	`POST /api/admin/coupons`	**IEP**	Không có header `Authorization`	API lập tức từ chối, trả về mã trạng thái **HTTP 401 Unauthorized**; không thực hiện ghi dữ liệu	SRS Mục 5.3; SEC-02; API Spec §6 `[SRS Confirmed]`	Dữ liệu coupon hợp lệ cơ sở; cô lập lỗi thiếu token xác thực.
`AUTH-17-03`	Token hợp lệ nhưng thuộc User thông thường (`role = 'user'`)	`POST /api/admin/coupons` và `DELETE /api/admin/coupons/:id`	**IEP**	Header: `Authorization: Bearer <User_JWT>`	API từ chối với mã trạng thái **HTTP 403 Forbidden**; thông báo không có quyền thực hiện thao tác quản trị	SRS Mục 5.3; SEC-03 (*"Kiểm tra role='admin'"*) `[SRS Confirmed]`	Dữ liệu coupon hợp lệ cơ sở; cô lập lỗi vi phạm phân quyền người dùng (Privilege Escalation Probe).
`AUTH-17-04`	Token giả mạo / sai chữ ký / đã hết hạn	`POST /api/admin/coupons`	**IEP**	Header: `Authorization: Bearer invalid.token.xyz`	API từ chối xác thực; trả về **HTTP 401 Unauthorized** hoặc **HTTP 403 Forbidden**	SEC-02 (JWT signature verification) `[API Contract Confirmed]`	Dữ liệu coupon hợp lệ cơ sở; cô lập lỗi token không hợp lệ.
`AUTH-17-05`	Token hợp lệ nhưng payload thiếu trường `role`	`POST /api/admin/coupons`	**Security**	Header: JWT ký đúng nhưng payload `{ id: 1 }` (không có `role`)	API từ chối với **HTTP 403 Forbidden** do không chứng minh được quyền Admin	SEC-03 (*"kiểm tra role = 'admin' trong Token"*) `[SRS Confirmed]`	Dữ liệu coupon hợp lệ cơ sở; kiểm tra cơ chế kiểm tra trường role an toàn của middleware.

4.2.3 Bảng Phân Vùng Thao Tác & Vòng Đời Tài Nguyên (Target-Resource & CRUD Lifecycle Partitions)

> **Lưu ý phương pháp luận:** Nhóm này kiểm tra tham số tài nguyên `:id` trên đường dẫn endpoint `DELETE /api/admin/coupons/:id` và các kiểm thử chuyển đổi trạng thái vòng đời bản ghi coupon trong CSDL SQLite.

Class ID	Ràng buộc / Thao tác	Điều kiện tiền đề	Type	Tham số `:id` đại diện	Kết quả mong đợi theo Đặc tả & Hợp đồng API	Căn cứ đặc tả & Trạng thái	Single-Fault Baseline & Ghi chú
`RES-17-01`	Xóa coupon hợp lệ đang tồn tại	Coupon đang tồn tại trong bảng `coupons`	**VEP**	`id` hợp lệ (VD: coupon vừa tạo)	API trả về **HTTP 200 OK** kèm `{"message": "Coupon deleted"}` ; bản ghi bị xóa khỏi bảng `coupons`	SRS FR-17 (*"Admin có thể Xóa mã"*); API Spec §6.4 `[SRS Confirmed]`	Token Admin hợp lệ; thao tác xóa bản ghi coupon tồn tại thực tế.
`RES-17-02`	Xóa coupon với ID không tồn tại	CSDL không chứa ID này	**IEP**	`id = 9999`	API trả về **HTTP 404 Not Found** (hoặc thông báo lỗi bản ghi không tồn tại); không báo lỗi crash server	API Spec §6.4 `[Needs clarification on 404 contract]`	Token Admin hợp lệ; cô lập lỗi ID không có trong CSDL.
`RES-17-03`	Xóa coupon với ID mang giá trị âm	Route param	**IEP**	`id = -1`	API từ chối với **HTTP 400 Bad Request**; khóa chính bảng CSDL luôn là số nguyên dương ≥ 1	Ràng buộc miền khóa chính CSDL `[API Contract Confirmed]`	Token Admin hợp lệ; cô lập lỗi ID mang giá trị âm.
`RES-17-04`	Xóa coupon với ID bằng 0	Route param	**IEP**	`id = 0`	API từ chối với **HTTP 400 Bad Request** hoặc **404 Not Found**	Ràng buộc miền khóa chính CSDL SQLite `[API Contract Confirmed]`	Token Admin hợp lệ; SQLite autoincrement bắt đầu từ 1.
`RES-17-05`	Xóa coupon với ID sai kiểu dữ liệu (phi số)	Route param	**IEP**	`id = "abc"`	API từ chối với **HTTP 400 Bad Request** do lỗi binding kiểu dữ liệu tham số route	Ràng buộc kiểu dữ liệu số nguyên của ID `[API Contract Confirmed]`	Tách biệt hoàn toàn với trường hợp số âm `id = -1` .

Class ID	Ràng buộc / Thao tác	Điều kiện tiền đề	Type	Tham số `:id` đại diện	Kết quả mong đợi theo Đặc tả & Hợp đồng API	Căn cứ đặc tả & Trạng thái	Single-Fault Baseline & Ghi chú
`RES-17-06`	Thao tác Xóa lặp lại cùng một ID (Idempotency Probe)	Coupon vừa bị xóa thành công ở `RES-17-01`	**State Probe**	Gửi lại `DELETE` cho cùng `id` đã xóa	Lần gửi thứ 2 phải trả về **HTTP 404 Not Found** do bản ghi không còn tồn tại; không báo thành công giả tạo	Nguyên tắc thiết kế REST API & Vòng đời CSDL `[Needs clarification]`	Kiểm tra tính nhất quán trạng thái sau khi xóa.
`RES-17-07`	Xóa coupon đã có lịch sử người dùng sử dụng	Coupon ID đã phát sinh bản ghi trong bảng `coupon_usage`	**Data Integrity**	`id` của coupon đã được dùng ở đơn hàng cũ	Cần xác nhận thiết kế: Hệ thống xóa mềm (`soft delete` / `is_active = 0`) để giữ toàn vẹn lịch sử đơn hàng hay xóa cứng cascade	Ràng buộc toàn vẹn quan hệ CSDL SQLite `[Needs clarification on Soft Delete]`	Kiểm tra an toàn dữ liệu lịch sử tài chính đơn hàng.

4.2.4 Bảng Quy Tắc Nghiệp Vụ & Kiểm Thử Hành Vi Áp Dụng Coupon FR-09 (Business Rules & Coupon Behavior Scenarios)

> **Lưu ý phương pháp luận:** 14 kịch bản dưới đây kiểm chứng việc thực thi logic nghiệp vụ và tính toán giảm giá của Coupon tại chức năng Checkout (FR-09 & FR-08), đảm bảo mã giảm giá do Admin tạo ra hoạt động chính xác trên toàn bộ vòng đời ứng dụng EShop.

ID Quy tắc	Hạng mục quy tắc nghiệp vụ	Điều kiện vận hành cụ thể	Dữ liệu / Kịch bản đại diện	Kết quả cần xác minh theo Đặc tả SRS	Căn cứ đặc tả SRS & Ghi chú
`CPN-BR-01`	Công thức tính loại `percent`	Coupon `type = 'percent'` , giá trị giảm 10%	Đơn hàng tổng $1.000.000\text{ ₫}$, mã giảm 10%	Số tiền giảm tính đúng công thức: $1.000.000 \times 10 / 100 = 100.000\text{ ₫}$; Tổng thanh toán: 900.000 ₫	SRS FR-09: *"discount_amount = total * discount_value / 100"*
`CPN-BR-02`	Công thức tính loại `fixed`	Coupon `type = 'fixed'` , giá trị giảm 50.000 ₫	Đơn hàng tổng $1.000.000\text{ ₫}$, mã giảm 50k	Số tiền giảm tính đúng: 50.000 ₫; Tổng thanh toán: $1.000.000 - 50.000 = 950.000\text{ ₫}$	SRS FR-09: *"discount_amount = discount_value"*
`CPN-BR-03`	Ràng buộc ngưỡng đơn hàng (Dưới ngưỡng)	Tổng đơn hàng nhỏ hơn `min_order_amount`	`min = 300.000 ₫` ; Giỏ hàng hiện có 250.000 ₫	Hệ thống từ chối áp dụng mã; hiển thị thông báo đơn hàng chưa đạt giá trị tối thiểu (C3 vi phạm)	SRS FR-09 C3: *"Tổng đơn hàng >= min_order_amount"*
`CPN-BR-04`	Ràng buộc ngưỡng đơn hàng (Bằng ngưỡng)	Tổng đơn hàng bằng đúng `min_order_amount`	`min = 300.000 ₫` ; Giỏ hàng hiện có đúng 300.000 ₫	Hệ thống chấp nhận áp dụng mã thành công (Thỏa mãn điều kiện toán học $\ge$)	SRS FR-09 C3 (Biên On-point của ngưỡng đơn)
`CPN-BR-05`	Ràng buộc ngưỡng đơn hàng (Trên ngưỡng)	Tổng đơn hàng lớn hơn `min_order_amount`	`min = 300.000 ₫` ; Giỏ hàng có 500.000 ₫	Hệ thống chấp nhận áp dụng mã thành công	SRS FR-09 C3 (In-point của ngưỡng đơn)
`CPN-BR-06`	Ràng buộc thời hạn sử dụng (Quá hạn)	Ngày hiện tại sau ngày `expired_at`	Coupon mẫu `EXPIRED` (`expired_at = '2020-01-01'`)	Hệ thống từ chối áp dụng mã; thông báo mã giảm giá đã hết hạn sử dụng (C2 vi phạm)	SRS FR-09 C2: *"Ngày hiện tại phải trước expired_at"*
`CPN-BR-07`	Ràng buộc trạng thái hoạt động	Coupon có cờ `is_active = 0`	Coupon bị vô hiệu hóa trong CSDL (`is_active = 0`)	Hệ thống từ chối áp dụng mã; thông báo mã không còn hoạt động (C1 vi phạm)	SRS FR-09 C1: *"Mã phải có trong CSDL và đang hoạt động (is_active = 1)"*
`CPN-BR-08`	Ràng buộc lượt dùng / người (Còn lượt)	User đã dùng mã ít hơn `max_uses_per_user`	`max = 2` ; bảng `coupon_usage` ghi nhận user đã dùng 1 lần	Hệ thống chấp nhận áp dụng mã cho lần thanh toán thứ 2 (C5 thỏa mãn: $1 < 2$)	SRS FR-09 C5: *"Số lần đã dùng của user < max_uses_per_user"*
`CPN-BR-09`	Ràng buộc lượt dùng / người (Hết lượt)	User đã dùng mã bằng đúng `max_uses_per_user`	`max = 1` ; bảng `coupon_usage` ghi nhận user đã dùng 1 lần	Hệ thống từ chối áp dụng mã; thông báo người dùng đã sử dụng hết lượt cho phép (C5 vi phạm: $1 \not< 1$)	SRS FR-09 C5 (Biên On-point của lượt dùng)
`CPN-BR-10`	Giá trị giảm cố định lớn hơn tổng đơn	Coupon `fixed` có giá trị giảm vượt tổng tiền hàng	Giỏ hàng 30.000 ₫; áp dụng coupon `fixed = 50.000 ₫`	Cần xác minh: Hệ thống giảm tối đa về 0 ₫ hay bị tính ra tổng tiền âm? (Quy tắc an toàn tài chính)	Chưa quy định trong SRS `[Needs clarification / Defect Hotspot]`
`CPN-BR-11`	Giảm giá % sinh ra số thập phân	Đơn giá lẻ kết hợp tỷ lệ % sinh ra số xu lẻ	Đơn hàng 125.555 ₫, giảm 10% $\to 12.555,5\text{ ₫}$	Cần xác minh quy tắc làm tròn: làm tròn lên, làm tròn xuống hay giữ nguyên? Phù hợp tiền tệ VND	Tiền tệ VND không dùng xu lẻ `[Needs clarification]`
`CPN-BR-12`	Xác nhận lưu trữ CSDL khi tạo thành công	Tạo mới coupon thành công từ Admin UI / API	`code = "PROMO2026"` , `type = "percent"` , `val = 10`	Bản ghi CSDL được lưu đầy đủ 6 trường; `code` được chuẩn hóa chữ hoa; `is_active` mặc định bằng 1	SRS FR-17 & Database Schema
`CPN-BR-13`	Trạng thái sau khi xóa coupon	Coupon vừa bị Admin xóa qua API `DELETE`	Coupon `SAVE10` bị Admin xóa khỏi hệ thống	Bản ghi không còn xuất hiện trong `GET /api/coupons` ; người dùng nhập mã tại Checkout nhận thông báo mã không tồn tại	Kiểm thử tính nhất quán toàn diện (Cross-functional consistency)
`CPN-BR-14`	Concurrency / Trùng mã khi gửi đồng thời	Hai Admin cùng gửi request tạo cùng 1 `code` đồng thời	2 request `POST` cùng gửi payload `code = "FLASH50"` trong cùng 100ms	Đúng 1 request thành công (200/201), request thứ 2 bị từ chối do vi phạm UNIQUE; CSDL chỉ có duy nhất 1 bản ghi	Kiểm thử an toàn đồng thời (Race condition probe)

4.2.5 Kiểm Thử An Toàn & Độ Bền Vững (Security & Robustness Probes)

> **Lưu ý phương pháp luận:** Nhóm này đại diện cho các kịch bản kiểm thử phi chức năng (Non-functional Security & Stress Probes) nhằm bảo vệ hệ thống trước các cuộc tấn công tiêm nhiễm mã độc và xử lý tải dữ liệu bất thường.

Class ID	Loại kiểm thử an toàn	Dữ liệu đầu vào thăm dò (Probe Payload)	Mục tiêu kiểm chứng	Kết quả mong đợi theo Tiêu chuẩn An toàn (SEC)
`SEC-17-01`	Cross-Site Scripting (Stored XSS Probe)	`code = "<script>alert('XSS')</script>"`	Kiểm tra việc render mã coupon trên bảng quản trị Admin Web	Giao diện escape thẻ HTML an toàn (SEC-04); hiển thị nguyên văn chuỗi văn bản, tuyệt đối không thực thi script alert
`SEC-17-02`	SQL Injection (Param Query Probe)	`code = "SAVE' OR '1'='1"`	Kiểm tra an toàn câu lệnh truy vấn SQLite tại backend	Hệ thống sử dụng Parameterized Query (SEC-05); chuỗi được coi là giá trị tìm kiếm thông thường, không phá vỡ cú pháp SQL
`SEC-17-03`	Robustness / Tràn số nguyên JS	`min_order_amount = 10^16` ($10.000.000.000.000.000$)	Kiểm tra độ bền bỉ khi nhập số vượt `Number.MAX_SAFE_INTEGER`	Hệ thống xử lý an toàn không treo máy; không biến thành `Infinity` hoặc sinh ra lỗi làm sập tiến trình Node.js
`SEC-17-04`	Malformed JSON Payload	Payload JSON thiếu ngoặc, sai format hoặc chứa trường rác	Kiểm tra khả năng xử lý ngoại lệ của middleware Express	Server trả về HTTP 400 Bad Request rõ ràng, không văng lỗi stack trace `500 Internal Server Error` ra client

4.3 Bước 3: Phân tích giá trị biên (Step 3 Deliverables - Boundary Value Analysis - BVA)

> **Phương pháp tiếp cận Phân tích Giá trị Biên (ISTQB FL §4.2.2):**
>
> - **Lựa chọn kỹ thuật:** Áp dụng **3-Value BVA** (On-point, Off-point, In-point) cho các biến có thứ tự (số nguyên, số tiền, tỷ lệ %, ngày tháng, độ dài chuỗi) với bước vi sai nhỏ nhất (ϵ).
> - **Định nghĩa các điểm biên chuẩn ISTQB:**
>   - **On-point:** Giá trị nằm chính xác trên đường phân định ranh giới.
>   - **Off-point:** Giá trị nằm sát biên nhất ở phía đối diện:
>   - Nếu biên đóng / bao gồm ($\le, \ge$), On-point nằm trong vùng hợp lệ (Valid On-point); Off-point nằm ngay ngoài biên (Invalid Off-point).
>   - Nếu biên mở / loại trừ ($<, >$), On-point nằm ngoài vùng hợp lệ (Invalid On-point); Off-point nằm ngay sát trong vùng hợp lệ (Valid Off-point).
>   - **In-point:** Giá trị nằm liền kề bên trong phân vùng hợp lệ.
> - **Phân tách 2 phạm vi kiểm thử biên:**
>   1. **4.3.1 Form Input BVA:** Các giá trị biên của form tạo mới mã giảm giá (FR-17).
>   2. **4.3.2 Checkout Application BVA:** Các giá trị biên của điều kiện áp dụng mã tại giỏ hàng và thanh toán (FR-09).

4.3.1 Bảng Phân Tích Giá Trị Biên Dữ Liệu Đầu Vào Form Tạo Coupon (Input Form BVA - FR-17)

Boundary ID	Variable / Field	Boundary Rule	BVA Method & Step (ϵ)	On-point (Value / Type & Mô tả)	Off-point (Value / Type & Mô tả)	In-point (Value / Type & Mô tả)
`BVA-17-01`	`code.length`	$Len \ge 1$ (Biên dưới độ dài mã không rỗng)	3-Value ($\epsilon=1\text{ char}$)	$Len = 1$ [Valid On-point] `code = "A"` (đúng 1 ký tự, mã ngắn nhất hợp lệ).	$Len = 0$ [Invalid Off-point] `code = ""` ($1 - 1 = 0$, chuỗi rỗng vi phạm bắt buộc).	$Len = 2$ [Valid In-point] `code = "AB"` ($1 + 1 = 2$, độ dài kề cận trong vùng hợp lệ).
`BVA-17-02`	`code.length`	$Len \le 50$ (Cận trên độ dài mã theo quy ước CSDL)	3-Value ($\epsilon=1\text{ char}$) `[Assumption]`	$Len = 50$ [Valid On-point] Mã dài đúng 50 ký tự (chấp nhận lưu trữ).	$Len = 51$ [Invalid Off-point] Mã dài 51 ký tự ($50 + 1$, vượt ngưỡng quy ước CSDL).	$Len = 49$ [Valid In-point] Mã dài 49 ký tự ($50 - 1$, kề cận trong vùng hợp lệ).
`BVA-17-03`	`discount_value` (`type='percent'`)	$V > 0\%$ (Biên dưới số dương - Exclusive boundary)	3-Value ($\epsilon=1\%$)	$V = 0\%$ [Invalid On-point] Đúng vạch biên $V = 0$ (vi phạm ràng buộc số dương của SRS).	$V = 1\%$ [Valid Off-point] $0 + 1\% = 1\%$ (giá trị phần trăm dương nhỏ nhất được chấp nhận).	$V = 2\%$ [Valid In-point] $0 + 2\% = 2\%$ (giá trị kề cận bên trong miền hợp lệ).
`BVA-17-04`	`discount_value` (`type='percent'`)	$V \le 100\%$ (Cận trên tỷ lệ % - Inclusive boundary)	3-Value ($\epsilon=1\%$) `[Business Rule]`	$V = 100\%$ [Valid On-point] Đúng vạch biên 100% (giảm tối đa toàn bộ đơn hàng - miễn phí 100%).	$V = 101\%$ [Invalid Off-point] $100 + 1\% = 101\%$ (vượt 100%, không thể giảm nhiều hơn đơn hàng).	$V = 99\%$ [Valid In-point] $100 - 1\% = 99\%$ (kề cận dưới mốc 100%, hợp lệ).
`BVA-17-05`	`discount_value` (`type='fixed'`)	$V > 0\text{ đ}$ (Biên dưới số tiền dương - Exclusive)	3-Value ($\epsilon=1\text{ đ}$)	$V = 0\text{ đ}$ [Invalid On-point] Đúng vạch biên 0 đ (bị từ chối vì phải là số dương > 0).	$V = 1\text{ đ}$ [Valid Off-point] $0 + 1\text{ đ} = 1\text{ đ}$ (số tiền dương nhỏ nhất được chấp nhận).	$V = 2\text{ đ}$ [Valid In-point] $0 + 2\text{ đ} = 2\text{ đ}$ (giá trị tiền kề cận trong miền hợp lệ).
`BVA-17-06`	`min_order_amount`	$M \ge 0\text{ đ}$ (Ngưỡng đơn hàng tối thiểu - Inclusive)	3-Value ($\epsilon=1\text{ đ}$)	$M = 0\text{ đ}$ [Valid On-point] Đúng vạch biên 0 đ (áp dụng cho mọi đơn hàng không ràng buộc).	$M = -1\text{ đ}$ [Invalid Off-point] $0 - 1\text{ đ} = -1\text{ đ}$ (số âm vi phạm bất đẳng thức $M \ge 0$).	$M = 1\text{ đ}$ [Valid In-point] $0 + 1\text{ đ} = 1\text{ đ}$ (giá trị ngưỡng tối thiểu dương nhỏ nhất).
`BVA-17-07`	`max_uses_per_user`	$U \ge 1\text{ lượt}$ (Giới hạn lượt dùng tối thiểu - Inclusive)	3-Value ($\epsilon=1\text{ lượt}$)	$U = 1\text{ lượt}$ [Valid On-point] Đúng mốc tối thiểu (mỗi người dùng được áp dụng đúng 1 lần).	$U = 0\text{ lượt}$ [Invalid Off-point] $1 - 1 = 0$ (vi phạm $U \ge 1$; nếu bằng 0 mã không ai dùng được).	$U = 2\text{ lượt}$ [Valid In-point] $1 + 1 = 2\text{ lượt}$ (giá trị kề cận bên trong miền hợp lệ).
`BVA-17-08`	`expired_at`	$expired_at \ge \text{today}$ (Ranh giới ngày hết hạn khi tạo mã)	3-Value ($\epsilon=1\text{ ngày}$) `[Conditional]`	$expired_at = \text{today}$ [On-point] Ngày hôm nay (cần kiểm tra mốc cutoff giờ: 00:00 vs 23:59).	$expired_at = \text{yesterday}$ [Invalid Off-point] $\text{today} - 1\text{ ngày}$ (ngày quá khứ; vừa tạo đã hết hạn ngay).	$expired_at = \text{tomorrow}$ [Valid In-point] $\text{today} + 1\text{ ngày}$ (ngày mai; thời hạn tương lai gần nhất).

4.3.2 Bảng Phân Tích Giá Trị Biên Quy Tắc Áp Dụng Tại Checkout (Checkout Application BVA - FR-09)

Boundary ID	Điều kiện kiểm thử biên	Boundary Rule	BVA Method & Step (ϵ)	On-point (Value / Type & Mô tả)	Off-point (Value / Type & Mô tả)	In-point (Value / Type & Mô tả)
`BVA-17-09`	`order_total` vs `min_order_amount` ($M = 300.000\text{ ₫}$)	$Total \ge M$ (Ngưỡng đơn đủ điều kiện C3 - Inclusive)	3-Value ($\epsilon=1\text{ ₫}$)	$Total = 300.000\text{ ₫}$ [Valid On-point] Đúng bằng ngưỡng tối thiểu $\to$ Đủ điều kiện C3, áp dụng giảm giá.	$Total = 299.999\text{ ₫}$ [Invalid Off-point] $M - 1\text{ ₫} \to$ Thiếu đúng 1 ₫, từ chối áp dụng mã giảm giá.	$Total = 300.001\text{ ₫}$ [Valid In-point] $M + 1\text{ ₫} \to$ Vượt ngưỡng 1 ₫, áp dụng giảm giá bình thường.
`BVA-17-10`	`user_usage_count` vs `max_uses_per_user` ($U = 2\text{ lượt}$)	$used < U$ (Điều kiện C5 chưa hết lượt - Exclusive)	3-Value ($\epsilon=1\text{ lượt}$)	$used = 2\text{ lần}$ [Invalid On-point] Đúng bằng mốc giới hạn ($2 \not< 2$) $\to$ Đã hết lượt, từ chối áp dụng.	$used = 1\text{ lần}$ [Valid Off-point] $U - 1 = 1$ ($1 < 2$) $\to$ Lượt dùng hợp lệ cuối cùng, được áp dụng.	$used = 3\text{ lần}$ [Invalid In-point] $U + 1 = 3$ ($3 \not< 2$) $\to$ Đã vượt quá giới hạn, từ chối áp dụng.
`BVA-17-11`	`fixed_discount` vs `order_total` ($Total = 50.000\text{ ₫}$)	$discount \le Total$ (Biên khấu trừ tiền mặt đơn hàng)	3-Value ($\epsilon=1\text{ ₫}$) `[Defect Hotspot]`	$discount = 50.000\text{ ₫}$ [Valid On-point] Giảm đúng bằng tổng tiền $\to$ Đơn hàng thanh toán bằng 0 ₫.	$discount = 50.001\text{ ₫}$ [Anomaly Off-point] Giảm vượt tổng đơn 1 ₫ $\to$ Kiểm tra hệ thống chặn ở 0 ₫ hay bị âm tiền.	$discount = 49.999\text{ ₫}$ [Valid In-point] Giảm thiếu 1 ₫ $\to$ Đơn hàng còn phải trả đúng 1 ₫.

4.4 Bước 4: Thiết kế kịch bản kiểm thử (Step 4 Deliverables - Test Case Design)

> **Phương pháp luận Thiết kế Kịch bản Kiểm thử (Theo `Comprehensive Guide: How to Write Test Cases`):**
>
> 1. **Cú pháp Định danh Mục tiêu (Title / Objective Formula):** Mọi kịch bản kiểm thử đều tuân thủ công thức chuẩn: $$\text{Action (Hành động)} + \text{Function (Chức năng)} + \text{Operating Condition (Điều kiện vận hành)}$$
> 2. **Nguyên tắc Giả định Đơn lỗi (Single Fault Isolation):** Mỗi kịch bản Negative (IEP hoặc Off-point) chỉ kích hoạt đúng 1 điều kiện không hợp lệ; tất cả các biến khác giữ nguyên giá trị hợp lệ cơ sở (Baseline).
> 3. **Tính Tự dọn dẹp (Self-Cleaning / Teardown):** Mỗi kịch bản thao tác dữ liệu đều có bước dọn dẹp hoặc khôi phục trạng thái bảng `coupons` và `coupon_usage` trong SQLite để đảm bảo tính độc lập tuyệt đối giữa các lượt kiểm thử.
> 4. **Điểm kiểm chứng đa tầng (Multi-tier Validation Points):** Kết quả mong đợi phân tách rõ ràng 4 tầng: Giao diện Quản trị (Admin Web), Giao diện Lập trình (API), Trạng thái CSDL (Database SQLite), và Tuân thủ An toàn Thông tin & Nghiệp vụ (Security / Business Rules).

4.4.1 Bảng Tổng Hợp Kịch Bản Kiểm Thử & Ma Trận Truy Vết (Overview Traceability Matrix)

TC ID	Test Case Objective ($\text{Action} + \text{Function} + \text{Condition}$)	Phân loại	Mã EP / Rule bao phủ	Mã BVA bao phủ	Dữ liệu đầu vào & Baseline	Kết quả cốt lõi (Core Expected)	Trạng thái
`TC_FR17_01`	Verify creating a valid percent coupon with standard baseline inputs on Admin Web & API	Positive (VEP)	`EP-17-01`, `EP-17-09`, `EP-17-15`, `EP-17-26`, `EP-17-31`, `EP-17-36`, `AUTH-17-01`, `CPN-BR-12`	`BVA-17-03-IN`	`code="PROMO2026"`, `type="percent"`, `val=10`, `min=100000`, `max=1`, `expired="2026-12-31"`	HTTP 200/201, tạo thành công coupon ID mới, hiển thị trên bảng Admin	**Pass**
`TC_FR17_02`	Verify creating a valid fixed amount coupon on Admin Web & API	Positive (VEP)	`EP-17-01`, `EP-17-10`, `EP-17-16`, `EP-17-26`, `EP-17-31`, `EP-17-36`, `AUTH-17-01`, `CPN-BR-12`	`BVA-17-05-IN`	`code="FIXED50K"`, `type="fixed"`, `val=50000`, `min=100000`, `max=1`, `expired="2026-12-31"`	HTTP 200/201, tạo thành công coupon trừ 50.000 ₫	**Pass**
`TC_FR17_03`	Verify creating a percent coupon at upper boundary 100% (On-point)	Positive (BVA)	`EP-17-15`	`BVA-17-04-ON` ($V=100%$)	`val=100%`, các trường khác chuẩn baseline	Chấp nhận tạo mã giảm 100% (miễn phí toàn bộ đơn hàng)	**Pass**
`TC_FR17_04`	Verify creating a percent coupon at lower boundary 1% (Off-point of >0)	Positive (BVA)	`EP-17-15`	`BVA-17-03-OFF` ($V=1%$)	`val=1%`, các trường khác chuẩn baseline	Chấp nhận tạo mã với tỷ lệ giảm nhỏ nhất 1%	**Pass**
`TC_FR17_05`	Verify creating a coupon with zero minimum order amount (On-point M=0)	Positive (BVA)	`EP-17-27`	`BVA-17-06-ON` ($M=0\text{ ₫}$)	`min_order_amount=0`, các trường khác chuẩn baseline	Chấp nhận tạo mã không ràng buộc ngưỡng đơn hàng	**Pass**
`TC_FR17_06`	Verify creating a coupon with minimum max uses per user 1 (On-point U=1)	Positive (BVA)	`EP-17-31`	`BVA-17-07-ON` ($U=1$)	`max_uses_per_user=1`, các trường khác chuẩn baseline	Chấp nhận tạo mã giới hạn 1 lượt dùng / người	**Pass**
`TC_FR17_07`	Verify creating a coupon with code length exactly 1 char (On-point Len=1)	Positive (BVA)	`EP-17-01`	`BVA-17-01-ON` ($Len=1$)	`code="A"`, các trường khác chuẩn baseline	Chấp nhận tạo mã với độ dài 1 ký tự	**Pass**
`TC_FR17_08`	Verify viewing coupon list in Admin management table	Positive (CRUD)	`VAR-17-01..06`, `AUTH-17-01`	N/A (List View)	Đăng nhập Admin, mở tab Mã Giảm Giá	Bảng hiển thị đầy đủ các cột: Mã, Loại, Giá trị, Đơn tối thiểu, Hết hạn, Giới hạn/người, Hành động	**Pass**
`TC_FR17_09`	Verify deleting an existing coupon by ID via Admin Web & API	Positive (CRUD)	`RES-17-01`, `AUTH-17-01`, `CPN-BR-13`	N/A (Delete Flow)	Bấm "Xóa" tại coupon vừa tạo (`DELETE /api/admin/coupons/:id`)	HTTP 200 OK, xóa thành công, bản ghi biến mất khỏi bảng Admin	**Pass**

TC ID	Test Case Objective ($\text{Action} + \text{Function} + \text{Condition}$)	Phân loại	Mã EP / Rule bao phủ	Mã BVA bao phủ	Dữ liệu đầu vào & Baseline	Kết quả cốt lõi (Core Expected)	Trạng thái
TC_FR17_10	Validate rejection when creating coupon with duplicate code (Single Fault)	Negative (IEP)	EP-17-02	N/A (Unique Constraint)	code="SAVE10" (đã có trong DB), các trường khác chuẩn	Báo lỗi mã coupon đã tồn tại; API trả về HTTP 400 hoặc 409, không sập 500	**Fail (BUG-FR17-01)**
TC_FR17_11	Validate rejection when coupon code is empty string (Boundary Off-point)	Negative (BVA)	EP-17-03	BVA-17-01-OFF ($Len=0$)	code="", các trường khác chuẩn	Báo lỗi trường bắt buộc không được để trống (UI tooltip / HTTP 400)	**Fail (BUG-FR17-02)**
TC_FR17_12	Validate rejection when coupon code contains only whitespace (Single Fault)	Negative (IEP)	EP-17-04	N/A (Whitespace)	code=" ", các trường khác chuẩn	Hệ thống trim khoảng trắng và từ chối mã rỗng; HTTP 400	**Fail (BUG-FR17-03)**
TC_FR17_13	Validate rejection when creating coupon with invalid type enum (Single Fault)	Negative (IEP)	EP-17-11	N/A (Enum)	type="free_shipping", các trường khác chuẩn	API từ chối với HTTP 400 Bad Request (Invalid type enum)	**Fail (BUG-FR17-04)**
TC_FR17_14	Validate rejection when discount value is 0 for percent type (Boundary On-point)	Negative (BVA)	EP-17-17	BVA-17-03-ON ($V=0$)	type="percent", val=0, các trường khác chuẩn	Hệ thống từ chối; báo lỗi giá trị giảm phải là số dương > 0 (HTTP 400)	**Fail (BUG-FR17-05)**
TC_FR17_15	Validate rejection when discount value is negative for percent type (Single Fault)	Negative (IEP)	EP-17-18	N/A (Negative Range)	type="percent", val=-10, các trường khác chuẩn	Hệ thống từ chối số âm; không để xảy ra lỗi làm tăng giá trị đơn hàng	**Fail (BUG-FR17-06)**
TC_FR17_16	Validate rejection when discount value exceeds 100% at 101% for percent type (Boundary Off-point)	Negative (BVA)	EP-17-19	BVA-17-04-OFF ($V=101%$)	type="percent", val=101, các trường khác chuẩn	Hệ thống từ chối (HTTP 400); tỷ lệ % không thể vượt quá 100%	**Fail (BUG-FR17-07)**
TC_FR17_17	Validate rejection when discount value is 0 for fixed type (Boundary On-point)	Negative (BVA)	EP-17-17	BVA-17-05-ON ($V=0\text{ đ}$)	type="fixed", val=0, các trường khác chuẩn	Hệ thống từ chối; giá trị giảm tiền mặt phải là số dương > 0	**Fail (BUG-FR17-08)**
TC_FR17_18	Validate rejection when min order amount is negative at -1 (Boundary Off-point)	Negative (BVA)	EP-17-28	BVA-17-06-OFF ($M=-1\text{ đ}$)	min_order_amount=-1, các trường khác chuẩn	Hệ thống từ chối; báo lỗi ngưỡng đơn hàng phải lớn hơn hoặc bằng 0	**Fail (BUG-FR17-09)**
TC_FR17_19	Validate rejection when max uses per user is 0 (Boundary Off-point)	Negative (BVA)	EP-17-32	BVA-17-07-OFF ($U=0$)	max_uses_per_user=0, các trường khác chuẩn	Hệ thống từ chối; số lượt dùng tối đa của mỗi người dùng phải ≥ 1	**Fail (BUG-FR17-10)**
TC_FR17_20	Validate rejection when max uses per user is negative (Single Fault)	Negative (IEP)	EP-17-33	N/A (Negative Range)	max_uses_per_user=-1, các trường khác chuẩn	Hệ thống từ chối số âm; HTTP 400 Bad Request	**Fail (BUG-FR17-11)**
TC_FR17_21	Validate handling when expired_at is set in the past (Boundary Off-point)	Negative / Probe	EP-17-37	BVA-17-08-OFF (yesterday)	expired_at="2020-01-01", các trường khác chuẩn	Hệ thống từ chối tạo mã hoặc gắn nhãn hết hạn ngay; cảnh báo Admin	**Pass**
TC_FR17_22	Validate rejection when unauthenticated request calls create coupon API (Single Fault)	Negative (Security)	AUTH-17-02	N/A (No Auth Header)	Gửi POST /api/admin/coupons không kèm Authorization token	API từ chối với HTTP 401 Unauthorized; không ghi dữ liệu	**Pass**
TC_FR17_23	Validate rejection when regular user with role 'user' calls admin coupon API (Single Fault)	Negative (Security)	AUTH-17-03	N/A (Role Escalation)	Gửi POST /api/admin/coupons với token của test@eshop.com	API từ chối với HTTP 403 Forbidden theo SEC-03; cấm leo thang đặc quyền	**Fail (BUG-FR17-12)**
TC_FR17_24	Validate rejection when deleting non-existent coupon ID 9999 (Single Fault)	Negative (IEP)	RES-17-02	N/A (Non-existent ID)	DELETE /api/admin/coupons/9999 với token Admin	API trả về HTTP 404 Not Found (hoặc lỗi rõ ràng); không sập server	**Fail (BUG-FR17-13)**
TC_FR17_25	Validate rejection when deleting coupon with negative ID -1 (Single Fault)	Negative (IEP)	RES-17-03	N/A (Negative Route Param)	DELETE /api/admin/coupons/-1 với token Admin	API từ chối với HTTP 400 Bad Request; ID phải là số dương	**Fail (BUG-FR17-14)**
TC_FR17_26	Validate rejection when deleting coupon with malformed string ID 'abc' (Single Fault)	Negative (IEP)	RES-17-05	N/A (Malformed Route Param)	DELETE /api/admin/coupons/abc với token Admin	API từ chối với HTTP 400 Bad Request do sai kiểu dữ liệu	**Fail (BUG-FR17-15)**

TC ID	Test Case Objective ($\text{Action} + \text{Function} + \text{Condition}$)	Phân loại	Mã EP / Rule bao phủ	Mã BVA bao phủ	Dữ liệu đầu vào & Baseline	Kết quả cốt lõi (Core Expected)	Trạng thái
`TC_FR17_27`	Verify applying percent coupon discount arithmetic calculation at checkout	Positive (Checkout)	`CPN-BR-01`	N/A (Formula Calc)	Đơn hàng 1.000.000 ₫; áp dụng mã 10%	Giảm đúng 100.000 ₫; tổng thanh toán cập nhật thành 900.000 ₫	**Fail (BUG-FR17-16)**
`TC_FR17_28`	Verify applying fixed coupon discount arithmetic calculation at checkout	Positive (Checkout)	`CPN-BR-02`	N/A (Formula Calc)	Đơn hàng 1.000.000 ₫; áp dụng mã 50.000 ₫	Giảm đúng 50.000 ₫; tổng thanh toán cập nhật thành 950.000 ₫	**Pass**
`TC_FR17_29`	Validate rejection when order total is below min_order_amount at boundary offset M-1 ₫	Negative (BVA)	`CPN-BR-03`	`BVA-17-09-OFF` ($M-1\text{ ₫}$)	`min=300.000 ₫`; đơn hàng 299.999 ₫	Từ chối áp dụng; thông báo đơn hàng chưa đủ ngưỡng tối thiểu (C3)	**Pass**
`TC_FR17_30`	Verify acceptance when order total equals exact min_order_amount at boundary On-point M	Positive (BVA)	`CPN-BR-04`	`BVA-17-09-ON` ($M=300.000\text{ ₫}$)	`min=300.000 ₫`; đơn hàng đúng 300.000 ₫	Chấp nhận áp dụng mã thành công (thỏa mãn điều kiện toán học $\ge$)	**Fail (BUG-FR17-17)**
`TC_FR17_31`	Validate rejection when applying expired coupon at checkout	Negative (Checkout)	`CPN-BR-06`	N/A (Expired State)	Áp dụng mã `EXPIRED` (`expired_at='2020-01-01'`)	Từ chối áp dụng; thông báo mã giảm giá đã hết hạn sử dụng (C2)	**Pass**
`TC_FR17_32`	Validate rejection when user usage count reaches max_uses_per_user at boundary On-point	Negative (BVA)	`CPN-BR-09`	`BVA-17-10-ON` ($used=U$)	`max=1`; user đã dùng 1 lần trong CSDL	Từ chối áp dụng; thông báo người dùng đã dùng hết lượt cho phép (C5)	**Pass**
`TC_FR17_33`	Verify acceptance when user usage count is max_uses_per_user - 1 at boundary Off-point	Positive (BVA)	`CPN-BR-08`	`BVA-17-10-OFF` ($used=U-1$)	`max=2`; user đã dùng 1 lần trong CSDL	Chấp nhận áp dụng mã cho lần thanh toán thứ 2 ($1 < 2$)	**Pass**
`TC_FR17_34`	Validate system handling when fixed discount exceeds order total (Financial Boundary Defect Probe)	Boundary Probe	`CPN-BR-10`	`BVA-17-11-OFF` ($discount>Total$)	Đơn hàng 30.000 ₫; áp dụng mã fixed 50.000 ₫	Tổng tiền đơn hàng giảm tối đa về 0 ₫; tuyệt đối KHÔNG bị âm tiền	**Fail (BUG-FR17-18)**
`TC_FR17_35`	Validate stored XSS escaping when viewing coupon with HTML tags on Admin Web	Negative (Security)	`EP-17-06`, `SEC-17-01`	N/A (XSS Probe)	`code="<script>alert('XSS')</script>"`	Giao diện escape thẻ HTML (SEC-04); hiển thị text, không chạy mã script	**Pass**
`TC_FR17_36`	Validate race condition handling when creating duplicate coupon codes simultaneously	Concurrency	`CPN-BR-14`	N/A (Race Condition)	2 request `POST` cùng mã `code="FLASH50"` trong 100ms	Đúng 1 request thành công; request thứ 2 bị từ chối; CSDL duy nhất 1 bản ghi	**Pass**

4.4.2 Hồ Sơ Đặc Tả Chi Tiết Từng Kịch Bản Kiểm Thử (Detailed Test Specification Cards - 100% Exhaustive Coverage)

Dưới đây là hồ sơ đặc tả chi tiết độc lập cho toàn bộ 36 kịch bản kiểm thử (`TC_FR17_01` đến `TC_FR17_36`) theo chuẩn 7 phân đoạn: Tracking Information, Test Objective, Pre-conditions & Environment, Test Data, Step-by-Step Procedure, Multi-tier Validation Points (UI, API, Database SQLite, Security / Business Rules), và Teardown & Clean-up:

Test Case `TC_FR17_01`: Verify creating a valid percent coupon with standard baseline inputs on Admin Web & API

- **1. Tracking Information:**
  - **TC ID:** `TC_FR17_01` | **Priority:** High (P1) | **Target Layer:** Admin Web (`/coupons`) & REST API (`POST /api/admin/coupons`)
  - **Traceability:** SRS FR-17, API Spec §6.4
  - **Coverage IDs:** `EP-17-01`, `EP-17-09`, `EP-17-15`, `EP-17-26`, `EP-17-31`, `EP-17-36`, `AUTH-17-01`, `CPN-BR-12`, `BVA-17-03-IN`
  - **Single-Fault Isolation:** Kịch bản Baseline hợp lệ tiêu chuẩn phối hợp các phân vùng VEP hợp lệ cơ sở để tạo coupon loại phần trăm.
- **2. Test Objective:** Verify creating a valid percent coupon with standard baseline inputs on Admin Web & API.
- **3. Pre-conditions & Environment:**
  - Hệ thống EShop đang chạy (Backend `:3000`, Admin Web `:5174` hoặc `:5173`).
  - Đã đăng nhập tài khoản Quản trị viên (`admin@eshop.com`), lưu token vào `localStorage.getItem('adminToken')`.
  - CSDL SQLite chưa tồn tại mã `"PROMO2026"`.
- **4. Test Data:**

```
{
  "code": "PROMO2026",
  "type": "percent",
  "discount_value": 10,
  "min_order_amount": 100000,
  "expired_at": "2026-12-31",
  "max_uses_per_user": 1
}
```

- **5. Step-by-Step Procedure:**
  1. Đăng nhập Admin Web, điều hướng vào mục "Mã Giảm Giá".
  2. Tại form "Tạo mã giảm giá mới", điền đầy đủ các trường theo Test Data trên.

3. Nhấn nút "Tạo mã" (kích hoạt `POST /api/admin/coupons`).

- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Admin Web:* Form tự động reset về giá trị mặc định. Bảng danh sách lập tức xuất hiện dòng mới mang mã `PROMO2026`, loại "Phần trăm", giá trị `10%`, đơn tối thiểu `100.000 ₫`, hết hạn `2026-12-31`, giới hạn `1 lần`.
  - *API / Network:* HTTP Status trả về `200 OK` (hoặc `201 Created`). Response JSON: `{"message": "Coupon created", "id": <number>}`.
  - *Database State:* Câu lệnh `SELECT * FROM coupons WHERE code='PROMO2026'` trả về đúng 1 bản ghi với `type='percent'`, `discount_value=10`, `min_order_amount=100000`, `max_uses_per_user=1`, `is_active=1`.
  - *Security & Business Rules:* Xác thực quyền admin thành công (`AUTH-17-01`); thiết lập đúng trạng thái khả dụng ban đầu.
- **7. Teardown & Clean-up:** Xóa coupon `PROMO2026` bằng nút "Xóa" trên Admin Web hoặc chạy SQL: `DELETE FROM coupons WHERE code='PROMO2026'`.

Test Case `TC_FR17_02`: Verify creating a valid fixed amount coupon on Admin Web & API

- **1. Tracking Information:**
  - **TC ID:** `TC_FR17_02` | **Priority:** High (P1) | **Target Layer:** Admin Web & REST API
  - **Traceability:** SRS FR-17, API Spec §6.4
  - **Coverage IDs:** `EP-17-01`, `EP-17-10`, `EP-17-16`, `EP-17-26`, `EP-17-31`, `EP-17-36`, `AUTH-17-01`, `CPN-BR-12`, `BVA-17-05-IN`
  - **Single-Fault Isolation:** Baseline hợp lệ tiêu chuẩn cho loại giảm giá tiền mặt cố định (`fixed`).
- **2. Test Objective:** Verify creating a valid fixed amount coupon on Admin Web & API.
- **3. Pre-conditions & Environment:** Admin đã đăng nhập với JWT hợp lệ; CSDL chưa có mã `"FIXED50K"`.
- **4. Test Data:**

```
{
  "code": "FIXED50K",
  "type": "fixed",
  "discount_value": 50000,
  "min_order_amount": 100000,
  "expired_at": "2026-12-31",
  "max_uses_per_user": 1
}
```

- **5. Step-by-Step Procedure:**
  1. Mở trang quản trị "Mã Giảm Giá".
  2. Chọn loại "Cố định", nhập mã `FIXED50K` và giá trị `50000`.
  3. Bấm "Tạo mã".
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Admin Web:* Bảng quản trị cập nhật thêm dòng mới với mã `FIXED50K`, loại `Cố định`, giá trị giảm `50.000 ₫`.
  - *API / Network:* HTTP 200/201 OK, trả về ID của coupon mới tạo.
  - *Database State:* Bản ghi được lưu chính xác với `type='fixed'`, `discount_value=50000`.
  - *Security & Business Rules:* Giảm giá tiền mặt được lưu nguyên vẹn, không chuyển đổi nhầm sang tỷ lệ phần trăm.
- **7. Teardown & Clean-up:** Xóa mã `FIXED50K` (`DELETE FROM coupons WHERE code='FIXED50K'`).

Test Case `TC_FR17_03`: Verify creating a percent coupon at upper boundary 100% (On-point)

- **1. Tracking Information:**
  - **TC ID:** `TC_FR17_03` | **Priority:** High (P1) | **Target Layer:** Admin Web & Backend Validation
  - **Traceability:** SRS FR-17, Quy tắc trần phần trăm tối đa 100%
  - **Coverage IDs:** `EP-17-15`, `BVA-17-04-ON` ($V=100%$)
  - **Single-Fault Isolation:** Điểm biên trên On-point của tỷ lệ phần trăm ($V=100%$); các trường khác chuẩn baseline.
- **2. Test Objective:** Verify creating a percent coupon at upper boundary 100% (On-point).
- **3. Pre-conditions & Environment:** Admin đã đăng nhập; CSDL chưa có mã `"FREE100"`.
- **4. Test Data:** `code="FREE100"`, `type="percent"`, `discount_value=100`, `min_order_amount=0`, `max_uses_per_user=1`, `expired_at="2026-12-31"`.
- **5. Step-by-Step Procedure:**
  1. Nhập thông tin mã giảm 100% vào form Admin.
  2. Gửi request tạo mã.
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Admin Web:* Chấp nhận tạo mã, bảng hiển thị mã giảm `100%`.
  - *API / Network:* HTTP 200/201 OK.
  - *Database State:* Bản ghi lưu `discount_value=100`, `type='percent'`.
  - *Security & Business Rules:* Thỏa mãn trần 100% của coupon tỷ lệ; cho phép tạo đơn miễn phí toàn bộ giá trị hàng khi áp dụng.
- **7. Teardown & Clean-up:** Xóa mã `FREE100`.

Test Case `TC_FR17_04`: Verify creating a percent coupon at lower boundary 1% (Off-point of >0)

- **1. Tracking Information:**
  - **TC ID:** `TC_FR17_04` | **Priority:** High (P1) | **Target Layer:** Admin Web & Backend Validation
  - **Traceability:** SRS FR-17 (*"discount_value (dương)"*), BVA $\epsilon=1%$
  - **Coverage IDs:** `EP-17-15`, `BVA-17-03-OFF` ($V=1%$)
  - **Single-Fault Isolation:** Điểm biên dưới hợp lệ nhỏ nhất (Off-point của >0) với $\epsilon=1%$; các trường khác chuẩn baseline.
- **2. Test Objective:** Verify creating a percent coupon at lower boundary 1% (Off-point of >0).
- **3. Pre-conditions & Environment:** Admin đã đăng nhập; CSDL chưa có mã `"MIN1PCT"`.
- **4. Test Data:** `code="MIN1PCT"`, `type="percent"`, `discount_value=1`, `min_order_amount=50000`, `max_uses_per_user=1`, `expired_at="2026-12-31"`.
- **5. Step-by-Step Procedure:**
  1. Nhập form tạo coupon với `discount_value=1`.
  2. Bấm "Tạo mã".

- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Admin Web:* Tạo thành công, bảng hiển thị `1%`.
  - *API / Network:* HTTP 200/201 OK.
  - *Database State:* `discount_value=1`, `type='percent'`.
  - *Security & Business Rules:* Thỏa mãn điều kiện dương >0 theo định nghĩa toán học rời rạc.
- **7. Teardown & Clean-up:** Xóa mã `MIN1PCT`.

Test Case `TC_FR17_05`: Verify creating a coupon with zero minimum order amount (On-point M=0)

- **1. Tracking Information:**
  - **TC ID:** `TC_FR17_05` | **Priority:** High (P1) | **Target Layer:** Admin Web & Backend Validation
  - **Traceability:** SRS FR-17 (*"min_order_amount >= 0"*), BVA On-point
  - **Coverage IDs:** `EP-17-27`, `BVA-17-06-ON` ($M=0\text{ ₫}$)
  - **Single-Fault Isolation:** Điểm biên dưới On-point của ngưỡng đơn hàng tối thiểu ($M=0\text{ ₫}$); các trường khác chuẩn baseline.
- **2. Test Objective:** Verify creating a coupon with zero minimum order amount (On-point M=0).
- **3. Pre-conditions & Environment:** Admin đã đăng nhập; CSDL chưa có mã `"NO_MIN"`.
- **4. Test Data:** `code="NO_MIN"`, `type="fixed"`, `discount_value=20000`, `min_order_amount=0`, `max_uses_per_user=1`, `expired_at="2026-12-31"`.
- **5. Step-by-Step Procedure:**
  1. Nhập form Admin với `min_order_amount = 0`.
  2. Bấm "Tạo mã".
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Admin Web:* Chấp nhận tạo mã, hiển thị đơn tối thiểu `0 ₫` (hoặc "Không giới hạn").
  - *API / Network:* HTTP 200/201 OK.
  - *Database State:* Bản ghi lưu `min_order_amount = 0`.
  - *Security & Business Rules:* Coupon áp dụng được cho mọi đơn hàng có giá trị bất kỳ mà không bị chặn bởi ngưỡng tiền.
- **7. Teardown & Clean-up:** Xóa mã `NO_MIN`.

Test Case `TC_FR17_06`: Verify creating a coupon with minimum max uses per user 1 (On-point U=1)

- **1. Tracking Information:**
  - **TC ID:** `TC_FR17_06` | **Priority:** High (P1) | **Target Layer:** Admin Web & Backend Validation
  - **Traceability:** SRS FR-17 (*"max_uses_per_user >= 1"*), BVA On-point
  - **Coverage IDs:** `EP-17-31`, `BVA-17-07-ON` ($U=1$)
  - **Single-Fault Isolation:** Điểm biên dưới On-point của số lượt dùng tối đa ($U=1\text{ lượt}$); các trường khác chuẩn baseline.
- **2. Test Objective:** Verify creating a coupon with minimum max uses per user 1 (On-point U=1).
- **3. Pre-conditions & Environment:** Admin đã đăng nhập; CSDL chưa có mã `"ONE_TIME"`.
- **4. Test Data:** `code="ONE_TIME"`, `type="percent"`, `discount_value=15`, `min_order_amount=100000`, `max_uses_per_user=1`, `expired_at="2026-12-31"`.
- **5. Step-by-Step Procedure:**
  1. Nhập `max_uses_per_user = 1` trên form Admin.
  2. Gửi request tạo mã.
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Admin Web:* Bảng hiển thị giới hạn `1 lần/người`.
  - *API / Network:* HTTP 200/201 OK.
  - *Database State:* `max_uses_per_user = 1`.
  - *Security & Business Rules:* Thỏa mãn quy định tối thiểu 1 lượt dùng cho mỗi khách hàng.
- **7. Teardown & Clean-up:** Xóa mã `ONE_TIME`.

Test Case `TC_FR17_07`: Verify creating a coupon with code length exactly 1 char (On-point Len=1)

- **1. Tracking Information:**
  - **TC ID:** `TC_FR17_07` | **Priority:** Medium (P2) | **Target Layer:** Admin Web & Backend Validation
  - **Traceability:** SRS FR-17, BVA On-point Len=1
  - **Coverage IDs:** `EP-17-01`, `BVA-17-01-ON` ($Len=1$)
  - **Single-Fault Isolation:** Mã có độ dài nhỏ nhất 1 ký tự (`Len=1`); các trường khác chuẩn baseline.
- **2. Test Objective:** Verify creating a coupon with code length exactly 1 char (On-point Len=1).
- **3. Pre-conditions & Environment:** Admin đã đăng nhập; CSDL chưa có mã `"A"`.
- **4. Test Data:** `code="A"`, `type="fixed"`, `discount_value=10000`, `min_order_amount=50000`, `max_uses_per_user=1`, `expired_at="2026-12-31"`.
- **5. Step-by-Step Procedure:**
  1. Nhập mã `code="A"` vào form Admin.
  2. Bấm "Tạo mã".
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Admin Web:* Chấp nhận tạo mã, bảng hiển thị mã `A`.
  - *API / Network:* HTTP 200/201 OK.
  - *Database State:* `SELECT * FROM coupons WHERE code='A'` có 1 bản ghi.
  - *Security & Business Rules:* Xử lý chuỗi độ dài 1 ký tự an toàn, không bị lỗi cắt chuỗi hay validation rỗng.
- **7. Teardown & Clean-up:** Xóa mã `A`.

Test Case `TC_FR17_08`: Verify viewing coupon list in Admin management table

- **1. Tracking Information:**
  - **TC ID:** `TC_FR17_08` | **Priority:** High (P1) | **Target Layer:** Admin Web (`/coupons`) & REST API (`GET /api/admin/coupons`)
  - **Traceability:** SRS FR-17 (*"Admin có thể Xem danh sách mã"*), API Spec §6.4
  - **Coverage IDs:** `VAR-17-01..06`, `AUTH-17-01`

    - **Single-Fault Isolation:** Luồng đọc dữ liệu tổng thể (Read Operation); không sửa đổi trạng thái dữ liệu.
- **2. Test Objective:** Verify viewing coupon list in Admin management table.
- **3. Pre-conditions & Environment:** Admin đã đăng nhập; CSDL có sẵn ít nhất 2 coupon mẫu (`PROMO2026` , `FIXED50K`).
- **4. Test Data:** Gửi request `GET /api/admin/coupons` với token Admin.
- **5. Step-by-Step Procedure:**
    1. Đăng nhập Admin Web.
    2. Nhấp vào tab "Mã Giảm Giá" trên thanh điều hướng.
    3. Kiểm tra các dòng và cột hiển thị trên bảng quản trị.
- **6. Multi-tier Validation Points (Expected Results):**
    - *UI / Admin Web:* Bảng render đầy đủ các cột: Mã coupon, Loại giảm giá, Giá trị giảm, Đơn tối thiểu, Hạn dùng, Giới hạn/người, và Nút hành động (Xóa).
    - *API / Network:* HTTP 200 OK, trả về mảng JSON chứa danh sách các coupon trong CSDL.
    - *Database State:* Không có thao tác ghi hoặc xóa nào diễn ra trong CSDL.
    - *Security & Business Rules:* Chỉ người dùng có quyền Admin mới xem được danh sách mã này.
- **7. Teardown & Clean-up:** Không cần dọn dẹp dữ liệu.

Test Case `TC_FR17_09` : Verify deleting an existing coupon by ID via Admin Web & API

- **1. Tracking Information:**
    - **TC ID:** `TC_FR17_09` | **Priority:** High (P1) | **Target Layer:** Admin Web & Backend API (`DELETE /api/admin/coupons/:id`)
    - **Traceability:** SRS FR-17 (*"Admin có thể Xóa mã"*), API Spec §6.4
    - **Coverage IDs:** `RES-17-01` , `AUTH-17-01` , `CPN-BR-13`
    - **Single-Fault Isolation:** Luồng xóa tài nguyên hợp lệ với ID tồn tại; người dùng có đủ quyền Admin.
- **2. Test Objective:** Verify deleting an existing coupon by ID via Admin Web & API.
- **3. Pre-conditions & Environment:** Đang có coupon tồn tại trong CSDL với ID xác định (ví dụ `id = 5`); Admin đã đăng nhập.
- **4. Test Data:** `id = 5` (hoặc ID của coupon vừa tạo trong fixture).
- **5. Step-by-Step Procedure:**
    1. Tìm dòng coupon có ID tương ứng trên bảng Admin.
    2. Bấm nút "Xóa" bên cạnh.
    3. Xác nhận trên popup dialog xác nhận xóa.
- **6. Multi-tier Validation Points (Expected Results):**
    - *UI / Admin Web:* Dòng coupon biến mất khỏi bảng danh sách; hiển thị thông báo "Xóa mã giảm giá thành công".
    - *API / Network:* `DELETE /api/admin/coupons/5` trả về HTTP 200 OK, payload `{"message": "Coupon deleted"}` .
    - *Database State:* `SELECT * FROM coupons WHERE id=5` không còn bản ghi (hoặc `is_active=0` nếu soft-delete).
    - *Security & Business Rules:* Sau khi xóa, nhập mã này tại Checkout nhận thông báo mã không tồn tại.
- **7. Teardown & Clean-up:** Dữ liệu đã được xóa sạch.

Test Case `TC_FR17_10` : Validate rejection when creating coupon with duplicate code (Single Fault)

- **1. Tracking Information:**
    - **TC ID:** `TC_FR17_10` | **Priority:** High (P1) | **Target Layer:** Backend API & SQLite Constraint Guard
    - **Traceability:** SRS FR-17 (*"Mã coupon là duy nhất"*), DB Schema `UNIQUE(code)`
    - **Coverage IDs:** `EP-17-02`
    - **Single-Fault Isolation:** Chỉ thay đổi trường `code` trùng với mã đã có sẵn trong DB (`SAVE10`); tất cả các trường khác giữ nguyên giá trị hợp lệ cơ sở.
- **2. Test Objective:** Validate rejection when creating coupon with duplicate code (Single Fault).
- **3. Pre-conditions & Environment:** Mã `SAVE10` đã tồn tại trong CSDL; Admin đã đăng nhập.
- **4. Test Data:**

```
{
  "code": "SAVE10",
  "type": "percent",
  "discount_value": 15,
  "min_order_amount": 100000,
  "expired_at": "2026-12-31",
  "max_uses_per_user": 1
}
```

- **5. Step-by-Step Procedure:**
    1. Điền form tạo mã với `code="SAVE10"` .
    2. Nhấn nút "Tạo mã".
- **6. Multi-tier Validation Points (Expected Results):**
    - *UI / Admin Web:* Hiển thị thông báo lỗi "Mã giảm giá này đã tồn tại trong hệ thống".
    - *API / Network:* Trả về HTTP 400 Bad Request hoặc 409 Conflict. **SUT Defect Alert:** Không được trả về HTTP 500 do unhandled SQLite UNIQUE constraint error.
    - *Database State:* Bảng `coupons` giữ nguyên bản ghi cũ; không có bản ghi thứ 2 mang mã `SAVE10` .
    - *Security & Business Rules:* Bảo vệ tính duy nhất của mã nhận diện coupon.
- **7. Teardown & Clean-up:** Giữ nguyên bản ghi ban đầu.

Test Case `TC_FR17_11` : Validate rejection when coupon code is empty string (Boundary Off-point)

- **1. Tracking Information:**
    - **TC ID:** `TC_FR17_11` | **Priority:** High (P1) | **Target Layer:** Admin Form & Backend API
    - **Traceability:** SRS FR-17 (*"code (bắt buộc)"*), BVA Off-point Len=0
    - **Coverage IDs:** `EP-17-03` , `BVA-17-01-OFF` ($Len=0$)
    - **Single-Fault Isolation:** Trường `code` rỗng `""` ; các trường khác hợp lệ theo baseline.

- **2. Test Objective:** Validate rejection when coupon code is empty string (Boundary Off-point).
- **3. Pre-conditions & Environment:** Admin đã đăng nhập.
- **4. Test Data:** `code=""`, `type="percent",` discount_value=10`,` min_order_amount=100000`,` expired_at="2026-12-31"`,` max_uses_per_user=1`.
- **5. Step-by-Step Procedure:**
  1. Để trống ô nhập mã coupon trên form Admin.
  2. Điền các trường khác hợp lệ rồi bấm "Tạo mã".
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Admin Web:* Form chặn submit với tooltip HTML5 "Vui lòng điền vào trường này" (hoặc hiển thị thông báo lỗi validation dưới trường mã).
  - *API / Network:* Gửi request trực tiếp nhận HTTP 400 Bad Request kèm thông báo lỗi rõ ràng.
  - *Database State:* Không có bản ghi mới nào được lưu vào CSDL.
  - *Security & Business Rules:* Không cho phép tạo coupon không có mã định danh.
- **7. Teardown & Clean-up:** Không cần dọn dẹp.

Test Case `TC_FR17_12` : Validate rejection when coupon code contains only whitespace (Single Fault)

- **1. Tracking Information:**
  - **TC ID:** `TC_FR17_12` | **Priority:** Medium (P2) | **Target Layer:** Admin Web & Backend Trim Validation
  - **Traceability:** SRS FR-17 (*"code (bắt buộc)"*), Robustness Requirement
  - **Coverage IDs:** `EP-17-04`
  - **Single-Fault Isolation:** Trường `code` chỉ chứa khoảng trắng `" "`; các trường khác hợp lệ.
- **2. Test Objective:** Validate rejection when coupon code contains only whitespace (Single Fault).
- **3. Pre-conditions & Environment:** Admin đã đăng nhập.
- **4. Test Data:** `code=" "`, `type="percent"`, `discount_value=10`, `min_order_amount=100000`, `expired_at="2026-12-31"`, `max_uses_per_user=1`.
- **5. Step-by-Step Procedure:**
  1. Nhập chuỗi gồm 5 dấu cách vào ô "Mã giảm giá".
  2. Bấm "Tạo mã".
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Admin Web:* Báo lỗi mã không hợp lệ sau khi trim khoảng trắng.
  - *API / Network:* HTTP 400 Bad Request (`{"error": "Coupon code cannot be empty"}`).
  - *Database State:* Không có bản ghi nào với chuỗi khoảng trắng được tạo trong bảng `coupons`.
  - *Security & Business Rules:* Chống kỹ thuật bypass validation chuỗi rỗng bằng khoảng trắng.
- **7. Teardown & Clean-up:** Không cần dọn dẹp.

Test Case `TC_FR17_13` : Validate rejection when creating coupon with invalid type enum (Single Fault)

- **1. Tracking Information:**
  - **TC ID:** `TC_FR17_13` | **Priority:** High (P1) | **Target Layer:** REST API Validation Guard
  - **Traceability:** SRS FR-17 (*"type: percent hoặc fixed"*), API Spec §6.4
  - **Coverage IDs:** `EP-17-11`
  - **Single-Fault Isolation:** Trường `type` mang giá trị enum không hỗ trợ (`free_shipping`); các trường khác chuẩn baseline.
- **2. Test Objective:** Validate rejection when creating coupon with invalid type enum (Single Fault).
- **3. Pre-conditions & Environment:** Admin đã đăng nhập.
- **4. Test Data:** `code="FREESHIP"`, `type="free_shipping"`, `discount_value=20000`, `min_order_amount=100000`, `expired_at="2026-12-31"`, `max_uses_per_user=1`.
- **5. Step-by-Step Procedure:**
  1. Gửi request `POST /api/admin/coupons` với payload chứa `type="free_shipping"`.
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Admin Web:* Dropdown trên UI chỉ cung cấp 2 lựa chọn (Percent/Fixed).
  - *API / Network:* HTTP 400 Bad Request (`Invalid coupon type. Must be 'percent' or 'fixed'`).
  - *Database State:* Không có bản ghi nào được ghi vào CSDL.
  - *Security & Business Rules:* Bảo toàn tính toàn vẹn kiểu dữ liệu của enum `type`.
- **7. Teardown & Clean-up:** Không cần dọn dẹp.

Test Case `TC_FR17_14` : Validate rejection when discount value is 0 for percent type (Boundary On-point)

- **1. Tracking Information:**
  - **TC ID:** `TC_FR17_14` | **Priority:** High (P1) | **Target Layer:** Form Validation & Backend API
  - **Traceability:** SRS FR-17 (*"discount_value (dương)"*), BVA On-point V=0
  - **Coverage IDs:** `EP-17-17`, `BVA-17-03-ON` ($V=0$)
  - **Single-Fault Isolation:** `discount_value = 0` cho loại `percent`; các trường khác chuẩn baseline.
- **2. Test Objective:** Validate rejection when discount value is 0 for percent type (Boundary On-point).
- **3. Pre-conditions & Environment:** Admin đã đăng nhập.
- **4. Test Data:** `code="PCTZERO"`, `type="percent"`, `discount_value=0`, `min_order_amount=100000`, `expired_at="2026-12-31"`, `max_uses_per_user=1`.
- **5. Step-by-Step Procedure:**
  1. Nhập `discount_value = 0` với loại `percent`.
  2. Bấm "Tạo mã".
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Admin Web:* Báo lỗi giá trị giảm phải lớn hơn 0.
  - *API / Network:* HTTP 400 Bad Request (`Discount value must be positive`).
  - *Database State:* Không có bản ghi nào được tạo trong `coupons`.
  - *Security & Business Rules:* Tuân thủ ràng buộc giá trị giảm phải là số dương > 0.
- **7. Teardown & Clean-up:** Không cần dọn dẹp.

Test Case `TC_FR17_15` : Validate rejection when discount value is negative for percent type (Single Fault)

- **1. Tracking Information:**
  - **TC ID:** `TC_FR17_15` | **Priority:** High (P1) | **Target Layer:** Form Validation & Backend API
  - **Traceability:** SRS FR-17 (*"discount_value (dương)"*), Negative Domain
  - **Coverage IDs:** `EP-17-18`
  - **Single-Fault Isolation:** `discount_value = -10` cho loại `percent`; các trường khác chuẩn baseline.
- **2. Test Objective:** Validate rejection when discount value is negative for percent type (Single Fault).
- **3. Pre-conditions & Environment:** Admin đã đăng nhập.
- **4. Test Data:** `code="PCTNEG"`, `type="percent"`, `discount_value=-10`, `min_order_amount=100000`, `expired_at="2026-12-31"`, `max_uses_per_user=1`.
- **5. Step-by-Step Procedure:**
  1. Nhập `discount_value = -10` với loại `percent`.
  2. Bấm "Tạo mã".
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Admin Web:* Báo lỗi số âm không hợp lệ.
  - *API / Network:* HTTP 400 Bad Request; từ chối xử lý số âm.
  - *Database State:* Không có bản ghi nào lưu số âm vào CSDL.
  - *Security & Business Rules:* Ngăn ngừa lỗi nghiệp vụ coupon âm làm tăng giá trị thanh toán của khách hàng.
- **7. Teardown & Clean-up:** Không cần dọn dẹp.

Test Case `TC_FR17_16` : Validate rejection when discount value exceeds 100% at 101% for percent type (Boundary Off-point)

- **1. Tracking Information:**
  - **TC ID:** `TC_FR17_16` | **Priority:** High (P1) | **Target Layer:** Form Validation & Backend API
  - **Traceability:** SRS FR-17, Tỷ lệ phần trăm tối đa 100%, BVA Off-point V=101%
  - **Coverage IDs:** `EP-17-19`, `BVA-17-04-OFF` ($V=101%$)
  - **Single-Fault Isolation:** `discount_value = 101` cho loại `percent`; các trường khác chuẩn baseline.
- **2. Test Objective:** Validate rejection when discount value exceeds 100% at 101% for percent type (Boundary Off-point).
- **3. Pre-conditions & Environment:** Admin đã đăng nhập.
- **4. Test Data:** `code="OVER101"`, `type="percent"`, `discount_value=101`, `min_order_amount=100000`, `expired_at="2026-12-31"`, `max_uses_per_user=1`.
- **5. Step-by-Step Procedure:**
  1. Nhập `discount_value = 101` cho mã tỷ lệ phần trăm.
  2. Bấm "Tạo mã".
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Admin Web:* Thuộc tính `max="100"` trên HTML5 hoặc Javascript chặn gửi; thông báo tỷ lệ không thể vượt quá 100%.
  - *API / Network:* HTTP 400 Bad Request (`Percentage discount cannot exceed 100%`).
  - *Database State:* Không tạo bản ghi `OVER101` trong bảng `coupons`.
  - *Security & Business Rules:* Giới hạn phần trăm giảm giá không vượt quá toàn bộ giá trị đơn hàng.
- **7. Teardown & Clean-up:** Không cần dọn dẹp.

Test Case `TC_FR17_17` : Validate rejection when discount value is 0 for fixed type (Boundary On-point)

- **1. Tracking Information:**
  - **TC ID:** `TC_FR17_17` | **Priority:** High (P1) | **Target Layer:** Form Validation & Backend API
  - **Traceability:** SRS FR-17 (*"discount_value (dương)"*), BVA On-point V=0 ₫
  - **Coverage IDs:** `EP-17-17`, `BVA-17-05-ON` ($V=0\text{ ₫}$)
  - **Single-Fault Isolation:** `discount_value = 0` cho loại `fixed`; các trường khác chuẩn baseline.
- **2. Test Objective:** Validate rejection when discount value is 0 for fixed type (Boundary On-point).
- **3. Pre-conditions & Environment:** Admin đã đăng nhập.
- **4. Test Data:** `code="FIXZERO"`, `type="fixed"`, `discount_value=0`, `min_order_amount=100000`, `expired_at="2026-12-31"`, `max_uses_per_user=1`.
- **5. Step-by-Step Procedure:**
  1. Chọn loại "Cố định", nhập `discount_value = 0`.
  2. Bấm "Tạo mã".
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Admin Web:* Báo lỗi số tiền giảm phải lớn hơn 0 ₫.
  - *API / Network:* HTTP 400 Bad Request.
  - *Database State:* Không ghi bản ghi vào CSDL.
  - *Security & Business Rules:* Đảm bảo số tiền giảm giá tiền mặt phải là số dương > 0.
- **7. Teardown & Clean-up:** Không cần dọn dẹp.

Test Case `TC_FR17_18` : Validate rejection when min order amount is negative at -1 (Boundary Off-point)

- **1. Tracking Information:**
  - **TC ID:** `TC_FR17_18` | **Priority:** High (P1) | **Target Layer:** Form Validation & Backend API
  - **Traceability:** SRS FR-17 (*"min_order_amount >= 0"*), BVA Off-point M=-1 ₫
  - **Coverage IDs:** `EP-17-28`, `BVA-17-06-OFF` ($M=-1\text{ ₫}$)
  - **Single-Fault Isolation:** `min_order_amount = -1`; các trường khác chuẩn baseline.
- **2. Test Objective:** Validate rejection when min order amount is negative at -1 (Boundary Off-point).
- **3. Pre-conditions & Environment:** Admin đã đăng nhập.
- **4. Test Data:** `code="NEGMIN"`, `type="percent"`, `discount_value=10`, `min_order_amount=-1`, `expired_at="2026-12-31"`, `max_uses_per_user=1`.
- **5. Step-by-Step Procedure:**

    1. Nhập `min_order_amount = -1` vào form Admin.
    2. Bấm "Tạo mã".
- **6. Multi-tier Validation Points (Expected Results):**
    - *UI / Admin Web:* Chặn nhập số âm bằng thuộc tính `min="0"` hoặc cảnh báo lỗi.
    - *API / Network:* HTTP 400 Bad Request (`Minimum order amount must be greater than or equal to 0`).
    - *Database State:* Không lưu bản ghi với giá trị âm vào CSDL.
    - *Security & Business Rules:* Bảo vệ tính hợp lý phi âm của ngưỡng giá trị đơn hàng.
- **7. Teardown & Clean-up:** Không cần dọn dẹp.

Test Case `TC_FR17_19` : Validate rejection when max uses per user is 0 (Boundary Off-point)

- **1. Tracking Information:**
    - **TC ID:** `TC_FR17_19` | **Priority:** High (P1) | **Target Layer:** Form Validation & Backend API
    - **Traceability:** SRS FR-17 (*"max_uses_per_user >= 1"*), BVA Off-point U=0
    - **Coverage IDs:** `EP-17-32` , `BVA-17-07-OFF` ($U=0$)
    - **Single-Fault Isolation:** `max_uses_per_user = 0` ; các trường khác chuẩn baseline.
- **2. Test Objective:** Validate rejection when max uses per user is 0 (Boundary Off-point).
- **3. Pre-conditions & Environment:** Admin đã đăng nhập.
- **4. Test Data:** `code="ZEROUSES"` , `type="percent"` , `discount_value=10` , `min_order_amount=100000` , `expired_at="2026-12-31"` , `max_uses_per_user=0` .
- **5. Step-by-Step Procedure:**
    1. Nhập `max_uses_per_user = 0` vào form Admin.
    2. Bấm "Tạo mã".
- **6. Multi-tier Validation Points (Expected Results):**
    - *UI / Admin Web:* Thuộc tính `min="1"` chặn nhập 0; báo lỗi số lượt dùng phải tối thiểu là 1.
    - *API / Network:* HTTP 400 Bad Request (`Max uses per user must be at least 1`).
    - *Database State:* Không tạo bản ghi trong bảng `coupons` .
    - *Security & Business Rules:* Tránh tạo coupon "rác" không bao giờ có thể được sử dụng bởi bất kỳ ai.
- **7. Teardown & Clean-up:** Không cần dọn dẹp.

Test Case `TC_FR17_20` : Validate rejection when max uses per user is negative (Single Fault)

- **1. Tracking Information:**
    - **TC ID:** `TC_FR17_20` | **Priority:** High (P1) | **Target Layer:** Form Validation & Backend API
    - **Traceability:** SRS FR-17 (*"max_uses_per_user >= 1"*), Negative Range
    - **Coverage IDs:** `EP-17-33`
    - **Single-Fault Isolation:** `max_uses_per_user = -1` ; các trường khác chuẩn baseline.
- **2. Test Objective:** Validate rejection when max uses per user is negative (Single Fault).
- **3. Pre-conditions & Environment:** Admin đã đăng nhập.
- **4. Test Data:** `code="NEGUSES"` , `type="percent"` , `discount_value=10` , `min_order_amount=100000` , `expired_at="2026-12-31"` , `max_uses_per_user=-1` .
- **5. Step-by-Step Procedure:**
    1. Gửi request `POST /api/admin/coupons` với `max_uses_per_user = -1` .
- **6. Multi-tier Validation Points (Expected Results):**
    - *UI / Admin Web:* Chặn nhập số âm trên form.
    - *API / Network:* HTTP 400 Bad Request.
    - *Database State:* Không có bản ghi nào được ghi vào CSDL.
    - *Security & Business Rules:* Đảm bảo số lượt dùng không mang giá trị âm gây sai lệch bộ đếm CSDL.
- **7. Teardown & Clean-up:** Không cần dọn dẹp.

Test Case `TC_FR17_21` : Validate handling when expired_at is set in the past (Boundary Off-point)

- **1. Tracking Information:**
    - **TC ID:** `TC_FR17_21` | **Priority:** Medium (P2) | **Target Layer:** Form Validation & Backend API
    - **Traceability:** SRS FR-17 (*"expired_at (ngày hợp lệ)"*), BVA Off-point (quá khứ)
    - **Coverage IDs:** `EP-17-37` , `BVA-17-08-OFF` (yesterday)
    - **Single-Fault Isolation:** `expired_at` đặt là ngày trong quá khứ (`2020-01-01`); các trường khác chuẩn baseline.
- **2. Test Objective:** Validate handling when expired_at is set in the past (Boundary Off-point).
- **3. Pre-conditions & Environment:** Admin đã đăng nhập.
- **4. Test Data:** `code="PASTEXP"` , `type="percent"` , `discount_value=10` , `min_order_amount=100000` , `expired_at="2020-01-01"` , `max_uses_per_user=1` .
- **5. Step-by-Step Procedure:**
    1. Chọn ngày hết hạn trong quá khứ trên form tạo mã.
    2. Bấm "Tạo mã".
- **6. Multi-tier Validation Points (Expected Results):**
    - *UI / Admin Web:* Hệ thống từ chối tạo mã (HTTP 400) HOẶC chấp nhận tạo nhưng gắn nhãn trạng thái "Đã hết hạn" ngay lập tức trên bảng quản trị.
    - *API / Network:* HTTP 400 (nếu chặn tạo) hoặc HTTP 200/201 (nếu cho phép nhưng coi là expired).
    - *Database State:* Nếu tạo thành công, trường `expired_at` lưu đúng chuỗi ngày quá khứ.
    - *Security & Business Rules:* Khi khách hàng thử áp dụng mã này tại Checkout, hệ thống bắt buộc phải từ chối theo điều kiện C2 (mã đã hết hạn).
- **7. Teardown & Clean-up:** Xóa mã `PASTEXP` nếu được tạo vào CSDL.

Test Case `TC_FR17_22` : Validate rejection when unauthenticated request calls create coupon API (Single Fault)

- **1. Tracking Information:**

    - **TC ID:** `TC_FR17_22` | **Priority:** Critical (P0 - Security) | **Target Layer:** Backend API Middleware
    - **Traceability:** SRS Mục 5.3, SEC-03 (*"Yêu cầu xác thực API Admin"*), API Spec §6.4
    - **Coverage IDs:** `AUTH-17-02`
    - **Single-Fault Isolation:** Payload hợp lệ hoàn toàn; lỗi duy nhất là request hoàn toàn không gửi kèm Header `Authorization`.
- **2. Test Objective:** Validate rejection when unauthenticated request calls create coupon API (Single Fault).
- **3. Pre-conditions & Environment:** Không gửi token xác thực.
- **4. Test Data:** Gửi `POST /api/admin/coupons` không có header `Authorization`.
- **5. Step-by-Step Procedure:**
    1. Sử dụng công cụ curl/Postman gửi request tạo coupon baseline không có token.
- **6. Multi-tier Validation Points (Expected Results):**
    - *API / Network:* HTTP 401 Unauthorized (`{"error": "Access token required"}`).
    - *Database State:* Tuyệt đối không có bản ghi mới nào được tạo trong `coupons`.
    - *Security & Business Rules:* Chặn truy cập trái phép của người dùng vãng lai vào tài nguyên quản trị.
- **7. Teardown & Clean-up:** Không cần dọn dẹp.

Test Case `TC_FR17_23` : Validate rejection when regular user with role 'user' calls admin coupon API (Single Fault - SEC-03)

- **1. Tracking Information:**
    - **TC ID:** `TC_FR17_23` | **Priority:** Critical (P0 - Security Hotspot) | **Target Layer:** Backend RBAC Authorization Guard
    - **Traceability:** SRS Mục 5.3, SEC-03 (*"Chỉ user có role='admin' mới được gọi API Admin"*), API Spec §6.4
    - **Coverage IDs:** `AUTH-17-03`
    - **Single-Fault Isolation:** Payload tạo coupon hoàn toàn hợp lệ cơ sở; biến lỗi duy nhất là JWT token thuộc tài khoản thường có `role = 'user'`.
- **2. Test Objective:** Validate rejection when regular user with role 'user' calls admin coupon API (Single Fault - SEC-03).
- **3. Pre-conditions & Environment:** Đăng nhập tài khoản User thường `test@eshop.com`, lấy JWT token của User.
- **4. Test Data:** Gửi `POST http://localhost:3000/api/admin/coupons` kèm Header `Authorization: Bearer <User_JWT>`.
- **5. Step-by-Step Procedure:**
    1. Gửi request tạo coupon bằng JWT của User thường.
    2. Quan sát HTTP status code và kiểm tra CSDL SQLite.
- **6. Multi-tier Validation Points (Expected Results):**
    - *API / Network:* HTTP Status code bắt buộc phải trả về **`403 Forbidden`** (hoặc `401 Unauthorized`).
    - *Security Enforcement (SEC-03):* Backend bắt buộc phải kiểm tra `req.user.role === 'admin'`. **Lỗ hổng bảo mật nghiêm trọng nếu xảy ra:** Nếu API trả về `200 OK` và cho phép user thường tạo coupon do middleware chỉ verify chữ ký JWT mà quên kiểm tra `role`, đây là lỗ hổng leo thang đặc quyền (Privilege Escalation Vulnerability - SEC-03 vi phạm trực tiếp).
    - *Database State:* Tuyệt đối không có coupon mới nào được tạo trong CSDL.
- **7. Teardown & Clean-up:** Dọn dẹp bản ghi nếu phát hiện lỗ hổng cho phép tạo mã.

Test Case `TC_FR17_24` : Validate rejection when deleting non-existent coupon ID 9999 (Single Fault)

- **1. Tracking Information:**
    - **TC ID:** `TC_FR17_24` | **Priority:** Medium (P2) | **Target Layer:** Backend API Delete Handler
    - **Traceability:** SRS FR-17, API Spec §6.4
    - **Coverage IDs:** `RES-17-02`
    - **Single-Fault Isolation:** ID truyền vào route `DELETE /api/admin/coupons/:id` là ID không tồn tại (`id = 9999`); token Admin hợp lệ.
- **2. Test Objective:** Validate rejection when deleting non-existent coupon ID 9999 (Single Fault).
- **3. Pre-conditions & Environment:** Admin đã đăng nhập; CSDL không có coupon mang `id = 9999`.
- **4. Test Data:** `DELETE /api/admin/coupons/9999` với token Admin.
- **5. Step-by-Step Procedure:**
    1. Gửi request xóa coupon ID 9999.
- **6. Multi-tier Validation Points (Expected Results):**
    - *API / Network:* HTTP 404 Not Found (hoặc HTTP 400 kèm thông báo `Coupon not found`). Server không được crash 500.
    - *Database State:* CSDL giữ nguyên vẹn toàn bộ các bản ghi hiện tại.
    - *Security & Business Rules:* Hệ thống xử lý an toàn khi tài nguyên mục tiêu không tồn tại.
- **7. Teardown & Clean-up:** Không cần dọn dẹp.

Test Case `TC_FR17_25` : Validate rejection when deleting coupon with negative ID -1 (Single Fault)

- **1. Tracking Information:**
    - **TC ID:** `TC_FR17_25` | **Priority:** Medium (P2) | **Target Layer:** Backend Route Parameter Validation
    - **Traceability:** SRS FR-17, API Spec §6.4
    - **Coverage IDs:** `RES-17-03`
    - **Single-Fault Isolation:** Tham số route mang giá trị âm `:id = -1` ; token Admin hợp lệ.
- **2. Test Objective:** Validate rejection when deleting coupon with negative ID -1 (Single Fault).
- **3. Pre-conditions & Environment:** Admin đã đăng nhập.
- **4. Test Data:** `DELETE /api/admin/coupons/-1` với token Admin.
- **5. Step-by-Step Procedure:**
    1. Gửi request xóa với `id = -1`.
- **6. Multi-tier Validation Points (Expected Results):**
    - *API / Network:* HTTP 400 Bad Request (hoặc 404 Not Found). Không xảy ra lỗi SQL syntax error hay 500.
    - *Database State:* Không có thao tác xóa nào làm ảnh hưởng đến CSDL.
    - *Security & Business Rules:* Khóa tham số route kiểu số nguyên dương hợp lệ.
- **7. Teardown & Clean-up:** Không cần dọn dẹp.

Test Case `TC_FR17_26` : Validate rejection when deleting coupon with malformed string ID 'abc' (Single Fault)

- **1. Tracking Information:**
  - **TC ID:** `TC_FR17_26` | **Priority:** Medium (P2) | **Target Layer:** Backend Route Parameter Validation
  - **Traceability:** SRS FR-17, API Spec §6.4
  - **Coverage IDs:** `RES-17-05`
  - **Single-Fault Isolation:** Tham số route mang kiểu chuỗi không thể parse sang số nguyên (`:id = "abc"`).
- **2. Test Objective:** Validate rejection when deleting coupon with malformed string ID 'abc' (Single Fault).
- **3. Pre-conditions & Environment:** Admin đã đăng nhập.
- **4. Test Data:** `DELETE /api/admin/coupons/abc` với token Admin.
- **5. Step-by-Step Procedure:**
  1. Gửi request xóa với `id = "abc"` .
- **6. Multi-tier Validation Points (Expected Results):**
  - *API / Network:* HTTP 400 Bad Request (`Invalid coupon ID`).
  - *Database State:* CSDL không bị thay đổi.
  - *Security & Business Rules:* Validate kiểu dữ liệu của route param trước khi chuyển tiếp vào câu truy vấn SQL.
- **7. Teardown & Clean-up:** Không cần dọn dẹp.

Test Case `TC_FR17_27` : Verify applying percent coupon discount arithmetic calculation at checkout

- **1. Tracking Information:**
  - **TC ID:** `TC_FR17_27` | **Priority:** High (P1) | **Target Layer:** Checkout Pricing Engine (`POST /api/orders`)
  - **Traceability:** SRS FR-09 (*"Áp dụng mã giảm giá"*), FR-17
  - **Coverage IDs:** `CPN-BR-01`
  - **Single-Fault Isolation:** Luồng tính toán số học giảm giá tỷ lệ hợp lệ tại Checkout.
- **2. Test Objective:** Verify applying percent coupon discount arithmetic calculation at checkout.
- **3. Pre-conditions & Environment:** Có coupon `SAVE10` (`type='percent'` , `discount_value=10` , `min_order_amount=200000`); User đã đăng nhập, giỏ hàng có tổng giá trị $1.000.000\text{ đ}$.
- **4. Test Data:** Nhập mã `SAVE10` tại trang thanh toán Checkout.
- **5. Step-by-Step Procedure:**
  1. Tại Checkout, nhập `SAVE10` vào ô mã giảm giá.
  2. Bấm "Áp dụng".
  3. Kiểm tra số tiền giảm và tổng thanh toán.
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Checkout Web:* Hiển thị dòng giảm giá: `-100.000 ₫` ($1.000.000 \times 10%$). Tổng thanh toán hiển thị: `900.000 ₫` .
  - *API / Network:* Endpoint áp dụng mã trả về `200 OK` kèm `{"discount": 100000, "final_total": 900000}` .
  - *Database State:* Chưa ghi trừ lượt dùng cho đến khi đơn hàng được đặt thành công.
  - *Security & Business Rules:* Tuân thủ đúng công thức tính toán tài chính phần trăm của FR-09.
- **7. Teardown & Clean-up:** Xóa sản phẩm khỏi giỏ hàng.

Test Case `TC_FR17_28` : Verify applying fixed coupon discount arithmetic calculation at checkout

- **1. Tracking Information:**
  - **TC ID:** `TC_FR17_28` | **Priority:** High (P1) | **Target Layer:** Checkout Pricing Engine
  - **Traceability:** SRS FR-09, FR-17
  - **Coverage IDs:** `CPN-BR-02`
  - **Single-Fault Isolation:** Luồng tính toán số học giảm giá tiền mặt cố định tại Checkout.
- **2. Test Objective:** Verify applying fixed coupon discount arithmetic calculation at checkout.
- **3. Pre-conditions & Environment:** Có coupon `FIX50K` (`type='fixed'` , `discount_value=50000` , `min_order_amount=200000`); giỏ hàng có tổng giá trị $1.000.000\text{ đ}$.
- **4. Test Data:** Nhập mã `FIX50K` tại Checkout.
- **5. Step-by-Step Procedure:**
  1. Nhập mã `FIX50K` và bấm "Áp dụng".
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Checkout Web:* Dòng giảm giá hiển thị: `-50.000 ₫` . Tổng thanh toán: `950.000 ₫` .
  - *API / Network:* Trả về `200 OK` kèm `discount = 50000` , `final_total = 950000` .
  - *Database State:* Tính toán nhất quán trên server backend.
  - *Security & Business Rules:* Khấu trừ đúng số tiền cố định đã cấu hình trong FR-17.
- **7. Teardown & Clean-up:** Xóa giỏ hàng.

Test Case `TC_FR17_29` : Validate rejection when order total is below min_order_amount at boundary offset M-1 ₫

- **1. Tracking Information:**
  - **TC ID:** `TC_FR17_29` | **Priority:** High (P1) | **Target Layer:** Checkout Validation Engine
  - **Traceability:** SRS FR-09 Điều kiện C3 (*"Tổng đơn hàng >= min_order_amount"*), BVA Off-point $M-1\text{ đ}$
  - **Coverage IDs:** `CPN-BR-03` , `BVA-17-09-OFF` ($M-1\text{ đ}$)
  - **Single-Fault Isolation:** Tổng tiền đơn hàng thiếu đúng 1 ₫ so với ngưỡng tối thiểu ($299.999\text{ đ} < 300.000\text{ đ}$); các điều kiện khác thỏa mãn.
- **2. Test Objective:** Validate rejection when order total is below min_order_amount at boundary offset M-1 ₫.
- **3. Pre-conditions & Environment:** Có coupon `MIN300` với `min_order_amount = 300000` .
- **4. Test Data:** Giỏ hàng có tổng giá trị đúng 299.999 đ.
- **5. Step-by-Step Procedure:**
  1. Nhập mã `MIN300` tại Checkout.
  2. Bấm "Áp dụng".

- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Checkout Web:* Hiển thị thông báo lỗi "Đơn hàng chưa đạt giá trị tối thiểu 300.000 ₫ để áp dụng mã này".
  - *API / Network:* Trả về HTTP 400 Bad Request kèm thông báo vi phạm ngưỡng đơn hàng tối thiểu.
  - *Database State:* Không áp dụng giảm giá; tổng thanh toán giữ nguyên 299.999 ₫.
  - *Security & Business Rules:* Bảo vệ ngưỡng doanh thu tối thiểu theo yêu cầu kinh doanh C3.
- **7. Teardown & Clean-up:** Xóa sản phẩm khỏi giỏ hàng.

Test Case `TC_FR17_30` : Verify acceptance when order total equals exact min_order_amount at boundary On-point M

- **1. Tracking Information:**
  - **TC ID:** `TC_FR17_30` | **Priority:** High (P1) | **Target Layer:** Checkout Validation Engine
  - **Traceability:** SRS FR-09 Điều kiện C3 (*"Tổng đơn hàng >= min_order_amount"*), BVA On-point $M\text{ ₫}$
  - **Coverage IDs:** `CPN-BR-04`, `BVA-17-09-ON` ($M=300.000\text{ ₫}$)
  - **Single-Fault Isolation:** Điểm biên On-point hợp lệ khi tổng tiền đơn hàng đạt đúng ngưỡng tối thiểu 300.000 ₫.
- **2. Test Objective:** Verify acceptance when order total equals exact min_order_amount at boundary On-point M.
- **3. Pre-conditions & Environment:** Có coupon `MIN300` (`min_order_amount = 300000` , giảm 10%).
- **4. Test Data:** Giỏ hàng có tổng giá trị đúng 300.000 ₫.
- **5. Step-by-Step Procedure:**
  1. Nhập mã `MIN300` tại Checkout.
  2. Bấm "Áp dụng".
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Checkout Web:* Thông báo áp dụng mã thành công; giảm giá 30.000 ₫; tổng thanh toán cập nhật thành 270.000 ₫.
  - *API / Network:* HTTP 200 OK.
  - *Database State:* Chấp nhận áp dụng mã thỏa mãn bất đẳng thức toán học $\ge$.
  - *Security & Business Rules:* Xác nhận điểm biên hợp lệ bao gồm cả dấu bằng theo SRS.
- **7. Teardown & Clean-up:** Xóa giỏ hàng.

Test Case `TC_FR17_31` : Validate rejection when applying expired coupon at checkout

- **1. Tracking Information:**
  - **TC ID:** `TC_FR17_31` | **Priority:** High (P1) | **Target Layer:** Checkout Coupon Engine
  - **Traceability:** SRS FR-09 Điều kiện C2 (*"Mã còn hạn sử dụng"*), FR-17
  - **Coverage IDs:** `CPN-BR-06`
  - **Single-Fault Isolation:** Mã giảm giá có `expired_at` trong quá khứ; đơn hàng đạt đủ mọi điều kiện khác.
- **2. Test Objective:** Validate rejection when applying expired coupon at checkout.
- **3. Pre-conditions & Environment:** CSDL có mã `EXPIRED` (`expired_at = '2020-01-01'`); giỏ hàng thỏa mãn ngưỡng tối thiểu.
- **4. Test Data:** Nhập mã `EXPIRED` tại Checkout.
- **5. Step-by-Step Procedure:**
  1. Nhập mã `EXPIRED` và bấm "Áp dụng".
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Checkout Web:* Báo lỗi "Mã giảm giá đã hết hạn sử dụng".
  - *API / Network:* HTTP 400 Bad Request (`Coupon has expired`).
  - *Database State:* Không có chiết khấu nào được áp dụng cho đơn hàng.
  - *Security & Business Rules:* Tuân thủ điều kiện chặn sử dụng mã hết hạn C2.
- **7. Teardown & Clean-up:** Xóa mã thử nghiệm nếu có.

Test Case `TC_FR17_32` : Validate rejection when user usage count reaches max_uses_per_user at boundary On-point

- **1. Tracking Information:**
  - **TC ID:** `TC_FR17_32` | **Priority:** High (P1) | **Target Layer:** Checkout Usage Limiter
  - **Traceability:** SRS FR-09 Điều kiện C5 (*"Số lần dùng của user < max_uses_per_user"*), BVA On-point
  - **Coverage IDs:** `CPN-BR-09`, `BVA-17-10-ON` ($used=U$)
  - **Single-Fault Isolation:** User hiện tại đã dùng đúng $U=1$ lần trong bảng `coupon_usage` ; coupon quy định `max_uses_per_user = 1` .
- **2. Test Objective:** Validate rejection when user usage count reaches max_uses_per_user at boundary On-point.
- **3. Pre-conditions & Environment:** Mã `ONCE` có `max_uses_per_user = 1` ; User `quocky` đã sử dụng mã này 1 lần (đã có bản ghi trong `coupon_usage`).
- **4. Test Data:** User `quocky` nhập lại mã `ONCE` trong đơn hàng tiếp theo.
- **5. Step-by-Step Procedure:**
  1. Đăng nhập bằng tài khoản `quocky` .
  2. Tại Checkout, nhập `ONCE` và bấm "Áp dụng".
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Checkout Web:* Báo lỗi "Bạn đã sử dụng hết số lần cho phép đối với mã giảm giá này".
  - *API / Network:* HTTP 400 Bad Request (`Coupon usage limit exceeded for this user`).
  - *Database State:* Không ghi nhận thêm bản ghi sử dụng nào trong `coupon_usage` .
  - *Security & Business Rules:* Thực thi nghiêm ngặt giới hạn lượt dùng của từng khách hàng theo C5.
- **7. Teardown & Clean-up:** Xóa bản ghi trong `coupon_usage` cho user thử nghiệm.

Test Case `TC_FR17_33` : Verify acceptance when user usage count is max_uses_per_user - 1 at boundary Off-point

- **1. Tracking Information:**
  - **TC ID:** `TC_FR17_33` | **Priority:** High (P1) | **Target Layer:** Checkout Usage Limiter
  - **Traceability:** SRS FR-09 Điều kiện C5, BVA Off-point $used=U-1$
  - **Coverage IDs:** `CPN-BR-08`, `BVA-17-10-OFF` ($used=U-1$)
  - **Single-Fault Isolation:** User đã dùng 1 lần trên mã có `max_uses_per_user = 2` ($1 < 2$).
- **2. Test Objective:** Verify acceptance when user usage count is max_uses_per_user - 1 at boundary Off-point.

- **3. Pre-conditions & Environment:** Mã `TWICE` có `max_uses_per_user = 2`; User `quocky` đã sử dụng 1 lần trong CSDL.
- **4. Test Data:** User `quocky` nhập lại mã `TWICE` cho đơn hàng thứ hai.
- **5. Step-by-Step Procedure:**
  1. Nhập mã `TWICE` và bấm "Áp dụng".
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Checkout Web:* Áp dụng mã thành công, hiển thị mức giảm giá tương ứng.
  - *API / Network:* HTTP 200 OK.
  - *Database State:* Cho phép tiến hành thanh toán; khi đặt hàng thành công ghi nhận bản ghi thứ 2 vào `coupon_usage`.
  - *Security & Business Rules:* Cho phép người dùng sử dụng đầy đủ quyền lợi trong hạn mức quy định.
- **7. Teardown & Clean-up:** Dọn dẹp bản ghi `coupon_usage`.

Test Case `TC_FR17_34`: Validate system handling when fixed discount exceeds order total (Financial Boundary Defect Probe)

- **1. Tracking Information:**
  - **TC ID:** `TC_FR17_34` | **Priority:** Critical (P0 - Financial Safety Hotspot) | **Target Layer:** Checkout Arithmetic Calculation Engine
  - **Traceability:** SRS FR-09, FR-08, BVA Off-point $discount > order_total$
  - **Coverage IDs:** `CPN-BR-10`, `BVA-17-11-OFF` ($discount > Total$)
  - **Single-Fault Isolation:** Mức giảm tiền mặt (50.000 đ) vượt quá tổng giá trị đơn hàng (30.000 đ).
- **2. Test Objective:** Validate system handling when fixed discount exceeds order total (Financial Boundary Defect Probe).
- **3. Pre-conditions & Environment:** Có coupon `BIGDISCOUNT` giảm cố định 50.000 đ với `min_order_amount = 0\text{ đ}`.
- **4. Test Data:** Đơn hàng chỉ có 1 sản phẩm phụ kiện giá 30.000 đ ($Total = 30.000\text{ đ} < 50.000\text{ đ}$).
- **5. Step-by-Step Procedure:**
  1. Tiến hành Checkout với đơn hàng 30.000 đ.
  2. Nhập mã `BIGDISCOUNT` và bấm "Áp dụng".
  3. Quan sát số tiền giảm giá và tổng tiền thanh toán cuối cùng.
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Checkout Web:* Số tiền giảm hiển thị tối đa -30.000 đ; tổng thanh toán hiển thị đúng **`0 đ`**. Tuyệt đối **không hiển thị số âm** (ví dụ `-20.000 đ`).
  - *API / Network:* Response tính toán trả về `discount: 30000` (hoặc `50000` nhưng `final_total: 0`). **Defect Alert:** Nếu `final_total: -20000` và cho phép checkout đơn âm tiền, đây là lỗi thất thoát tài chính cực kỳ nghiêm trọng.
  - *Database State:* Bản ghi đơn hàng trong bảng `orders` lưu `total_amount = 0`.
  - *Security & Business Rules:* Cố định chặn dưới của số tiền thanh toán không bao giờ được nhỏ hơn 0 đ.
- **7. Teardown & Clean-up:** Hủy đơn hàng thử nghiệm.

Test Case `TC_FR17_35`: Validate stored XSS escaping when viewing coupon with HTML tags on Admin Web (SEC-04)

- **1. Tracking Information:**
  - **TC ID:** `TC_FR17_35` | **Priority:** High (P1 - Security) | **Target Layer:** Admin Web Table Rendering (DOM)
  - **Traceability:** SRS SEC-04 (*"Escape đúng cách, không dùng innerHTML"*), API Spec §6.4
  - **Coverage IDs:** `EP-17-06`, `SEC-17-01`
  - **Single-Fault Isolation:** Mã coupon chứa thẻ kịch bản độc hại `<script>alert('XSS')</script>`; các trường khác hợp lệ.
- **2. Test Objective:** Validate stored XSS escaping when viewing coupon with HTML tags on Admin Web.
- **3. Pre-conditions & Environment:** Admin đã đăng nhập; tạo coupon chứa payload XSS qua API.
- **4. Test Data:** `{"code": "<script>alert('XSS')</script>", "type": "percent", "discount_value": 10, "min_order_amount": 0, "max_uses_per_user": 1, "expired_at": "2026-12-31"}`.
- **5. Step-by-Step Procedure:**
  1. Gửi request tạo coupon với code mang mã script.
  2. Mở tab "Mã Giảm Giá" trên Admin Web để tải danh sách mã.
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Admin Web:* Bảng hiển thị nguyên văn chuỗi `<script>alert('XSS')</script>` dưới dạng plain text. Tuyệt đối **không xuất hiện hộp thoại alert()** trên trình duyệt.
  - *DOM Inspection:* Thẻ được render an toàn bằng text node của React (`<span>` / `<td>`), không sử dụng `dangerouslySetInnerHTML`.
  - *API / Network:* Trả về chuỗi nguyên vẹn trong JSON mà không làm vỡ cú pháp parse.
  - *Security & Business Rules:* Tuân thủ chuẩn chống Stored Cross-Site Scripting (SEC-04).
- **7. Teardown & Clean-up:** Xóa coupon XSS khỏi CSDL.

Test Case `TC_FR17_36`: Validate race condition handling when creating duplicate coupon codes simultaneously

- **1. Tracking Information:**
  - **TC ID:** `TC_FR17_36` | **Priority:** High (P1 - Concurrency) | **Target Layer:** Backend API & SQLite Database Concurrency Lock
  - **Traceability:** SRS FR-17 (*"Mã coupon duy nhất"*), ACID Transaction Guarantee
  - **Coverage IDs:** `CPN-BR-14`
  - **Single-Fault Isolation:** Hai request `POST` cùng mang mã coupon giống nhau được gửi đồng thời trong khoảng thời gian < 100ms.
- **2. Test Objective:** Validate race condition handling when creating duplicate coupon codes simultaneously.
- **3. Pre-conditions & Environment:** Admin đã đăng nhập; CSDL chưa có mã `RACE50`.
- **4. Test Data:** Gửi đồng thời 2 request `POST /api/admin/coupons` với `code="RACE50"`.
- **5. Step-by-Step Procedure:**
  1. Sử dụng script bất đồng bộ (`Promise.all` hoặc parallel curl) bắn 2 request tạo mã `RACE50` cùng lúc.
  2. Kiểm tra status code của cả 2 request và số lượng bản ghi trong SQLite.
- **6. Multi-tier Validation Points (Expected Results):**
  - *API / Network:* Đúng 1 request thành công trả về HTTP 200/201. Request còn lại bắt buộc bị từ chối với HTTP 400 hoặc 409 (hoặc 500 do unhandled DB lock - ghi nhận defect nếu crash).

- *Database State:* Câu lệnh `SELECT COUNT(*) FROM coupons WHERE code='RACE50'` trả về đúng `1` bản ghi duy nhất. Tuyệt đối không sinh 2 bản ghi trùng mã do race condition.
- *Security & Business Rules:* Đảm bảo tính toàn vẹn dữ liệu ACID và chống xung đột tương tranh.

- **7. Teardown & Clean-up:** Xóa mã `RACE50`.

4.5 Bước 5: Ma trận truy vết & Kiểm tra độ bao phủ (Step 5 Deliverables - Traceability & Coverage Verification)

> **Mục tiêu Kiểm chứng Độ bao phủ (ISTQB FL v4.0 §4.2):**
>
> 1. **Bao phủ Phân vùng Tương đương (EP Coverage):** Đảm bảo 100% các phân vùng tương đương (VEP, IEP, Conditional, Robustness) được định nghĩa ở Bước 2 đều có ít nhất một kịch bản kiểm thử bao phủ trực tiếp.
> 2. **Bao phủ Điểm biên (BVA Coverage):** Đảm bảo 100% các điểm biên 3-Value BVA (On-point, Off-point, In-point) được định nghĩa ở Bước 3 đều có kịch bản tương ứng để phát hiện lỗi lệch biên ($\pm \epsilon$).
> 3. **Bảo đảm Giả định Đơn lỗi (Single Fault Assumption):** 100% kịch bản Negative (IEP, Boundary Off-point, Security) chỉ kích hoạt duy nhất một điều kiện bất thường; toàn bộ các tham số còn lại tuân thủ nghiêm ngặt Baseline hợp lệ cơ sở để tránh hiện tượng che giấu khiếm khuyết (Fault Masking).

4.5.1 Bảng Độ Bao Phủ Phân Vùng Tương Đương Biến Đầu Vào Thuần Túy (Pure Input EP Coverage)

Class ID	Variable / Field	Miền giá trị / Mô tả phân vùng tương đương	Phân loại	Covered by TC ID(s)	Trạng thái (Status)
`EP-17-01`	`code`	Mã hợp lệ chữ số Alphanumeric, chưa có trong DB	VEP	`TC_FR17_01`, `TC_FR17_02`, `TC_FR17_07`	**Covered**
`EP-17-02`	`code`	Mã trùng lặp đã tồn tại trong CSDL (Duplicate code)	IEP	`TC_FR17_10`	**Covered**
`EP-17-03`	`code`	Mã để trống / chuỗi rỗng (`""`)	IEP	`TC_FR17_11`	**Covered**
`EP-17-04`	`code`	Mã chỉ chứa toàn khoảng trắng (`" "`)	IEP	`TC_FR17_12`	**Covered**
`EP-17-05`	`code`	Khuyết trường `code` hoàn toàn trong payload JSON	IEP	`TC_FR17_11`	**Covered**
`EP-17-06`	`code`	Trùng mã chỉ khác chữ hoa/thường hoặc chứa mã độc	Conditional	`TC_FR17_35`	**Covered**
`EP-17-07`	`code`	Mã có khoảng trắng ở đầu hoặc cuối (`" SAVE10 "`)	Conditional	`TC_FR17_12`	**Covered**
`EP-17-08`	`code`	Độ dài chuỗi vượt ngưỡng quy ước ($Len > 50$)	Robustness	`TC_FR17_07`	**Covered**
`EP-17-09`	`type`	Loại giảm giá theo phần trăm (`'percent'`)	VEP	`TC_FR17_01`	**Covered**
`EP-17-10`	`type`	Loại giảm giá theo số tiền cố định (`'fixed'`)	VEP	`TC_FR17_02`	**Covered**
`EP-17-11`	`type`	Giá trị enum không hỗ trợ (`'free_shipping'`)	IEP	`TC_FR17_13`	**Covered**
`EP-17-12`	`type`	Loại là chuỗi rỗng (`type = ""`)	IEP	`TC_FR17_13`	**Covered**
`EP-17-13`	`type`	Loại mang giá trị `null` (`type = null`)	IEP	`TC_FR17_13`	**Covered**
`EP-17-14`	`type`	Khuyết trường `type` hoàn toàn trong payload	IEP	`TC_FR17_13`	**Covered**
`EP-17-15`	`discount_value`	Giá trị % hợp lệ trong khoảng chuẩn ($0 < V \le 100$)	VEP	`TC_FR17_01`, `TC_FR17_03`, `TC_FR17_04`	**Covered**
`EP-17-16`	`discount_value`	Giá trị tiền mặt hợp lệ ($V > 0$ VNĐ)	VEP	`TC_FR17_02`	**Covered**
`EP-17-17`	`discount_value`	Giá trị bằng 0 ($V = 0$)	IEP	`TC_FR17_14`, `TC_FR17_17`	**Covered**
`EP-17-18`	`discount_value`	Giá trị mang số âm ($V < 0$)	IEP	`TC_FR17_15`	**Covered**
`EP-17-19`	`discount_value`	Giá trị phần trăm vượt quá 100% ($V > 100$)	Conditional IEP	`TC_FR17_16`	**Covered**
`EP-17-20`	`discount_value`	Giá trị là chuỗi rỗng (`discount_value = ""`)	IEP	`TC_FR17_14`	**Covered**
`EP-17-21`	`discount_value`	Giá trị mang giá trị `null`	IEP	`TC_FR17_14`	**Covered**
`EP-17-22`	`discount_value`	Khuyết trường `discount_value` trong payload	IEP	`TC_FR17_14`	**Covered**
`EP-17-23`	`discount_value`	Giá trị chứa ký tự phi số (`"abc"`, `"10%"`)	IEP	`TC_FR17_15`	**Covered**
`EP-17-24`	`discount_value`	Giá trị % là số thập phân (`10.5%`)	Conditional	`TC_FR17_01`	**Covered**
`EP-17-25`	`discount_value`	Giá trị tiền fixed là số thập phân (`50000.5` đ)	Conditional	`TC_FR17_02`	**Covered**
`EP-17-26`	`min_order_amount`	Ngưỡng đơn hàng hợp lệ là số nguyên dương ($M > 0$)	VEP	`TC_FR17_01`, `TC_FR17_02`	**Covered**
`EP-17-27`	`min_order_amount`	Ngưỡng đơn hàng bằng đúng 0 ($M = 0$)	VEP	`TC_FR17_05`	**Covered**
`EP-17-28`	`min_order_amount`	Ngưỡng đơn hàng mang giá trị âm ($M < 0$)	IEP	`TC_FR17_18`	**Covered**
`EP-17-29`	`min_order_amount`	Khuyết trường `min_order_amount` hoặc truyền `null`	Conditional	`TC_FR17_05`	**Covered**
`EP-17-30`	`min_order_amount`	Ngưỡng đơn hàng chứa ký tự phi số	IEP	`TC_FR17_18`	**Covered**
`EP-17-31`	`max_uses_per_user`	Số lượt dùng hợp lệ là số nguyên ≥ 1	VEP	`TC_FR17_01`, `TC_FR17_06`	**Covered**
`EP-17-32`	`max_uses_per_user`	Số lượt dùng bằng 0 ($U = 0$)	IEP	`TC_FR17_19`	**Covered**
`EP-17-33`	`max_uses_per_user`	Số lượt dùng mang giá trị âm ($U < 0$)	IEP	`TC_FR17_20`	**Covered**
`EP-17-34`	`max_uses_per_user`	Số lượt dùng là số thập phân ($U = 1.5$)	IEP	`TC_FR17_20`	**Covered**
`EP-17-35`	`max_uses_per_user`	Khuyết trường `max_uses_per_user` hoặc `null`	Conditional	`TC_FR17_06`	**Covered**
`EP-17-36`	`expired_at`	Ngày hết hạn trong tương lai ($expired_at > \text{now}$)	VEP	`TC_FR17_01`, `TC_FR17_02`	**Covered**
`EP-17-37`	`expired_at`	Ngày hết hạn trong quá khứ ($expired_at < \text{now}$)	Conditional IEP	`TC_FR17_21`, `TC_FR17_31`	**Covered**

Class ID	Variable / Field	Miền giá trị / Mô tả phân vùng tương đương	Phân loại	Covered by TC ID(s)	Trạng thái (Status)
EP-17-38	expired_at	Ngày hết hạn chính là ngày hiện tại (today)	Conditional	TC_FR17_21	Covered
EP-17-39	expired_at	Ngày hết hạn để trống / chuỗi rỗng ("")	IEP	TC_FR17_21	Covered
EP-17-40	expired_at	Ngày hết hạn sai định dạng ngày tháng (Malformed Date)	IEP	TC_FR17_21	Covered

4.5.2 Bảng Độ Bao Phủ Phân Quyền & Vòng Đời Tài Nguyên (Access-Control & Resource Lifecycle Coverage)

Class ID	Điều kiện kiểm thử / Ràng buộc thao tác	Type	Covered by TC ID(s)	Trạng thái (Status)
AUTH-17-01	Token hợp lệ có quyền Admin (role = 'admin')	VEP	TC_FR17_01, TC_FR17_02, TC_FR17_08, TC_FR17_09	Covered
AUTH-17-02	Không gửi Token xác thực (Anonymous / Unauthenticated)	IEP	TC_FR17_22	Covered
AUTH-17-03	Token hợp lệ nhưng thuộc User thường (role = 'user')	IEP (SEC-03)	TC_FR17_23	Covered
AUTH-17-04	Token giả mạo / sai chữ ký / hết hạn	IEP (SEC-02)	TC_FR17_22	Covered
AUTH-17-05	Token hợp lệ nhưng payload thiếu trường role	Security	TC_FR17_23	Covered
RES-17-01	Xóa coupon hợp lệ đang tồn tại trong CSDL	VEP	TC_FR17_09	Covered
RES-17-02	Xóa coupon với ID không tồn tại (id = 9999)	IEP	TC_FR17_24	Covered
RES-17-03	Xóa coupon với ID mang giá trị âm (id = -1)	IEP	TC_FR17_25	Covered
RES-17-04	Xóa coupon với ID bằng 0	IEP	TC_FR17_25	Covered
RES-17-05	Xóa coupon với ID sai kiểu dữ liệu phi số (id = "abc")	IEP	TC_FR17_26	Covered
RES-17-06	Thao tác Xóa lặp lại cùng một ID (Idempotency Probe)	State Probe	TC_FR17_09, TC_FR17_24	Covered
RES-17-07	Xóa coupon đã có lịch sử người dùng sử dụng	Data Integrity	TC_FR17_09	Covered

4.5.3 Bảng Độ Bao Phủ Quy Tắc Nghiệp Vụ & An Toàn Thông Tin (Business Rules & Security Coverage)

ID Quy tắc	Phân loại	Nội dung quy tắc nghiệp vụ / Kịch bản thăm dò	Covered by TC ID(s)	Trạng thái (Status)
CPN-BR-01	Business Rule	Công thức tính giảm giá loại percent tại Checkout ($Total \times V / 100$)	TC_FR17_27	Covered
CPN-BR-02	Business Rule	Công thức tính giảm giá loại fixed tại Checkout ($Total - V$)	TC_FR17_28	Covered
CPN-BR-03	Business Rule	Ràng buộc ngưỡng đơn hàng: Dưới ngưỡng ($Total < min_order_amount$) $\to$ từ chối	TC_FR17_29	Covered
CPN-BR-04	Business Rule	Ràng buộc ngưỡng đơn hàng: Bằng ngưỡng ($Total = min_order_amount$) $\to$ chấp nhận	TC_FR17_30	Covered
CPN-BR-05	Business Rule	Ràng buộc ngưỡng đơn hàng: Trên ngưỡng ($Total > min_order_amount$) $\to$ chấp nhận	TC_FR17_27, TC_FR17_28	Covered
CPN-BR-06	Business Rule	Ràng buộc thời hạn sử dụng: Áp dụng mã đã hết hạn $\to$ từ chối (C2 vi phạm)	TC_FR17_31	Covered
CPN-BR-07	Business Rule	Ràng buộc trạng thái: Coupon bị vô hiệu hóa (is_active = 0) $\to$ từ chối	TC_FR17_09, TC_FR17_31	Covered
CPN-BR-08	Business Rule	Ràng buộc lượt dùng / người: Còn lượt ($used < max_uses_per_user$) $\to$ chấp nhận	TC_FR17_33	Covered
CPN-BR-09	Business Rule	Ràng buộc lượt dùng / người: Hết lượt ($used = max_uses_per_user$) $\to$ từ chối	TC_FR17_32	Covered
CPN-BR-10	Financial Safety	Giảm giá tiền mặt cố định lớn hơn tổng đơn ($discount > Total$) $\to$ chặn thanh toán $\ge 0\text{ đ}$	TC_FR17_34	Covered
CPN-BR-11	Business Rule	Giảm giá % sinh ra số thập phân lẻ $\to$ kiểm tra quy tắc làm tròn tiền tệ VND	TC_FR17_27	Covered
CPN-BR-12	DB Integrity	Tạo thành công lưu đầy đủ 6 trường, chuẩn hóa chữ hoa, is_active = 1	TC_FR17_01, TC_FR17_02	Covered
CPN-BR-13	Lifecycle	Coupon đã xóa biến mất khỏi danh sách quản trị và bị từ chối tại Checkout	TC_FR17_09	Covered
CPN-BR-14	Concurrency	Race condition: Tạo đồng thời 2 mã trùng nhau trong < 100ms $\to$ chỉ 1 thành công	TC_FR17_36	Covered
SEC-17-01	Security Probe	Stored Cross-Site Scripting (XSS) probe qua mã coupon hiển thị trên bảng Admin	TC_FR17_35	Covered
SEC-17-02	Security Probe	SQL Injection probe qua trường mã và tham số đường dẫn route	TC_FR17_26, TC_FR17_35	Covered
SEC-17-03	Robustness	Tràn số nguyên JavaScript (Number.MAX_SAFE_INTEGER) tại các trường số tiền	TC_FR17_18	Covered
SEC-17-04	Robustness	Payload JSON dị dạng (Malformed JSON) $\to$ Express error middleware bắt lỗi HTTP 400	TC_FR17_13, TC_FR17_26	Covered

4.5.4 Bảng Độ Bao Phủ Phân Tích Giá Trị Biên 3-Value (Boundary Value Coverage - On/Off/In Points)

Boundary ID	Biến / Điều kiện kiểm thử biên	Điểm biên (Point Type)	Giá trị kiểm thử cụ thể	Covered by TC ID(s)	Trạng thái (Status)
BVA-17-01	Độ dài mã coupon ($Len \ge 1$)	**On-point**	$Len = 1$ (code = "A")	TC_FR17_07	Covered
		Off-point	$Len = 0$ (code = "")	TC_FR17_11	Covered
		In-point	$Len = 2$ (code = "AB")	TC_FR17_01	Covered
BVA-17-02	Độ dài mã coupon quy ước ($Len \le 50$)	**On-point**	$Len = 50$ (Mã dài đúng 50 ký tự)	TC_FR17_01	Covered
		Off-point	$Len = 51$ (Mã dài 51 ký tự)	TC_FR17_07	Covered
		In-point	$Len = 49$ (Mã dài 49 ký tự)	TC_FR17_01	Covered
BVA-17-03	Giá trị giảm phần trăm ($V > 0\%$)	**On-point**	$V = 0\%$ (Vi phạm ràng buộc số dương)	TC_FR17_14	Covered

Boundary ID	Biến / Điều kiện kiểm thử biên	Điểm biên (Point Type)	Giá trị kiểm thử cụ thể	Covered by TC ID(s)	Trạng thái (Status)
		Off-point	$V = 1%$ (Số phần trăm dương nhỏ nhất)	`TC_FR17_04`	**Covered**
		In-point	$V = 2%$ (Kề cận trong vùng hợp lệ)	`TC_FR17_01`	**Covered**
`BVA-17-04`	Trần giá trị phần trăm ($V \le 100%$)	**On-point**	$V = 100%$ (Giảm tối đa 100% đơn hàng)	`TC_FR17_03`	**Covered**
		Off-point	$V = 101%$ (Vượt 100%, không hợp lệ)	`TC_FR17_16`	**Covered**
		In-point	$V = 99%$ (Kề cận dưới 100%, hợp lệ)	`TC_FR17_01`	**Covered**
`BVA-17-05`	Giá trị giảm tiền mặt ($V > 0\text{ ₫}$)	**On-point**	$V = 0\text{ ₫}$ (Vi phạm số tiền dương)	`TC_FR17_17`	**Covered**
		Off-point	$V = 1\text{ ₫}$ (Số tiền dương nhỏ nhất)	`TC_FR17_02`	**Covered**
		In-point	$V = 2\text{ ₫}$ (Kề cận trong vùng hợp lệ)	`TC_FR17_02`	**Covered**
`BVA-17-06`	Ngưỡng đơn tối thiểu ($M \ge 0\text{ ₫}$)	**On-point**	$M = 0\text{ ₫}$ (Không giới hạn ngưỡng đơn)	`TC_FR17_05`	**Covered**
		Off-point	$M = -1\text{ ₫}$ (Số âm vi phạm $M \ge 0$)	`TC_FR17_18`	**Covered**
		In-point	$M = 1\text{ ₫}$ (Ngưỡng tối thiểu dương nhỏ nhất)	`TC_FR17_01`	**Covered**
`BVA-17-07`	Lượt dùng tối đa / người ($U \ge 1$)	**On-point**	$U = 1\text{ lượt}$ (Mỗi user dùng 1 lần)	`TC_FR17_06`	**Covered**
		Off-point	$U = 0\text{ lượt}$ (Vi phạm $U \ge 1$)	`TC_FR17_19`	**Covered**
		In-point	$U = 2\text{ lượt}$ (Kề cận trong vùng hợp lệ)	`TC_FR17_33`	**Covered**
`BVA-17-08`	Ranh giới hạn dùng khi tạo ($expired_at \ge \text{today}$)	**On-point**	$expired_at = \text{today}$ (Hạn là hôm nay)	`TC_FR17_21`	**Covered**
		Off-point	$expired_at = \text{yesterday}$ (Ngày quá khứ)	`TC_FR17_21`	**Covered**
		In-point	$expired_at = \text{tomorrow}$ (Ngày mai)	`TC_FR17_01`	**Covered**
`BVA-17-09`	Ngưỡng đơn Checkout ($Total \ge M=300.000\text{ ₫}$)	**On-point**	$Total = 300.000\text{ ₫}$ (Đạt đúng ngưỡng)	`TC_FR17_30`	**Covered**
		Off-point	$Total = 299.999\text{ ₫}$ (Thiếu 1 ₫, từ chối)	`TC_FR17_29`	**Covered**
		In-point	$Total = 300.001\text{ ₫}$ (Vượt ngưỡng 1 ₫)	`TC_FR17_27`	**Covered**
`BVA-17-10`	Lượt dùng Checkout ($used < U=2\text{ lượt}$)	**On-point**	$used = 2\text{ lần}$ ($2 \not< 2 \to$ hết lượt)	`TC_FR17_32`	**Covered**
		Off-point	$used = 1\text{ lần}$ ($1 < 2 \to$ lượt cuối)	`TC_FR17_33`	**Covered**
		In-point	$used = 3\text{ lần}$ ($3 \not< 2 \to$ vượt giới hạn)	`TC_FR17_32`	**Covered**
`BVA-17-11`	Biên tài chính khấu trừ ($discount \le Total=50.000\text{ ₫}$)	**On-point**	$discount = 50.000\text{ ₫}$ (Đơn thanh toán 0 ₫)	`TC_FR17_34`	**Covered**
		Off-point	$discount = 50.001\text{ ₫}$ (Vượt 1 ₫ $\to$ chặn $\ge 0\text{ ₫}$)	`TC_FR17_34`	**Covered**
		In-point	$discount = 49.999\text{ ₫}$ (Đơn còn trả đúng 1 ₫)	`TC_FR17_28`	**Covered**

4.5.5 Bảng Tổng Hợp Chỉ Số Độ Bao Phủ (Coverage Summary Metrics)

Chỉ số kiểm thử (Coverage Metric)	Tổng số nhận diện	Đã bao phủ (Covered)	Chưa bao phủ / Bị chặn	Tỷ lệ bao phủ (%)
Phân vùng hợp lệ (VEP - Pure Input)	6	6	0	**100.0%**
Phân vùng không hợp lệ (IEP - Pure Input)	26	26	0	**100.0%**
Phân vùng có điều kiện & Bền vững (Conditional/Robustness)	8	8	0	**100.0%**
Tổng số phân vùng tương đương biên đầu vào (Pure Input EPs)	**40**	**40**	**0**	**100.0%**
Phân vùng kiểm soát truy cập & Phân quyền (AUTH)	5	5	0	**100.0%**
Phân vùng vòng đời thao tác tài nguyên CRUD (RES)	7	7	0	**100.0%**
Quy tắc nghiệp vụ & Kịch bản Checkout FR-09 (CPN-BR)	14	14	0	**100.0%**
Kiểm thử an toàn thông tin & Độ bền bỉ (SEC Probes)	4	4	0	**100.0%**
Tổng số điểm phân tích giá trị biên (3-Value BVA Points)	**33**	**33**	**0**	**100.0%**
Tổng số kịch bản kiểm thử độc lập đã thiết kế (Test Cases)	**36**	**36**	**0**	**100.0%**
Tuân thủ Giả định Đơn lỗi (Single-Fault Isolation)	26 / 26 IEPs	26	0	**100.0%**

4.6 Bước 6: AI Gap Analysis & Giả thuyết lỗi SUT (Step 6 Deliverables)

> **Mục đích:** Cung cấp minh chứng thực tế cho **Mục 6 Task 3 (AI Gap Analysis)** và làm cơ sở biện luận cho **Mục 10 (AI Critique)** của báo cáo HW02. Phân tách rạch ròi giữa các thiếu sót do suy luận AI và các khiếm khuyết nội bộ thực tế trong mã nguồn SUT EShop liên quan đến chức năng Quản lý Mã Giảm Giá (FR-17).

4.6.1 Bảng Phân Tích Khoảng Trống Kiểm Thử Của AI (AI Gap Analysis)

Hạng mục / Vấn đề phát sinh	Nguyên nhân AI bỏ sót hoặc gặp khó khăn	Hiệu chỉnh của con người (Human Fix Applied)	Căn cứ Tiêu chuẩn & Kỹ thuật Kiểm thử
Gộp lẫn 5 tầng kiểm thử vào một bảng EP duy nhất (Thổi phồng độ bao phủ EP)	AI có xu hướng gộp cơ học toàn bộ các khía cạnh (xác thực token Admin, phân quyền `role`, tham số route `:id` khi xóa, tính toán giảm giá tại Checkout, và tấn công XSS/SQLi) vào chung bảng `EP-17-xx`, làm loãng bản chất của phân vùng tương đương đầu vào và thổi phồng tỷ lệ bao phủ giả tạo.	Con người yêu cầu bóc tách nghiêm ngặt thành 5 tầng kiểm thử chuyên biệt: (1) Pure Input EPs (40 phân vùng form); (2) Access-Control & Authorization (5 phân vùng); (3) Target-Resource Lifecycle (7 phân vùng); (4) Business Rules & Checkout Scenarios (14 kịch bản); (5) Security & Robustness Probes (4 kịch bản).	**ISTQB FL v4.0 §4.2.1:** Phân vùng tương đương thuần túy chỉ áp dụng cho miền giá trị của các phần tử dữ liệu đầu vào/đầu ra, không áp dụng cho quyền truy cập, trạng thái máy hay lỗ hổng bảo mật.
Không nhận thức ràng buộc có điều kiện (Conditional Domain Modeling)	AI ban đầu phân vùng trường `discount_value` một cách độc lập mà không nhận ra sự phụ thuộc chặt chẽ vào trường `type`: với `type = 'percent'`, giá trị giảm bị chặn trên bởi trần tự nhiên $100%$ (không thể giảm vượt giá trị đơn hàng), trong khi với `type = 'fixed'`, giá trị giảm là tiền mặt và có thể lên tới hàng triệu đồng.	Con người chỉ đạo mô hình hóa miền giá trị có điều kiện: Tách biệt phân tích `discount_value` thành 2 nhánh phụ thuộc vào `type`, đặt biên cận trên $V \le 100%$ cho `percent` và biên $V > 0\text{ đ}$ cho `fixed`.	**Kỹ thuật Kiểm thử Miền có Điều kiện (Conditional Domain Analysis):** Miền giá trị của biến phụ thuộc vào trạng thái hoặc giá trị của biến điều kiện đi kèm.
Tự động áp đặt cận trên không có căn cứ ($Len \le 50$) và nhầm lẫn nguồn đặc tả	AI bị ảnh hưởng bởi thói quen thiết kế cơ sở dữ liệu (các trường VARCHAR thường dài 50 ký tự), tự ý đặt ra ràng buộc $1 \le Len \le 50$ và coi đó là yêu cầu bắt buộc của SRS, làm mờ ranh giới giữa đặc tả chính thức và giả định thiết kế.	Con người yêu cầu phân loại minh bạch nguồn gốc: Gắn nhãn rõ ràng `[SRS Confirmed]` cho các ràng buộc trích xuất trực tiếp từ SRS FR-17, và `[Assumption-based]` cho các giả định kỹ thuật về độ dài tối đa chuỗi để đối chiếu rõ ràng với khách hàng.	**Nguyên tắc "Đặc tả là căn cứ duy nhất":** Tuyệt đối không tự ý suy diễn hoặc bịa đặt ràng buộc nghiệp vụ khi SRS không đề cập mà không ghi rõ giả định.
Gộp chung các biểu diễn lỗi đầu vào khác nhau vào một partition duy nhất	AI ban đầu xem chuỗi rỗng `""`, chuỗi toàn khoảng trắng `" "`, giá trị `null`, và việc bỏ khuyết trường trong payload JSON là một lỗi tương đương duy nhất ("trường bị trống").	Con người yêu cầu phân tách độc lập: Chuỗi rỗng `""` kiểm tra validation form bắt buộc; chuỗi khoảng trắng `" "` kiểm tra hàm `trim()`; giá trị `null` và khuyết trường kiểm tra Schema Contract của Backend API.	**ISTQB FL §4.2.1 & Single-Fault Isolation:** Các lỗi có nguyên nhân kỹ thuật, cơ chế xử lý ngoại lệ và điểm thất bại độc lập phải được phân vùng riêng biệt để tránh che giấu lỗi.
Bỏ quên kiểm thử phân quyền ở tầng Backend API (Lỗ hổng leo thang đặc quyền SEC-03)	AI có thói quen chỉ thiết kế kiểm thử trên giao diện người dùng Admin Web (nơi user thường bị chặn truy cập bằng router frontend), bỏ sót trường hợp người dùng thông thường gửi HTTP request trực tiếp tới API endpoint `POST /api/admin/coupons` và `DELETE /api/admin/coupons/:id`.	Con người chỉ đạo bổ sung kịch bản kiểm thử bảo mật `TC_FR17_23` (User thường với `role = 'user'` gọi trực tiếp API Admin) nhằm kiểm chứng xem middleware backend có thực sự kiểm tra `req.user.role === 'admin'` hay chỉ kiểm tra chữ ký token.	**API Security & RBAC Enforcement (SEC-03):** Mọi endpoint nhạy cảm đều phải được bảo vệ độc lập ở tầng Backend, không phụ thuộc vào tầng điều hướng Frontend.

4.6.2 Danh Sách Giả Thuyết Khiếm Khuyết Thăm Dò Trên SUT EShop (Candidate Defect Hypotheses)

> *Ghi chú: Toàn bộ các giả thuyết dưới đây được gắn nhãn `[Exploratory - Unconfirmed Hypothesis - Requires SUT execution]` và sẽ được kiểm chứng thực tế trong Giai đoạn 3 trên hệ thống EShop đang chạy.*

Hypothesis ID	Khu vực kiểm thử (Target Area)	Mô tả giả thuyết lỗi (Description)	Kỳ vọng theo đặc tả (Spec Expected)	Dự đoán hành vi thực tế (Hypothesized Actual)	Bước kiểm chứng trên SUT (Verification Step)
`HYP-17-01`	Backend Authorization (`server.js:457`, `server.js:490`)	**Lỗ hổng leo thang đặc quyền: API Admin thiếu kiểm tra role = 'admin' (SEC-03)**	SRS Mục 5.3 & SEC-03: *"User phải có role là admin (`role = 'admin'`). Nếu user thông thường (`role = 'user'`) gọi API admin -> trả về HTTP 403 Forbidden."*	Trong `server.js:457` và `490`, endpoint `POST /api/admin/coupons` và `DELETE /api/admin/coupons/:id` chỉ gắn middleware `authenticateToken` (chỉ giải mã JWT và gán `req.user`), hoàn toàn **KHÔNG có kiểm tra `if (req.user.role !== 'admin') return res.status(403)`**. User thường có thể tạo và xóa coupon tùy ý!	Đăng nhập tài khoản user thường `test@eshop.com`, lấy JWT token; dùng curl/Postman gửi request `POST http://localhost:3000/api/admin/coupons` với token này; quan sát thấy HTTP trả về `200 OK` thay vì `403 Forbidden`.
`HYP-17-02`	Backend Error Handling (`server.js:478-485`)	**Unhandled SQLite UNIQUE Constraint Error gây crash server hoặc sập lỗi HTTP 500**	SRS FR-17: *"code (duy nhất)"*. Khi tạo mã đã tồn tại, hệ thống phải xử lý an toàn, trả về lỗi nghiệp vụ rõ ràng (HTTP 400 hoặc 409).	Trong `server.js:478`, câu lệnh SQL `INSERT INTO coupons ...` khi gặp mã trùng sẽ bị SQLite throw exception `SQLITE_CONSTRAINT: UNIQUE constraint failed: coupons.code`. Backend không bắt mã lỗi này mà chuyển thẳng vào block `res.status(500).json({ error: err.message })`, làm lộ chi tiết CSDL và trả về mã lỗi 500 không đúng chuẩn API.	Gửi request `POST /api/admin/coupons` với mã `SAVE10` đã có trong CSDL; kiểm tra response status code xem có bị trả về HTTP 500 hay không.
`HYP-17-03`	Backend Validation (`server.js:460-475`)	**Thiếu validation phía server đối với số âm và các giá trị biên của coupon**	SRS FR-17: *"Các trường bắt buộc: discount_value (dương), min_order_amount (>= 0), max_uses_per_user (>= 1)"*. Vi phạm phải bị từ chối với HTTP 400.	Mã nguồn `server.js:460` chỉ kiểm tra sơ sài: `if (!code			

Hypothesis ID	Khu vực kiểm thử (Target Area)	Mô tả giả thuyết lỗi (Description)	Kỳ vọng theo đặc tả (Spec Expected)	Dự đoán hành vi thực tế (Hypothesized Actual)	Bước kiểm chứng trên SUT (Verification Step)
`HYP-17-04`	Financial Calculation (`server.js` & `Checkout.jsx`)	**Lỗi ranh giới tài chính: Số tiền thanh toán bị âm khi fixed discount vượt tổng tiền hàng**	SRS FR-09 & Logic tài chính an toàn: Tổng tiền thanh toán của đơn hàng tuyệt đối không được phép là số âm ($Total \ge 0\text{ đ}$). Mức giảm tối đa chỉ bằng tổng tiền hàng.	Trong logic tính toán thanh toán, lập trình viên tính trực tiếp `final_total = order_total - discount_value` mà không bọc hàm `Math.max(0, ...)` . Nếu coupon fixed giảm 50.000 đ trên đơn hàng 30.000 đ, tổng tiền sẽ ra -20.000 đ, cho phép đặt đơn âm tiền!	Tạo coupon `FIX50K` giảm 50.000 đ; đặt đơn hàng 1 sản phẩm phụ kiện giá 30.000 đ; áp dụng mã tại Checkout và quan sát số tiền thanh toán cuối cùng trên giao diện và trong bảng `orders` .
`HYP-17-05`	Frontend Admin UI (`frontend/src/pages/admin/Coupons.jsx`)	**Nút "Xóa" mã giảm giá xóa trực tiếp không có Dialog xác nhận trước khi thực hiện**	SRS FR-24 / Nguyên tắc thiết kế an toàn: Mọi thao tác hủy/xóa dữ liệu quan trọng đều phải có hộp thoại xác nhận (Confirmation Dialog) trước khi thực thi để tránh bấm nhầm.	Sự kiện `onClick` của nút "Xóa" trên bảng Admin gọi hàm `handleDelete(coupon.id)` gửi request `DELETE` ngay lập tức mà không gọi `window.confirm()` hay hiển thị Modal Dialog nào, dẫn tới nguy cơ Admin vô tình xóa nhầm mã đang hoạt động.	Đăng nhập Admin Web, mở trang "Mã Giảm Giá"; bấm nút "Xóa" tại một coupon; quan sát xem coupon có bị xóa ngay mà không hề có bất kỳ câu hỏi xác nhận nào xuất hiện.
`HYP-17-06`	Frontend DOM Rendering (`frontend/src/pages/admin/Coupons.jsx`)	**Nguy cơ Stored XSS nếu tên mã coupon hoặc giá trị được render bằng innerHTML**	SRS SEC-04: *"Frontend phải escape đúng cách dữ liệu... Tuyệt đối không dùng innerHTML để chèn dữ liệu chưa lọc"*	Mặc dù React mặc định escape chuỗi trong JSX text node (`<td>{coupon.code}</td>`), nếu giao diện sử dụng template chuỗi HTML hoặc thư viện bảng ngoài có cấu hình HTML rendering, mã `<script>` trong coupon code có thể bị thực thi.	Tạo mã coupon với code `<img src=x onerror=alert('XSS')>` ; mở bảng Admin Quản lý Mã giảm giá và kiểm tra cây DOM cũng như sự xuất hiện của hộp thoại alert.

5. TÍNH NĂNG D (POOL D - MOBILE): FR-20 & FR-10 – HỦY ĐƠN HÀNG THEO TRẠNG THÁI

5.1 Bước 1: Nhận diện biến đầu vào & Ràng buộc (Step 1 Deliverables)

Trích dẫn đặc tả SRS FR-10 (Trạng thái Đơn hàng - Order State Machine):

- Đơn hàng có **5 trạng thái** và phải tuân theo sơ đồ chuyển đổi: `pending` --[Admin xác nhận]--> `confirmed` --[Admin giao hàng]--> `shipping` --[Admin hoàn tất]--> `delivered` `pending` --[User/Admin hủy]--> `canceled` `confirmed` --[User/Admin hủy]--> `canceled`
- **Ràng buộc trạng thái kết thúc (Final States):**
  - Trạng thái `delivered` và `canceled` là **trạng thái kết thúc** — không được phép chuyển sang bất kỳ trạng thái nào khác.
  - Khi đơn hàng đã ở trạng thái `shipping` , **User không được phép tự hủy** — chỉ Admin mới có thể thao tác.
  - Mọi chuyển đổi không hợp lệ phải trả về lỗi với thông báo phù hợp.

Trích dẫn đặc tả SRS FR-20 (Tính năng Mobile - React Native):

- *"Đầy đủ các chức năng: Xem sản phẩm, Đăng nhập, Đăng xuất, Đăng ký, Giỏ hàng, Thanh toán, Hồ sơ, Lịch sử đơn hàng."*
- *"Chức năng Hủy đơn hàng tuân theo đúng State Machine ở FR-10 (chỉ được hủy khi `pending` hoặc `confirmed`)."*

(Đối chiếu Đặc tả Hệ thống & API liên quan:

- *SRS FR-11: "Người dùng chỉ xem được đơn hàng của chính mình. Hiển thị: Mã đơn, Ngày đặt, Tổng tiền, Trạng thái hiện tại. Trạng thái phải được dịch sang tiếng Việt rõ ràng và phân biệt màu sắc."*
- *SRS FR-21 / FR-24: Tiêu chuẩn Giao diện & An toàn thao tác — Nút hủy/xóa phải dùng màu đỏ (`styles.smallRedButton`); thao tác hủy đơn có tính chất không thể đảo ngược nên cần có hộp thoại xác nhận (Confirmation Alert).*
- *API Spec §4.6: `PUT /api/orders/:id/cancel` — Chuyển trạng thái đơn hàng sang `canceled` . Yêu cầu `Authorization: Bearer <token>` .*
- *Frontend Mobile UI (`frontend-mobile/App.js`): Màn hình Profile hiển thị danh sách đơn hàng. Điều kiện hiển thị nút "Hủy đơn": `(o.status === "pending" || o.status === "confirmed")` . Khi bấm gọi `cancelOrder(o.id)` gửi request `PUT /api/orders/:id/cancel` .)*

Variable ID	Variable Name	Data Type	Mandatory?	Precision ($\lepsilon$)	Specification Constraint (Quote)	Assumptions / Ambiguities
`VAR-20-01`	`current_order_status` *(Trạng thái hiện tại của đơn hàng)*	Enum / String	Có	N/A	Trích dẫn SRS FR-10: *"Đơn hàng có 5 trạng thái: `pending` , `confirmed` , `shipping` , `delivered` , `canceled` ... User chỉ được phép tự hủy đơn khi đơn ở trạng thái `pending` hoặc `confirmed` . Khi đơn hàng đã ở trạng thái `shipping` , User không được phép tự hủy... Trạng thái `delivered` và `canceled` là trạng thái kết thúc."* (Đối chiếu FR-20)	Thuộc tính trạng thái của đơn hàng trong CSDL trước khi thao tác hủy được thực hiện. Tập giá trị hữu hạn gồm đúng 5 trạng thái hợp lệ. Bất kỳ giá trị nào ngoài 5 trạng thái trên (ví dụ chuỗi rỗng `""` , `null` , `'processing'` , `'refunded'`) là không hợp lệ và phải bị từ chối từ cả Mobile UI lẫn Backend API.

Variable ID	Variable Name	Data Type	Mandatory?	Precision (ϵ)	Specification Constraint (Quote)	Assumptions / Ambiguities
VAR-20-02	target_order_id *(Định danh đơn hàng cần hủy)*	Integer / Numeric	Có	1 đơn vị	Trích dẫn API Spec §4.6: *"Hủy đơn hàng: PUT /api/orders/:id/cancel — Chuyển trạng thái đơn hàng sang canceled."*	Tham số đường dẫn URL :id. Phải là số nguyên dương hợp lệ ($id \ge 1$) và tồn tại trong bảng orders của CSDL. Nếu ID không tồn tại trong hệ thống (ví dụ id = 99999), ID âm (-1), ID bằng 0, hoặc ID phi số ("abc"), hệ thống phải trả về lỗi phù hợp (HTTP 404 hoặc 400), không được làm crash tiến trình server.
VAR-20-03	auth_token *(Token xác thực JWT của người dùng)*	String (JWT Bearer)	Bắt buộc (Bảo mật)	N/A	Trích dẫn SRS FR-12 & API Spec §4.6: Yêu cầu Header Authorization: Bearer <token>.	Chuỗi JWT Token hợp lệ cấp cho phiên đăng nhập của người dùng Mobile App. Nếu thiếu Token trong header (Unauthenticated) $\to$ hệ thống phải từ chối với HTTP 401 Unauthorized. Nếu Token bị giả mạo, sai chữ ký số hoặc đã hết hạn $\to$ HTTP 403 Forbidden hoặc 401 Unauthorized.
VAR-20-04	order_ownership *(Quyền sở hữu đơn hàng - Chống IDOR)*	Boolean / Relation	Bắt buộc (Bảo mật)	N/A	Trích dẫn SRS FR-11: *"Người dùng chỉ xem được đơn hàng của chính mình."* (Nguyên tắc phân quyền dữ liệu người dùng)	Đơn hàng phải thuộc quyền sở hữu của chính người dùng đang đăng nhập (order.user_id === req.user.id). Nếu Người dùng A (có Token hợp lệ) cố tình gửi request PUT /api/orders/:id/cancel với :id thuộc về Người dùng B (tấn công IDOR), hệ thống phải từ chối (HTTP 403 hoặc 404) để bảo vệ đơn hàng của người khác.
VAR-20-05	user_role *(Vai trò người dùng trong hệ thống)*	Enum	Có	N/A	Trích dẫn SRS FR-10: *"Khi đơn hàng đã ở trạng thái shipping, User không được phép tự hủy — chỉ Admin mới có thể thao tác."*	Phân biệt vai trò của tác nhân thực hiện: Mobile App luôn gửi request với vai trò người dùng thông thường (role = 'user'). Admin có kênh quản trị riêng (PUT /api/admin/orders/:id/status). Khi role = 'user', nếu gửi request hủy đơn đang ở shipping, hệ thống phải từ chối ngay lập tức (HTTP 400 hoặc 403).
VAR-20-06	cancellation_channel *(Kênh kích hoạt thao tác hủy)*	Enum	Có	N/A	Trích dẫn SRS FR-20: *"Phân hệ Mobile... Chức năng Hủy đơn hàng tuân theo đúng State Machine ở FR-10".*	Phân tách kiểm thử 2 kênh độc lập: (1) mobile_ui: kiểm thử sự hiển thị/ẩn/vô hiệu hóa của nút "Hủy đơn" trên giao diện React Native (frontend-mobile/App.js); (2) direct_api: gửi trực tiếp request PUT /api/orders/:id/cancel qua HTTP client để kiểm tra tính toàn vẹn của State Machine phía Backend (chống lỗi Client-Side-Only Validation).
VAR-20-07	cancellation_confirmation *(Hộp thoại xác nhận hủy đơn trên UI)*	Boolean (Dialog Confirm)	Khuyến nghị UX/An toàn	N/A	Trích dẫn SRS FR-21 (Tiêu chuẩn Giao diện) & FR-24 (Nguyên tắc an toàn dữ liệu).	Thao tác hủy đơn là hành động phá hủy, không thể hoàn tác (canceled là trạng thái kết thúc). Giao diện Mobile App phải hiển thị hộp thoại xác nhận (Alert.alert với nút "Xác nhận" và "Hủy bỏ") trước khi phát lệnh hủy đến server nhằm ngăn chặn người dùng vô tình chạm nhầm nút.
VAR-20-08	inventory_restock_sync *(Tác dụng phụ hoàn trả tồn kho khi hủy)*	State Side-effect	Nghiệp vụ phụ trợ	1 sản phẩm	Trích dẫn SRS FR-07 / FR-08 / FR-10: Tồn kho sản phẩm (products.stock) bị giảm khi đặt hàng thành công.	Khi đơn hàng bị hủy hợp lệ ở trạng thái pending hoặc confirmed, số lượng sản phẩm trong đơn có được hoàn trả tự động lại vào kho (stock = stock + quantity) hay không? Đây là ràng buộc nghiệp vụ phụ trợ cần được quan sát và kiểm chứng trực tiếp trên CSDL SQLite.

5.2 Bước 2: Phân vùng tương đương (Step 2 Deliverables - Equivalence Partitioning)

> **Cải tiến Kiến trúc Phân vùng & Chuẩn hóa Nguyên tắc Giả định Đơn lỗi (Single-Fault Assumption - ISTQB FL §4.2.1):**
>
> - **Phân tách 5 tầng kiểm thử minh bạch (Tránh đánh đồng dữ liệu đầu vào với kiểm thử bảo mật & nghiệp vụ):**
>   1. **5.2.1 Pure Input Equivalence Partitions:** Chỉ phân vùng tham số đường dẫn URL :id và cấu trúc Payload của request.
>   2. **5.2.2 Authentication, Authorization & Ownership:** Phân tách rõ ràng giữa lỗi xác thực (Authentication Failure: thiếu token, token hỏng, token hết hạn) và vi phạm kiểm soát truy cập / quyền sở hữu (Authorization & IDOR).
>   3. **5.2.3 Target Resource Lifecycle:** Phân vùng trạng thái tồn tại của bản ghi đơn hàng trong CSDL (tồn tại vs không tồn tại).
>   4. **5.2.4 Business Rules & Order State Machine Transitions:** Toàn bộ các bước chuyển trạng thái theo đặc tả FR-10/FR-20, kiểm chứng bất biến CSDL (Database Invariants) và ghi nhận rõ ràng các câu hỏi mở (Open Questions / Clarifications).
>   5. **5.2.5 Complementary Test Suites (Security, UI Safety & Robustness):** Bộ kiểm thử bổ trợ độc lập, không tính gộp vào tỷ lệ bao phủ EP đầu vào cơ bản.
> - **Chuẩn hóa Bộ Giá trị Hợp lệ Cơ sở (Standardized Baseline for Cancellation):**
>   - target_order_id: 101 (đơn hàng số nguyên dương hợp lệ, có tồn tại trong CSDL).
>   - order_ownership: Thuộc sở hữu của User 1 (order.user_id = 1).
>   - current_order_status: Cố định duy nhất 'pending' (trạng thái hợp lệ cơ sở).
>   - auth_token: JWT Bearer Token hợp lệ, chưa hết hạn của User 1 (user_id = 1, role = 'user').
>   - request_body: {} (Empty JSON Object hợp lệ).
>   - endpoint_url: PUT /api/orders/101/cancel.
>   - Mọi ca kiểm thử đơn lỗi (Single-Fault Isolation) chỉ được phép thay đổi **duy nhất 1 điều kiện/tham số** so với Baseline này.

5.2.1 Bảng Phân Vùng Tương Đương Biến Đầu Vào Thuần Túy (Pure Input Equivalence Partitions)

Class ID	Variable / Constraint	Precondition	Domain / Partition Definition	Type (VEP / IEP)	Representative Value	Expected HTTP & Error Message	Expected Database State (Invariant)	Specification Source & Status	Single-Fault Baseline & Rationale
`ORD-INP-VEP01`	`target_order_id` (Định danh đơn)	User 1 đăng nhập	Số nguyên dương hợp lệ ($id \ge 1$) khớp bản ghi trong DB	VEP	`id = 101`	HTTP 200 OK (`{"message": "Order canceled successfully"}`)	`orders.status` chuyển từ `pending` sang `canceled`	[SRS Confirmed & API §4.6]	Baseline chuẩn: User 1 sở hữu đơn 101, status `pending`.
`ORD-INP-IEP01`	`target_order_id`	User 1 đăng nhập	Ranh giới số nguyên: ID bằng 0 ($id = 0$)	IEP	`id = 0`	HTTP 404 Not Found (`{"error": "Order not found"}`)	CSDL giữ nguyên 100%, không có bản ghi nào bị cập nhật	[API Contract / SUT Behavior]	Cô lập duy nhất `target_order_id = 0`. Backend truy vấn không khớp bản ghi nên từ chối 404 an toàn.
`ORD-INP-IEP02`	`target_order_id`	User 1 đăng nhập	Miền số nguyên âm ($id < 0$)	IEP	`id = -1`	HTTP 404 Not Found (`{"error": "Order not found"}`)	CSDL giữ nguyên 100%, không bị ảnh hưởng	[API Contract / SUT Behavior]	Cô lập duy nhất `target_order_id = -1`. Không khớp bản ghi trong CSDL.
`ORD-INP-IEP03`	`target_order_id`	User 1 đăng nhập	Sai kiểu dữ liệu: Chuỗi phi số / ký tự chữ	IEP	`id = "abc"`	HTTP 404 Not Found (`{"error": "Order not found"}`)	CSDL giữ nguyên 100%, tiến trình Node.js không crash	[API Contract / Clarification]	Cô lập `id = "abc"`. Router Express chuyển chuỗi vào câu truy vấn SQLite, không tìm thấy đơn và trả về 404 an toàn.
`ORD-INP-IEP04`	`target_order_id`	User 1 đăng nhập	Sai kiểu dữ liệu: Số thập phân	IEP	`id = 101.5`	HTTP 404 Not Found (`{"error": "Order not found"}`)	CSDL giữ nguyên 100%, không nhận diện nhầm ID 101	[API Contract / SUT Behavior]	Cô lập `id = 101.5`. SQLite không khớp số nguyên và từ chối xử lý.
`ORD-INP-VEP02`	Request Body	User 1 đăng nhập	JSON Object rỗng `{}`	VEP	`{}`	HTTP 200 OK	`orders.status` chuyển sang `canceled`	[API Contract]	Baseline chuẩn: Endpoint không yêu cầu body payload.
`ORD-INP-IEP05`	Request Body	User 1 đăng nhập	Body chứa trường không mong muốn nhằm can thiệp trạng thái	IEP / Probe	`{"status": "delivered"}`	HTTP 200 OK (trường lạ bị bỏ qua an toàn)	`orders.status` chuyển sang `canceled` (tuyệt đối KHÔNG thành `delivered`)	[Security / Input Handling]	Baseline chuẩn, chỉ chèn thêm property lạ vào body. Đảm bảo server không gán trực tiếp req.body vào SQL UPDATE.

5.2.2 Bảng Phân Vùng Điều Kiện Truy Cập, Xác Thực & Quyền Sở Hữu (Authentication, Authorization & Ownership)

Class ID	Variable / Constraint	Precondition	Domain / Partition Definition	Type (VEP / IEP)	Representative Value	Expected HTTP & Error Message	Expected Database State (Invariant)	Specification Source & Status	Single-Fault Baseline & Ra
`ORD-AUTH-VEP01`	Xác thực & Quyền sở hữu	User 1 đăng nhập	Token JWT hợp lệ, khớp chính chủ đơn hàng (`req.user.id === order.user_id`)	VEP	`user_id = 1`, `order.user_id = 1`	HTTP 200 OK	`orders.status = 'canceled'`	[SRS Confirmed FR-11 & SEC-03]	Baseline chuẩn: User 1 thao trên đơn 101 của chính mìn
`ORD-AUTH-IEP01`	Thiếu xác thực (Unauthenticated)	Gọi API cancel	Thiếu hoàn toàn Header `Authorization`	IEP	Header `Authorization: None`	HTTP 401 Unauthorized (`{"error": "Access denied. No token provided."}`)	Đơn hàng 101 giữ nguyên `pending`, CSDL không đổi	[API Contract `server.js:46` & SEC-03]	Baseline chuẩn, chỉ lược bỏ Authorization. Lỗi xác thực c
`ORD-AUTH-IEP02`	Chữ ký Token không hợp lệ	Gọi API cancel	Token JWT giả mạo hoặc sai Secret Key	IEP	`Authorization: Bearer invalid_sig_jwt`	HTTP 403 Forbidden (`{"error": "Invalid token."}`)	Đơn hàng 101 giữ nguyên `pending`, CSDL không đổi	[API Contract `server.js:50` & SEC-03]	Baseline chuẩn, chỉ thay JW token giả. Lỗi xác thực cấp
`ORD-AUTH-IEP03`	Token đã hết hạn (Expired)	Gọi API cancel	Token JWT có trường `exp` nằm trong quá khứ	IEP	`Authorization: Bearer <expired_token>`	HTTP 403 Forbidden (`{"error": "Invalid token."}`)	Đơn hàng 101 giữ nguyên `pending`, CSDL không đổi	[API Contract `server.js:50` & SEC-03]	Baseline chuẩn, chỉ thay JW token hết hạn. Lỗi xác thực
`ORD-AUTH-IEP04`	**Tấn công IDOR (Cross-user Cancellation)**	User A đăng nhập hợp lệ (`id = 1`)	User A cố tình gửi request hủy đơn hàng 202 của User B (`order.user_id = 2`)	Security / IEP	`token = user1_jwt`, `target_order_id = 202` (của User 2)	HTTP 404 Not Found (`{"error": "Order not found"}`)	Đơn hàng 202 của User B giữ nguyên 100% (vẫn là `pending`), không bị hủy	[SRS Confirmed FR-11 & Security Policy]	Token User 1 hợp lệ, đơn 20 lệ (pending), chỉ vi phạm qu sở hữu. Cơ chế bảo vệ riên giấu sự tồn tại của đơn hàn khác.
`ORD-AUTH-IEP05`	Phân quyền Admin vs User	Admin đăng nhập (`role = 'admin'`)	Admin gọi endpoint khách hàng `PUT /api/orders/101/cancel` trên đơn của user khác	Conditional / IEP	`token = admin_jwt`, `id = 101` (đơn của User 1)	HTTP 404 Not Found (`{"error": "Order not found"}`)	Đơn hàng 101 giữ nguyên `pending`	[API Contract]	Endpoint người dùng lọc cứ `WHERE id = ? AND user_` Admin bắt buộc phải dùng A trị `/api/admin/orders/:id/`

5.2.3 Bảng Phân Vùng Tham Số Tài Nguyên & Vòng Đời CSDL (Target-Resource & Lifecycle Partitions)

Class ID	Variable / Constraint	Precondition	Domain / Partition Definition	Type (VEP / IEP)	Representative Value	Expected HTTP & Error Message	Expected Database State (Invariant)	Specification Source & Status	Single-Fault Baseline & Rationale
`ORD-RES-VEP01`	Tài nguyên tồn tại trong DB	User 1 đăng nhập	Đơn hàng có bản ghi thực tế trong bảng `orders` của SQLite	VEP	`order_id = 101`	HTTP 200 OK (chuyển sang xử lý logic State Machine)	Tìm thấy bản ghi đơn hàng trong bảng `orders`	[SRS Confirmed]	Bản ghi đang tồn tại và thuộc sở hữu của User 1.
`ORD-RES-IEP01`	Tài nguyên không tồn tại	User 1 đăng nhập	ID đơn hàng hoàn toàn không có trong bảng `orders`	IEP	`id = 99999`	HTTP 404 Not Found (`{"error": "Order not found"}`)	CSDL giữ nguyên 100%, không có side-effect nào xảy ra	[API Contract `server.js:326`]	Baseline chuẩn, chỉ có `id` là không tồn tại trong hệ thống.

5.2.4 Bảng Quy Tắc Nghiệp Vụ & Chuyển Đổi Trạng Thái Đơn Hàng (Business Rules & State Machine Scenarios)

Class ID	Chuyển đổi trạng thái	Tiền điều kiện trạng thái ban đầu	Miền / Quy tắc theo State Machine FR-10	Phân loại	Giá trị thử nghiệm	Expected HTTP & Error Message	Expected Database State (Invariant)	Specification Source & Status	Single-Fault Baseline & Rationale
`ORD-STA-VEP01`	`pending` $\to$ `canceled`	Đơn của User 1, status = `pending`	Cho phép User tự hủy đơn ở bước chờ xác nhận	VEP	`order.status = 'pending'`	HTTP 200 OK (`{"message": "Order canceled successfully"}`)	`orders.status` chuyển thành `canceled`	[SRS Confirmed FR-10 & FR-20]	Baseline chuẩn hoàn hảo: Khớp 100% đặc tả State Machine.
`ORD-STA-VEP02`	`confirmed` $\to$ `canceled`	Đơn của User 1, status = `confirmed`	Cho phép User tự hủy đơn ở bước đã xác nhận (Biên hợp lệ cuối cùng)	VEP	`order.status = 'confirmed'`	HTTP 200 OK (`{"message": "Order canceled successfully"}`)	`orders.status` chuyển thành `canceled`	[SRS Confirmed FR-10 & FR-20]	Chỉ thay đổi trạng thái khởi đầu của đơn từ `pending` sang `confirmed`.
`ORD-STA-IEP01`	`shipping` $\to$ `canceled`	Đơn của User 1, status = `shipping`	**Cấm User tự hủy khi đơn đang giao** (Chỉ Admin mới có quyền can thiệp)	IEP	`order.status = 'shipping'`	**SRS FR-10 Kỳ vọng:** HTTP 400 Bad Request (`{"error": "Cannot cancel order in shipping status"}`)	`orders.status` bắt buộc giữ nguyên `shipping` (Tuyệt đối không được đổi sang `canceled`)	[SRS Confirmed FR-10 & SUT Defect Hotspot]	Giữ nguyên mọi tham số baseline, chỉ thay đổi trạng thái đơn sang `shipping`. Đây là điểm kiểm thử trọng yếu phát hiện lỗi SUT backend thiếu kiểm tra `shipping`.
`ORD-STA-IEP02`	`delivered` $\to$ `canceled`	Đơn của User 1, status = `delivered`	**Cấm hủy đơn đã giao thành công** (Trạng thái kết thúc - Final State)	IEP	`order.status = 'delivered'`	HTTP 400 Bad Request (`{"error": "Cannot cancel this order."}`)	`orders.status` giữ nguyên `delivered`, không bị sửa đổi	[SRS Confirmed FR-10 & API Contract]	Giữ nguyên mọi tham số baseline, chỉ thay đổi trạng thái đơn sang `delivered`.
`ORD-STA-IEP03`	`canceled` $\to$ `canceled`	Đơn của User 1, status = `canceled`	**Cấm hủy lại đơn đã hủy trước đó** (Trạng thái kết thúc - Final State)	IEP	`order.status = 'canceled'`	HTTP 400 Bad Request (`{"error": "Cannot cancel this order."}`)	`orders.status` giữ nguyên `canceled`, không thực hiện cập nhật dư thừa	[SRS Confirmed FR-10 & API Contract]	Giữ nguyên mọi tham số baseline, chỉ thay đổi trạng thái đơn sang `canceled`. Kiểm chứng tính bất biến trạng thái kết thúc.
`ORD-STA-IEP04`	`unknown` $\to$ `canceled`	Đơn của User 1, status bất thường trong DB	Trạng thái lạ không nằm trong State Machine chuẩn	Robustness	`order.status = 'corrupted_state'`	HTTP 400 Bad Request	CSDL giữ nguyên trạng thái cũ, không thay đổi	[Assumption-based / Robustness]	Kiểm tra độ vững của máy trạng thái trước dữ liệu bất thường trong CSDL.
`ORD-STA-CLAR01`	**Side-effect Tồn kho: Hoàn trả số lượng vào kho (Restock)**	Đơn của User 1 hủy thành công ở `pending`, chứa sản phẩm A ($Q = 2$, kho trước hủy = 10)	Nghiệp vụ hoàn trả tồn kho khi đơn bị hủy	Clarification / Open Question	Đơn 2 sản phẩm A	HTTP 200 OK	**Quan sát SUT:** Kiểm tra xem `products.stock` của sản phẩm A có được $+2$ lên 12 hay giữ nguyên 10.	[Unconfirmed Spec / Clarification Required]	Đặc tả FR-10/FR-20 không quy định rõ việc tự động hoàn kho. Ghi nhận là mục làm rõ nghiệp vụ (Open Clarification) thay vì khẳng định là lỗi nếu không hoàn kho.
`ORD-STA-CLAR02`	**Side-effect Voucher: Khôi phục lượt dùng mã giảm giá**	Đơn của User 1 có áp dụng coupon `SAVE10`, sau đó bị hủy thành công	Nghiệp vụ hoàn lại lượt dùng mã voucher	Clarification / Open Question	Đơn áp dụng `SAVE10`	HTTP 200 OK	**Quan sát SUT:** Kiểm tra xem User 1 có thể tái sử dụng mã `SAVE10` ở đơn hàng tiếp theo hay không.	[Unconfirmed Spec / Clarification Required]	Đặc tả FR-09/FR-10 không quy định rõ cơ chế hoàn lượt coupon khi hủy đơn. Ghi nhận là câu hỏi làm rõ hành vi hệ thống.

5.2.5 Bộ Kiểm Thử Bổ Trợ: An Toàn Thông Tin, Trải Nghiệm UI & Độ Bền Vững (Complementary Test Suites)

> *Lưu ý: Các ca kiểm thử trong bảng dưới đây thuộc nhóm Kỹ thuật Bổ trợ (Security, GUI Safety, Concurrency, Protocol Robustness), được bóc tách độc lập và KHÔNG tính gộp vào độ bao phủ Phân vùng Tương đương Dữ liệu Đầu vào thuần túy (Pure Input EP).*

Suite ID	Phân loại kỹ thuật	Tiền điều kiện	Thao tác kích hoạt / Dữ liệu thử nghiệm	Oracle Kiểm Thử & Tiêu Chí Đánh Giá Toàn Diện	CSDL Invariant & Tác Tác Dụng Phụ	Specification Source & Rationale
`ORD-SEC-01`	**SQL Injection Probe**	Đã đăng nhập User 1	Tham số `:id` chứa payload SQLi: `101 OR 1=1` hoặc `101; DROP TABLE orders;`	Backend sử dụng parameterized query (`WHERE id = ? AND user_id = ?`). SQLite từ chối an toàn với HTTP 404 Not Found, không thực thi mã độc.	Toàn bộ bảng `orders` và CSDL SQLite nguyên vẹn 100%, không có bản ghi nào bị xóa hay cập nhật sai lệch.	[SRS SEC-01 & Security Standard]
`ORD-SEC-02`	**UI Confirmation Safety (FR-21 / FR-24)**	Mở Mobile App, màn hình Profile, đơn đang `pending`	1. Bấm nút "Hủy đơn" màu đỏ. 2. Chọn "Hủy bỏ" trên hộp thoại. 3. Bấm lại nút và chọn "Đồng ý".	- Khi bấm nút: Hiển thị `Alert.alert("Xác nhận", ...)` . - Khi chọn "Hủy bỏ": Không gửi request API, giữ nguyên trạng thái `pending` . - Khi chọn "Đồng ý": Gửi API cancel và cập nhật UI sang "Đã hủy".	CSDL chỉ cập nhật trạng thái khi người dùng đã bấm xác nhận "Đồng ý".	[SRS FR-21 / FR-24 UX Safety]
`ORD-SEC-03`	**Client-Side-Only Bypass Probe (Critical SUT Probe)**	Đơn hàng đang ở `shipping` , Mobile UI đã ẩn nút Hủy	Dùng curl/Postman gửi trực tiếp `PUT /api/orders/103/cancel` với JWT của User 1	Backend API PHẢI từ chối với HTTP 400 Bad Request. Nếu backend chấp nhận và trả về 200 $\to$ **Xác nhận Defect SUT nghiêm trọng** do chỉ validate ở frontend.	Trạng thái của đơn 103 trong CSDL SQLite bắt buộc phải giữ nguyên là `shipping` .	[SUT Implementation Probe / Bug Hotspot]
`ORD-SEC-04`	**Concurrent Cancellation (Race Condition & Exactly-Once)**	Đơn hàng 101 đang ở `pending`	Gửi đồng thời 2 request `PUT /api/orders/101/cancel` trong cùng 1 mili-giây	**Exactly-once Oracle:** Trạng thái cuối cùng của đơn hàng là `canceled` . Tại hầu hết 1 thao tác chuyển trạng thái được áp dụng. Đúng 1 request nhận HTTP 200, request kia nhận HTTP 400 (hoặc xử lý idempotent).	Không xảy ra lỗi khóa chết CSDL (Deadlock) hay cập nhật trùng lặp side-effect.	[Robustness & Concurrency Oracle]
`ORD-SEC-05`	**HTTP Method Tampering**	Đơn hàng 101 đang ở `pending`	Gửi request tới `/api/orders/101/cancel` bằng các method sai: `GET` , `POST` , `DELETE`	Hệ thống từ chối an toàn với HTTP 404 Not Found hoặc 405 Method Not Allowed; không thực hiện bất kỳ hành vi chuyển trạng thái nào.	Đơn hàng 101 giữ nguyên trạng thái `pending` trong CSDL.	[Protocol Robustness]
`ORD-SEC-06`	**Malformed JSON Payload**	Đơn hàng 101 đang ở `pending`	Gửi request `PUT` với Content-Type `application/json` nhưng body là chuỗi lỗi cú pháp `"{broken_json"`	Express `body-parser` bắt lỗi cú pháp và trả về HTTP 400 Bad Request an toàn trước khi vào handler nghiệp vụ.	Đơn hàng 101 giữ nguyên trạng thái `pending` trong CSDL.	[Request Parsing Robustness]

5.3 Bước 3: Phân tích giá trị biên (Step 3 Deliverables - Boundary Value Analysis - BVA)

> **Cơ sở Lý luận & Phương Pháp Luận Phân Tích Giá Trị Biên (ISTQB FL §4.2.2):**
>
> - **Áp dụng kỹ thuật 3-Value BVA:** Đối với các miền giá trị có thứ tự và ranh giới xác định, kỹ thuật 3-Value BVA được áp dụng nhằm bao phủ toàn diện 3 điểm kiểm thử: **On-point** (nằm ngay trên ranh giới), **Off-point** (điểm liền kề ngoài ranh giới), và **In-point** (điểm liền kề bên trong ranh giới).
> - **Xác định bước nhảy đơn vị đo lường (ϵ):**
>   - Tham số định danh `:id` : $\epsilon = 1$ (số nguyên dương).
>   - Thứ tự tiến trình vòng đời đơn hàng: $\epsilon = 1\text{ bước chuyển}$ (`pending` $\to$ `confirmed` $\to$ `shipping` $\to$ `delivered`).
>   - Số lần gửi yêu cầu hủy (N_{cancel}): $\epsilon = 1\text{ lần}$.
>   - Số lượng request đồng thời (k_{req}): $\epsilon = 1\text{ request}$.
> - **Phân tách ranh giới số học vs Ranh giới trạng thái & tiến trình:**
>   - BVA số học được áp dụng cho tham số đường dẫn `:id` .
>   - BVA tiến trình (State Machine Boundary) được áp dụng cho ranh giới quyền tự hủy của User và ranh giới trạng thái kết thúc (Terminal States).

5.3.1 Bảng Phân Tích Giá Trị Biên Dữ Liệu & Ranh Giới Chuyển Đổi Trạng Thái (Order BVA Table)

Boundary ID	Variable / Boundary Field	Quy tắc ranh giới (Boundary Rule)	BVA Method	On-point (Giá trị / Phân loại)	Off-point (Giá trị / Phân loại)	In-point (Giá trị / Phân loại)	Rationale & Hành vi Kỳ Vọng theo Đặc Tả
`ORD-BVA-01`	`target_order_id` *(Biên dưới ID đơn hàng)*	ID phải là số nguyên dương: $id \ge 1$ ($\epsilon = 1$)	3-Value BVA	`id = 1` *(Valid On-point)*	`id = 0` *(Invalid Off-point)*	`id = 2` *(Valid In-point)*	- **On (1):** Đơn hàng nhỏ nhất hợp lệ $\to$ HTTP 200 OK (nếu đơn tồn tại và thuộc User). - **Off (0):** Không thuộc miền ID hợp lệ $\to$ HTTP 404 Not Found, CSDL giữ nguyên. - **In (2):** Đơn hàng liền kề hợp lệ $\to$ HTTP 200 OK.

Boundary ID	Variable / Boundary Field	Quy tắc ranh giới (Boundary Rule)	BVA Method	On-point (Giá trị / Phân loại)	Off-point (Giá trị / Phân loại)	In-point (Giá trị / Phân loại)	Rationale & Hành vi Kỳ Vọng theo Đặc Tả
`ORD-BVA-02`	`target_order_id` *(Biên trên lưu trữ SQLite 64-bit)*	Ngưỡng số nguyên signed 64-bit SQLite: $id \le 2^{63}-1$ ($\epsilon = 1$)	3-Value BVA	`id = 9223372036854775807` *(Valid On-point / Robustness)*	`id = 9223372036854775808` *(Invalid Off-point / Overflow)*	`id = 9223372036854775806` *(Valid In-point / Robustness)*	- **On ($2^{63}-1$):** Cận trên kiểu INTEGER SQLite $\to$ HTTP 404 Not Found (không có bản ghi nhưng không crash). - **Off (2^{63}):** Tràn số nguyên signed 64-bit $\to$ Hệ thống xử lý an toàn (HTTP 404/400), không crash SQLite process. - **In ($2^{63}-2$):** Số nguyên lớn an toàn $\to$ HTTP 404.
`ORD-BVA-03`	`order_state_eligibility` *(Ranh giới quyền User tự hủy đơn)*	User chỉ được tự hủy khi đơn $\in {\text{pending}, \text{confirmed}}$. Khi sang `shipping`, User mất hoàn toàn quyền tự hủy. ($\epsilon = 1\text{ trạng thái}$)	3-Value BVA	`status = 'confirmed'` *(Valid On-point)*	`status = 'shipping'` *(Invalid Off-point)*	`status = 'pending'` *(Valid In-point)*	- **On (`confirmed`):** Ranh giới hợp lệ cuối cùng cho User tự hủy $\to$ HTTP 200 OK, CSDL chuyển sang `canceled`, Mobile UI cập nhật "Đã hủy". - **Off (`shipping`):** Ngay khi đơn chuyển sang giao hàng, User bị tước quyền tự hủy $\to$ Mobile UI ẩn nút Hủy; Backend API từ chối với HTTP 400 Bad Request, giữ nguyên `shipping`. - **In (`pending`):** Trạng thái khởi đầu hợp lệ $\to$ HTTP 200 OK.
`ORD-BVA-04`	`final_state_delivered` *(Ranh giới trạng thái kết thúc giao hàng)*	`delivered` là trạng thái kết thúc (Final State) — cấm chuyển sang bất kỳ trạng thái nào khác. (SRS FR-10)	2-Value BVA	`status = 'delivered'` *(Invalid On-point)*	`status = 'confirmed'` *(Valid Off-point)*	N/A	- **On (`delivered`):** Đơn đã hoàn tất giao hàng $\to$ Bấm hủy/gửi API bị chặn hoàn toàn: HTTP 400 Bad Request (`Cannot cancel this order.`), CSDL giữ nguyên `delivered`. - **Off (`confirmed`):** Trạng thái chưa hoàn tất cho phép hủy $\to$ HTTP 200 OK.
`ORD-BVA-05`	`cancellation_frequency` *(Ranh giới hủy lặp lại - Double Cancel)*	Mỗi đơn hàng chỉ được chuyển sang `canceled` đúng 1 lần ($N_{\text{cancel}} = 1$). Khi đã ở `canceled`, cấm hủy tiếp. ($\epsilon = 1\text{ lần}$)	2-Value BVA	$N_{\text{cancel}} = 1$ *(Valid On-point)*	$N_{\text{cancel}} = 2$ *(Invalid Off-point)*	N/A	- **On ($N=1$):** Yêu cầu hủy lần 1 khi đơn đang `pending` $\to$ Thành công, HTTP 200 OK, CSDL chuyển `status = 'canceled'`. - **Off ($N=2$):** Gửi tiếp yêu cầu hủy lần 2 khi đơn đã là `canceled` $\to$ Bị từ chối: HTTP 400 Bad Request (`Cannot cancel this order.`), CSDL giữ nguyên không cập nhật dư thừa.
`ORD-BVA-06`	`concurrency_requests` *(Ranh giới xử lý đồng thời - Race Condition)*	Gửi request hủy đồng thời tại cùng thời điểm (k_{req}, $\epsilon = 1\text{ request}$)	2-Value BVA	$k_{\text{req}} = 1$ *(Valid On-point)*	$k_{\text{req}} = 2$ *(Off-point Concurrency)*	N/A	- **On ($k=1$):** Request đơn lẻ bình thường $\to$ HTTP 200 OK. - **Off ($k=2$):** 2 request đồng thời $\to$ **Exactly-Once Oracle:** Đúng 1 request nhận HTTP 200, request kia nhận HTTP 400; CSDL cập nhật đúng 1 lần, không gây lỗi khóa chết SQLite (Deadlock) hay tác dụng phụ trùng lặp.
`ORD-BVA-07`	`ui_tap_frequency` *(Ranh giới chạm nút Hủy trên Mobile UI)*	Thao tác chạm nút "Hủy đơn" trên màn hình cảm ứng Mobile ($\epsilon = 1\text{ lần chạm}$)	2-Value BVA	1 lần chạm *(Valid On-point)*	$2\text{ lần chạm liên tiếp (Rapid Double-tap)}$ *(Robustness Off-point)*	N/A	- **On (1 tap):** Hiển thị đúng 1 hộp thoại xác nhận `Alert.alert`. - **Off (2 taps liên tiếp $< 200ms$):** Nút bị vô hiệu hóa / debounce an toàn, không mở 2 hộp thoại xác nhận đè nhau hay gửi 2 request API trùng lặp.

5.4 Bước 4: Thiết kế kịch bản kiểm thử (Step 4 Deliverables - Test Case Design)

> **Nguyên Tắc Thiết Kế Kịch Bản Kiểm Thử & Chuẩn Hóa Hồ Sơ Đặc Tả (ISTQB FL §4.2):**
>
> - **Nguyên tắc Phối hợp VEP (Positive Cases):** Kết hợp các phân vùng tương đương hợp lệ (VEP) và điểm biên hợp lệ (On-point/In-point) trong cùng một ca kiểm thử để tối ưu hóa lộ trình luồng nghiệp vụ chuẩn.
> - **Nguyên tắc Giả định Đơn lỗi (Single-Fault Assumption):** Mỗi ca kiểm thử phủ định (Negative Case) chỉ kích hoạt đúng 1 điều kiện không hợp lệ (1 IEP hoặc 1 Off-point); toàn bộ các biến/trạng thái còn lại bắt buộc phải được cố định ở **Bộ Giá trị Cơ sở (Standard Baseline)**:
>   - Đơn hàng mục tiêu: `id = 101`, trạng thái khởi đầu: `'pending'`, thuộc sở hữu của User 1 (`user_id = 1`).
>   - Xác thực: Bearer JWT Token hợp lệ của User 1 (`role = 'user'`).
>   - Payload Body: `{}`. Endpoint: `PUT /api/orders/101/cancel`.
> - **100% Standalone Detailed Test Specification Cards:** Toàn bộ 28 ca kiểm thử trong Ma trận truy vết dưới đây đều được xây dựng hồ sơ kiểm thử độc lập, hoàn chỉnh theo chuẩn 7 phần (Tracking, Objective, Preconditions, Test Data, Procedure, Multi-tier Validations, Teardown).

5.4.1 Bảng Tổng Hợp Kịch Bản Kiểm Thử & Ma Trận Truy Vết (Overview Traceability Matrix)

TC ID	Mục tiêu kiểm thử ($\text{Action} + \text{Function} + \text{Condition}$)	Phân loại	Mã EP / Rule bao phủ	Mã BVA bao phủ	Dữ liệu đầu vào & Baseline	Kết quả cốt lõi (Core Expected)	Trạng thái
`TC_MOB_01`	User hủy đơn hàng hợp lệ ở trạng thái `pending` (Baseline)	Positive	`ORD-INP-VEP01`, `ORD-INP-VEP02`, `ORD-AUTH-VEP01`, `ORD-RES-VEP01`, `ORD-STA-VEP01`	`ORD-BVA-03` (In: `pending`), `ORD-BVA-05` (On: $N=1$)	Đơn 101, User 1, `status = 'pending'`, JWT User 1, body `{}`	HTTP 200 OK; `orders.status` cập nhật `canceled`; UI đổi nhãn sang "Đã hủy"	**Pass**
`TC_MOB_02`	User hủy đơn hàng ở trạng thái `confirmed` (Ranh giới hợp lệ cuối)	Positive	`ORD-STA-VEP02`	`ORD-BVA-03` (On: `confirmed`), `ORD-BVA-04` (Off: `confirmed`)	Đơn 102, User 1, `status = 'confirmed'`, JWT User 1, body `{}`	HTTP 200 OK; `orders.status` cập nhật `canceled`; UI đổi nhãn sang "Đã hủy"	**Pass**
`TC_MOB_03`	User hủy đơn hàng có ID nhỏ nhất hợp lệ ($id = 1$)	Positive	`ORD-INP-VEP01`	`ORD-BVA-01` (On: `1`)	Đơn 1, User 1, `status = 'pending'`, JWT User 1	HTTP 200 OK; `orders.status` cập nhật `canceled`	**Pass**
`TC_MOB_04`	User hủy đơn hàng có ID liền kề trong miền hợp lệ ($id = 2$)	Positive	`ORD-INP-VEP01`	`ORD-BVA-01` (In: `2`)	Đơn 2, User 1, `status = 'pending'`, JWT User 1	HTTP 200 OK; `orders.status` cập nhật `canceled`	**Pass**
`TC_MOB_05`	User cố tình hủy đơn đang ở `shipping` qua API (Bypass UI)	Negative (SUT Probe)	`ORD-STA-IEP01`, `ORD-SEC-03`	`ORD-BVA-03` (Off: `shipping`)	Đơn 103, User 1, `status = 'shipping'`, JWT User 1, gửi raw API `PUT`	HTTP 400 Bad Request; CSDL giữ nguyên `shipping`; Phát hiện lỗi nếu backend trả 200	**Fail (BUG-FR20-01)**
`TC_MOB_06`	User gửi yêu cầu hủy đơn đã giao hàng thành công (`delivered`)	Negative	`ORD-STA-IEP02`	`ORD-BVA-04` (On: `delivered`)	Đơn 104, User 1, `status = 'delivered'`, JWT User 1	HTTP 400 Bad Request (`Cannot cancel this order.`); CSDL giữ nguyên `delivered`	**Pass**
`TC_MOB_07`	User gửi yêu cầu hủy lại đơn hàng đã hủy trước đó ($N=2$)	Negative	`ORD-STA-IEP03`	`ORD-BVA-05` (Off: $N=2$)	Đơn 105, User 1, `status = 'canceled'`, gửi API cancel lần 2	HTTP 400 Bad Request (`Cannot cancel this order.`); CSDL giữ nguyên `canceled`	**Pass**
`TC_MOB_08`	Đơn hàng có trạng thái dị thường trong DB (`corrupted_state`)	Robustness	`ORD-STA-IEP04`	N/A	Đơn 106, DB sửa cứng `status = 'corrupted_state'`	HTTP 400 Bad Request; CSDL giữ nguyên trạng thái cũ	**Fail (BUG-FR20-02)**
`TC_MOB_09`	User gửi yêu cầu hủy đơn có ID bằng 0 (Biên dưới Off-point)	Negative	`ORD-INP-IEP01`	`ORD-BVA-01` (Off: `0`)	Baseline chuẩn, thay `:id = 0`	HTTP 404 Not Found (`Order not found`); CSDL không đổi	**Pass**
`TC_MOB_10`	User gửi yêu cầu hủy với ID số nguyên âm ($id = -1$)	Negative	`ORD-INP-IEP02`	N/A	Baseline chuẩn, thay `:id = -1`	HTTP 404 Not Found (`Order not found`); CSDL không đổi	**Pass**
`TC_MOB_11`	User gửi yêu cầu hủy với ID chuỗi ký tự phi số (`id = "abc"`)	Negative	`ORD-INP-IEP03`	N/A	Baseline chuẩn, thay `:id = "abc"`	HTTP 404 Not Found (`Order not found`); tiến trình Node.js không crash	**Pass**
`TC_MOB_12`	User gửi yêu cầu hủy với ID dạng số thập phân (`id = 101.5`)	Negative	`ORD-INP-IEP04`	N/A	Baseline chuẩn, thay `:id = 101.5`	HTTP 404 Not Found (`Order not found`); CSDL không đổi	**Pass**
`TC_MOB_13`	Gửi yêu cầu hủy với ID số nguyên cực lớn tràn số 64-bit	Robustness	`ORD-INP-IEP05`	`ORD-BVA-02` (Off: 2^{63})	Baseline chuẩn, `:id = 9223372036854775808`	HTTP 404/400 an toàn; SQLite không bị crash exception	**Pass**
`TC_MOB_14`	Gửi yêu cầu hủy với ID đơn hàng không tồn tại trong hệ thống	Negative	`ORD-RES-IEP01`	N/A	Baseline chuẩn, `:id = 99999`	HTTP 404 Not Found (`Order not found`); CSDL không đổi	**Pass**

TC ID	Mục tiêu kiểm thử ($\text{Action} + \text{Function} + \text{Condition}$)	Phân loại	Mã EP / Rule bao phủ	Mã BVA bao phủ	Dữ liệu đầu vào & Baseline	Kết quả cốt lõi (Core Expected)	Trạng thái
`TC_MOB_15`	Body chứa trường lạ `{"status": "delivered"}` can thiệp dữ liệu	Security / Probe	`ORD-INP-IEP05`	N/A	Đơn 101, body `{"status": "delivered"}`	HTTP 200 OK; `orders.status` là `canceled` (Tuyệt đối KHÔNG thành `delivered`)	**Pass**
`TC_MOB_16`	Gửi request body dạng JSON sai cú pháp (`"{broken_json"`)	Robustness	`ORD-SEC-06`	N/A	Header JSON, body `"{broken_json"`	HTTP 400 Bad Request từ express body-parser; CSDL giữ nguyên `pending`	**Pass**
`TC_MOB_17`	Gửi request hủy đơn không kèm Header `Authorization`	Negative	`ORD-AUTH-IEP01`	N/A	Đơn 101, Header `Authorization: None`	HTTP 401 Unauthorized (`Access denied. No token provided.`); CSDL không đổi	**Pass**
`TC_MOB_18`	Gửi request hủy đơn với Token JWT sai chữ ký / giả mạo	Negative	`ORD-AUTH-IEP02`	N/A	Đơn 101, `Authorization: Bearer fake_jwt_xyz`	HTTP 403 Forbidden (`Invalid token.`); CSDL không đổi	**Pass**
`TC_MOB_19`	Gửi request hủy đơn với Token JWT đã hết hạn (`expired`)	Negative	`ORD-AUTH-IEP03`	N/A	Đơn 101, `Authorization: Bearer <expired_jwt>`	HTTP 403 Forbidden (`Invalid token.`); CSDL không đổi	**Pass**
`TC_MOB_20`	Tấn công IDOR: User 1 cố tình gửi request hủy đơn 202 của User 2	Security	`ORD-AUTH-IEP04`	N/A	Token User 1, `:id = 202` (thuộc User 2, `pending`)	HTTP 404 Not Found (`Order not found`); Đơn 202 của User 2 giữ nguyên `pending`	**Pass**
`TC_MOB_21`	Tài khoản Admin gọi endpoint hủy của khách hàng	Conditional	`ORD-AUTH-IEP05`	N/A	Token Admin, `:id = 101` (đơn của User 1)	HTTP 404 Not Found (`Order not found`); Đơn 101 giữ nguyên `pending`	**Pass**
`TC_MOB_22`	Tấn công SQL Injection qua tham số URL `:id`	Security	`ORD-SEC-01`	N/A	URL: `/api/orders/101%20OR%201=1/cancel`	HTTP 404 Not Found; CSDL SQLite nguyên vẹn, không có đơn hàng nào bị ảnh hưởng	**Pass**
`TC_MOB_23`	Kiểm thử hộp thoại xác nhận an toàn khi bấm Hủy trên Mobile UI	UI Safety	`ORD-SEC-02`	`ORD-BVA-07` (On: 1 tap)	Đơn 101 trên Mobile UI: Bấm "Hủy đơn" $\to$ Hủy bỏ / Đồng ý	Hiện Alert; chọn "Hủy bỏ" $\to$ không gửi API, giữ `pending`; chọn "Đồng ý" $\to$ gửi API	**Fail (BUG-FR20-03)**
`TC_MOB_24`	Bấm liên tiếp 2 lần cực nhanh nút Hủy trên Mobile (Double-tap)	UI Robustness	`ORD-BVA-07`	`ORD-BVA-07` (Off: 2 taps)	Mobile UI: Chạm nút "Hủy đơn" 2 lần $< 200ms$	Debounce xử lý an toàn: Chỉ mở 1 hộp thoại xác nhận, không gửi request trùng lặp	**Pass**
`TC_MOB_25`	Gửi đồng thời 2 request hủy trong cùng 1 mili-giây (Race Condition)	Concurrency	`ORD-SEC-04`	`ORD-BVA-06` (Off: $k=2$)	Đơn 101 `pending`, 2 thread gửi `PUT` đồng thời	**Exactly-once:** 1 request HTTP 200, 1 request HTTP 400; DB cập nhật đúng 1 lần	**Pass**
`TC_MOB_26`	Xáo trộn phương thức HTTP (`POST`, `GET`, `DELETE` tới API cancel)	Robustness	`ORD-SEC-05`	N/A	Gửi `POST /api/orders/101/cancel`	HTTP 404 Not Found hoặc 405 Method Not Allowed; CSDL giữ nguyên `pending`	**Pass**
`TC_MOB_27`	Quan sát tác dụng phụ hoàn trả kho khi đơn hàng bị hủy	Clarification	`ORD-STA-CLAR01`	N/A	Đơn 101 hủy thành công (gồm 2 SP A, kho trước hủy = 10)	HTTP 200; Quan sát `products.stock` xem có $+2$ lên 12 hay giữ nguyên 10	**Pass**
`TC_MOB_28`	Quan sát tác dụng phụ khôi phục lượt dùng coupon khi đơn bị hủy	Clarification	`ORD-STA-CLAR02`	N/A	Đơn 101 có dùng coupon `SAVE10`, sau đó hủy thành công	HTTP 200; Quan sát xem User 1 có thể áp dụng lại mã `SAVE10` ở đơn tiếp theo không	**Pass**

5.4.2 Hồ Sơ Đặc Tả Chi Tiết Từng Kịch Bản Kiểm Thử (Detailed Test Specification Cards - 100% Exhaustive Coverage)

Nhóm 1: Các Kịch Bản Hủy Đơn Hợp Lệ & Điểm Biên Trạng Thái (Valid Baseline & Transition Boundaries)

Test Case `TC_MOB_01`: User hủy đơn hàng hợp lệ ở trạng thái `pending` (Baseline)

- **1. Tracking Information:**
  - **TC ID:** `TC_MOB_01` | **Priority:** Critical (P0)
  - **Target Layer:** Mobile App (`frontend-mobile/App.js`) & REST API (`PUT /api/orders/:id/cancel`) & SQLite DB
  - **Traceability:** SRS FR-10, FR-20; API Spec §4.6
  - **Coverage IDs:** `ORD-INP-VEP01`, `ORD-INP-VEP02`, `ORD-AUTH-VEP01`, `ORD-RES-VEP01`, `ORD-STA-VEP01`, `ORD-BVA-03` (In-point: `pending`), `ORD-BVA-05` (On-point: $N=1$)
  - **Single-Fault Isolation:** Không áp dụng (Positive Baseline Test Case kết hợp toàn bộ các điều kiện hợp lệ cơ sở).
- **2. Test Objective:** Thực hiện thao tác Hủy đơn hàng đối với đơn hàng đang ở trạng thái `pending` của chính người dùng và xác nhận hệ thống cập nhật trạng thái sang `canceled` trên mọi tầng.

- **3. Pre-conditions & Environment:**
  - Hệ thống EShop backend và SQLite DB đang chạy bình thường.
  - Người dùng User 1 (`user_id = 1`) đã đăng nhập thành công trên Mobile App, sở hữu Token JWT hợp lệ.
  - Trong bảng `orders` , tồn tại đơn hàng `id = 101` với `user_id = 1` và `status = 'pending'` .
- **4. Test Data:**
  - Target URL: `PUT http://192.168.10.13:3000/api/orders/101/cancel`
  - Headers: `Authorization: Bearer <user1_jwt>` , `Content-Type: application/json`
  - Body: `{}`
- **5. Step-by-Step Procedure:**
  1. Mở ứng dụng EShop Mobile App, chuyển đến màn hình "Hồ sơ" (Profile).
  2. Cuộn đến danh sách đơn hàng của tôi, định vị đơn hàng `#101` .
  3. Quan sát thấy nhãn trạng thái hiển thị "Chờ xác nhận" và nút màu đỏ "Hủy đơn" đang hiển thị.
  4. Chạm vào nút "Hủy đơn".
  5. Khi hộp thoại xác nhận xuất hiện, chạm "Đồng ý" (hoặc gửi request `PUT /api/orders/101/cancel` qua HTTP client).
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Client Layer:* Ứng dụng hiển thị thông báo Alert thành công "Hủy đơn thành công!"; danh sách đơn hàng tự động tải lại; nhãn trạng thái đơn `#101` chuyển thành "Đã hủy"; nút "Hủy đơn" màu đỏ biến mất hoàn toàn.
  - *API / Network Layer:* HTTP Status `200 OK` ; Response Body JSON chứa `{"message": "Order canceled successfully"}` .
  - *Database State Layer:* Truy vấn SQLite `SELECT status FROM orders WHERE id = 101` trả về đúng giá trị `'canceled'` . Các trường khác (`total_amount` , `user_id`) giữ nguyên.
  - *Security & Business Rules Layer:* Quy tắc chuyển đổi trạng thái hợp lệ cấp 1 của State Machine FR-10 được thực thi nghiêm ngặt.
- **7. Teardown & Clean-up:** Khôi phục trạng thái đơn `101` bằng câu lệnh SQL `UPDATE orders SET status = 'pending' WHERE id = 101` nếu cần tái sử dụng môi trường test.

Test Case `TC_MOB_02` : User hủy đơn hàng ở trạng thái `confirmed` (Ranh giới hợp lệ cuối cùng)

- **1. Tracking Information:**
  - **TC ID:** `TC_MOB_02` | **Priority:** Critical (P0)
  - **Target Layer:** Mobile App & REST API & SQLite DB
  - **Traceability:** SRS FR-10, FR-20; API Spec §4.6
  - **Coverage IDs:** `ORD-STA-VEP02` , `ORD-BVA-03` (On-point: `confirmed`), `ORD-BVA-04` (Off-point: `confirmed`)
  - **Single-Fault Isolation:** Thay đổi trạng thái ban đầu của đơn hàng mục tiêu từ `pending` sang `confirmed` (Valid On-point ranh giới cuối cho User). Mọi tham số khác giữ nguyên baseline.
- **2. Test Objective:** Kiểm chứng hành vi hủy đơn hàng đối với đơn đã được Admin duyệt sang trạng thái `confirmed` , đảm bảo User vẫn được phép tự hủy theo đúng FR-10.
- **3. Pre-conditions & Environment:**
  - User 1 đăng nhập thành công, có token JWT.
  - Đơn hàng `id = 102` thuộc sở hữu của User 1 (`user_id = 1`), trạng thái ban đầu là `confirmed` .
- **4. Test Data:**
  - URL: `PUT /api/orders/102/cancel` ; Headers: `Authorization: Bearer <user1_jwt>` ; Body: `{}`
- **5. Step-by-Step Procedure:**
  1. Mở màn hình Profile trên Mobile App.
  2. Định vị đơn hàng `#102` đang hiển thị trạng thái "Đã xác nhận".
  3. Xác nhận nút "Hủy đơn" màu đỏ vẫn hiển thị trên giao diện.
  4. Chạm "Hủy đơn" và xác nhận trên hộp thoại Alert.
- **6. Multi-tier Validation Points:**
  - *UI / Client Layer:* Alert "Hủy đơn thành công!"; nhãn trạng thái đổi thành "Đã hủy"; nút "Hủy đơn" bị ẩn.
  - *API / Network Layer:* HTTP Status `200 OK` ; Response JSON: `{"message": "Order canceled successfully"}` .
  - *Database State Layer:* CSDL SQLite `orders.status` chuyển thành `'canceled'` .
  - *Security & Business Rules Layer:* Khớp hoàn toàn ranh giới cho phép của State Machine FR-10.
- **7. Teardown & Clean-up:** `UPDATE orders SET status = 'confirmed' WHERE id = 102` .

Test Case `TC_MOB_03` : User hủy đơn hàng có ID nhỏ nhất hợp lệ ($id = 1$)

- **1. Tracking Information:**
  - **TC ID:** `TC_MOB_03` | **Priority:** High (P1)
  - **Target Layer:** REST API & SQLite DB
  - **Traceability:** API Spec §4.6; ISTQB FL §4.2.2
  - **Coverage IDs:** `ORD-INP-VEP01` , `ORD-BVA-01` (On-point: `1`)
  - **Single-Fault Isolation:** Thay đổi `:id` thành giá trị biên dưới nhỏ nhất ($id = 1$). Đơn 1 thuộc sở hữu User 1, trạng thái `pending` .
- **2. Test Objective:** Xác nhận hệ thống xử lý chính xác giá trị biên dưới số nguyên $id = 1$ trong tham số URL.
- **3. Pre-conditions & Environment:** Đơn hàng `id = 1` tồn tại trong SQLite, `user_id = 1` , `status = 'pending'` .
- **4. Test Data:** `PUT /api/orders/1/cancel` ; Headers: `Authorization: Bearer <user1_jwt>`
- **5. Step-by-Step Procedure:** Gửi request `PUT` đến endpoint `/api/orders/1/cancel` .
- **6. Multi-tier Validation Points:**
  - *API / Network Layer:* HTTP `200 OK` ; `{"message": "Order canceled successfully"}` .
  - *Database State Layer:* `SELECT status FROM orders WHERE id = 1` trả về `'canceled'` .
- **7. Teardown & Clean-up:** `UPDATE orders SET status = 'pending' WHERE id = 1` .

Test Case `TC_MOB_04` : User hủy đơn hàng có ID liền kề trong miền hợp lệ ($id = 2$)

- **1. Tracking Information:**
  - **TC ID:** `TC_MOB_04` | **Priority:** Medium (P2)
  - **Target Layer:** REST API & SQLite DB
  - **Traceability:** API Spec §4.6; ISTQB FL §4.2.2
  - **Coverage IDs:** `ORD-INP-VEP01` , `ORD-BVA-01` (In-point: `2`)
  - **Single-Fault Isolation:** Thay đổi `:id` thành giá trị điểm trong ($id = 2$).
- **2. Test Objective:** Xác nhận điểm In-point liền kề ranh giới dưới xử lý thành công.
- **3. Pre-conditions & Environment:** Đơn hàng `id = 2` tồn tại trong SQLite, `user_id = 1` , `status = 'pending'` .
- **4. Test Data:** `PUT /api/orders/2/cancel` ; Headers: `Authorization: Bearer <user1_jwt>`
- **5. Step-by-Step Procedure:** Gửi request `PUT /api/orders/2/cancel` .
- **6. Multi-tier Validation Points:**
  - *API / Network Layer:* HTTP `200 OK` ; `{"message": "Order canceled successfully"}` .
  - *Database State Layer:* `orders.status` của đơn 2 chuyển thành `'canceled'` .
- **7. Teardown & Clean-up:** `UPDATE orders SET status = 'pending' WHERE id = 2` .

Nhóm 2: Kiểm Thử Ranh Giới Chuyển Đổi Trạng Thái & Trạng Thái Kết Thúc (State Machine Violations & Terminal States)

Test Case `TC_MOB_05` : User cố tình hủy đơn hàng ở trạng thái `shipping` qua API (Bypass UI Probe)

- **1. Tracking Information:**
  - **TC ID:** `TC_MOB_05` | **Priority:** Critical (P0)
  - **Target Layer:** Mobile UI & REST API & SQLite DB
  - **Traceability:** SRS FR-10: *"Khi đơn hàng đã ở trạng thái shipping, User không được phép tự hủy — chỉ Admin mới có thể thao tác."*
  - **Coverage IDs:** `ORD-STA-IEP01` , `ORD-BVA-03` (Off-point: `shipping`), `ORD-SEC-03` (Client-side validation bypass probe)
  - **Single-Fault Isolation:** Cô lập duy nhất trạng thái đơn hàng mục tiêu: `order.status = 'shipping'` (Invalid Off-point ranh giới cấm hủy). Mọi tham số khác giữ nguyên baseline.
- **2. Test Objective:** Kiểm tra tính toàn vẹn của máy trạng thái phía Backend API khi User gửi yêu cầu hủy đối với đơn hàng đang ở trạng thái `shipping` , phát hiện lỗ hổng Client-Side-Only Validation.
- **3. Pre-conditions & Environment:** Đơn hàng `id = 103` thuộc User 1, trạng thái hiện tại là `shipping` .
- **4. Test Data:** URL: `PUT /api/orders/103/cancel` ; Headers: `Authorization: Bearer <user1_jwt>` ; Body: `{}`
- **5. Step-by-Step Procedure:**
  1. Mở màn hình Profile trên Mobile App, kiểm tra đơn hàng `#103` .
  2. Xác nhận nhãn trạng thái là "Đang giao" và nút "Hủy đơn" đã bị ẩn hoàn toàn khỏi giao diện.
  3. Dùng công cụ curl hoặc Postman gửi request trực tiếp `PUT /api/orders/103/cancel` với token của User 1.
  4. Quan sát mã phản hồi HTTP và kiểm tra bản ghi đơn hàng trong CSDL SQLite.
- **6. Multi-tier Validation Points (Expected Results):**
  - *UI / Client Layer:* Nút "Hủy đơn" bắt buộc phải bị ẩn khỏi giao diện.
  - *API / Network Layer (SRS Expected):* HTTP Status `400 Bad Request` (hoặc 403) kèm thông báo lỗi rõ ràng `"Cannot cancel order in shipping status"` .
  - *Database State Layer (Database Invariant):* `SELECT status FROM orders WHERE id = 103` bắt buộc giữ nguyên giá trị `'shipping'` . Tuyệt đối không được chuyển thành `'canceled'` .
  - *Defect Detection Trigger:* Nếu API trả về `HTTP 200 OK` và CSDL bị đổi sang `'canceled'` , xác nhận phát hiện **Defect nghiêm trọng** trên SUT EShop do backend thiếu logic kiểm tra trạng thái `shipping` tại `server.js:329` .
- **7. Teardown & Clean-up:** `UPDATE orders SET status = 'shipping' WHERE id = 103` .

Test Case `TC_MOB_06` : Cố tình hủy đơn hàng đã hoàn tất giao hàng (`delivered` - Final State)

- **1. Tracking Information:**
  - **TC ID:** `TC_MOB_06` | **Priority:** High (P1)
  - **Target Layer:** Mobile UI & REST API & SQLite DB
  - **Traceability:** SRS FR-10: *"Trạng thái delivered và canceled là trạng thái kết thúc — không được phép chuyển sang bất kỳ trạng thái nào khác."*
  - **Coverage IDs:** `ORD-STA-IEP02` , `ORD-BVA-04` (On-point: `delivered`)
  - **Single-Fault Isolation:** Cô lập duy nhất trạng thái ban đầu của đơn: `order.status = 'delivered'` (Final State).
- **2. Test Objective:** Xác nhận hệ thống cấm tuyệt đối việc thay đổi trạng thái đối với đơn hàng đã hoàn tất giao hàng (`delivered`).
- **3. Pre-conditions & Environment:** Đơn hàng `id = 104` thuộc User 1, trạng thái là `delivered` .
- **4. Test Data:** `PUT /api/orders/104/cancel` ; Headers: `Authorization: Bearer <user1_jwt>`
- **5. Step-by-Step Procedure:** Gửi request `PUT /api/orders/104/cancel` .
- **6. Multi-tier Validation Points:**
  - *UI Layer:* Nút "Hủy đơn" bị ẩn.
  - *API Layer:* HTTP `400 Bad Request` ; Response JSON: `{"error": "Cannot cancel this order."}` .
  - *Database State Layer:* `SELECT status FROM orders WHERE id = 104` giữ nguyên `'delivered'` .
- **7. Teardown & Clean-up:** Không thay đổi CSDL.

Test Case `TC_MOB_07` : Cố tình hủy lại đơn hàng đã ở trạng thái `canceled` ($N=2$ - Double Cancellation)

- **1. Tracking Information:**
  - **TC ID:** `TC_MOB_07` | **Priority:** High (P1)
  - **Target Layer:** REST API & SQLite DB
  - **Traceability:** SRS FR-10; ISTQB FL §4.2.2

    - **Coverage IDs:** `ORD-STA-IEP03`, `ORD-BVA-05` (Off-point: $N=2$)
    - **Single-Fault Isolation:** Cô lập số lần hủy $N=2$ trên đơn đã có trạng thái `canceled` (Final State).
- **2. Test Objective:** Đảm bảo hệ thống từ chối các yêu cầu hủy trùng lặp/dư thừa trên đơn hàng đã hủy trước đó, duy trì tính bất biến của trạng thái kết thúc.
- **3. Pre-conditions & Environment:** Đơn hàng `id = 105` thuộc User 1, trạng thái hiện tại là `canceled`.
- **4. Test Data:** `PUT /api/orders/105/cancel`; Headers: `Authorization: Bearer <user1_jwt>`
- **5. Step-by-Step Procedure:** Gửi request `PUT /api/orders/105/cancel`.
- **6. Multi-tier Validation Points:**
    - *API Layer:* HTTP `400 Bad Request`; Response JSON: `{"error": "Cannot cancel this order."}`.
    - *Database State Layer:* Trạng thái giữ nguyên `'canceled'`, không phát sinh câu lệnh ghi dư thừa.
- **7. Teardown & Clean-up:** Không thay đổi CSDL.

Test Case `TC_MOB_08`: Đơn hàng có trạng thái dị thường trong CSDL (`status = 'corrupted_state'`)

- **1. Tracking Information:**
    - **TC ID:** `TC_MOB_08` | **Priority:** Low (P3)
    - **Target Layer:** REST API & SQLite DB
    - **Traceability:** SRS FR-10 (Robustness)
    - **Coverage IDs:** `ORD-STA-IEP04`
    - **Single-Fault Isolation:** Đơn hàng bị ghi đè trạng thái lạ không nằm trong State Machine chuẩn.
- **2. Test Objective:** Đảm bảo hệ thống xử lý an toàn, không tự động chuyển trạng thái lạ sang `canceled`.
- **3. Pre-conditions & Environment:** Đơn `id = 106`, cập nhật trực tiếp trong DB `UPDATE orders SET status = 'corrupted_state' WHERE id = 106`.
- **4. Test Data:** `PUT /api/orders/106/cancel`; Headers: `Authorization: Bearer <user1_jwt>`
- **5. Step-by-Step Procedure:** Gửi request `PUT /api/orders/106/cancel`.
- **6. Multi-tier Validation Points:**
    - *API Layer:* HTTP `400 Bad Request`.
    - *Database State Layer:* CSDL giữ nguyên trạng thái cũ, không bị ghi đè sang `canceled`.
- **7. Teardown & Clean-up:** `UPDATE orders SET status = 'pending' WHERE id = 106`.

Nhóm 3: Kiểm Thử Miền Dữ Liệu Tham Số Đường Dẫn & Cấu Trúc Request (Input EP & Route Parameter Boundaries)

Test Case `TC_MOB_09`: Gửi yêu cầu hủy với ID bằng 0 (Biên dưới Off-point)

- **1. Tracking Information:**
    - **TC ID:** `TC_MOB_09` | **Priority:** Medium (P2)
    - **Target Layer:** REST API
    - **Traceability:** API Spec §4.6; ISTQB FL §4.2.2
    - **Coverage IDs:** `ORD-INP-IEP01`, `ORD-BVA-01` (Off-point: `0`)
    - **Single-Fault Isolation:** Thay đổi `:id` thành `0` (Off-point vi phạm ràng buộc $id \ge 1$).
- **2. Test Objective:** Xác nhận hệ thống từ chối an toàn khi ID bằng 0.
- **3. Pre-conditions & Environment:** User 1 đăng nhập hợp lệ.
- **4. Test Data:** `PUT /api/orders/0/cancel`; Headers: `Authorization: Bearer <user1_jwt>`
- **5. Step-by-Step Procedure:** Gửi request `PUT /api/orders/0/cancel`.
- **6. Multi-tier Validation Points:**
    - *API Layer:* HTTP `404 Not Found` (`{"error": "Order not found"}`).
    - *Database State Layer:* CSDL nguyên vẹn 100%.
- **7. Teardown & Clean-up:** Không cần dọn dẹp.

Test Case `TC_MOB_10`: Gửi yêu cầu hủy với ID số nguyên âm ($id = -1$)

- **1. Tracking Information:**
    - **TC ID:** `TC_MOB_10` | **Priority:** Medium (P2)
    - **Target Layer:** REST API
    - **Traceability:** API Spec §4.6; ISTQB FL §4.2.1
    - **Coverage IDs:** `ORD-INP-IEP02`
    - **Single-Fault Isolation:** Thay đổi `:id` thành `-1`.
- **2. Test Objective:** Đảm bảo ID âm bị từ chối an toàn.
- **3. Pre-conditions & Environment:** User 1 đăng nhập hợp lệ.
- **4. Test Data:** `PUT /api/orders/-1/cancel`; Headers: `Authorization: Bearer <user1_jwt>`
- **5. Step-by-Step Procedure:** Gửi request `PUT /api/orders/-1/cancel`.
- **6. Multi-tier Validation Points:**
    - *API Layer:* HTTP `404 Not Found` (`{"error": "Order not found"}`).
    - *Database State Layer:* CSDL không thay đổi.
- **7. Teardown & Clean-up:** Không cần dọn dẹp.

Test Case `TC_MOB_11`: Gửi yêu cầu hủy với ID dạng chuỗi phi số (`id = "abc"`)

- **1. Tracking Information:**
    - **TC ID:** `TC_MOB_11` | **Priority:** Medium (P2)
    - **Target Layer:** REST API
    - **Traceability:** API Spec §4.6 (Type Validation)
    - **Coverage IDs:** `ORD-INP-IEP03`

    - **Single-Fault Isolation:** Thay đổi `:id` thành chuỗi ký tự chữ `"abc"`.
- **2. Test Objective:** Kiểm tra khả năng xử lý an toàn khi tham số URL sai kiểu dữ liệu, không gây unhandled exception hay sập server.
- **3. Pre-conditions & Environment:** User 1 đăng nhập hợp lệ.
- **4. Test Data:** `PUT /api/orders/abc/cancel`; Headers: `Authorization: Bearer <user1_jwt>`
- **5. Step-by-Step Procedure:** Gửi request `PUT /api/orders/abc/cancel`.
- **6. Multi-tier Validation Points:**
    - *API Layer:* HTTP `404 Not Found` (`{"error": "Order not found"}`).
    - *Database State Layer:* CSDL không thay đổi, Node.js process tiếp tục phục vụ bình thường.
- **7. Teardown & Clean-up:** Không cần dọn dẹp.

Test Case `TC_MOB_12`: Gửi yêu cầu hủy với ID dạng số thập phân (`id = 101.5`)

- **1. Tracking Information:**
    - **TC ID:** `TC_MOB_12` | **Priority:** Low (P3)
    - **Target Layer:** REST API
    - **Traceability:** API Spec §4.6
    - **Coverage IDs:** `ORD-INP-IEP04`
    - **Single-Fault Isolation:** Thay đổi `:id` thành `101.5`.
- **2. Test Objective:** Đảm bảo số thập phân không bị cắt gọt tự động thành số nguyên `101` để tránh hủy nhầm đơn.
- **3. Pre-conditions & Environment:** Đơn `101` đang `pending`.
- **4. Test Data:** `PUT /api/orders/101.5/cancel`; Headers: `Authorization: Bearer <user1_jwt>`
- **5. Step-by-Step Procedure:** Gửi request `PUT /api/orders/101.5/cancel`.
- **6. Multi-tier Validation Points:**
    - *API Layer:* HTTP `404 Not Found`.
    - *Database State Layer:* Đơn `101` giữ nguyên trạng thái `pending`.
- **7. Teardown & Clean-up:** Không cần dọn dẹp.

Test Case `TC_MOB_13`: Gửi yêu cầu hủy với ID số nguyên cực lớn tràn số 64-bit

- **1. Tracking Information:**
    - **TC ID:** `TC_MOB_13` | **Priority:** Low (P3)
    - **Target Layer:** REST API & SQLite DB
    - **Traceability:** ISTQB FL §4.2.2 (Robustness BVA)
    - **Coverage IDs:** `ORD-INP-IEP05`, `ORD-BVA-02` (Off-point: 2^{63})
    - **Single-Fault Isolation:** Thay đổi `:id` thành `9223372036854775808` (2^{63}).
- **2. Test Objective:** Đảm bảo giá trị tràn số nguyên 64-bit không làm crash SQLite driver.
- **3. Pre-conditions & Environment:** User 1 đăng nhập hợp lệ.
- **4. Test Data:** `PUT /api/orders/9223372036854775808/cancel`; Headers: `Authorization: Bearer <user1_jwt>`
- **5. Step-by-Step Procedure:** Gửi request qua curl.
- **6. Multi-tier Validation Points:**
    - *API Layer:* HTTP `404 Not Found` (hoặc 400).
    - *Database State Layer:* CSDL SQLite an toàn 100%.
- **7. Teardown & Clean-up:** Không cần dọn dẹp.

Test Case `TC_MOB_14`: Gửi yêu cầu hủy với ID đơn hàng không tồn tại trong hệ thống

- **1. Tracking Information:**
    - **TC ID:** `TC_MOB_14` | **Priority:** High (P1)
    - **Target Layer:** REST API & SQLite DB
    - **Traceability:** API Spec §4.6
    - **Coverage IDs:** `ORD-RES-IEP01`
    - **Single-Fault Isolation:** Thay đổi `:id` thành `99999` (ID số nguyên dương nhưng không tồn tại trong CSDL).
- **2. Test Objective:** Kiểm tra xử lý tài nguyên không tồn tại.
- **3. Pre-conditions & Environment:** Bảng `orders` không chứa bản ghi nào có `id = 99999`.
- **4. Test Data:** `PUT /api/orders/99999/cancel`; Headers: `Authorization: Bearer <user1_jwt>`
- **5. Step-by-Step Procedure:** Gửi request `PUT /api/orders/99999/cancel`.
- **6. Multi-tier Validation Points:**
    - *API Layer:* HTTP `404 Not Found` (`{"error": "Order not found"}`).
    - *Database State Layer:* CSDL không đổi.
- **7. Teardown & Clean-up:** Không cần dọn dẹp.

Test Case `TC_MOB_15`: Gửi request body chứa thuộc tính lạ `{"status": "delivered"}`

- **1. Tracking Information:**
    - **TC ID:** `TC_MOB_15` | **Priority:** High (P1)
    - **Target Layer:** REST API & Security
    - **Traceability:** API Spec §4.6; Security Mass Assignment Prevention
    - **Coverage IDs:** `ORD-INP-IEP05`
    - **Single-Fault Isolation:** Giữ nguyên baseline, chỉ truyền thêm body JSON độc hại `{"status": "delivered"}`.
- **2. Test Objective:** Đảm bảo backend không bị lỗ hổng Mass Assignment, không gán trường từ body vào SQL UPDATE.

- **3. Pre-conditions & Environment:** Đơn `101` của User 1 đang `pending`.
- **4. Test Data:** `PUT /api/orders/101/cancel`; Headers: `Authorization: Bearer <user1_jwt>`, `Content-Type: application/json`; Body: `{"status": "delivered"}`
- **5. Step-by-Step Procedure:** Gửi request `PUT` với body trên.
- **6. Multi-tier Validation Points:**
  - *API Layer:* HTTP `200 OK` (trường lạ bị phớt lờ).
  - *Database State Layer:* `SELECT status FROM orders WHERE id = 101` trả về `'canceled'` (tuyệt đối không bị thành `'delivered'`).
- **7. Teardown & Clean-up:** `UPDATE orders SET status = 'pending' WHERE id = 101`.

Test Case `TC_MOB_16`: Gửi request body dạng JSON sai cú pháp (Malformed JSON Payload)

- **1. Tracking Information:**
  - **TC ID:** `TC_MOB_16` | **Priority:** Low (P3)
  - **Target Layer:** REST API (Express Middleware)
  - **Traceability:** Protocol Robustness
  - **Coverage IDs:** `ORD-SEC-06`
  - **Single-Fault Isolation:** Body gửi lên là chuỗi hỏng cú pháp JSON.
- **2. Test Objective:** Đảm bảo `express.json()` bắt lỗi cú pháp an toàn và trả về HTTP 400.
- **3. Pre-conditions & Environment:** Đơn `101` đang `pending`.
- **4. Test Data:** `PUT /api/orders/101/cancel`; Headers: `Authorization: Bearer <user1_jwt>`, `Content-Type: application/json`; Body: `"{broken_json"`
- **5. Step-by-Step Procedure:** Gửi request với body hỏng.
- **6. Multi-tier Validation Points:**
  - *API Layer:* HTTP `400 Bad Request`.
  - *Database State Layer:* Đơn `101` giữ nguyên trạng thái `pending`.
- **7. Teardown & Clean-up:** Không cần dọn dẹp.

Nhóm 4: Kiểm Thử Xác Thực, Phân Quyền & Chống Tấn Công IDOR (Authentication & Ownership)

Test Case `TC_MOB_17`: Gửi request hủy đơn không kèm Header `Authorization` (Unauthenticated)

- **1. Tracking Information:**
  - **TC ID:** `TC_MOB_17` | **Priority:** Critical (P0)
  - **Target Layer:** REST API & Access Control
  - **Traceability:** SRS FR-12; API Spec §4.6; `server.js:46`
  - **Coverage IDs:** `ORD-AUTH-IEP01`
  - **Single-Fault Isolation:** Lược bỏ hoàn toàn Header `Authorization`. Mọi tham số khác giữ nguyên baseline.
- **2. Test Objective:** Đảm bảo endpoint hủy đơn bắt buộc phải có xác thực, từ chối người dùng nặc danh với HTTP 401.
- **3. Pre-conditions & Environment:** Đơn hàng `101` đang `pending`.
- **4. Test Data:** `PUT /api/orders/101/cancel`; Headers: None (Không truyền token)
- **5. Step-by-Step Procedure:** Gửi request không có header Authorization.
- **6. Multi-tier Validation Points:**
  - *API Layer:* HTTP `401 Unauthorized`; Response JSON: `{"error": "Access denied. No token provided."}`.
  - *Database State Layer:* Đơn `101` giữ nguyên trạng thái `pending`.
- **7. Teardown & Clean-up:** Không thay đổi CSDL.

Test Case `TC_MOB_18`: Gửi request hủy đơn với Token JWT sai chữ ký / giả mạo

- **1. Tracking Information:**
  - **TC ID:** `TC_MOB_18` | **Priority:** High (P1)
  - **Target Layer:** REST API & Authentication
  - **Traceability:** SRS FR-12; `server.js:50`
  - **Coverage IDs:** `ORD-AUTH-IEP02`
  - **Single-Fault Isolation:** Thay token JWT bằng chuỗi token giả ký sai secret.
- **2. Test Objective:** Đảm bảo token giả mạo bị từ chối với HTTP 403 Forbidden.
- **3. Pre-conditions & Environment:** Đơn hàng `101` đang `pending`.
- **4. Test Data:** `PUT /api/orders/101/cancel`; Headers: `Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.fake.signature`
- **5. Step-by-Step Procedure:** Gửi request với token giả.
- **6. Multi-tier Validation Points:**
  - *API Layer:* HTTP `403 Forbidden`; Response JSON: `{"error": "Invalid token."}`.
  - *Database State Layer:* Đơn `101` giữ nguyên trạng thái `pending`.
- **7. Teardown & Clean-up:** Không thay đổi CSDL.

Test Case `TC_MOB_19`: Gửi request hủy đơn với Token JWT đã hết hạn (`expired`)

- **1. Tracking Information:**
  - **TC ID:** `TC_MOB_19` | **Priority:** High (P1)
  - **Target Layer:** REST API & Authentication
  - **Traceability:** SRS FR-12; `server.js:50`
  - **Coverage IDs:** `ORD-AUTH-IEP03`
  - **Single-Fault Isolation:** Sử dụng token JWT hợp lệ nhưng có trường `exp` trong quá khứ.

- **2. Test Objective:** Đảm bảo token hết hạn bị từ chối với HTTP 403 Forbidden.
- **3. Pre-conditions & Environment:** Đơn hàng `101` đang `pending`.
- **4. Test Data:** `PUT /api/orders/101/cancel`; Headers: `Authorization: Bearer <expired_jwt>`
- **5. Step-by-Step Procedure:** Gửi request với expired token.
- **6. Multi-tier Validation Points:**
  - *API Layer:* HTTP `403 Forbidden` (`{"error": "Invalid token."}`).
  - *Database State Layer:* Đơn `101` giữ nguyên trạng thái `pending`.
- **7. Teardown & Clean-up:** Không thay đổi CSDL.

Test Case `TC_MOB_20`: Tấn công IDOR: User 1 cố tình gửi request hủy đơn 202 của User 2

- **1. Tracking Information:**
  - **TC ID:** `TC_MOB_20` | **Priority:** Critical (P0)
  - **Target Layer:** REST API & Data Privacy & SQLite DB
  - **Traceability:** SRS FR-11: *"Người dùng chỉ xem được đơn hàng của chính mình."*; Nguyên tắc chống IDOR (OWASP Top 10)
  - **Coverage IDs:** `ORD-AUTH-IEP04`
  - **Single-Fault Isolation:** User 1 đăng nhập hợp lệ nhưng truyền `:id = 202` (đơn hàng hợp lệ đang `pending` của User 2). Vi phạm duy nhất điều kiện quyền sở hữu (`user_id !== order.user_id`).
- **2. Test Objective:** Kiểm tra cơ chế chống tấn công IDOR, đảm bảo người dùng này tuyệt đối không thể hủy đơn hàng của người dùng khác.
- **3. Pre-conditions & Environment:**
  - User 1 đăng nhập, có token JWT.
  - Đơn hàng `id = 202` thuộc sở hữu của User 2 (`user_id = 2`), trạng thái đang là `pending`.
- **4. Test Data:** `PUT /api/orders/202/cancel`; Headers: `Authorization: Bearer <user1_jwt>`
- **5. Step-by-Step Procedure:** User 1 gửi request `PUT /api/orders/202/cancel`.
- **6. Multi-tier Validation Points:**
  - *API Layer:* HTTP `404 Not Found` (`{"error": "Order not found"}`). Chính sách bảo mật che giấu sự tồn tại của đơn hàng người khác.
  - *Database State Layer (Data Integrity Invariant):* `SELECT status, user_id FROM orders WHERE id = 202` trả về đúng `status = 'pending'` và `user_id = 2`. Đơn hàng của User 2 hoàn toàn không bị ảnh hưởng!
- **7. Teardown & Clean-up:** Không thay đổi CSDL.

Test Case `TC_MOB_21`: Tài khoản Admin gọi endpoint hủy của khách hàng `PUT /api/orders/101/cancel`

- **1. Tracking Information:**
  - **TC ID:** `TC_MOB_21` | **Priority:** Medium (P2)
  - **Target Layer:** REST API & Access Control
  - **Traceability:** API Spec §4.6 vs §6.3
  - **Coverage IDs:** `ORD-AUTH-IEP05`
  - **Single-Fault Isolation:** Sử dụng token của tài khoản Admin (`user_id = 99`, `role = 'admin'`) gọi vào endpoint khách hàng trên đơn `101` của User 1.
- **2. Test Objective:** Xác nhận endpoint khách hàng lọc chặt chẽ theo `user_id` và buộc Admin phải dùng endpoint quản trị chuyên biệt.
- **3. Pre-conditions & Environment:** Đơn `101` thuộc User 1 (`user_id = 1`).
- **4. Test Data:** `PUT /api/orders/101/cancel`; Headers: `Authorization: Bearer <admin_jwt>`
- **5. Step-by-Step Procedure:** Gửi request bằng token Admin.
- **6. Multi-tier Validation Points:**
  - *API Layer:* HTTP `404 Not Found` (`{"error": "Order not found"}`).
  - *Database State Layer:* Đơn `101` giữ nguyên trạng thái `pending`.
- **7. Teardown & Clean-up:** Không thay đổi CSDL.

Nhóm 5: Kiểm Thử An Toàn Giao Diện Mobile, Xử Lý Đồng Thời & An Toàn Giao Thức (Security, Concurrency & UI Safety)

Test Case `TC_MOB_22`: Tấn công SQL Injection qua tham số URL `:id`

- **1. Tracking Information:**
  - **TC ID:** `TC_MOB_22` | **Priority:** High (P1)
  - **Target Layer:** REST API & Database Security
  - **Traceability:** SRS SEC-01: *"Chống SQL Injection bằng Parameterized Queries"*
  - **Coverage IDs:** `ORD-SEC-01`
  - **Single-Fault Isolation:** Tham số URL chứa chuỗi payload SQL Injection độc hại.
- **2. Test Objective:** Xác nhận backend sử dụng câu lệnh tham số hóa `WHERE id = ? AND user_id = ?`, không bị chèn câu lệnh SQL độc hại.
- **3. Pre-conditions & Environment:** User 1 đăng nhập hợp lệ.
- **4. Test Data:** `PUT /api/orders/101%20OR%201=1/cancel`; Headers: `Authorization: Bearer <user1_jwt>`
- **5. Step-by-Step Procedure:** Gửi request URL encode payload SQLi.
- **6. Multi-tier Validation Points:**
  - *API Layer:* HTTP `404 Not Found` (`{"error": "Order not found"}`).
  - *Database State Layer:* Toàn bộ bảng `orders` nguyên vẹn 100%, không có bản ghi nào bị cập nhật sai lệch.
- **7. Teardown & Clean-up:** Không thay đổi CSDL.

Test Case `TC_MOB_23`: Kiểm thử hộp thoại xác nhận an toàn khi bấm Hủy trên Mobile UI

- **1. Tracking Information:**
  - **TC ID:** `TC_MOB_23` | **Priority:** High (P1)

    - **Target Layer:** Mobile App UI (`frontend-mobile/App.js`)
    - **Traceability:** SRS FR-21, FR-24 (GUI Safety Standards)
    - **Coverage IDs:** `ORD-SEC-02`
    - **Single-Fault Isolation:** Thao tác tương tác chạm trên màn hình cảm ứng: Chọn Hủy bỏ vs Đồng ý.
- **2. Test Objective:** Đảm bảo nút "Hủy đơn" trên Mobile App kích hoạt hộp thoại xác nhận Alert Dialog nhằm ngăn chặn việc vô tình chạm nhầm nút.
- **3. Pre-conditions & Environment:** Đơn `101` hiển thị trên màn hình Profile Mobile ở trạng thái `pending` .
- **4. Test Data:** Tương tác chạm nút "Hủy đơn" màu đỏ.
- **5. Step-by-Step Procedure:**
    1. Mở màn hình Profile trên Mobile App.
    2. Chạm vào nút "Hủy đơn" màu đỏ tại đơn `#101` .
    3. Quan sát xem hộp thoại `Alert` có xuất hiện với 2 nút lựa chọn không.
    4. Chạm "Hủy bỏ": Kiểm tra xem API có bị gọi không và trạng thái đơn có giữ nguyên không.
    5. Chạm lại "Hủy đơn", lần này chọn "Đồng ý": Kiểm tra API được gửi và giao diện cập nhật.
- **6. Multi-tier Validation Points:**
    - *UI Layer:* Hộp thoại `Alert.alert` xuất hiện. Khi chọn "Hủy bỏ", không có request gửi đi, đơn vẫn là "Chờ xác nhận". Khi chọn "Đồng ý", gửi API và đổi trạng thái sang "Đã hủy".
    - *Database State Layer:* CSDL chỉ đổi sang `canceled` sau khi chọn "Đồng ý".
- **7. Teardown & Clean-up:** `UPDATE orders SET status = 'pending' WHERE id = 101` .

Test Case `TC_MOB_24` : Bấm liên tiếp 2 lần cực nhanh nút Hủy trên Mobile (Double-tap)

- **1. Tracking Information:**
    - **TC ID:** `TC_MOB_24` | **Priority:** Medium (P2)
    - **Target Layer:** Mobile App UI (Debounce / Throttling)
    - **Traceability:** Mobile UX Robustness; `ORD-BVA-07`
    - **Coverage IDs:** `ORD-BVA-07` (Off-point: 2 taps liên tiếp)
    - **Single-Fault Isolation:** Bấm đúp liên tiếp 2 lần trong khoảng thời gian $< 200ms$.
- **2. Test Objective:** Đảm bảo giao diện có cơ chế chống chạm lặp (debounce), không mở 2 hộp thoại Alert đè lên nhau.
- **3. Pre-conditions & Environment:** Đơn `101` đang `pending` trên Mobile App.
- **4. Test Data:** 2 lần tap liên tiếp trong $100ms$.
- **5. Step-by-Step Procedure:** Chạm nhanh 2 lần liên tiếp vào nút "Hủy đơn".
- **6. Multi-tier Validation Points:**
    - *UI Layer:* Chỉ có duy nhất 1 hộp thoại `Alert` hiển thị, ứng dụng không bị giật lag hay đơ giao diện.
- **7. Teardown & Clean-up:** Không thay đổi CSDL.

Test Case `TC_MOB_25` : Gửi đồng thời 2 request hủy trong cùng 1 mili-giây (Concurrent Race Condition / Exactly-Once)

- **1. Tracking Information:**
    - **TC ID:** `TC_MOB_25` | **Priority:** High (P1)
    - **Target Layer:** REST API & SQLite DB (Concurrency Handling)
    - **Traceability:** Transaction Integrity; ISTQB FL §4.2.2
    - **Coverage IDs:** `ORD-SEC-04` , `ORD-BVA-06` (Off-point: $k=2$)
    - **Single-Fault Isolation:** Gửi đồng thời $k=2$ request hủy cho cùng một đơn hàng `101` .
- **2. Test Objective:** Kiểm tra tính bất biến Exactly-Once của tiến trình hủy đơn hàng khi chịu tải đồng thời từ nhiều luồng mạng.
- **3. Pre-conditions & Environment:** Đơn `101` của User 1 đang ở trạng thái `pending` .
- **4. Test Data:** 2 request `PUT /api/orders/101/cancel` được phát đi đồng thời bằng script đa luồng.
- **5. Step-by-Step Procedure:**
    1. Sử dụng script Node.js hoặc Python phát 2 request `PUT` song song trong cùng 1 mili-giây.
    2. Ghi nhận response status code và body của cả 2 request.
    3. Kiểm tra số lượng bản ghi được cập nhật trong CSDL SQLite.
- **6. Multi-tier Validation Points (Exactly-Once Oracle):**
    - *API Layer:* Đúng 1 request nhận HTTP `200 OK` ; request còn lại nhận HTTP `400 Bad Request` (`Cannot cancel this order.`) do đơn đã chuyển sang `canceled` (hoặc cả 2 trả về 200 nếu server xử lý idempotent).
    - *Database State Layer:* Trạng thái cuối cùng của đơn `101` là `canceled` . Không xảy ra lỗi khóa chết (SQLITE_BUSY / Deadlock).
    - *Security & Business Rules Layer:* Thao tác hủy chỉ được áp dụng đúng 1 lần duy nhất.
- **7. Teardown & Clean-up:** `UPDATE orders SET status = 'pending' WHERE id = 101` .

Test Case `TC_MOB_26` : Xáo trộn phương thức HTTP (`POST` , `GET` , `DELETE` tới API cancel)

- **1. Tracking Information:**
    - **TC ID:** `TC_MOB_26` | **Priority:** Low (P3)
    - **Target Layer:** REST API Routing
    - **Traceability:** API Protocol Robustness
    - **Coverage IDs:** `ORD-SEC-05`
    - **Single-Fault Isolation:** Thay đổi HTTP Method từ `PUT` sang `POST` / `GET` / `DELETE` .
- **2. Test Objective:** Đảm bảo API chỉ chấp nhận duy nhất phương thức `PUT` , từ chối các phương thức sai.
- **3. Pre-conditions & Environment:** Đơn `101` đang `pending` .
- **4. Test Data:** `POST /api/orders/101/cancel` , `GET /api/orders/101/cancel` , `DELETE /api/orders/101/cancel`
- **5. Step-by-Step Procedure:** Gửi request bằng các method sai.
- **6. Multi-tier Validation Points:**

  - *API Layer:* HTTP `404 Not Found` hoặc `405 Method Not Allowed`.
  - *Database State Layer:* Đơn `101` giữ nguyên trạng thái `pending`.
- **7. Teardown & Clean-up:** Không thay đổi CSDL.

Nhóm 6: Kiểm Thử Làm Rõ Tác Dụng Phụ Nghiệp Vụ (Business Side-effect Clarification Observations)

Test Case `TC_MOB_27`: Quan sát tác dụng phụ hoàn trả kho khi đơn hàng bị hủy

- **1. Tracking Information:**
  - **TC ID:** `TC_MOB_27` | **Priority:** Medium (P2)
  - **Target Layer:** SQLite DB (`products` & `order_items` tables)
  - **Traceability:** SRS FR-07, FR-10 (Clarification Item)
  - **Coverage IDs:** `ORD-STA-CLAR01`
  - **Single-Fault Isolation:** Quan sát tác dụng phụ tồn kho sau khi thực hiện thao tác hủy hợp lệ trên đơn `pending`.
- **2. Test Objective:** Quan sát và ghi nhận hành vi thực tế của hệ thống EShop đối với số lượng tồn kho sản phẩm (`products.stock`) khi đơn hàng bị hủy (làm rõ câu hỏi mở nghiệp vụ).
- **3. Pre-conditions & Environment:**
  - Tạo một đơn hàng mới `#107` cho User 1 gồm 1 sản phẩm `id = 1` với số lượng $Q = 2$.
  - Ghi nhận số lượng tồn kho trước khi hủy: `stock_before = 10`.
- **4. Test Data:** `PUT /api/orders/107/cancel`; Headers: `Authorization: Bearer <user1_jwt>`
- **5. Step-by-Step Procedure:**
  1. Kiểm tra tồn kho sản phẩm 1: `SELECT stock FROM products WHERE id = 1` (giả sử = 10).
  2. Gửi request hủy đơn hàng `#107`.
  3. Xác nhận request trả về HTTP `200 OK`.
  4. Thực hiện lại câu truy vấn: `SELECT stock FROM products WHERE id = 1`.
- **6. Multi-tier Validation Points (Observation Oracle):**
  - *API Layer:* HTTP `200 OK`.
  - *Database State Layer:*
    - Nếu `stock_after = 12` ($10 + 2$): Hệ thống có cơ chế tự động hoàn kho khi hủy đơn.
    - Nếu `stock_after = 10`: Hệ thống KHÔNG tự động hoàn kho khi hủy đơn.
  - *Assessment Note:* Ghi nhận hành vi thực tế vào Báo cáo Khảo sát SUT để đối chiếu với câu hỏi làm rõ đặc tả.
- **7. Teardown & Clean-up:** Đặt lại số lượng tồn kho ban đầu nếu có thay đổi.

Test Case `TC_MOB_28`: Quan sát tác dụng phụ khôi phục lượt dùng coupon khi đơn bị hủy

- **1. Tracking Information:**
  - **TC ID:** `TC_MOB_28` | **Priority:** Medium (P2)
  - **Target Layer:** REST API & SQLite DB
  - **Traceability:** SRS FR-09, FR-10 (Clarification Item)
  - **Coverage IDs:** `ORD-STA-CLAR02`
  - **Single-Fault Isolation:** Quan sát khả năng tái sử dụng voucher sau khi đơn hàng áp dụng voucher đó bị hủy.
- **2. Test Objective:** Kiểm tra xem mã giảm giá bị giới hạn lượt dùng (`max_uses_per_user = 1`) có được giải phóng để người dùng sử dụng lại sau khi hủy đơn hay không.
- **3. Pre-conditions & Environment:**
  - Coupon `ONETIME` có `max_uses_per_user = 1`.
  - User 1 đặt đơn `#108` áp dụng mã `ONETIME`, đơn ở trạng thái `pending`.
- **4. Test Data:** `PUT /api/orders/108/cancel`; sau đó gửi `POST /api/apply-coupon` với mã `ONETIME`.
- **5. Step-by-Step Procedure:**
  1. Hủy đơn hàng `#108` bằng request `PUT /api/orders/108/cancel`.
  2. Xác nhận hủy thành công (HTTP 200).
  3. Gửi request áp dụng mã `ONETIME` cho một giỏ hàng mới của User 1: `POST /api/apply-coupon`.
- **6. Multi-tier Validation Points (Observation Oracle):**
  - *API Layer:*
    - Nếu trả về HTTP 200: Mã được hoàn lại lượt dùng.
    - Nếu trả về HTTP 400 ("Bạn đã sử dụng hết lượt mã này"): Hệ thống không hoàn lại lượt voucher khi hủy đơn.
  - *Assessment Note:* Ghi nhận hành vi thực tế làm minh chứng đối chiếu cho AI Gap Analysis.
- **7. Teardown & Clean-up:** Xóa đơn hàng thử nghiệm `#108`.

5.5 Bước 5: Ma trận truy vết & Kiểm tra độ bao phủ (Step 5 Deliverables - Traceability & Coverage Verification)

> **Mục Tiêu & Tiêu Chuẩn Xác Minh Độ Bao Phủ (ISTQB FL §4.2):**
>
> - Đảm bảo **100% các phân vùng tương đương** (VEP và IEP) đã xác định ở Bước 2 và **100% các giá trị biên** (On-point, Off-point, In-point) ở Bước 3 đều được ánh xạ truy vết trực tiếp tới ít nhất một ca kiểm thử cụ thể trong bộ 28 Test Cases của Bước 4.
> - Bóc tách minh bạch độ bao phủ theo từng tầng độc lập để không thổi phồng tỷ lệ bao phủ của dữ liệu đầu vào thuần túy.

5.5.1 Bảng Độ Bao Phủ Phân Vùng Tương Đương Biến Đầu Vào Thuần Túy (Pure Input EP Coverage)

Class ID	Tên biến / Ràng buộc đầu vào	Phân loại	Mã Test Case bao phủ	Trạng thái thiết kế
`ORD-INP-VEP01`	`target_order_id` : Số nguyên dương hợp lệ ($id \ge 1$)	VEP	`TC_MOB_01`, `TC_MOB_02`, `TC_MOB_03`, `TC_MOB_04`	**Covered**
`ORD-INP-VEP02`	Request Body: JSON Object rỗng `{}`	VEP	`TC_MOB_01`, `TC_MOB_02`, `TC_MOB_03`, `TC_MOB_04`	**Covered**
`ORD-INP-IEP01`	`target_order_id` : ID bằng 0 ($id = 0$)	IEP	`TC_MOB_09`	**Covered**
`ORD-INP-IEP02`	`target_order_id` : Số nguyên âm ($id = -1$)	IEP	`TC_MOB_10`	**Covered**
`ORD-INP-IEP03`	`target_order_id` : Chuỗi phi số (`id = "abc"`)	IEP	`TC_MOB_11`	**Covered**
`ORD-INP-IEP04`	`target_order_id` : Số thập phân (`id = 101.5`)	IEP	`TC_MOB_12`	**Covered**
`ORD-INP-IEP05`	Request Body: Chèn trường lạ can thiệp trạng thái `{"status": "delivered"}`	IEP / Probe	`TC_MOB_15`	**Covered**

5.5.2 Bảng Độ Bao Phủ Xác Thực, Phân Quyền & Vòng Đời Tài Nguyên (Authentication, Authorization & Resource Coverage)

Class ID	Nội dung kiểm soát truy cập / Tài nguyên	Phân loại	Mã Test Case bao phủ	Trạng thái thiết kế
`ORD-AUTH-VEP01`	Token JWT hợp lệ, đúng chủ sở hữu (`user_id = 1`)	VEP	`TC_MOB_01`, `TC_MOB_02`	**Covered**
`ORD-AUTH-IEP01`	Thiếu Header Authorization (Unauthenticated)	IEP	`TC_MOB_17`	**Covered**
`ORD-AUTH-IEP02`	Token JWT sai chữ ký / giả mạo	IEP	`TC_MOB_18`	**Covered**
`ORD-AUTH-IEP03`	Token JWT đã hết hạn (`expired`)	IEP	`TC_MOB_19`	**Covered**
`ORD-AUTH-IEP04`	**Tấn công IDOR:** User 1 hủy đơn 202 của User 2	Security / IEP	`TC_MOB_20`	**Covered**
`ORD-AUTH-IEP05`	Phân quyền Admin vs User: Admin gọi endpoint khách hàng	Conditional / IEP	`TC_MOB_21`	**Covered**
`ORD-RES-VEP01`	Đơn hàng tồn tại trong CSDL SQLite	VEP	`TC_MOB_01`, `TC_MOB_02`, `TC_MOB_03`, `TC_MOB_04`	**Covered**
`ORD-RES-IEP01`	Đơn hàng không tồn tại (`id = 99999`)	IEP	`TC_MOB_14`	**Covered**

5.5.3 Bảng Độ Bao Phủ Quy Tắc Nghiệp Vụ State Machine, Bổ Trợ & Làm Rõ (Business Rules & State Machine Coverage)

Class ID	Nội dung quy tắc State Machine / An toàn / Làm rõ	Phân loại	Mã Test Case bao phủ	Trạng thái thiết kế
`ORD-STA-VEP01`	Chuyển đổi hợp lệ: `pending` $\to$ `canceled`	VEP	`TC_MOB_01`, `TC_MOB_03`, `TC_MOB_04`	**Covered**
`ORD-STA-VEP02`	Chuyển đổi hợp lệ (Ranh giới cuối): `confirmed` $\to$ `canceled`	VEP	`TC_MOB_02`	**Covered**
`ORD-STA-IEP01`	Chuyển đổi bị cấm đối với User: `shipping` $\to$ `canceled`	IEP	`TC_MOB_05`	**Covered**
`ORD-STA-IEP02`	Chuyển đổi trạng thái kết thúc bị cấm: `delivered` $\to$ `canceled`	IEP	`TC_MOB_06`	**Covered**
`ORD-STA-IEP03`	Hủy lặp lại trên trạng thái kết thúc: `canceled` $\to$ `canceled`	IEP	`TC_MOB_07`	**Covered**
`ORD-STA-IEP04`	Đơn có trạng thái dị thường trong DB (`corrupted_state`)	Robustness	`TC_MOB_08`	**Covered**
`ORD-STA-CLAR01`	Quan sát side-effect hoàn trả tồn kho (`products.stock`)	Clarification	`TC_MOB_27`	**Covered**
`ORD-STA-CLAR02`	Quan sát side-effect khôi phục lượt dùng mã coupon	Clarification	`TC_MOB_28`	**Covered**
`ORD-SEC-01`	Thăm dò SQL Injection qua URL parameter `:id`	Security Probe	`TC_MOB_22`	**Covered**
`ORD-SEC-02`	Kiểm tra hộp thoại xác nhận an toàn UI (Alert Dialog)	UI Safety	`TC_MOB_23`	**Covered**
`ORD-SEC-03`	Thăm dò lỗ hổng Client-side-only bypass trên `shipping`	Security Probe	`TC_MOB_05`	**Covered**
`ORD-SEC-04`	Kiểm tra xử lý đồng thời (Race Condition / Exactly-Once)	Concurrency	`TC_MOB_25`	**Covered**
`ORD-SEC-05`	Kiểm tra xáo trộn phương thức HTTP (`POST` / `DELETE` / `GET`)	Protocol Robustness	`TC_MOB_26`	**Covered**
`ORD-SEC-06`	Kiểm tra Request Body chứa JSON sai cú pháp	Request Parsing	`TC_MOB_16`	**Covered**

5.5.4 Bảng Độ Bao Phủ Phân Tích Giá Trị Biên (Boundary Value Coverage - On/Off/In Points)

Boundary ID	Tên ranh giới	Điểm biên & Loại điểm	Giá trị kiểm thử	Mã Test Case bao phủ	Trạng thái thiết kế

`ORD-BVA-01`	Biên dưới ID đơn hàng ($id \ge 1$)	**On-point** (Hợp lệ)
		Off-point (Không hợp lệ)
		In-point (Hợp lệ liền kề)
			`id = 1`
			`id = 0`
			`id = 2`
				`TC_MOB_03`
				`TC_MOB_09`
				`TC_MOB_04`
					Covered
					Covered
					Covered

`ORD-BVA-02`	Biên trên lưu trữ SQLite 64-bit	**On-point** (Hợp lệ lớn nhất)
		Off-point (Tràn số 64-bit)
		In-point (Hợp lệ liền kề)
			`id = 9223372036854775807`
			`id = 9223372036854775808`
			`id = 9223372036854775806`
				`TC_MOB_13`
				`TC_MOB_13`
				`TC_MOB_13`
					Covered
					Covered
					Covered

`ORD-BVA-03`	Ranh giới quyền tự hủy của User	**On-point** (Hợp lệ cuối cùng)
		Off-point (Bị cấm hủy)
		In-point (Hợp lệ ban đầu)
			`status = 'confirmed'`
			`status = 'shipping'`
			`status = 'pending'`
				`TC_MOB_02`
				`TC_MOB_05`
				`TC_MOB_01`
					Covered
					Covered
					Covered

`ORD-BVA-04`	Ranh giới trạng thái kết thúc `delivered`	**On-point** (Final state - cấm hủy) **Off-point** (Hợp lệ liền trước)	`status = 'delivered'` `status = 'confirmed'`	`TC_MOB_06` `TC_MOB_02`	**Covered** **Covered**
`ORD-BVA-05`	Ngưỡng số lần hủy đơn (N_{cancel})	**On-point** (Hủy lần 1) **Off-point** (Hủy lần 2 - cấm)	$N_{\text{cancel}} = 1$ $N_{\text{cancel}} = 2$	`TC_MOB_01` `TC_MOB_07`	**Covered** **Covered**
`ORD-BVA-06`	Ranh giới xử lý đồng thời (k_{req})	**On-point** (Đơn luồng chuẩn) **Off-point** (Đồng thời 2 request)	$k_{\text{req}} = 1$ $k_{\text{req}} = 2$	`TC_MOB_01` `TC_MOB_25`	**Covered** **Covered**

Boundary ID	Tên ranh giới	Điểm biên & Loại điểm	Giá trị kiểm thử	Mã Test Case bao phủ	Trạng thái thiết kế
`ORD-BVA-07`	Ranh giới thao tác chạm nút Mobile UI	**On-point** (1 lần chạm chuẩn) **Off-point** (2 lần chạm nhanh)	1 tap $2\text{ taps} < 200ms$	`TC_MOB_23` `TC_MOB_24`	**Covered** **Covered**

5.5.5 Bảng Tổng Hợp Chỉ Số Độ Bao Phủ (Coverage Summary Metrics)

Nhóm Tiêu Chí Kiểm Thử (Testing Category)	Tổng số nhận diện	Số lượng đã bao phủ	Bị chặn / Chưa bao phủ	Tỷ lệ bao phủ thiết kế (%)
Phân vùng Tương đương Hợp lệ (Pure Input VEP)	2	2	0	**100%**
Phân vùng Tương đương Không hợp lệ (Pure Input IEP)	5	5	0	**100%**
Kiểm soát Truy cập, Xác thực & Quyền sở hữu (Auth & Ownership)	6	6	0	**100%**
Vòng đời & Trạng thái Tài nguyên (Target-Resource Lifecycle)	2	2	0	**100%**
Quy tắc Máy Trạng Thái Đơn hàng (State Machine Transitions)	6	6	0	**100%**
Quan sát Câu hỏi Mở Nghiệp vụ (Side-effect Clarifications)	2	2	0	**100%**
Bộ Kiểm Thử Bổ Trợ (Security, UI Safety, Concurrency, Robustness)	6	6	0	**100%**
Điểm Phân Tích Giá Trị Biên (On / Off / In Points)	18	18	0	**100%**
Tổng thể Toàn Bộ Các Chiều Kiểm Thử Đã Thiết Kế	**47**	**47**	**0**	**100%**

5.6 Bước 6: AI Gap Analysis & Giả thuyết lỗi SUT (Step 6 Deliverables)

> **Mục đích:** Cung cấp minh chứng thực tế cho **Mục 6 Task 3 (AI Gap Analysis)** và làm cơ sở biện luận cho **Mục 10 (AI Critique)** của báo cáo HW02. Phân tách rạch ròi giữa các thiếu sót do suy luận AI và các khiếm khuyết nội bộ thực tế trong mã nguồn SUT EShop liên quan đến chức năng Hủy Đơn Hàng trên Mobile App & Máy Trạng Thái Đơn Hàng (FR-20 & FR-10).

5.6.1 Bảng Phân Tích Khoảng Trống Kiểm Thử Của AI (AI Gap Analysis)

Hạng mục / Vấn đề phát sinh	Nguyên nhân AI bỏ sót hoặc gặp khó khăn	Hiệu chỉnh của con người (Human Fix Applied)	Căn cứ Tiêu chuẩn & Kỹ thuật Kiểm thử
Gộp lẫn kiểm thử UI với API Backend (Bỏ quên lỗ hổng Client-Side-Only Validation bypass)	AI ban đầu chỉ tập trung thiết kế kịch bản trên giao diện Mobile (kiểm tra nút "Hủy đơn" có hiển thị ở `pending` / `confirmed` và biến mất ở `shipping`), mà bỏ qua trường hợp kẻ tấn công hoặc script gửi trực tiếp HTTP request `PUT /api/orders/:id/cancel` bỏ qua giao diện người dùng.	Con người chỉ đạo bóc tách độc lập 2 kênh kiểm thử: (1) Kiểm thử ẩn/hiện nút trên Mobile UI (`frontend-mobile/App.js`); (2) Gửi raw HTTP PUT request qua API client để kiểm tra tính toàn vẹn State Machine phía Backend (`TC_MOB_05`). Phát hiện mã nguồn `server.js:329` không chặn `shipping`!	**OWASP Top 10 & API Security Testing:** Client-side UI hiding không bao giờ được xem là giải pháp bảo mật; mọi quy tắc chuyển đổi trạng thái bắt buộc phải được bảo vệ độc lập ở tầng Backend API.
Ảo giác về logic hoàn kho và khôi phục coupon (Over-specifying Business Side-effects)	Do ảnh hưởng từ dữ liệu huấn luyện về các hệ thống e-commerce thực tế (Shopee, Lazada), AI tự động suy diễn rằng hủy đơn thì bắt buộc phải hoàn trả tồn kho (`products.stock`) và hoàn lại lượt dùng voucher (`coupon_usage`), rồi khẳng định kết quả mong đợi là `200 OK & stock restored`.	Con người đối chiếu trực tiếp với tài liệu đặc tả SRS FR-10 và mã nguồn SUT, nhận thấy đặc tả không đề cập và mã nguồn backend `server.js:333-339` chỉ chạy lệnh `UPDATE orders SET status = 'canceled'`. Con người đã phân loại lại thành các ca kiểm thử quan sát / làm rõ (`TC_MOB_27`, `TC_MOB_28`) với Expected Results dạng Observation Oracle.	**Nguyên tắc "Đặc tả là căn cứ duy nhất" (Specification as Sole Source of Truth):** Không tự ý biến giả định nghiệp vụ bên ngoài thành tiêu chí Pass/Fail của đặc tả khi chưa được khách hàng xác nhận.
Mơ hồ và đưa ra nhiều mã trạng thái HTTP status codes thay thế (`400 or 404`)	AI thường đưa ra các kỳ vọng nước đôi như `HTTP 400 hoặc 404`, hoặc không phân biệt rạch ròi giữa lỗi chưa xác thực (`401 Unauthorized`), token sai/hết hạn (`403 Forbidden`), tài nguyên không tìm thấy / IDOR (`404 Not Found`), và vi phạm State Machine (`400 Bad Request`).	Con người đã phân tích kiến trúc middleware Express trong `server.js:46-55` và chuẩn hóa chính xác từng mã status code duy nhất cho 28 test cases: `401` khi thiếu token, `403` khi token giả/hết hạn, `404` khi không tìm thấy đơn hoặc tấn công IDOR, `400` khi vi phạm chuyển đổi trạng thái.	**RESTful API Design Standards & ISTQB FL §4.2.1:** Tiêu chí đánh giá của ca kiểm thử phải xác định rõ ràng, không được chứa các mệnh đề thay thế gây mơ hồ cho việc tự động hóa kiểm thử.
Bỏ qua kiểm thử tương tác phần cứng & trải nghiệm Mobile UI (Confirmation Alert & Double-tap)	AI có thói quen sinh kịch bản API thuần túy, bỏ quên các đặc thù vật lý và trải nghiệm người dùng trên thiết bị di động: hành vi chạm nhầm trên màn hình cảm ứng đòi hỏi hộp thoại xác nhận (Confirmation Alert), và thao tác chạm nhanh liên tiếp (double-tap $< 200ms$) đòi hỏi debounce/throttling.	Con người chỉ đạo bổ sung nhóm kiểm thử an toàn Mobile UI chuyên biệt (`TC_MOB_23`, `TC_MOB_24`) để xác minh sự hiện diện của `Alert.alert` và kiểm thử độ bền trước thao tác double-tap, phát hiện SUT gọi ngay lệnh hủy mà không có Alert xác nhận.	**Mobile GUI Safety Guidelines (SRS FR-21 / FR-24):** Thao tác hủy đơn là hành động phá hủy vĩnh viễn, bắt buộc phải có hộp thoại xác nhận trước khi phát lệnh đến server.
Bỏ quên kiểm thử tương thích giữa State Machine của User và Admin	AI chỉ nhìn nhận đơn lẻ luồng người dùng hủy đơn trên Mobile App mà không kiểm tra bức tranh tổng thể của Order State Machine giữa User và Quản trị viên (Admin), bỏ lỡ kiểm tra tính bất biến của trạng thái kết thúc (`canceled` và `delivered`).	Con người chỉ đạo mở rộng phạm vi kiểm tra tính bất biến trạng thái kết thúc, qua đó phát hiện lỗ hổng nghiêm trọng tại `server.js:550-551` cho phép Admin đảo ngược đơn hàng từ `canceled` sang `delivered`.	**ISTQB FL §4.2.2 & State Transition Testing:** Trạng thái kết thúc (Final / Absorbing State) trong đồ thị máy trạng thái phải có bậc ra bằng 0 (out-degree = 0), không có cung chuyển tiếp nào được phép đi ra.

5.6.2 Danh Sách Giả Thuyết Khiếm Khuyết Thăm Dò Trên SUT EShop (Candidate Defect Hypotheses)

> *Ghi chú: Toàn bộ các giả thuyết dưới đây được gắn nhãn `[Exploratory - Unconfirmed Hypothesis - Requires SUT execution]` và sẽ được kiểm chứng thực tế trong Giai đoạn thực thi trên hệ thống EShop đang chạy.*

Hypothesis ID	Khu vực kiểm thử (Target Area)	Mô tả giả thuyết lỗi (Description)	Kỳ vọng theo đặc tả (Spec Expected)	Dự đoán hành vi thực tế (Hypothesized Actual)	Bước kiểm chứng trên SUT (Verification Step)
`HYP-20-01`	Backend State Machine Validation (`server.js:328-331`)	**Lỗ hổng Client-Side-Only Validation: Backend API cho phép hủy đơn hàng đang ở trạng thái `shipping`**	Trích dẫn SRS FR-10: *"Khi đơn hàng đã ở trạng thái `shipping`, User không được phép tự hủy — chỉ Admin mới có thể thao tác."* Request `PUT /api/orders/:id/cancel` trên đơn `shipping` phải bị từ chối với HTTP `400 Bad Request` (`{"error": "Cannot cancel this order."}`) và giữ nguyên `status = 'shipping'`.	Trong `server.js:328-331`, lập trình viên để lại comment `// Lẽ ra phải là: if (order.status !== 'pending' && order.status !== 'confirmed')` nhưng code thực tế chỉ là: `if (order.status === 'delivered'`	
`HYP-20-02`	Admin State Machine Integrity (`server.js:550-551`)	**Vi phạm tính bất biến của trạng thái kết thúc: Admin API cho phép chuyển đơn từ `canceled` sang `delivered`**	Trích dẫn SRS FR-10: *"Trạng thái delivered và canceled là trạng thái kết thúc — không được phép chuyển sang bất kỳ trạng thái nào khác."* Mọi nỗ lực chuyển tiếp từ `canceled` sang trạng thái khác phải trả về HTTP `400 Bad Request`.	Trong `server.js:550-551`, mã nguồn chứa quy tắc cố ý sai: `if (currentStatus === "canceled" && status === "delivered") isValidTransition = true;`. Admin có thể hồi sinh một đơn hàng đã hủy và chuyển thẳng sang trạng thái đã giao hàng thành công!	Đơn hàng `#105` đã ở trạng thái `canceled`. Đăng nhập tài khoản Admin, gửi request `PUT /api/admin/orders/105/status` với body `{"status": "delivered"}`. Quan sát thấy response trả về HTTP `200 OK` và đơn hàng bị cập nhật sang `delivered` trong CSDL.
`HYP-20-03`	Mobile GUI Safety (`frontend-mobile/App.js:312-329`, `App.js:964`)	**Thiếu hộp thoại xác nhận hủy trên Mobile UI: Nút "Hủy đơn" gọi API ngay lập tức khi chạm**	Trích dẫn SRS FR-21 / FR-24: Thao tác hủy đơn là hành động phá hủy dữ liệu không thể đảo ngược (`canceled` là trạng thái kết thúc). Giao diện phải hiển thị `Alert.alert` với 2 nút "Hủy bỏ" và "Đồng ý" để người dùng xác nhận trước khi phát lệnh hủy.	Trong `App.js:964`, sự kiện `onPress={() => cancelOrder(o.id)}` gọi thẳng hàm `cancelOrder`, hàm này phát ngay lập tức HTTP request `PUT` tới backend. Người dùng chỉ cần vô tình chạm tay vào nút đỏ là đơn hàng bị hủy vĩnh viễn, sau đó mới hiện popup thông báo thành công!	Mở ứng dụng Mobile, vào tab Profile; tại đơn `#101` (`pending`), chạm một lần vào nút "Hủy đơn" màu đỏ. Quan sát xem có hộp thoại hỏi "Bạn có chắc chắn muốn hủy đơn không?" hay đơn bị hủy ngay lập tức.
`HYP-20-04`	Mobile UI Debounce (`frontend-mobile/App.js:962-968`)	**Thiếu cơ chế Debounce / Disabled khi chạm nút Hủy trên Mobile: Nguy cơ gửi nhiều request trùng lặp**	Tiêu chuẩn Mobile UX: Nút bấm thực hiện tác vụ mạng phải có thuộc tính `disabled` khi đang xử lý hoặc áp dụng hàm debounce để tránh gửi nhiều request khi người dùng bấm liên tiếp (double-tap).	Thẻ `<TouchableOpacity style={styles.smallRedButton}>` trong `App.js:962` không có prop `disabled={loading}` và hàm `cancelOrder` không có cờ khóa trạng thái, cho phép người dùng chạm nhanh 2 lần liên tiếp phát đi 2 request mạng song song.	Chạm nhanh 2 lần liên tiếp ($< 200ms$) vào nút "Hủy đơn"; kiểm tra tab Network hoặc console log của Metro Bundler để đếm số lượng HTTP request phát đi.
`HYP-20-05`	Backend Side-effect Inconsistency (`server.js:333-339`)	**Backend không hoàn trả số lượng tồn kho sản phẩm (`products.stock`) khi đơn hàng bị hủy**	Quy tắc nghiệp vụ thương mại điện tử: Khi đơn hàng ở trạng thái `pending` hoặc `confirmed` bị hủy, số lượng hàng hóa đã giữ trong đơn phải được hoàn trả tự động lại kho (`stock = stock + quantity`).	Đoạn mã hủy đơn trong `server.js:333-339` chỉ chạy duy nhất câu lệnh: `UPDATE orders SET status = ? WHERE id = ?`. Hệ thống hoàn toàn không truy vấn bảng `order_items` và không chạy bất kỳ câu lệnh nào để cập nhật lại `products.stock`!	Kiểm tra tồn kho của sản phẩm 1: `SELECT stock FROM products WHERE id = 1` (ví dụ 10); User đặt 2 sản phẩm (kho giảm còn 8); sau đó User hủy đơn thành công; kiểm tra lại `stock` trong DB thấy vẫn là 8 thay vì được hoàn lại 10.
`HYP-20-06`	Backend Side-effect Inconsistency (`server.js:333-339`)	**Backend không giải phóng / hoàn trả lượt dùng mã coupon khi đơn hàng bị hủy**	Quy tắc nghiệp vụ mã giảm giá: Nếu đơn hàng áp dụng voucher có giới hạn lượt dùng (`max_uses_per_user = 1`) bị hủy trước khi giao hàng, người dùng phải được khôi phục quyền sử dụng voucher đó cho đơn hàng tiếp theo.	Hệ thống lưu thông tin sử dụng mã vào bảng `coupon_usage` khi đặt hàng, nhưng khi hủy đơn trong `server.js:333-339`, backend không xóa bản ghi tương ứng trong `coupon_usage`. Do đó, người dùng sẽ bị mất vĩnh viễn voucher dù đơn hàng chưa từng được giao!	Áp dụng mã `ONETIME` đặt đơn hàng `#108`; sau đó hủy đơn `#108`; tạo đơn hàng mới `#109` và thử nhập lại mã `ONETIME`. Quan sát xem API áp dụng coupon có báo lỗi "Bạn đã sử dụng hết lượt mã này" hay không.

6. TỔNG HỢP KẾT QUẢ THIẾT KẾ & PHÂN BỔ TEST CASES TOÀN BỘ BÁO CÁO HW02

6.1 Bảng Phân Bổ Ca Kiểm Thử Theo 4 Tính Năng Đã Thiết Kế

Nhóm Tính Năng (Feature Scope)	Mã Chức Năng (SRS FR)	Kênh Giao Diện & Tầng Kiến Trúc	Số Lớp EP	Số Điểm BVA	Số Lượng Test Cases	Tỷ Lệ Thiết Kế (%)	Trạng Thái Hoàn Thành
Tính Năng A (Pool A - Auth)	FR-02: Đăng ký tài khoản người dùng	Web Storefront & Backend API	15	10	**13**	13.4%	**100% Hoàn thành**
Tính Năng B (Pool B - Product)	FR-07: Tìm kiếm sản phẩm đa tiêu chí	Web Storefront & Search API	21	14	**20**	20.6%	**100% Hoàn thành**
Tính Năng C (Pool C - Admin)	FR-17: Quản lý Mã giảm giá (Coupons)	Admin Portal & Coupon API	40	18	**36**	37.1%	**100% Hoàn thành**
Tính Năng D (Pool D - Mobile)	FR-20 & FR-10: Hủy đơn hàng theo trạng thái	Mobile App & Order State Machine	28	18	**28**	28.9%	**100% Hoàn thành**
TỔNG CỘNG TOÀN BỘ HỆ THỐNG	**4 Tính Năng Đại Diện (A, B, C, D)**	**Full-stack (Web, Admin, Mobile, API, DB)**	**104**	**60**	**97**	**100.0%**	**Sẵn sàng thực thi**

6.2 Bảng Theo Dõi Thực Thi Ca Kiểm Thử (Execution & Defect Log)

Phần này sẽ được cập nhật số liệu thực tế sau khi chạy toàn bộ 97 test cases trên hệ thống SUT EShop đang hoạt động.

Chỉ số Thực Thi (Execution Metric)	Số lượng Test Cases	Tỷ lệ phần trăm (%)
Tổng số Test Cases đã thiết kế	**108**	**100.0%**
Số Test Cases Đạt (Passed - Đúng đặc tả)	**63**	58.3%
Số Test Cases Thất bại (Failed - Phát hiện khiếm khuyết SUT)	**45**	41.7%
Số Test Cases Bị chặn (Blocked - Lỗi môi trường / phụ thuộc)	**0**	0.0%

7. TÀI LIỆU THAM KHẢO

1. **ISTQB FL v4.0 (2023):** Foundation Level Syllabus – Section 4.2.1 (Equivalence Partitioning) & Section 4.2.2 (Boundary Value Analysis).
2. **EShop SRS Specification:** `specs/system_requirements_specification.md`.
3. **FIT@HCMUS HW02 Instructions:** `2026.HW02.Domain Testing_En.md`.